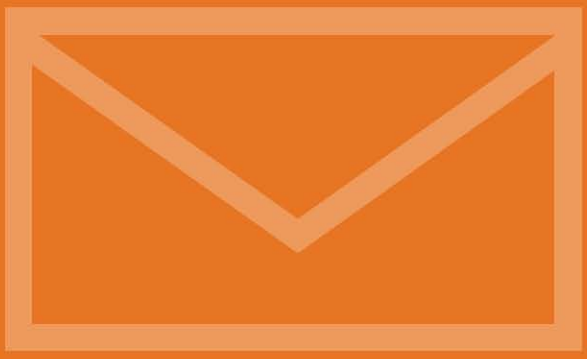


END-TO-END ENCRYPTED MESSAGING

ROLF OPPLIGER

End-to-End Encrypted Messaging

For a complete listing of titles in the
Artech House Information Security and Privacy Series,
turn to the back of this book.

End-to-End Encrypted Messaging

Rolf Oppliger



ARTECH HOUSE

BOSTON | LONDON

artechhouse.com

Library of Congress Cataloging-in-Publication Data
A catalog record for this book is available from the U.S. Library of Congress.

British Library Cataloguing in Publication Data
A catalogue record for this book is available from the British Library.

Cover design by John Gomes

ISBN 13: 978-1-63081-732-9

© 2020 Artech House
685 Canton Street
Norwood, MA 02062

All rights reserved. Printed and bound in the United States of America. No part of this book may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage and retrieval system, without permission in writing from the publisher.

All terms mentioned in this book that are known to be trademarks or service marks have been appropriately capitalized. Artech House cannot attest to the accuracy of this information. Use of a term in this book should not be regarded as affecting the validity of any trademark or service mark.

10 9 8 7 6 5 4 3 2 1

To Marc

Contents

Preface xi

Acknowledgments xv

Chapter 1 Introduction 1

Chapter 2 Internet Messaging 11
- 2.1 Introduction 11
- 2.2 E-Mail 12
  - 2.2.1 Internet Message Format 16
  - 2.2.2 E-Mail Protocols 24
  - 2.2.3 Recent Enhancements 31
- 2.3 Instant Messaging 33
- 2.4 Final Remarks 36

Chapter 3 Cryptographic Techniques 41
- 3.1 Introduction 41
  - 3.1.1 Cryptology 41
  - 3.1.2 Cryptographic Systems 43
  - 3.1.3 Historical Background Information 55
- 3.2 Cryptosystems 57
  - 3.2.1 Unkeyed Cryptosystems 57
  - 3.2.2 Secret Key Cryptosystems 63
  - 3.2.3 Public Key Cryptosystems 72
- 3.3 Certif cate Management 85
  - 3.3.1 Introduction 85

3.3.2 X.509 Certif cates 89
3.3.3 OpenPGP Certif cates 95
3.3.4 State of the Art 97
3.4 Final Remarks 99

Chapter 4 Secure Messaging 105
4.1 Threats and Attacks 105
4.1.1 Passive Attacks 106
4.1.2 Active Attacks 108
4.2 Aspects and Notions of Security 110
4.3 Final Remarks 114

Chapter 5 OpenPGP 117
5.1 Origins and History 117
5.2 Technology 120
5.2.1 Preliminary Remarks 121
5.2.2 Key ID 122
5.2.3 Message Format 123
5.2.4 PGP/MIME 128
5.2.5 Cryptographic Algorithms 131
5.2.6 Message Processing 138
5.2.7 Key Management 144
5.3 Web of Trust 146
5.3.1 Keyrings 146
5.3.2 Trust Establishment 147
5.3.3 Key Revocation 152
5.3.4 Key Servers 154
5.4 Security Analysis 156
5.4.1 Specif cation 156
5.4.2 Implementations 157
5.5 Final Remarks 159

Chapter 6 S/MIME 163
6.1 Origins and History 163
6.2 Technology 166
6.2.1 Message Formats 167
6.2.2 Cryptographic Algorithms 176
6.2.3 Signer Attributes 178
6.2.4 Enhanced Security Services 179
6.3 Certif cates 182

6.4 Security Analysis 184
6.5 Final Remarks 185

Chapter 7 Evolutionary Improvements 189
7.1 WKD and WKS 189
7.2 DNS-Based Distribution of Public Keys 191
7.3 Opportunistic Encryption 191
7.3.1 Autocrypt 192
7.3.2 p≡p 194
7.4 Web-Based Solutions 196
7.5 Final Remarks 198

Chapter 8 OTR 201
8.1 Origins and History 201
8.2 Technology 205
8.2.1 OTR AKE 206
8.2.2 Diff e-Hellman Ratchet 213
8.2.3 Message Processing 215
8.3 Security Analysis 217
8.4 Final Remarks 217

Chapter 9 Signal 221
9.1 Origins and History 221
9.2 Technology 224
9.2.1 Key Agreement and Session Establishment 225
9.2.2 Double Ratchet Mechanism 231
9.2.3 Authentication Ceremony 237
9.2.4 Group Messaging 239
9.3 Security Analysis 241
9.4 Implementations 243
9.4.1 Viber 243
9.4.2 Wire 250
9.4.3 Riot 252
9.5 Final Remarks 255

Chapter 10 WhatsApp 261
10.1 Origins and History 261
10.2 Implementation Details 262
10.2.1 Transport Layer Security and Complementary Technologies 263
10.2.2 Cryptographic Algorithms and Key Generation 264

10.2.3 Message Attachments 264
10.2.4 Group Messaging 265
10.3 Security Analysis 268
10.4 Final Remarks 269

Chapter 11 Other E2EE Messengers 271
11.1 iMessage 272
11.2 Wickr 275
11.3 Threema 281
11.4 Telegram 289

Chapter 12 Privacy Issues 297
12.1 Introduction 297
12.2 Self-Destructing Messaging 299
12.3 Online Presence Indication 301
12.4 Final Remarks 302

Chapter 13 Conclusions and Outlook 305

Appendix A Mathematical Notation 313

Appendix B Abbreviations and Acronyms 315

About the Author 325

Index 327

Preface

In 2001, I wrote *Secure Messaging with PGP and S/MIME* that was published as the fourth title in the then newly established *Information Security and Privacy* book series of Artech House.[1] At this point in time, the topic of the book—secure messaging—was largely dominated by PGP and S/MIME, and both technologies were sufficiently stable and significant to be addressed in a book of its own. Due to their maturity and significance, we decided to include the acronyms PGP and S/MIME in the book title.

But secure messaging with PGP and S/MIME turned out to be not as successful as originally anticipated. When I revisited the topic in 2014, I had to realize that I could not produce a second edition of *Secure Messaging with PGP and S/MIME*. Instead, several trends had changed the field substantially:

- Purely text-based messaging had been replaced or at least complemented by multimedia messaging, simultaneously comprising text, voice, and video;[2]
- The asynchronous nature of messaging (e-mail) had been replaced or at least complemented by synchronous messaging called instant messaging (IM);
- People had realized that hybrid encryption and digital signatures are not the only cryptographic techniques in town, and that certain use cases require other techniques and properties, such as forward secret encryption and plausible deniability;

1 The book series was then called *Computer Security*.

2 This trend is also illustrated by the fact that the formerly popular *short message service* (SMS) has been replaced with a service called *multimedia messaging service* (MMS). Most E2EE messengers in use today don't support encrypted SMS/MMS anymore, but always transmit encrypted messages over the Internet.

- The distributed and open nature of Internet messaging had been challenged by large companies providing centralized and proprietary messaging services (that are very convenient to use).

All of these trends led to a situation in which PGP and S/MIME were not the only technologies for secure messaging, and I therefore had to expand the scope of the revised book a little bit. The resulting book was entitled *Secure Messaging on the Internet*, and it was published as the 39th book in the series.

In the past six years, the above-itemized trends have intensif ed, and several new approaches and respective messaging protocols have evolved over time. Similar to e-mail, some of the respective protocols are based on standards, while others are based on nonstandard and proprietary protocols. Like PGP and S/MIME, some protocols provide *end-to-end encrypted* (E2EE) messaging using very similar technologies. But some protocols go one step further and provide additional features that are more in line with the requirements of today's messaging users, such as off-the-record (OTR) messaging that provides forward secret encryption and plausible deniability. Also, some large companies have come up with E2EE-enabled messengers, such as Apple with iMessage and Google with Allo's Incognito Mode.[3] Furthermore—and even more after the revelations of Edward Snowden in 2013—several E2EE messengers have been launched, such as Threema, Viber, Wickr, Telegram, Wire, and maybe most importantly, TextSecure, which has been the starting point for Signal and the E2EE messaging feature of WhatsApp. The cryptographic protocol that was originally developed for TextSecure and later used in Signal and WhatsApp was originally called *Axolotl* and later renamed to *Signal*. Today, Signal is the protocol of choice for most E2EE messengers and respective apps in use. As PGP and S/MIME dominated the f eld in the 1990s and 2000s, the Signal protocol clearly dominates the f eld in E2EE messaging today, and this is not likely to change anytime soon.

Against this background, I had to realize that the f eld had again changed substantially, and that the topic, secure messaging on the Internet, deserved another update. This insight was even fortif ed by EFAIL[4] and some related attacks that demonstrated that the cryptographic primitives used in most S/MIME and OpenPGP implementations was buggy and somewhat out of date. Since 2017, the S/MIME and OpenPGP specif cations have been adapted to comprise more modern cryptographic primitives, such as authenticated encryption and elliptic curve cryptography (ECC). This has improved the situation considerably, but it has not led to a revitalization of OpenPGP or S/MIME.

3 The development of Google Allo was stopped in 2018.
4 https://efail.de.

The evolution and mode of operation of the Signal protocol is key to understand E2EE messaging as it stands today. Any book about this topic needs to explain the Signal protocol from scratch and explain the rationale behind its design in greater detail. This is the major purpose of this book: In addition to the conventional approaches to secure messaging, it explains the modern approaches messengers like Signal are based on. The resulting book is entitled *End-to-End Encrypted Messaging*. OpenPGP and S/MIME are still addressed to explain the roots and origins of secure messaging, but the focal point of the book is really the Signal protocol and its implementation and use in WhatsApp. For the sake of completeness, some other E2EE messengers are explained, as well. Some of them may not withstand the proof of time.

The bottom line is that *End-to-End Encrypted Messaging* is an entirely new book. In some sense, it can be seen as a third edition of *Secure Messaging with PGP and S/MIME* or a second edition of *Secure Messaging on the Internet*. This means that there are some parts of these books that have been reused, but most parts are new and written from scratch (this even applies to the parts that refer to OpenPGP and S/MIME). I hope that the new structure of the book better ref ects the shift in industry, and that the book better serves the needs of today's practitioners working in the messaging f eld. Most books are written to be used in practice, and this also applies to *End-to-End Encrypted Messaging*—I hope it serves its intended purpose and the needs of its readers.

I would like to take the opportunity to invite you as a reader of this book to let me know your opinion and thoughts. If you have something to correct or add, please let me know. If I haven't expressed myself clearly, please let me know, too. I appreciate and sincerely welcome any comments or suggestions to improve the book and possibly update it in a couple of years. The best way to reach me is to send an e-mail—whether cryptographically protected or not—to `rolf.oppliger@esecurity.ch`. You may also visit the book's website at `https://www.esecurity.ch/Books/e2ee.html` to f nd the latest information about the book, or visit my blogs at `https://blog.esecurity.ch` for information security and privacy, `https://cryptolog.esecurity.ch` for cryptology, and `esecurity.academy` for courses and seminars related to the topic. In any case, I'd like to take the opportunity to thank you for choosing this book and for hopefully reading it. Note that this book can only serve its purpose if it is actually read and taken into account when solving real-world problems in the realm of E2EE messaging. This book has not been written for the bookcase, and you are inivted to challenge the book and actively work with it as much as possible.

Acknowledgments

It is a pleasure to acknowledge the people who have contributed to the conception, research, writing, and production of a book. First of all, I want to thank the people who were involved in the publication of *Secure Messaging with PGP and S/MIME* (2001) and *Secure Messaging on the Internet* (2014), and the people who have provided feedback to these books. The feedback has found its way into *End-to-End Encrypted Messaging*, and has helped to improve it. Next, I want to thank the people who have directly contributed to *End-to-End Encrypted Messaging* by cooperating with me, sharing ideas, or answering (sometimes silly) questions. Standing representatively for many others, I want to name Phil Zimmermann and Manuel Kasper. I am particularly grateful to Stefan Rass, who reviewed the entire manuscript in a timely manner and provided exceptionally useful comments. Also, I sincerely thank the people at Artech House, who have been enormously helpful and appreciative. I owe a lot to Aileen Storry, Soraya Nair, and David Michelson. Last but not least (but above all), I want to thank my wife, Isabelle, for her love and support during the period of time when the book was produced. I am fully aware that I was overworked and unsupportive, and that bearing with me was not easy. Like the previous books on secure messaging, this book is again dedicated to our son Marc, who—by the way—has grown up and become an active Internet messaging user himself.

Chapter 1

Introduction

Electronic mail has been—and still is—one of the most important and widely deployed network applications in use today. More commonly called *e-mail*, or *mail* in short, it enables users to send and receive written correspondence over wide area or even global networks, such as the Internet [1]. A big percentage of all correspondence that has previously gone via physical media and traditional communication channels, such as postal delivery, is currently being exchanged via e-mail. However, in spite of its importance for private and business communications, e-mail used natively must still be considered to be insecure. This is particularly true if the Internet is used for message delivery. An attacker can read, spoof, modify, or even delete messages while they are stored, processed, or transmitted between computer systems. This is because the entire e-mail system—including the message user agents (MUAs) and message transfer agents (MTAs)—has not been designed with security in mind or even with security being a priority.

In the late 1980s and early 1990s, there was some effort to put strong security features into message handling systems (MHSs) based on the X.400 series of recommendations issued by the Telecommunication Standardization Sector of the International Telecommunication Union (ITU-T).[1] The resulting security architecture for X.400-based MHSs has been extensively described and discussed in the literature [2]. But in the real world, security only plays a minor role in the commercial value and success of a standard or product, and this rule of thumb also applies to MHSs (a respective discussion can, for example, be found in [3]). Hence, there is no market for X.400-based MHSs with built-in security features (at least not outside military environments), and this book does not even address them. The same is true for the Message Security Protocol (MSP[2]) that has been specified by the U.S. Department

1 The X.400 series of ITU-T recommendations was first published in 1984. In the 1988 revision, however, a comprehensive set of security features was added.

2 The MSP is sometimes also called P42.

of Defense (DoD) for its Defense Messaging System (DMS) [4].[3] Both are irrelevant for commercial applications, and we therefore ignore them in this book as well.

The e-mail systems that are used in the f eld either depend on standardized and open Internet messaging protocols (e.g., SMTP, MIME, POP3, and IMAP4) or use proprietary protocols (e.g., Microsoft Exchange).[4] In either case, additional software must be used to provide security services at or rather above the application layer in a way that is transparent to the underlying network(s) and e-mail system(s).[5] This transparency is important for the commercial value of secure messaging. A message that is secured above the application layer can, in principle, be transported by any e-mail system, including Internet messaging systems, Microsoft Exchange, or even the DMS and X.400-based MHSs mentioned above. The resulting independence from message transfer is important and key for the large-scale deployment and success of secure messaging.

Historically, there have been three primary schemes for secure e-mail on the Internet:

- Privacy enhanced mail (PEM) and MIME object security services (MOSS);
- Pretty Good Privacy (PGP) and OpenPGP;
- Secure MIME (S/MIME).

PEM was an early standardization effort initiated by the Internet Research Task Force (IRTF) Privacy and Security Research Group, and later continued by the Internet Engineering Task Force (IETF) Privacy Enhanced Mail (PEM) Working Group (WG) [5–9].[6] Unfortunately, the PEM specif cation was limited to 7-bit ASCII text messages and a three-layer hierarchy of certif cation authorities (CAs) that constituted the public key infrastructure (PKI) for PEM. Both limitations are overly restrictive, and MOSS was a later attempt to overcome them [10–12]. As its name suggests, MOSS was designed to additionally handle messages that make use of the multipurpose Internet mail extensions (MIME) and to be more liberal with

3 https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nistir90-4250.pdf.

4 The terms open and *proprietary* are often used without precise def nitions. In this book, we use the term proprietary to refer to a computer software product or system that is created, developed, and controlled by a single company. This can be achieved by treating various aspects of the design as trade secrets or through explicit legal protection in the form of patents and copyrights. Contrary to that, the details about an open computer software product or system are available for anyone to read and use, ideally without paying royalties.

5 This is in contrast to network security protocols that operate at the lower layers in the TCP/IP protocol stack, such as the IP security (IPsec) protocol suite or the Secure Sockets Layer (SSL) and Transport Layer Security (TLS) protocols.

6 Both groups no longer exist. The IETF PEM WG was off cially chartered on August 1, 1991, and it was concluded on February 9, 1996; it was active for roughly four and a half years.

regard to PKI requirements. However, MOSS had so many implementation options that it was possible and likely for two independent software developers to come up with MOSS implementations that would not interoperate. MOSS can be thought of as a framework rather than a specif cation, and considerable work in implementation prof ling still needed to be done. Unfortunately, this work was never accomplished.

While PEM and MOSS failed to become commercially successful and silently sank into oblivion, the remaining two secure e-mail schemes (i.e., PGP/OpenPGP and S/MIME) adapted some of the successful features of their predecessors and avoided the less successful ones. Hence, PGP/OpenPGP and S/MIME are the way to go for secure messaging on the Internet—at least when it comes to asynchronous messaging in the form of e-mail. While S/MIME is actually a specif cation, OpenPGP can be thought of as both a specif cation and a software (or a collection of software packages, respectively). OpenPGP and S/MIME are very similar in nature. For example, they both use public key cryptography to digitally sign and envelope messages (as explained later in the book). But there are (at least) two fundamental differences that lead to a situation in which OpenPGP and S/MIME implementations do not interoperate.

- First, OpenPGP and S/MIME use different message formats.
- Second, OpenPGP and S/MIME handle public keys and respective public key certif cates in fundamentally different ways.
  - OpenPGP relies on users exchanging public keys and establishing trust in each other.[7] This informal approach to establish a so-called "web of trust" works well for small workgroups, but it does not scale, meaning that it is prohibitively diff cult to manage a web of trust in large groups.
  - Contrary to that, S/MIME relies on public key certif cates that are issued by off cial—or at least off cially looking—and hierarchically organized CAs, and may be distributed by respective directory services.

Beginning in 1997, OpenPGP and S/MIME had both been standardized by two distinct IETF WGs within the Security Area of the IETF, namely the Open Specif cation for Pretty Good Privacy (OpenPGP) WG[8] and the S/MIME Mail Security (SMIME) WG.[9] Both WGs are concluded[10] and have come up with

7 The public key exchange can occur directly or through PGP key servers.
8 http://datatracker.ietf.org/wg/openpgp/.
9 http://datatracker.ietf.org/wg/smime/.
10 The IETF OpenPGP WG was concluded on March 18, 2008, whereas the SMIME WG was concluded on October 12, 2010. The latter was therefore active for two and a half years longer than the former.

respective Request for Comments (RFC) documents that can be used to implement the technologies.

- In the case of OpenPGP, the relevant documents are RFC 4880 [13], specifying the OpenPGP message format, and RFC 3156 [14], specifying ways to integrate OpenPGP with MIME. Either RFC document has been submitted to the Internet standards track and are currently Proposed Standards.
- In the case of S/MIME, the situation is more involved. In fact, there is a huge quantity of RFC documents that refer to different versions of S/MIME, such as RFC 5652 [15] for the cryptographic message syntax, RFC 8550 [16] for the certif cate handling, RFC 8551 [17] for the message specif cation, and many more (cf. Chapter 6). All RFC documents have been submitted to the Internet standards track. While RFC 5652 became an Internet Standard (STD 70) in June 2013, all other RFCs still refer to Proposed Standards.

In addition to these RFC documents, there is hardly any literature that addresses secure messaging on the Internet in general, and OpenPGP and S/MIME in particular. There are some manuals that describe the installation, conf guration, and use of respective plug-ins for MUAs or e-mail clients, but there is hardly any literature that goes beyond the graphical user interfaces (GUIs) of these software packages and also addresses the conceptual and technical approaches followed by OpenPGP and S/MIME.

The same was true almost twenty years ago, when I decided to write a book about secure messaging using PGP and S/MIME. As already pointed out in the Preface, the result of this decision was the book *Secure Messaging with PGP and S/MIME* that appeared in 2001 [18]. In 2014, I updated the book to take into account the emerging trends mentioned in the Preface, namely the increasing use of multimedia and instant messaging, new cryptographic techniques, and centrally-operated and proprietary messengers. The resulting book, *Secure Messaging on the Internet* [19], addresses these trends and describes the respective technologies and solutions from a relatively high level of abstraction, without going into much detail.

In the recent past, the trends mentioned above have continued and amplif ed themselves in a way that several new approaches and respective messaging protocols have evolved. Similar to e-mail, some of the resulting (instant) messaging protocols are based on standards, such as the extensible messaging and presence protocol (XMPP) formerly known as Jabber, while others are based on nonstandard and proprietary protocols. You may refer to Section 2.3 for a brief survey of the instant messaging protocols that are relevant in the f eld.

Like OpenPGP and S/MIME, some of the protocols use cryptographic techniques to provide *end-to-end encryption* (E2EE).[11] But some protocols go one step further and provide additional features that are more in line with the requirements of today's messaging users, such as off-the-record (OTR) messaging that provides forward secrecy and plausible deniability (these terms and the rationale behind them are explained later in the book). OTR messaging was proposed in the early 2000s and challenged common wisdom of only using digital envelopes and signatures in secure messaging. The proposal led to a situation in which new people came up with new proposals to provide secure and E2EE messaging on the Internet. Some of these proposals were preliminary and not thought to the end; but some proposals were sophisticated and built into mainstream products, such as Apple's E2EE feature built into iMessage.

In the early 2010s, however, E2EE messaging on the Internet still lived a shadowy existence. This changed entirely when Edward Snowden went public in 2013. After his revelations, everybody asked for E2EE and wanted to use E2EE messengers and respective messenger apps. Examples include Threema, Viber, Wickr, Telegram, Wire, and—maybe most importantly—TextSecure. Some of these messengers were inspired by OTR and tried to use and combine some new cryptographic techniques—in addition to digital envelopes and signatures—to provide new security properties. Probably the most mature and sophisticated messenger was TextSecure. The cryptographic protocol that had originally been designed for TextSecure was called *Axolotl* and was later renamed to *Signal*—mainly because TextSecure was also renamed and merged with RedPhone to become a messenger called Signal. The Signal protocol is nowadays used in many E2EE messengers, including WhatsApp, Facebook Messenger, and Skype. It is either used by default or as an added value feature that can be activated by the user at will. As OpenPGP and S/MIME dominated the f eld in the 1990s and 2000s, the Signal protocol clearly dominates the f eld in E2EE messaging today. A respective overview and *systematization of knowledge* (SoK) is, for example, provided in [20]. Other surveys are available in [21, 22].[12] In spite of the proliferation of E2EE messaging, there are still a few widely deployed messengers that do not support it, such as WeChat[13] that has almost one billion users mainly in China.

The coexistence of asynchronous (e-mail) and synchronous (instant messaging) messaging today, paired with the dominance of the Signal protocol in E2EE messaging has made it necessary to write a new book. Instead of PGP/OpenPGP

11 While the idea of end-to-end encryption is not new, the term and the respective acronym are newly coined and used mainly in the f eld of secure messaging. The importance of the term is also ref ected in the title of the book.

12 While the focus of [21] is e-mail, the focus of [22] is more related to instant messaging.

13 https://www.wechat.com.

and S/MIME, the focus of this new book is the Signal protocol—its evolution and mode of operation. The aim is to provide a comprehensive introduction into secure and E2EE messaging on the Internet as it stands today. The resulting book, *End-to-End Encrypted Messaging*, is an attempt to bring together and put into perspective all relevant information that is needed to understand E2EE messaging in general, and the Signal protocol (as well as its use in WhatsApp) in particular.

Due to asymmetry in information between providers and users, the market for security products and services is—what economists usually call—a lemon market, in which users lack the possibility to distinguish between secure and insecure products and services. There are several ways to improve the situation for users, ranging from providing a better understanding of technology to regulation. In this book, we clearly follow the first way and try to provide a better understanding of technology used in secure and E2EE messaging on the Internet. We don't think that regulation works in this area.

Unfortunately and due to the limited space in a book, we have to make some assumptions. In particular, we have to assume that the reader is familiar with both the fundamentals of TCP/IP networking and the basic concepts of cryptology. Some points are briefly mentioned in this book (e.g., the protocols that are used for Internet messaging), but most aspects are assumed to be known by the reader. Refer to [23, 24] for a comprehensive introduction to TCP/IP networking, or Chapter 2 of [25] for a respective summary. Also, refer to [26] for a comprehensive introduction to contemporary cryptography, or Chapter 3 of this book for a brief summary. Note, however, that this summary is not comprehensive, and that some additional sources of knowledge are needed to properly understand the working principles and the current state of the art in E2EE messaging.

End-to-End Encrypted Messaging is primarily intended for security managers, network practitioners, professional system and network administrators, software engineers, students, and users who want to learn more about the rationale behind E2EE messaging on the Internet. It can be used for self-study or to teach classes and courses. The rest of the book is organized as follows:

- Chapter 2, "Internet Messaging," introduces and briefly summarizes the core technologies that are used for asynchronous (e-mail) and synchronous (instant) messaging. As such, it provides the fundamentals for the book and is only loosely related to security.
- Chapter 3, "Cryptographic Techniques," provides a very brief summary of the cryptographic techniques and building blocks that are used for secure and E2EE messaging on the Internet.

- As its title suggests, Chapter 4, "Secure Messaging," introduces, discusses, and puts into perspective the notions of secure and E2EE messaging. It also introduces and explains terms like (perfect) forward secrecy and post-compromise security that are very important for the topic of the book.
- Chapter 5, "OpenPGP," provides a comprehensive introduction and outline of PGP and OpenPGP.
- Chapter 6, "S/MIME," does the same for S/MIME.
- Chapter 7, "Evolutionary Improvements," elaborates on some improvements that have been made regarding the use of OpenPGP and S/MIME in the f eld. The respective changes are neither fundamental nor radical, and hence the improvements are called evolutionary.
- Chapter 8, "OTR," introduces, discusses, and puts into perspective the f rst fundamental and radical change in secure messaging. It is called OTR, an acronym standing for off-the-record, and it provides features that are closely related to the ones we know from private conversations held in real life.
- Chapter 9, "Signal," explains in detail the Signal messenger and the protocol it uses. The protocol is quite involved and sophisticated, that's why we spend some time explaining it. This chapter is probably the core of the book and the most important one to read.
- Chapter 10, "WhatsApp," elaborates on the way the Signal protocol is used in WhatsApp. This topic is addressed in a distinct chapter, mainly because WhatsApp is by far the most widely deployed and used E2EE messenger in the f eld (at least outside China).
- Chapter 11, "Other E2EE Messengers," overviews several other E2EE messengers that are available and used in the f eld, such as iMessage, Wickr, Threema, and Telegram.
- Chapter 12, "Privacy Issues," addresses two important topics related to privacy: Self-destructing messages (sometimes called disappearing messages) and online presence indication. Both topics are getting more and more important in practice.
- Finally, Chapter 13, "Conclusions and Outlook," rounds up the book by drawing some conclusions and providing an outlook. It also addresses a recent IETF proposal for a message layer security (MLS) protocol that may become important in the future. Many features of Signal will be incorporated into this protocol.

As usual, you may find a list of abbreviations and acronyms, an *About the Author* page, and an index at the end of the book.

While time brings new technologies and outdates current technologies, I have attempted to focus primarily on the conceptual and technical approaches for E2EE messaging on the Internet. The Internet is changing so rapidly that any book is out of date by the time it hits the shelves. *End-to-End Encrypted Messaging* is no exception here. By the time you read this book, several of my comments will probably have moved from the future to the present, and from the present to the past, resulting in inevitable anachronisms. It will even be the case that some comments have shown to be incorrect.

Whenever possible, I have added some uniform resource locators (URLs) as footnotes to the text. The URLs point to corresponding information pages provided on the Web. While care has been taken to ensure that the URLs are valid, due to the dynamic nature of the Web, these URLs, as well as their contents, may not remain valid forever.

If you want to implement and market products or services that employ technologies or techniques mentioned in this book, you have to be cautious and note that the entire fields of cryptography and E2EE messaging are tied up in patents and intellectual property rights. In fact, there are companies that make a living from suing other companies for patent infringements. The situation is complicated and sometimes even bizarre. You must make sure that you have appropriate licenses or good lawyers—or preferably both. The situation regarding software patents is out of control, and there is no simple patch for it. Unfortunately, this book is not able to change this highly unsatisfactory situation.

References

[1] Hughes, L., *Internet E-Mail: Protocols, Standards, and Implementations*, Artech House, Norwood, MA, 1998.

[2] Ford, W., *Computer Communications Security: Principles, Standard Protocols and Techniques*, Prentice Hall, Upper Saddle River, NJ, 1994.

[3] Rhoton, J., *X.400 and SMTP: Battle of the E-Mail Protocols*, Butterworth-Heinemann (Digital Press), Woburn, MA, 1997.

[4] Dinkel, C. (Ed.), "Secure Data Network System (SDNS) Network, Transport, and Message Security Protocols," U.S. Department of Commerce, NIST Internal/Interagency Report NISTIR 90-4250, 1990.

[5] Linn, J., "Privacy Enhancement for Internet Electronic Mail: Part I — Message Encryption and Authentication Procedures," RFC 1421, February 1993.

[6] Kent, S.T., "Privacy Enhancement for Internet Electronic Mail: Part II — Certificate-Based Key Management," RFC 1422, February 1993.

[7] Balenson, D., "Privacy Enhancement for Internet Electronic Mail: Part III — Algorithms, Modes, and Identif ers," RFC 1423, February 1993.

[8] Kaliski, B., "Privacy Enhancement for Internet Electronic Mail: Part IV — Key Certif cation and Related Services," RFC 1424, February 1993.

[9] Kent, S.T. "Internet Privacy Enhanced Mail," *Communications of the ACM*, 36(8), August 1993, pp. 48 – 60.

[10] Galvin, J., and M.S. Feldman, "MIME object security services: Issues in a multi-user environment," *Proceedings of the 5th USENIX UNIX Security Symposium*, Salt Lake City, Utah, June 1995, https://www.usenix.org/legacy/publications/library/proceedings/security95/galvin.html.

[11] Galvin, J., Murphy, S., Crocker, S., and N. Freed, "Security Multiparts for MIME: Multipart/Signed and Multipart/Encrypted," RFC 1847, October 1995.

[12] Crocker, S., Freed, N., Galvin, J., and S. Murphy, "MIME Object Security Services," RFC 1848, October 1995.

[13] Callas, J., et al., "OpenPGP Message Format," RFC 4880, November 2007.

[14] Elkins, M., Del Torto, D., Levien, R., and T. Roessler, "MIME Security with OpenPGP," RFC 3156, August 2001.

[15] Housley, R., "Cryptographic Message Syntax (CMS)," RFC 5652, September 2009.

[16] Ramsdell, B., and S. Turner, "Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Certif cate Handling," RFC 8550, April 2019.

[17] Ramsdell, B., and S. Turner, "Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specif cation," RFC 8551, April 2019.

[18] Oppliger, R., *Secure Messaging with PGP and S/MIME*. Artech House, Norwood, MA, 2001.

[19] Oppliger, R., *Secure Messaging on the Internet*. Artech House, Norwood, MA, 2014.

[20] Unger, N., et al., "SoK: Secure Messaging," *Proceedings of the 2015 IEEE Symposium on Security and Privacy*, 2015, pp. 232–249.

[21] Clark, J., et al., "Securing Email," arXiv 1804.07706, 2018, https://arxiv.org/abs/1804.07706.

[22] Johansen, C., et al., "The Snowden Phone: A Comparative Survey of Secure Instant Messaging Mobile Applications," arXiv 1807.07952, 2018, https://arxiv.org/abs/1807.07952.

[23] Comer, D., *Computer Networks and Internets*, 6th Edition, Pearson India, 2018.

[24] Tanenbaum, A.S., and D.J. Wetherall, *Computer Networks*, 5th Edition, Prentice-Hall, Upper Saddle River, NJ, 2010.

[25] Oppliger, R., *Internet and Intranet Security*, 2nd Edition, Artech House, Norwood, MA, 2002.

[26] Oppliger, R., *Contemporary Cryptography*, 2nd Edition, Artech House, Norwood, MA, 2011.

Chapter 2

Internet Messaging

In this chapter, we introduce and brief y overview the core technologies used for Internet messaging (not yet related to security). More specif cally, we introduce the topic in Section 2.1, elaborate on e-mail and instant messaging in Sections 2.2 and 2.3, and conclude with some f nal remarks in Section 2.4. Note that this chapter is intentionally kept short, and that it only provides a broad and superf cial overview (or summary, respectively). If you want to get more details, then you may refer to the documents referenced throughout the chapter.

2.1 INTRODUCTION

Generally speaking, the term *messaging* refers to the transmission and exchange of messages over a communication network. If the network is the Internet, then the more precise term *Internet messaging* is used. The f rst Internet messaging application that has become popular is e-mail; it is used to send and receive mostly text-based messages in an asynchronous (store and forward) manner. But as already mentioned in the Preface, there are several trends that have changed the nature of Internet messaging fundamentally. Most importantly, text-based messaging has been replaced by multimedia messaging, simultaneously comprising text, voice, image, and video, and the asynchronous nature of e-mail has been replaced or complemented by real-time and synchronous forms of messaging—collectively referred to as *instant messaging*. These trends (and the other trends mentioned in the Preface) have had and continue to have a deep impact on the way people use the Internet for messaging.

While the world of e-mail is clearly dominated by open standards and decentralized (or federated) implementations, the world of instant messaging is more

dominated by proprietary protocols and centralized (or nonfederated) implementations. Large companies like Facebook (with its own messenger and its subsidiaries WhatsApp and Instagram) clearly dominate this f eld; they have a huge installed user base and an impressive market power.

Internet messaging did not start with security in mind. In fact, the various security solutions for e-mail had only been developed once e-mail had proven to be successful in the f rst place. The same is true for instant messaging: In the early days, instant messaging apps were launched without providing support for encryption. This changed when the technology proved to be successful, and especially after the revelations of Edward Snowden. Since then, many software developers have started to build security features into their messengers and messenger apps, and there is a huge proliferation of such apps today. The magic word is encryption in general, and E2EE in particular, and there is hardly any new app that does not provide support for E2EE in one way or another. Needless to say, this development is welcome and highly appreciated from a security perspective.

The aim of this chapter is to introduce and brief y overview the core technologies used for Internet messaging today. We do this separately for e-mail and instant messaging, but—due to a lack of widely deployed standards in instant messaging—the focus is primarily on e-mail. We often refer to standards specif ed by the IETF (e.g., [1]). The Internet standardization process is not addressed in this book. It is, for example, outlined in [2] and partly updated in [3].

2.2 E-MAIL

As mentioned in the previous chapter, e-mail started its breakthrough and triumphal procession with TCP/IP networking in general, and the Internet in particular. In contrast to instant messaging, e-mail typically conforms to IETF standards that address various aspects of a messaging infrastructure, such as a particular message format and various protocols for the transfer of messages (i.e., messaging protocols). As discussed later in this chapter, the realm of instant messaging is more dominated by proprietary protocols.

The *Internet mail architecture* is specif ed in informational RFC 5598 [4]. This architecture has its roots in the specif cations of the X.400 series of ITU-T recommendations, but it has evolved and has been further ref ned within the Internet community. While the f rst standardized architecture for Internet mail was relatively simple and only distinguished between the user world, represented by user agents (UAs) or MUAs, and the message transfer world, represented by the MHS that basically consists of message transfer agents (MTAs) and message stores (MSs), the current Internet mail architecture is slightly more involved and f ne-grained,

and comprises some additional components. The aim of this section is to introduce, brief y discuss, and put into perspective this architecture and its main components. As such, the following components are the most relevant ones:

- A message (or e-mail message) is a data unit that is transferred and delivered through an MHS.
- A message usually has one originating user (i.e., the originator) and one or several receiving users (i.e., the recipients).
- A user—be it an originator or a recipient—is not directly operating on messages, but employs a piece of application software to do so. Historically, people have used the term UA to refer to such software, but nowadays people prefer and more commonly use the term MUA. This is in line with the current version of the Internet mail architecture. An MUA is typically employed by a user to prepare, send, receive, and read messages. It may be a stand-alone application software—sometimes called a mail client or mailer—or it may be integrated into another application software, such as one for the Web. In fact, Web-based messaging is very popular today. In this case, the functionality of the MUA is mostly provided by a Web server, and the Web browser is only used to display messages. In either case, the MUA provides the user interface to e-mail and the respective MHS.
- A message transfer system (MTS) basically consists of a collection of MTAs. A message is submitted by an MUA at the originating MTA and then stored and forwarded along a message delivery path to the receiving MTA.
- Each MTA may contain one or several MSs to store e-mail messages on the users' behalf. The users, in turn, employ their MUAs to access their MSs.

In addition to these functional components, there are three technologies at the core of Internet messaging and Internet-based MHSs:

- The Simple Mail Transfer Protocol (SMTP) specif ed in RFC 5321 [5] is used to transfer messages through the Internet—most notably between MTAs. Note that SMTP is a protocol that addresses the transfer of a message and not its format (the format is addressed in the companion RFC 5322 [6]). There are many implementations of SMTP that can be used to operate an MTA. Examples include Sendmail (now called MeTA1[1]), Postf x,[2] and qmail,[3] as well as many commercial implementations from software vendors, such as

1 http://www.meta1.org.
2 http://www.postf x.org.
3 http://cr.yp.to/qmail.html.

Microsoft and Oracle. For the purpose of this book, we ignore the details and just talk about MTAs. There are entire books on the conf guration and proper operation of a single MTA, such as [7] in the case of Sendmail.

- As just mentioned, the Internet message format (IMF) is specif ed in RFC 5322 [6] and updated in RFC 6854 [8] for group addresses. In essence, the IMF def nes the format of messages or message objects that are to be transferred through the Internet.
- The multipurpose Internet mail extensions (MIME) def ne enhancements to message objects that permit using multimedia attachments [9–13]. As such, the use of MIME is not restricted to e-mail and has many applications beyond Internet messaging. In fact, many applications started being text-based and later evolved to support multimedia data. The bottom line is that MIME is a core technology for the Internet as it stands today.

While MTAs use SMTP to send and receive messages, MUAs typically use SMTP only to send messages. To receive messages, they usually employ a message store access protocol, such as the Post Off ce Protocol (POP) currently in version 3 (POP3) or the Internet Message Access Protocol (IMAP) currently in version 4 (IMAP4). As further addressed in Section 2.2.2.2, the main difference between POP3 and IMAP4 is that the former typically downloads the messages from an MS to an MUA, whereas the latter leaves the messages on the MS. This is in line with the current trend towards service-oriented architectures (SOA) and cloud computing.

Originally, SMTP servers and respective MTAs were located at the border of an organization, typically receiving messages for the organization from the outside world and relaying messages from the organization to the outside world. However, as time went on, these MTAs were expanding their roles to actually become message submission agents for users located outside the organization (e.g., employees who wished to send messages while being on a business trip). This led to a situation in which SMTP had to include specif c rules and methods for relaying messages and authenticating users to prevent abuse, such as the relaying of unsolicited bulk e-mail (UBE)—also known as *spam*. During the 1990s, the separation of message submission and relay became a best (security) practice for Internet messaging [14, 15], and this f nally culminated in RFC 6409 [16] (that made [14] obsolete). According to this RFC, it is required that MUAs are properly authenticated and authorized before they can make use of the mail submission service provided by so-called message submission agents (MSAs). There are many possibilities for handling MUA authentication and authorization. In the simplest case, the MUA simply provides some credentials, like a username and password, on the user's behalf. This means that the user conf gures his or her MUA with his or her credentials, and that

the MUA then provides these credentials whenever appropriate (or required by the server, respectively).

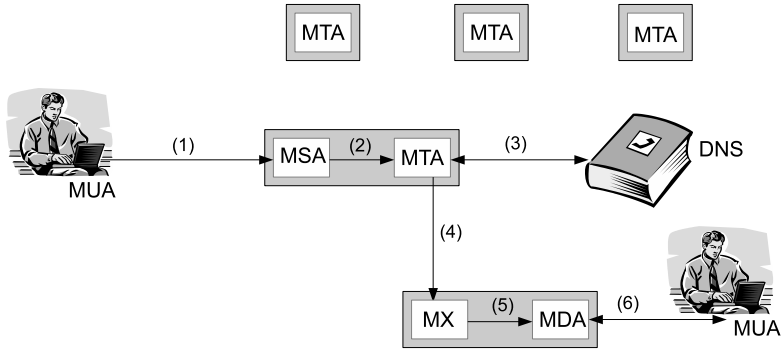


Figure 2.1 A simplif ed version of the Internet mail architecture according to RFC 5598.

A simplif ed version of the Internet mail architecture according to RFC 5598 [4] is illustrated in Figure 2.1. The user on the left side wants to send a message to the recipient on the right side. He or she therefore uses an MUA to prepare and submit the message to his or her MSA in step (1). As outlined above, the protocol of choice for this submission is a variant of SMTP specif ed in RFC 6409 [16], where the MSA typically resides on port 587 (instead of the "normal" SMTP port 25). The MSA, in turn, verif es the format of the message and, if needed, modif es or extends some header f elds. The MSA then forwards the message to the MTA that typically resides on the same machine in step (2).[4] There are many other MTAs available on the Internet (as illustrated at the top of Figure 2.1), but a specif c MSA always uses the same MTA—the one it has been conf gured to use. In step (3), the MTA employs the domain name system (DNS) to f nd the server system that is conf gured to act as a mail exchanger (MX) for the recipient's domain. The message is then delivered to this MX in step (4). The MX forwards the message to the appropriate message delivery agent (MDA) in step (5),[5] where it is put into the recipient's MS or

4 Often, the MSA and the MTA are different instances of the same software launched with different options.

5 An MDA is typically able to save messages in the preferred format of the recipient's mailbox. It may deliver messages directly to storage, but it may also forward them over a network using SMTP, or any other means, including, for example, the local mail transfer protocol (LMTP), a derivate of SMTP specif cally designed for this purpose.

mailbox, respectively. From there, the receiving user employs an MUA to access his or her MS or—as in the case of POP3—to retrieve the message in step (6). Again, this access requires proper user authentication and authorization (depending on the message store access protocol in use). But this time, it is the recipient of the message that needs to be authenticated and authorized (in the previous case, it has been the originator of the message). The bottom line is that the simple process of delivering a message can be broken into many pieces that require different technologies to implement. The result is inherently involved and diff cult to outline in a few words. Things would even get worse, if lawful interception were considered and taken into account. This is not done in this book.

2.2.1 Internet Message Format

As mentioned above, the IMF is specif ed in [6] and updated (for group addresses) in [8]. An IMF-compliant e-mail message is illustrated in Figure 2.2. It consists of two parts that are separated with an empty (or null) line: a header section and a message body. As their names suggest, the header section comprises the message headers, whereas the message body comprises the actual contents of the message (note that a message may have multiple contents).

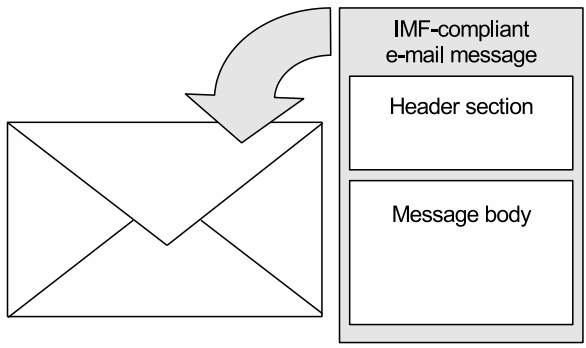


Figure 2.2 An IMF-compliant e-mail message.

Before we delve more deeply into the header section and the body of a message, we have to say a few words about the notion of an e-mail address. In fact, there are many possibilities to specify such an address. It can always be written in angle brackets (i.e., < and >). More specif cally, if a substring is delimited

with angle brackets, then just that substring is interpreted as e-mail address, and anything else is ignored (i.e., treated as a comment). If no substring is delimited with angle brackets, then the entire string is interpreted as e-mail address. Also, any substrings that are delimited by parentheses are considered to be comments and are ignored, as well. For example, the following e-mail addresses are all equivalent to `rolf.oppliger@esecurity.ch`:

```
<rolf.oppliger@esecurity.ch>
Rolf Oppliger <rolf.oppliger@esecurity.ch>
"Rolf Oppliger" <rolf.oppliger@esecurity.ch>
rolf.oppliger@esecurity.ch (Rolf Oppliger)
```

An MUA can use any of these possibilities to refer to a particular recipient. If an e-mail address refers to a group, then it is dissolved into a set of e-mail addresses and every e-mail address of that set receives the same copy of the message.

2.2.1.1 Header Section

According to RFC 5322 [6], the header section of an e-mail message includes an arbitrary number of header fields in no particular order. Each header field occupies one line of characters[6] beginning with a field name, followed by a colon (:), and terminated by a field body that holds one or more parameters for that particular field. The only header fields that are mandatory are the origination date field and at least one originator field.

- The origination date field is named `Date` and carries a timestamp for the message that is generated by the originator of the message. An example may look like this:

  ```
  Date: Tue, 19 Mar 2019 23:25:00 -0400 (EDT)
  ```

  In this example, the message was compiled and submitted on Tuesday, March 19, 2019, shortly before midnight, according to eastern daylight time (EDT). EDT, in turn, derives minus 4 hours from universal time coordinated (UTC).[7]

- The originator fields specify the e-mail addresses or mailbox(es) that represent the source(s) of the message. They consist of at least a from field, but may optionally comprise a sender and a reply-to field. The from field is named

6 Each line of characters must not be longer than 998 characters, and should even not be longer than 78 characters, excluding the closing carriage return (CR) and line feed (LF) characters.

7 UTC is the primary time standard by which the world regulates clocks and time. It is one of several closely related successors to Greenwich mean time (GMT). For most purposes, UTC is synonymous with GMT, but GMT is no longer precisely defined by the scientific community.

`From` and carries a comma-separated list of e-mail addresses (for individuals or groups). A simple example may look as follows:

```
From: alice@esecurity.ch, bob@esecurity.ch
```

In this example, the from field carries two e-mail addresses (for individuals), so the message is originated from either of them and they are both responsible for the writing of the message. In addition, a sender field may be used to specify an agent who is responsible for the actual transmission of the message. For example, if a secretary were to send a message for another person, then the secretary's e-mail address would appear in the sender field and the e-mail address of the actual author would appear in the from field. In the example given above, this field may look like this (if the message was sent out by `carol@esecurity.ch`):

```
Sender: carol@esecurity.ch
```

If the originator of the message can be indicated by a single entity and the author and transmitter are identical, the sender field should not be used. Also, an optional reply-to field may be included to refer to the e-mail address to which a reply should be sent. This field is named `Reply-To` and carries a comma-separated list of one or more mailboxes (note that these mailboxes can be distinct from the ones specified in the from and sender fields). In general, there are many possibilities to combine the various originator fields. The details can be found in RFC 5322.

In addition to the origination date and originator fields (that are mandatory), there are many header fields that are optional (but sometimes strongly recommended) and can be set where appropriate. For example, there are several destination address fields that can be used to specify the recipient(s) of a message. There are three such fields, all of them comprising a field name, a colon (:), and a comma-separated list of one or more e-mail addresses. The fields are as follows:

- The `To` field contains the e-mail address(es) of the primary recipient(s) of the message.
- The `Cc` field contains the e-mail address(es) of other recipient(s) of the message (i.e., the recipient(s) who have a legitimate reason to know the message and all other recipient(s) can be aware of this fact).[8]

8 The term cc stands for carbon copy in the sense of making a copy on a typewriter using carbon paper.

- The `Bcc` field is to contain the e-mail address(es) of yet other recipient(s) of the message (i.e., the recipient(s) who have a legitimate reason to know the message but all other recipient(s) should not be aware of this fact).[9]

Note that any meaningful message must at least include one destination address field—otherwise it may not be delivered.

Next, the identification fields are used to identify messages. Most importantly, every message should have a message identifier field named `Message-ID` that carries a unique[10] character string. This string is intended to be machine readable and not necessarily meaningful to humans; this means that it can be arbitrarily long and look cryptic. It is used to keep track of messages and to link reply messages to them. The following fields are used for this purpose:

- The `In-Reply-To` field contains the identifier of the message(s) to which the message is a reply—the so-called "parent message(s)". Note that there may be multiple parent messages, in which case the `In-Reply-To` field contains all respective message identifiers.
- The `References` field contains the contents of a parent message's `References` field (if any) followed by the parent message's `Message-ID` field (if any).

From the user's point of view, there are several fields that are optional but seem to be important. All of them are intended to have human-readable contents and comprise information about the message. The subject field is the most important one. It is named `Subject` and carries an arbitrary string chosen by the sender that identifies, in some sense, the topic of the message. Similarly, the comments field is named `Comments` and may be used by the sender to add some additional information to the message, whereas the keywords field named `Keywords` may be used to carry a comma-separated list of important words and phrases that might be useful for the recipient of the message.

The trace fields refer to header fields that carry information about the trace of message delivery. There are basically two trace fields, namely an optional `Return-Path` field and one or several `Received` fields.

- The `Return-Path` field is used to specify an e-mail address to which a reply message can be sent. In general, this address is the same as the one specified in the `From` or `Sender` field.

9 The term bcc stands for blind carbon copy.

10 The uniqueness of the message identifier must be guaranteed by the host that generates it.

- The `Received` fields are added by the MTAs during message delivery. This means that each MTA that receives a message prepends a `Received` field before it forwards the message towards its destination. The `Received` field, in turn, may contain information about the originating and receiving MTAs (DNS names and IP addresses), the message transfer protocol, as well as the date and time of the message delivery. Note, however, that this information simply represents text that can be modified at will. It is therefore just informational and cannot be used to provide a proof of message delivery.

Also, there are a number of resent fields that should be added to any message that is reintroduced into the MHS by the user. When resent fields are used, then the `Resent-From` and `Resent-Date` header fields are mandatory, whereas all other fields (e.g., `Resent-Sender`, `Resent-To`, `Resent-Cc`, `Resent-Bcc`, and `Resent-Message-ID`) are optional.

Finally, there is room for optional header fields that must conform to the syntax specified in RFC 5322 but can otherwise contain any information that might be useful. By convention, the names of these header fields begin with the prefix `X-`. If, for example, antispam software is invoked, then the respective header fields are named `X-Spam-Checker-Version`, `X-Spam-Level`, and `X-Spam-Status`. They carry information about the checks performed by the antispam software. Other software may use different `X-`prefixed header fields.

Taking into account the variety of header fields, there are many possible ways to form a header section for a particular message. Hence, one could fill pages and pages with exemplary messages. We don't want to go through this exercise in this book. Instead, you can always have a look at the source code of the messages in your own mailbox or refer to Appendix A of RFC 5322. The examples compiled in there are instructive and give you a good feeling about the expressiveness of the current header fields.

2.2.1.2 Message Body

Following the header section and an empty line, an RFC 5322- and hence IMF-compliant e-mail message must include a message body that consists of zero or more lines of ASCII characters. The only two limitations on the body are that <CR> and <LF> must not appear independently in the message body (i.e., they must only occur together as <CR><LF>), and that lines of characters must be limited to 998 characters, and should be limited to 78 characters, excluding the <CR> and <LF> characters. Except from these limitations, everything is possible in the message body—so there is no need to give examples here.

2.2.1.3 MIME

The IMF specified in RFC 5322 applies to 7-bit ASCII text messages. There are two trends that have led to situations in which the transmission of such messages is overly restrictive:

- On the one hand, today's messages often comprise multimedia data, such as images, sound, and video (in addition to text);
- On the other hand, today's messages often comprise multiple (independent) parts.

To enable the transmission of such messages, people have developed and come up with MIME [9–13]. MIME addresses the problem of transporting arbitrary binary (8-bit) data possibly consisting of multiple parts as 7-bit ASCII text, and hence extends RFC 5322 accordingly. Note that MIME is not specific to Internet messaging, and that it can be used for many other network applications, as well. Most importantly, MIME is heavily used in Web applications.

The MIME specifications introduce six new header fields that can be used by the originator of a message to instruct the recipient(s) on how to interpret data.

- The `Mime-Version` field is used to specify the MIME version in use. The current version is 1.0, so this field typically looks like this:

  ```
  MIME-Version: 1.0
  ```

- The `Content-Type` field is used to specify the MIME type and subtype of the data contained in the message body or any of its body parts. The aim is to enable the receiving MUA to pick the appropriate application to render or represent the data to the user or otherwise deal with it. As illustrated in Table 2.1, many content types and subtypes are possible and all of them require different parameters. For a plain text message using character set ISO/IEC 8859-1, for example, this field may look like this:

  ```
  Content-Type: text/plain; charset="iso-8859-1"
  ```

  So the character set is specified as an additional parameter (i.e., `charset="iso-8859-1"`) separated with a semicolon. The additional parameters that are required depend on the MIME content type and subtype in use. For text messages, for example, it is important to specify a character set as done above. In addition to `iso-8859-1`, many other character sets are possible. If more than one parameter needs to be added, then they must be separated with semicolons.

Table 2.1
MIME Content Types and Subtypes

Type	Subtype	Description
`Text`	`plain`	Unformatted text (e.g., ASCII or ISO-8859).
	`enriched`	MIME enriched text (according to RFC 1896).
	`html`	HTML text (according to RFC 2854).
`Multipart`	`mixed`	The message includes multiple subparts with no particular relationship between them.
	`alternative`	Similar to `multipart/mixed`, except that the various subparts are different versions of the same message (e.g., one ASCII f le and one RTF f le with the same contents).
	`parallel`	Similar to `multipart/mixed`, except that all the subparts are intended to be displayed together (e.g., one audio and one video f le).
	`digest`	Similar to `multipart/mixed`, except that each subpart is an RFC 822-compliant message of its own.
`Message`	`rfc822`	E-mail message that conforms to RFC 822.
	`partial`	Used to allow fragmentation of large messages into a number of parts that must be reassembled at the destination.
	`external-body`	Pointer to an object that exists elsewhere.
`Image`	`gif`	Data in GIF image format.
	`jpeg`	Data in JPEG image format.
`Video`	`mpeg`	Data in MPEG video format.
`Audio`	`basic`	Data in standard audio format.
`Application`	`postscript`	Data in Postscript format.
	`octet-stream`	Binary data consisting of 8-bit bytes.

- The `Content-Transfer-Encoding` f eld is used to specify the transfer encoding for the message content. Possible values are `7bit`, `8bit`, `binary`, `quoted-printable`, `base64`, and `x-token` (standing for a nonstandard vendor-specif c or application-specif c encoding scheme). Obviously, three of these values, namely `7bit`, `8bit`, and `binary`, indicate that no encoding has been used, but provide some further information about the nature of the transported data. Only `quoted-printable`, `base64`, and maybe `x-token` refer to actual encoding schemes. As an example, this f eld may look like this:

  ```
  Content-Transfer-Encoding: 7bit
  ```

  In this case, 7-bit ASCII is used to represent the message content.

- An optional header field named `Content-ID` may be used to uniquely identify a MIME entity.
- An optional header field named `Content-Description` may be used to further describe the body of a MIME entity (e.g., a caption that might be displayed along with an image file).
- An optional header field named `Content-Disposition` may be used to specify the presentation style (i.e., inline or attachment), and to provide some information about the name of a file, the creation date, and the modification date. These additional parameters may be used by the MUA to properly display or store the MIME entity. Unfortunately, many MUAs ignore the contents of the content-disposition headers and take decisions on their own.

Any or all of these header fields may appear in a header section. Any implementation that is compliant with the MIME specifications must at least support the `MIME-Version`, `Content-Type`, and `Content-Transfer-Encoding` header fields. As mentioned above, all other header fields are optional and may be ignored by the receiving MUA.

As summarized in Table 2.1, the MIME specifications define a number of content types and subtypes that can be used to represent multimedia data. The content type specifies the general type of data, whereas the subtype specifies a particular format of that type. The MIME `multipart` type indicates that the message body contains multiple parts. In this case, the `Content-Type` header includes a parameter, called the boundary, that actually defines a delimiter string for the separation of the various body parts of the message (it goes without saying that this delimiter string should not appear elsewhere in the message). Each boundary starts on a new line and consists of two hyphens followed by the delimiter string. The final boundary, which also indicates the end of the last part, also has a suffix of two hyphens. Within each part, MIME headers that are specific for this part may occur.

In an exemplary message, the `Content-Type` header may look like this:

```
Content-Type: multipart/mixed;
   boundary="_005_75FF21C22146D441B7B6551E7FE5B7ED55B48
   30FSB00105Aadbintr_"
```

Afterwards, every MIME entity is separated with the following boundary:

```
--_005_75FF21C22146D441B7B6551E7FE5B7ED55B4830FSB00105A
adbintr_
```

followed by a series of header fields.

As multimedia messaging evolves, the MIME specif cations have also become a moving target. This is particularly true for the MIME types and subtypes. So people have created a central registry to update and provide accurate and up-to-date information about MIME content and respective media types.[11]

2.2.2 E-Mail Protocols

In this section, we brief y overview and put into perspective the various protocols that are used for e-mail. Again, we remain short and superf cial here. Whenever you need more information about a particular protocol, you may go to the respective protocol specif cations—most notably RFC documents. In our exposition, we separately address protocols for message transfer and delivery, message store access, and directory access. All of these protocols are required for an Internet-based MHS to be fully operational.

2.2.2.1 Message Transfer and Delivery

In theory, there are many protocols that can be used for message transfer and delivery. In practice, however, the main protocol in use is SMTP [5] and a few others that are mostly used in proprietary environments, such as Microsoft Exchange. While extended SMTP (ESMTP) was independently specif ed in RFC 1869 [17], the current version of SMTP comprises ESMTP and has made RFC 1869 obsolete. So SMTP is the Internet standard application layer protocol for transferring and delivering e-mail messages. More specif cally, SMTP is used to upload e-mail messages from MUAs to MSAs or MTAs, and to transfer them between MTAs. The f nal MTAs deliver the messages to the appropriate MDUs where they may be accessed and eventually retrieved by the recipients or the receiving MUAs, respectively, either in (near) real-time or at some later point in time.

SMTP is a simple client/server protocol layered on top of TCP, meaning that the underlying transport layer protocol must provide a connection-oriented and reliable data delivery service. An SMTP client may be an MUA or a peer MSA/MTA, whereas an SMTP server is always an MSA/MTA—with or without MSs. By default, an SMTP server (or daemon) listens at the well-known port 25 or 587 in the case of an MSA that requires user authentication. If SMTP runs over SSL/TLS using Secure SMTP (SSMTP), then the default server-side port is 465. If an SMTP client has successfully established a TCP connection to one of these ports, then it can send arbitrary SMTP command messages to the server. The server, in turn, executes the commands and optionally sends back response messages.

11 http://www.iana.org/assignments/media-types.

SMTP command and response messages are ASCII-encoded and not case sensitive. The SMTP command messages consist of a four-letter code usually followed by a string that represents one or several arguments (for the SMTP command). The SMTP response messages, in turn, consist of a three-digit numeric response code, followed by some optional explanatory text, such as:

```
250 OK
```

In this case, the SMTP server signals to the client that it has accepted a command and that everything is thus f ne. The four SMTP response code classes are summarized in Table 2.2.

Table 2.2
SMTP Response Code Classes

Code	Explanatory text
2xx	Request accepted and processed
3xx	Ready to receive message text
4xx	Some service unavailable, possibly temporarily
5xx	Error, request rejected

In general, there are many SMTP commands that a client can use to interact with a server. For example, using the HELO command, a client must f rst specify its domain name (and, optionally, its host name). This command must be the f rst command that follows a TCP connection establishment to the appropriate server port (usually 25). For example, an MUA from domain `esecurity.ch` may send

```
HELO esecurity.ch
```

to the server (without host name). With the introduction of ESMTP, the HELO command was replaced with an extended HELO (EHLO) command that is to identify the sender as supporting ESMTP. If the SMTP server supports EHLO, it sends back a series of `250` messages, one for each extension it actually supports. If the server does not support EHLO, then the client is to continue with SMTP. In either case, we note that a server can be conf gured to accept only particular domains (for security reasons).

After this initial handshake, the MUA may want to send a message on a user's behalf. In this case, it sends a MAIL command to the server. This command basically specif es the originator of the message. In the simplest case, a MAIL command may look like this:

```
MAIL FROM: <sender@senderdomain.com>
```

If the command is accepted, then the server sends back a `250 OK` response message, and the MUA can then specify the recipient(s) of the message using the RCPT command. For every recipient, the MUA must issue a distinct RCPT command that specif es this particular recipient (or a respective forward path). Such an RCPT command may look like this:

```
RCPT TO: <recipient@recipientdomain.com>
```

Again, if the command is accepted, then the server sends back a `250 OK` response message. The next step for the MUA is to use the DATA command to provide the content of the message that may comprise any number of text lines. The only requirement is that the f nal text line consists only of a period or full stop (.).

In former times, SMTP servers were often conf gured in a way that they were open for arbitrary clients to establish a TCP connection to port 25 and compile an e-mail message that was then sent out without further verif cation. To spoof a message, it was then suff cient to use a Telnet client to connect to port 25 of such an SMTP server, wait for the server's response code, and then type in the following command sequence:

```
MAIL FROM: <sender@senderdomain.com>
250 OK
RCPT TO: <recipient@recipientdomain.com>
250 OK
DATA
...
Arbitrary text ...
.
250 OK
QUIT
```

For each command, the server sends back a `250 OK` message (in the positive case). In the end, the server generates a message that originates from `sender@sender-domain.com` and is sent to `recipient@recipientdomain.com`. For such a message, it is very diff cult for the recipient to recognize that it is spoofed. Depending on the actual content of the message, it may be used to mount a social engineering attack. Imagine, for example, what happens if a user receives a (spoofed) message that reads as follows:

```
Dear user,

Due to some necessary system update and reconfiguration,
we ask you to set your password to the temporary value
```

```
"er45w.jk." As soon as we have finished work, we'll let
you know and you can change your password to the old
value. Thanks for your cooperation.

Your system administrator
```

If the adversary spoofs the local system or security administrator's e-mail address in the MAIL command, then it is possible and very likely that the user adheres to the request and changes his or her password to the requested value (that is known to the adversary). If the SMTP server is open, then such a spoof ng attack is simple to mount and highly effective. Luckily, it is also simple to detect and defeat such an attack. In addition to requiring proper user authentication when submitting a message, the following additional countermeasures are applicable:

- The SMTP server can be conf gured to accept only local e-mail addresses as arguments to the MAIL command (this basically means that the actually used domain in verif ed by the server).
- An SMTP proxy server (running at the f rewall of the intranet) can enforce a policy that MAIL commands for outgoing messages cannot comprise external e-mail addresses as arguments.[12]
- Similarly, the same SMTP proxy server can enforce a policy that MAIL commands for incoming messages cannot comprise internal e-mail addresses as arguments.
- The recipient of a spoofed e-mail can sometimes detect the attack by having a closer look at the message source and the `Received` headers. Remember from our previous discussion that the `Received` headers def ne a reverse path to the message originator. If this path is not adapted by the attacker, then it is possible to decide whether a reverse path matches a claimed sender address.

Note, however, that neither the message nor its headers are authenticated by using standard Internet messaging techniques. So it may be the case that somebody is spoof ng entire messages (with all headers) that look f ne and cannot be detected as forgeries. If you want to protect yourself against these kinds of spoof ng attacks, then you must enter the f eld of cryptography and cryptographic protocols.

In addition to the basic SMTP commands mentioned so far, there are many optional SMTP commands that may serve specif c purposes. Examples include the VRFY command that can be used to conf rm that a given address matches an existing mail account; the EXPN command that can be used to expand a mailing list name

12 Obviously, this only works if the attacker does not control the proxy server. Otherwise, the attacker can simply disable the enforcement of this policy.

to a list of subscribed e-mail addresses; the HELP command that can be used to help users who interactively access the SMTP server (using, for example, a Telnet client);[13] the RSET command that can be used to abort a current mail transaction; the NOOP command that does nothing other than verify that the receiving SMTP server is still alive or keep it from timing out; the QUIT command that immediately terminates an SMTP session; and a number of other commands (not even mentioned here).

There are SMTP extensions that have originated from ESMTP and found their way into the current specif cation of SMTP. Table 2.3 summarizes some SMTP extensions that are frequently used in the f eld. We do not delve more deeply into the topic, as more information is available in [17], [18], and the references itemized in Table 2.3.

Table 2.3
Some SMTP Extensions

Keyword	Explanatory text	References
`SIZE`	Declaration of message size	[18]
`PIPELINING`	Command pipelining	[19]
`8BITMIME`	Use 8-bit MIME data	[20]
`STARTTLS`	Invoke SSL/TLS	[21]
`AUTH`	Authentication	[22]
`MTRK`	Message tracking	[23]
`DSN`	Delivery status notif cation	[24]

2.2.2.2 Message Store Access

Besides the protocols that are used in proprietary environments, such as Microsoft Exchange, there are two standard protocols that can be used by an MUA to access a user-specif c MS: POP and IMAP.

POP

POP was the f rst standard protocol to access an MS. It has gone through various versions,[14] where the current version is version 3 (POP3) specif ed in RFC 1939

13 For security reasons, the VRFY, EXPN, and HELP commands are most disabled by default.

14 The f rst version of POP (POP1) was described in RFC 918 back in 1984. The second version of POP (POP2) was specif ed in RFC 937 and off cially released in 1985. The currently used third version of POP (POP3) was released in 1996.

[25], and some extension mechanisms specif ed in RFC 2449 [26]. Similar to SMTP, POP3 is a simple client/server protocol that is layered on top of a reliable transport service, such as the one provided by TCP, and that uses ASCII-encoded messages to serve as command and response strings. Standard commands, like USER, PASS, STAT, LIST, RETR, DELE, NOOP, RSET, and QUIT, are supported by all POP3 servers, whereas optional commands, like APOP (see below), TOP, and UIDL, may be supported at will.

A POP3 server usually listens at port 110. If a client (which is usually an MUA) establishes a TCP connection to this port, the server responds with a status message. The client then authenticates the user with the USER and PASS commands. As their names suggest, the f rst command is to identify the user, whereas the second command is to specify the user password. The actual username and password represent parameters to these commands. Unfortunately—and this is the major security concern regarding POP3—these commands (together with their parameters) may be sent unencrypted to the server, meaning that any passive adversary can easily extract them from the data stream. This is arguably the most serious security vulnerability of POP3, and there are a few possible ways to improve it.

- First, some POP3 servers support the APOP command mentioned above to provide a strong authentication mechanism. In this case, the client does not transmit the password in the clear. Instead, the server provides a timestamp that is combined by the client with the user password to provide an MD5 hash value. This hash value is then transmitted to the server (instead of the password sent in the clear). Hence, the APOP command implements a simple challenge-response mechanism.
- Second, some POP3 servers provide support for the Simple Authentication and Security Layer (SASL) [27] that yields a framework for providing authentication and data security services in connection-oriented protocols via replaceable mechanisms, such as Kerberos. The use of SASL in the realm of POP3 is further addressed in [28].
- Third, it is possible to layer POP3 on top of SSL/TLS to cryptographically protect it. Either the server can listen at a specif c port (the default port number is 995) to take SSL/TLS connections and transparently secure POP3 traff c using such a connection, or the server continues to listen at the normal port and uses SSL/TLS on the f y [29, 30].

Any of these possibilities is f ne and can be used to cryptographically protect POP3 traff c and mitigate respective attacks.

IMAP

IMAP is the second and—in some sense—more advanced MS access protocol. While POP3 is typically used to retrieve messages from the MS and download them to the MUA for local storage, IMAP is often used to manage messages directly in the MS (so the messages reside on the server). This is particularly useful if multiple devices are used to access a particular MS. It is also useful, because the centralized storage simplif es backup considerably.

The current version of IMAP is 4 (IMAP4), which was published (in a revised form) in 2003 [31]. Like SMTP and POP3, IMAP4 is layered on top of a connection-oriented and reliable transport layer service, such as the one provided by TCP, and uses ASCII-encoded commands and responses. An IMAP4 server usually listens at port 143 (instead of port 110 used for POP3).

Similar to POP3, an IMAP4 server needs to authenticate a user prior to serving his or her requests. Because the messages often continue to be stored on the server side, user authentication is even more important in the case of IMAP4 than it is in the case of POP3. Hence, IMAP4 servers usually support many (strong) user authentication mechanisms, as outlined, for example, in RFC 1731 [32]. Also, similar to POP3, IMAP4 can be layered on top of the SSL/TLS protocol [29], either using IMAP over SSL/TLS (IMAPS) on a new port number (default is 993) or using a mechanism to dynamically invoke the SSL/TLS for IMAP4 traff c (on the normal port number 143).

In the past, there has been some work trying to combine IMAP and SMTP in a new protocol named Simple Mail Access Protocol (SMAP).[15] But so far, SMAP has not been particularly successful in the f eld, meaning that there is hardly any deployment of it. We therefore don't expand it in this book and we only mention it for the sake of completeness here.

2.2.2.3 Directory Access

Like many other network applications, e-mail requires that the message originators have access to the addresses of the potential receivers. Hence, there is room for respective directory services. For example, as illustrated in Figure 2.1, when an MTA is to deliver a message to a recipient, it needs to request the DNS to retrieve the respective MX server registered for the recipient's domain. Hence, the DNS serves as the directory service of choice for information regarding hosts and domains. Support for DNS is therefore integrated in all TCP/IP protocol stacks, so there is no need to implement any supplementary directory access protocol. However, when it comes to user-specif c information, the situation is less clear. In fact,

15 http://www.courier-mta.org/cone/smap1.html.

there are many directory services and corresponding implementations. The greatest common divisor of all these services and implementations is that they all provide support for the *Lightweight Directory Access Protocol* (LDAP), of which version 3 is specif ed in RFC 4511 [33].[16] LDAP has evolved from the *Directory Access Protocol* (DAP) that has its roots in directory services that conform to the ITU-T X.500 recommendations. An LDAP server usually listens at default port 389.

From a security and privacy perspective, directory access is crucial, and hence LDAP must provide support for user authentication and authorization. In addition to passwords transmitted in the clear, LDAP also provides support for SASL and LDAP over SSL/TLS (LDAPS). A respective LDAPS server usually listens at the default port number 636 (instead of 389).

2.2.3 Recent Enhancements

More recently, the Internet mail architecture has been enhanced in many regards, such as spam protection and transport layer security. Many of the respective technologies and techniques are, for example, addressed in U.S. NIST SP 800-177 [34] and brief y summarized here.

2.2.3.1 Spam Protection

There are several technologies and techniques that have been developed to protect e-mail users against UBE and *spam*. Examples include *sender policy framework* (SPF), *DomainKeys identf ed mail* (DKIM), *domain-based message authentication, reporting, and conformance* (DMARC), and *greylisting*. None of these technologies and techniques is able to alone protect against spam, but they are not mutually exclusive and complement each other to achieve a reasonable level of protection.

SPF

SPF [35] was developed in the early 2000s as a simple mechanism to protect against spam. The basic idea is that the owner of a domain can specify in DNS (TXT or SPF) records what hosts (in terms of IP addresses) are authorized to act as MTA and send out e-mail messages on the domain's behalf. It is then up to the receiving mail server to look up the respective DNS records and check whether the message originates from a valid mail server. Note that SPF is mainly based on IP addresses, and that it does not employ any form of cryptography.

16 The LDAP is addressed in an entire series of RFC documents (i.e., ranging from RFC 4510 to RFC 4520).

DKIM

Shortly after SPF, Cisco and Yahoo jointly developed a spam protection mechanism that employs cryptography—more specif cally, public key cryptography. The technology is called DKIM [36], and it allows a sending mail server or MTA to digitally sign selected headers and the body of a message with a domain-specif c key. This means that the message is reliably associated with the domain, and hence that the recipient can be sure that the message is originating from the claimed domain. To generate the signatures, the sending mail server must have access to the domain-specif c private key, whereas the receiving mail server must have access to the respective domain-specif c public key. This is usually achieved by storing and making available the domain-specif c public keys in respective DNS TXT records.

DMARC

SPF and DKIM may provide some evidence whether a particular message originates from a claimed domain. Neither of the mechanisms specif es what should be done with this evidence and how it can be taken into account. This is where DMARC [37] comes into play: It aggregates and complements SPF and DKIM in the sense that it can express domain-level policies and preferences for message validation, disposition, and reporting. As such, DMARC is important for the deployment and actual use of SPF and DKIM. Recently, an experimental protocol named *authenticated received chain* (ARC) has been specif ed [38] to solve some practical problems related to SPF, DKIM, and DMARC, especially when it comes to forwarding e-mails and using mailing lists.

Greylisting

Greylisting refers to a very simple but effective spam protection mechanism that starts from the observation that most spammers implement a f re-and-forget strategy, meaning that they don't queue and retry to send out spam messages after an unsuccessful try. The ability to queue and retry is what distinguishes a legitimate mail server from a compromised one (that acts as a spammer). This can be exploited by having a receiving mail server abort a connection establishment from a new and not yet known mail server, and have this server reconnect after a short period of time. If it does, then it may be a legitimate server; otherwise, it is probably not, and the respective messages can be considered to be spam. Note that this mechanism does not slow down the normal behavior of a server. It only introduces some latency for new and not yet known mail servers, and these tend to be only exceptional cases.

2.2.3.2 Transport Layer Encryption

The SSL/TLS protocols are the technology of choice to implement transport layer encryption.[17] As mentioned above (Table 2.3), STARTTLS [21] is an SMTP security extension that enables an SMTP client and server to opportunistically invoke and negotiate the use of SSL/TLS. In its native form, STARTTLS does not require authentication and is susceptible to *man-in-the-middle* (MITM) attacks (Section 3.2.3.3). It is therefore important to reliably authenticate the peers, using, for example, *DNS-based authentication of named entities* (DANE) [39, 40] in conjunction with the *DNS security* (DNSSEC) extensions [41–45].[18]

STARTTLS is opportunistically invoked, and this means that TLS is not always used. To enforce a stricter use of TLS, people have specif ed an SMTP option called *REQUIRETLS* (that is ongoing work and specif ed in an Internet-Draft) and—maybe more importantly—*MTA Strict Transport Security* (MTA-STS) [46]. MTA-STS is likely going to be the standard to secure SMTP data exchanged between MTAs. It is conceptually similar to HTTP strict transport security (HSTS) in the case of HTTP.

Last but not least, there are many things that can go wrong when STARTTLS, DANE, or MTA-STS is invoked. [47] provides a reporting mechanism and format by which sending systems can share statistics and specif c information about potential failures with recipient domains. These domains can then use this information to both detect potential attacks and diagnose unintentional misconf gurations.

2.3 INSTANT MESSAGING

Instant messaging started its success story in the late 1980s with *Internet relay chat* (IRC) that was experimentally specif ed in RFC 1459 [48] and later informationally specif ed in RFCs 2810–2813 [49–52]. IRC peaked in popularity in the 1990s, but continues to have thousands of users.[19] In 1996, an Israeli company called Mirabilis launched ICQ—a homophone standing for I seek you. ICQ was the f rst instant messaging application that allowed users to search for other users, chat in a peer-to-peer or group-wise fashion, and exchange f les. Mirabilis was acquired by America Online (AOL) in 1998. But at its peak in 2001, ICQ held over 100 million user accounts.

17 The current version is TLS version 1.3 that has been available since August 2018.

18 https://www.dnssec.net.

19 Note that the start of IRC was before the f rst SMS message was sent over a GSM network in December 1992.

Soon after the launch of ICQ, several competitors appeared on the market: AOL launched the *AOL Instant Messenger* (AIM) with its buddy list in 1997 (that was even before AOL acquired Mirabilis),[20] Yahoo launched the *Yahoo!Messenger* in 1998, and Microsoft came up with the *MSN Messenger*. At the dawn of the 21st century, all of these instant messaging platforms were competing for market share. It was the Golden Age of instant messaging, and sharing photos, making voice or video calls, and playing games became common features as device technology became more advanced. The market was even enriched when Apple launched *iChat* in 2002,[21] Skype appeared in 2003, Facebook released a chat feature in 2008, and WhatsApp opened the mass market in 2009.

In the first decade of the 2000s, several proprietary and hence incompatible instant messaging platforms evolved. AOL developed a proprietary protocol named *Open System for Communication in Realtime* (OSCAR) that was used in ICQ and AIM. Contrary to its name, the protocol was not open until 2008, so people had to reverse-engineer it. Other companies used other proprietary protocols for their instant messaging platforms. It therefore became clear that the community required open standards. This was the moment for the IETF to specify the requirements for an instant messaging and presence protocol [53] and to charter a respective working group (WG). Quite naturally, the name of the WG became *extensible messaging and presence protocol* (XMPP), and it was active from 2009 to 2015. The task of the WG was to specify a protocol that was first named *Jabber* and later renamed to XMPP. The XMPP specification is open and made available in a triple of RFC documents [54–56]. It is the basis for a wide range of applications in the field of instant messaging, presence, and collaboration.

XMPP as its stands does not employ cryptography. Similar to other messaging protocols,[22] it can be layered on top of SSL/TLS to provide transport layer security. Note that transport layer security is technically sound and assumed to be secure, but it does not always provide E2EE. There is also a complementary RFC [58] that specifies how to invoke S/MIME for message signing and encryption in XMPP (using AES-128 in CBC mode for encryption and RSA for authentication). But S/MIME—as outlined and discussed in Chapter 6—has turned out to be less successful in the field than originally anticipated, and hence the rush to use it to secure XMPP is relatively moderate (to say the least) and there are only a few implementations of the specification, such as the SixChat secure messaging app. In contrast to S/MIME, the combined use and integration of OpenPGP in XMPP has been done entirely outside the IETF, and hence there is no RFC document to refer to. Instead, the work has been

20 By the mid 2000s, AIM had the largest share of the instant messaging market in North America with 52%.

21 At this time, iChat was intentionally compatible with AIM.

22 The use of SSL/TLS for IRC is, for example, specified in [57].

done within the XMPP Standards Foundation (XSF)—formerly known as Jabber Software Foundation—and has culminated in a pair of respective XMPP extension protocol (XEP) proposals: XEP-0373[23] entitled "OpenPGP for XMPP" and XEP-0374[24] entitled "OpenPGP for XMPP Instant Messaging." Both XEP proposals are experimental in nature, and have not been particularly successful in the f eld either. Other XEP proposals for E2EE are more enthusiastically received by the XMPP community, such as OMEMO that is based on Signal (and will be addressed in the respective chapter about Signal).

In addition to XMPP, there are only a few open technologies and protocols that can be used for instant messaging, presence, and/or collaboration. Examples include IRC (as mentioned above), WebRTC standardized by both the World Wide Web Consortium (W3C) and the IETF Real-Time Communication in WEB-browsers (RTCWEB) WG,[25] SIP for instant messaging and presence leveraging extensions (SIMPLE) [59], Matrix,[26] and Tox.[27] Due to some documented and serious security vulnerabilities, however, the use of Tox is discouraged. None of these alternatives to XMPP is particularly successful, and hence the f eld of instant messaging, presence, and collaboration is still dominated by proprietary protocols based on a simple (and centralized) architecture. This is fundamentally different from e-mail, where interoperable technologies and protocols are omnipresent and have been in existence from the very beginning.

More recently, the telecommunication operators (telcos) have started to revitalize the success story of SMS with the *Rich Communication Services* (RCS) standardized by the GSM Association. On the technical side, RCS build on many well-established technologies, such as HTTP and HTTPS, the session initiation protocol (SIP) [60] and SIP security (SIPS), the real-time transport protocol (RTP) [61] and secure RTP (SRTP) [62], as well as the message session relay protocol (MSRP) [63, 64] and MSRP security (MSRPS). Note that the secure version of either of these protocols refers to the protocol layered on top of SSL/TLS. Also note that many of these technologies are also used in E2EE messaging. This is particularly true for HTTPS and SRTP. In addition to various telcos, Google is also heading towards RCS with its Jibe platform and Android Messages that is going to replace Allo. The outcome of this battle is open, although it is going to be very challenging for the promoters of RCS to beat out the many messengers used in the f eld.

23 https://xmpp.org/extensions/xep-0373.html.
24 https://xmpp.org/extensions/xep-0374.html.
25 http://webrtc.org.
26 Matrix refers to a family of protocols for instant messaging, voice over IP (VoIP), and Internet of things (IoT) applications. In fact, there are many implementations of the Matrix protocols, and more information is available at https://matrix.org.
27 https://tox.chat.

Last but not least, the IETF has also become active in this f eld and has launched a *message layer security* (MLS) WG. This WG and some of its preliminary results are summarized at the end of the book (providing an outlook on how the f eld may evolve in the future).

2.4 FINAL REMARKS

In this chapter, we have introduced, brief y overviewed, and put into perspective the core technologies that are used for Internet messaging—both in terms of e-mail and instant messaging. This includes many protocols that are widely deployed, such as SMTP, MIME, POP3, IMAP4, and LDAP in the case of e-mail and XMPP in the case of instant messaging. In the latter case, the f eld is dominated by proprietary technologies and protocols. This is in contrast to the E2EE extensions to these protocols that are mostly based on the Signal protocol (as we will see later in the book).

Due to its own success, Internet messaging has undergone (and still undergoes) a steady and profound evolution. Existing standards are being revised and new features are being introduced and added on a regular basis—possibly replacing old ones. There are even attempts to change the overall architecture and protocols disruptively. An example of this type is the Dark Internet Mail Environment (DIME) developed and specif ed [65] by the Dark Mail Alliance[28] in the aftermath of the Snowden revelations and the temporary shutdown of Lavabit.[29] DIME uses new and distinct protocols, such as the Dark Mail Transfer Protocol (DMTP) that is to replace SMTP and the Dark Mail Access Protocol (DMAP) that is to replace IMAP. As of this writing, Lavabit provides support for DIME,[30] but no other company seems to support DIME. It is therefore possible and likely that it will turn out to be a dead end and that it will silently sink into oblivion in the future.

References

[1] Hughes, L., *Internet E-Mail: Protocols, Standards, and Implementations*, Artech House, Norwood, MA, 1998.

[2] Oppliger, R., *Internet and Intranet Security*, 2nd Edition, Artech House, Norwood, MA, 2002.

28 https://darkmail.info.

29 Lavabit LLC (https://lavabit.com) is an e-mail provider that was founded in 2004 by Ladar Levison. It was used by Edward Snowden and temporarily shut down from 2013 to 2017.

30 There is a client software called *Flow* and an open source server software called *Magma*. Furthermore, Lavabit has also announced an open source client software called *Volcano*.

[3] Housley, R., Crocker, D., and E. Burger, "Reducing the Standards Track to Two Maturity Levels," RFC 6410, October 2011.

[4] Crocker, D., "Internet Mail Architecture," RFC 5598, July 2009.

[5] Klensin, J., "Simple Mail Transfer Protocol," RFC 5321, October 2008.

[6] Resnick, P. (Ed.), "Internet Message Format," RFC 5322, October 2008.

[7] Costales, B., et al., *Sendmail*, 4th edition, O'Reilly Media, Sebastopol, CA, 2007.

[8] Leiba, B., "Update to Internet Message Format to Allow Group Syntax in the 'From:' and 'Sender:' Header Fields," RFC 6854, March 2013.

[9] Freed, N., and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part One: Format of Internet Message Bodies," RFC 2045, November 1996.

[10] Freed, N., and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Two: Media Types," RFC 2046, November 1996.

[11] Moore, K., "MIME (Multipurpose Internet Mail Extensions) Part Three: Message Header Extensions for Non-ASCII Text," RFC 2047, November 1996.

[12] Freed, N., and J. Klensin, "Multipurpose Internet Mail Extensions (MIME) Part Four: Registration Procedures," BCP 13, RFC 4289, December 2005.

[13] Freed, N., and N. Borenstein, "Multipurpose Internet Mail Extensions (MIME) Part Five: Conformance Criteria and Examples," RFC 2049, November 1996.

[14] Gellens, R., and J. Klensin, "Message Submission," RFC 2476, December 1998.

[15] Myers, J., "SMTP Service Extension for Authentication," RFC 2554, March 1999.

[16] Gellens, R., and J. Klensin, "Message Submission for Mail," RFC 6409, November 2011.

[17] Klensin, J., et al., "SMTP Service Extensions," RFC 1869, November 1995.

[18] Klensin, J., Freed, N., and K. Moore, "SMTP Service Extensions for Message Size Declaration," RFC 1870, November 1995.

[19] Freed, N., "SMTP Service Extensions for Command Pipelining," STD 60, RFC 2197, September 2000.

[20] Freed, N., Rose, M., and D. Crocker, "SMTP Service Extension for 8-bit MIME Transport," STD 71, RFC 6152, March 2011.

[21] Hoffman, P., "SMTP Service Extension for Secure SMTP over Transport Layer Security," RFC 3207, February 2002.

[22] Siemborski, R., and A. Melnikov, "SMTP Service Extension for Authentication," RFC 4954, July 2007.

[23] Allman, E., and T. Hansen, "SMTP Service Extension for Message Tracking," RFC 3885, September 2004.

[24] Moore, K., “Simple Mail Transfer Protocol (SMTP) Service Extension for Delivery Status Notif cations (DSNs),” RFC 3461, January 2003.

[25] Myers, J., and M. Rose, “Post Off ce Protocol—Version 3,” STD 53, RFC 1939, May 1996.

[26] Gellens, R., Newman, C., and L. Lundblade, “POP3 Extension Mechanism,” RFC 2449, November 1998.

[27] Melnikov, A., and K. Zeilenga (Eds.), “Simple Authentication and Security Layer (SASL),” RFC 4422, June 2006.

[28] Siemborski, R., and A. Menon-Sen, “The Post Off ce Protocol (POP3)—Simple Authentication and Security Layer (SASL) Authentication Mechanism,” RFC 5034, July 2007.

[29] Newman, C., “Using TLS with IMAP, POP3 and ACAP,” RFC 2595, June 1999.

[30] Zeilenga, K., “The PLAIN Simple Authentication and Security Layer (SASL) Mechanism,” RFC 4616, August 2006.

[31] Crispin, M., “Internet Message Access Protocol—Version 4rev1,” RFC 3501, March 2003.

[32] Myers, J., “IMAP4 Authentication Mechanisms,” RFC 1731, December 1994.

[33] Sermersheim, J. (Ed.), “Lightweight Directory Access Protocol (LDAP): The Protocol,” RFC 4511, June 2006.

[34] Chandramouli, R., et al., “Trustworthy Email,” NIST Special Publication 800-177, September 2016 (revised in February 2019).

[35] Kitterman, S., “Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1,” RFC 7208, April 2014.

[36] Crocker, D., Hansen, T., and M. Kucherawy (Eds.), “DomainKeys Identif ed Mail (DKIM) Signatures,” RFC 6376, September 2011.

[37] Kucherawy, M., and E. Zwicky (Eds.), “Domain-based Message Authentication, Reporting, and Conformance (DMARC),” RFC 7489, March 2015.

[38] Andersen, K., et al., “The Authenticated Received Chain (ARC) Protocol,” RFC 8617, July 2019.

[39] Barnes, R., “Use Cases and Requirements for DNS-Based Authentication of Named Entities (DANE),” RFC 6394, October 2011.

[40] Hoffman, P., and J. Schlyter, “The DNS-Based Authentication of Named Entities (DANE) Transport Layer Security (TLS) Protocol: TLSA,” RFC 6698, August 2012.

[41] Arends, R., et al., “DNS Security Introduction and Requirements,” RFC 4033, March 2005.

[42] Arends, R., et al., “Resource Records for the DNS Security Extensions,” RFC 4034, March 2005.

[43] Arends, R., et al., “Protocol Modif cations for the DNS Security Extensions,” RFC 4035, March 2005.

[44] StJohns, M., “Automated Updates of DNS Security (DNSSEC) Trust Anchors,” RFC 5011, September 2007.

[45] Laurie, B., et al., "DNS Security (DNSSEC) Hashed Authenticated Denial of Existence," RFC 5155, March 2008.

[46] Margolis, D., et al., "SMTP MTA Strict Transport Security (MTA-STS)," RFC 8461, September 2018.

[47] Margolis, D., et al., "SMTP TLS Reporting," RFC 8460, September 2018.

[48] Oikarinen, J., and D. Reed, "Internet Relay Chat Protocol," RFC 1459, May 1993.

[49] Kalt, C., "Internet Relay Chat: Architecture," RFC 2810, April 2000.

[50] Kalt, C., "Internet Relay Chat: Channel Management," RFC 2811, April 2000.

[51] Kalt, C., "Internet Relay Chat: Client Protocol," RFC 2812, April 2000.

[52] Kalt, C., "Internet Relay Chat: Server Protocol," RFC 2813, April 2000.

[53] Day, M., et al., "Instant Messaging / Presence Protocol Requirements," RFC 2779, February 2000.

[54] Saint-Andre, P., "Extensible Messaging and Presence Protocol (XMPP): Core," RFC 6120, March 2011.

[55] Saint-Andre, P., "Extensible Messaging and Presence Protocol (XMPP): Instant Messaging and Presence," RFC 6121, March 2011.

[56] Saint-Andre, P., "Extensible Messaging and Presence Protocol (XMPP): Address Format," RFC 7622, September 2015.

[57] Hartmann, R., "Default Port for Internet Relay Chat (IRC) via TLS/SSL," RFC 7194, August 2014.

[58] Saint-Andre, P., "End-to-End Signing and Object Encryption for the Extensible Messaging and Presence Protocol (XMPP)," RFC 3923, October 2004.

[59] Rosenberg, J., "SIMPLE Made Simple: An Overview of the IETF Specifications for Instant Messaging and Presence Using the Session Initiation Protocol (SIP)," RFC 6914, April 2013.

[60] Rosenberg, J., et. al., "SIP: Session Initiation Protocol," RFC 3261, June 2002.

[61] Schulzrinne, H., et. al., "RTP: A Transport Protocol for Real-Time Applications," RFC 3550, July 2003.

[62] Baugher, M., et. al., "The Secure Real-time Transport Protocol (SRTP)," RFC 3711, March 2004.

[63] Campbell, B., Mahy, R., and C. Jennings (Eds.), "The Message Session Relay Protocol (MSRP)," RFC 4975, September 2007.

[64] Campbell, B., Mahy, R., and A.B. Roach, "Relay Extensions for the Message Session Relay Protocol (MSRP)," RFC 4976, September 2007.

[65] Dark Mail Alliance, "Dark Internet Mail Environment—Architecture and Specifications," June 2018.

Chapter 3

Cryptographic Techniques

In this chapter, we provide a summary of the cryptographic techniques used in the f eld in general, and E2EE messaging in particular. We do this only brief y, because cryptography is a huge topic that f lls many books on its own (e.g., [1–34] itemized in alphabetical order, and [24] being particularly recommended for obvious reasons), and we want to be as short as possible here. If you are already familiar with the basics of cryptography, then you can easily skip this chapter and jump directly to the next one. Otherwise, the rest of the chapter is organized as follows: Section 3.1 introduces the topic, Section 3.2 overviews and puts into perspective the cryptosystems used in the f eld, Section 3.3 outlines certif cate management, and Section 3.4 concludes with some f nal remarks.

3.1 INTRODUCTION

In this section, we introduce the topic by f rst elaborating on cryptology, then classifying the cryptographic systems, and f nally providing some historical background information.

3.1.1 Cryptology

The term *cryptology* is derived from the Greek words kryptós, standing for hidden, and lógos, standing for word. Consequently, the meaning of the term cryptology can be paraphrased as hidden word. This refers to the original intent of cryptology, namely to hide the meaning of words and to protect the conf dentiality and secrecy of the respective data accordingly. This viewpoint is too narrow, and the term cryptology is nowadays used for many other security-related purposes and applications—in addition to the protection of the conf dentiality and secrecy of data.

More specif cally, cryptology refers to the mathematical science and f eld of study that comprises cryptography and cryptanalysis.

- The term *cryptography* is derived from the Greek words kryptós (see above) and gráphein, standing for to write. Consequently, the meaning of the term cryptography can be paraphrased as hidden writing. According to [35], cryptography refers to the "mathematical science that deals with transforming data to render its meaning unintelligible (i.e., to hide its semantic content), prevent its undetected alteration, or prevent its unauthorized use. If the transformation is reversible, cryptography also deals with restoring encrypted data to intelligible form." Consequently, cryptography refers to the process of protecting data in a very broad sense.
- The term *cryptanalysis* is derived from the Greek words kryptós (see above) and analýein, standing for to loosen. Consequently, the meaning of the term can be paraphrased as to loosen the hidden word. This paraphrase refers to the process of destroying the cryptographic protection, or—more generally—to study the security properties and possibilities to break cryptographic techniques and systems. According to [35], the term cryptanalysis refers to the "mathematical science that deals with analysis of a cryptographic system in order to gain knowledge needed to break or circumvent[1] the protection that the system is designed to provide." As such, the cryptanalyst is the antagonist of the cryptographer, meaning that his or her job is to break or—more likely—circumvent the protection that the cryptographer has designed and implemented in the f rst place. Quite naturally, there is an arms race going on between the cryptographers and the cryptanalysts (but note that an individual person may have both skills, cryptographic and cryptanalytical ones).

Many other def nitions for the terms cryptology, cryptography, and cryptanalysis exist and can be found in the literature (or on the Internet, respectively). For example, the term cryptography is sometimes said to more broadly refer to the study of mathematical techniques related to all aspects of information security (e.g., [20]). These aspects include (but are not restricted to) data conf dentiality, data integrity, entity authentication, data origin authentication, nonrepudiation, and/or many more. Again, this def nition is broad and comprises anything that is directly or indirectly related to information security.

1 In practice, circumventing (bypassing) the protection is much more common than breaking it. In his 2002 ACM Turing Award Lecture (https://www.youtube.com/watch?v=KUHaLQFJ6Cc), for example, Adi Shamir—a coinventor of the RSA public key cryptosystem (cf. Section 3.2.3.1)—made the point that "cryptography is typically bypassed, not penetrated," and this point was so important to him that he put it as a third law of security (in addition to "absolutely secure systems do not exist" and "to halve your vulnerability you have to double your expenditure").

In some literature, the term cryptology is said to also include steganography (in addition to cryptography and cryptanalysis).

- The term *steganography* is derived from the Greek words "steganos," standing for "impenetrable," and "gráphein" (see above). Consequently, the meaning of the term can be paraphrased as "impenetrable writing." According to [35], the term refers to "methods of hiding the existence of a message or other data. This is different than cryptography, which hides the meaning of a message but does not hide the message itself." Let us consider an analogy to clarify the difference between steganography and cryptography: if we have money to protect or safeguard, then we can either hide its existence (by putting it, for example, under a mattress), or we can put it in a safe that is as burglarproof as possible. In the first case, we are referring to steganographic methods, whereas in the second case, we are referring to cryptographic ones. An example of a formerly widely used steganographic method is invisible ink. A message remains invisible, unless the ink is subject to some chemical reaction that makes the message to reappear and become visible again. Currently deployed steganographic methods are more sophisticated, and can, for example, be used to hide information in electronic files. In general, this information is arbitrary, but it is typically used to identify the owner or the recipient of a file. In the first case, one refers to *digital watermarking*, whereas in the second case one refers to *digital fingerprinting*. Digital watermarking and fingerprinting are active areas of research today.

Interestingly, cryptographic and steganographic technologies and techniques are not mutually exclusive and can be combined in a particular application.

3.1.2 Cryptographic Systems

According to [35], the term *cryptographic system*[2]—or *cryptosystem* in short—refers to "a set of cryptographic algorithms together with the key management processes that support use of the algorithms in some application context." Again, this definition is fairly broad and comprises all kinds of cryptographic algorithms and protocols, where an *algorithm* refers to a "finite set of step-by-step instructions for a

2 In some literature, the term *cryptographic scheme* is used to refer to a cryptographic system. Unfortunately, it is seldom explained what the difference is between a (cryptographic) scheme and a system. So for the purpose of this book, we don't make a distinction, and we use the term cryptographic system to refer to either of them. We hope that this simplification is not too confusing for you. In the realm of digital signatures, for example, people often use the term digital signature scheme that is not used in this book. Instead, we consistently use the term digital signature system to refer to the same construct.

problem-solving or computation procedure, especially one that can be implemented by a computer" [35], and a *protocol* just refers to a distributed algorithm (i.e., an algorithm that is executed by multiple entities). Algorithms (and protocols) can be deterministic or probabilistic:

- An algorithm is *deterministic*, if its behavior is completely determined by its input. This also means that the algorithm always generates the same output for the same input (if executed multiple times).
- An algorithm is *probabilistic* (or *randomized*), if its behavior is not completely determined by its input, meaning that the algorithm internally employs some (pseudo)random values.[3] Consequently, a probabilistic algorithm may generate different outputs each time it is executed with the same input.

An algorithm may be implemented by a computer program that is written in a specif c programming language, such as Pascal, C, or Java. Whenever we describe algorithms in this book, we don't use a specif c programming language, but we use a simpler and more formal notation instead.

A cryptographic application may consist of multiple (cryptographic) algorithms and protocols. The protocols and their concurrent execution may interact in sometimes subtle ways, and the respective interdependencies may be susceptible to *multiprotocol attacks*.[4] As its name suggests, more than one protocol is involved in such an attack, and the adversary may employ messages from one protocol execution to construct valid-looking messages for other protocols or protocol executions. If, for example, one protocol uses digital signatures for random-looking data and another protocol is an authentication protocol in which an entity must sign a number used once (nonce) to authenticate itself, then an adversary can misuse the f rst protocol to sign a nonce from the second protocol. This is a simple and straightforward attack that can be mitigated easily (for example, by separating the keys). However, more involved interactions and interdependencies may be possible and are likely to exist, and hence multi-protocol attacks tend to be very powerful and diff cult to mitigate. Fortunately, many such attacks have been described in scientif c papers, but to the best of our knowledge only few have been mounted in the f eld so far .

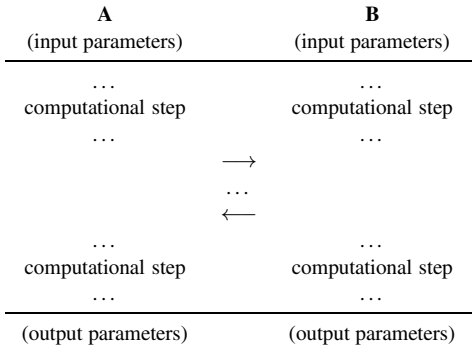
In the cryptographic literature, it is common to use human names to refer to entities that participate in cryptographic protocols. For example, in a two-party protocol the participating entities are usually called *Alice* and *Bob*. This is a convenient way of making things unambiguous with relatively few words, since the pronoun *she* can be used for Alice, and *he* can be used for Bob. The disadvantage

3 A value is random (pseudorandom), if it is randomly (pseudorandomly) generated.

4 The notion of a *chosen protocol* or *multiprotocol attack* f rst appeared in a 1997 paper [36], but the problem had preexisted before that.

of this naming scheme is that people automatically assume that the entities refer to people. This need not be the case, and Alice, Bob, and all other entities are rather computer systems, cryptographic devices, or something similar. In this book, we don't follow the tradition of using Alice, Bob, and the rest of the gang. Instead, we use single-letter characters (such as A, B, etc.) to refer to the entities that take part and participate in a cryptographic protocol. This is admittedly less fun, but more appropriate (see, for example, [37] for a more comprehensive reasoning). In reality, the entities refer to social-technical systems that may have a user interface, and the question of how to properly design and implement these interfaces is very important for the overall security of the systems. If these interfaces are not appropriate, then phishing and other social engineering attacks become trivial to mount.

The cryptographic literature provides many examples of more or less useful cryptographic protocols. Some of these protocols—especially the ones used in E2EE messaging—are overviewed, discussed, and put into perspective in this book. To formally describe a (cryptographic) protocol in which A and B take part, the following notation is used:



Some input parameters may be required on either side of the protocol (note that the input parameters need not be the same). The protocol then includes a sequence of computational and communication steps. Each computational step may occur only on one side of the protocol, whereas each communication step requires data to be transferred from one side to the other. In this case, the direction of the data f ow is indicated by the arrow. Finally, some parameters may be output on either side of the protocol. These output parameters actually represent the result of the protocol execution. Similar to the input parameters, the output parameters need not be the same on either side. In many cases, however, the output parameters are the same. In the case of the Diff e-Hellman key exchange, for example, the output of a protocol execution is a session key that can afterwards be used to secure communications.

3.1.2.1 Classes of Cryptographic Systems

Cryptographic systems may or may not use secret parameters (e.g., cryptographic keys). If secret parameters are used, then they may or may not be shared among the participating entities. Consequently, there are three classes of cryptographic systems that can be distinguished:[5]

- An *unkeyed cryptosystem* is a cryptographic system that uses no secret parameter.
- A *secret key cryptosystem* is a cryptographic system that uses secret parameters that are shared among the participating entities. Because the parameters are shared, they occur symmetrically, and hence the respective cryptosystems are also called *symmetric*.
- A *public key cryptosystem* is a cryptographic system that uses secret parameters that are not shared among the participating entities. Because the parameters are not shared, they occur asymmetrically, and hence the respective cryptosystems are also called *asymmetric*.

In Section 3.2, we overview the most important representatives of unkeyed, secret key, and public key cryptosystems. But before that we want to informally argue about the security of cryptographic systems and the different perspectives one may take to look at them.

3.1.2.2 Secure Cryptographic Systems

The goal of cryptography is to design, implement, and employ cryptographic systems that are secure in some meaningful way. But to make precise statements about the security of a particular cryptosystem, one must formally def ne the term security. Unfortunately, reality looks different, and the literature is full of cryptographic systems that are claimed to be secure without providing an appropriate def nition for it. This is dissatisfactory, mainly because anything can be claimed to be secure, unless its meaning is precisely nailed down.

Instead of properly def ning the term security and analyzing whether a cryptographic system meets this def nition, people often argue about key lengths. This is because the key length is a simple and very intuitive security parameter. So people frequently use it to characterize the cryptographic strength of a system. This is clearly an oversimplif cation, and the key length is a suitable (and meaningful) measure of security, if and only if an exhaustive key search is the most eff cient way

5 This classif cation scheme is due to Ueli M. Maurer.

to break it. In practice, this is seldom the case, and there are often simpler ways to break the security of a system (e.g., by reading out some keying material from the memory). In this book, we avoid discussing too much about the key lengths of cryptographic systems; instead, we refer to the recommendations of BlueKrypt.[6] They provide advice to decide what key lengths are appropriate for any given cryptosystem.

In order to discuss the security of a cryptosystem, there are two perspectives one may take: a theoretical one and a practical one. Unfortunately, the two perspectives are inherently different, and one may have a cryptosystem that is theoretically secure but practically insecure (e.g., due to a poor implementation), or—vice versa—a cryptosystem that provides a suff cient level of security in practice but is not very sophisticated from a theoretical viewpoint.

Theoretical Perspective

In theory, one has to start with a precise def nition of the term security when it comes to a particular cryptosystem. What does it mean for such a system to be secure? What properties does it have to fulf ll? In general, there are two questions that need to be answered here:

1. Who is the adversary, that is, what are his or her capabilities and how powerful is he or she?
2. What is the task the adversary has to solve in order to be successful, that is, to break the security of the system?

An answer to the f rst question comprises the specif cation of several parameters related to the adversary, such as his or her computing power, available memory, available time, types of feasible attacks, and access to a-priori information. For some of these parameters, the statements can be coarse, such as the computing power and the available time are f nite. The result is a *threats model* (i.e., a model of the adversary one has in mind and against whom one wants to protect oneself).

An answer to the second question is more tricky. In general, the adversary's task is to f nd (i.e., compute, guess, or otherwise determine) one or several pieces of information that he or she should not be able to know. If, for example, the adversary is able to determine the cryptographic key used to encrypt a message, then he or she must clearly be considered to be successful. But what if he or she is able to determine only half of the key, or—maybe even more controversial—a single bit of the key? Similar diff culties occur in other cryptosystems that are used for other purposes than conf dentiality protection. One possibility to deal with these diff culties is to

6 http://www.keylength.com.

def ne a theoretically perfect system—a so-called *ideal system*—and to state that the adversary is successful, if he or she is able to tell it apart from the real system (i.e., decide whether he or she is interacting with the real system or an ideal one). If he or she cannot tell the two systems apart, then the real system has all the relevant properties of the ideal system, and hence the real system is arguably as secure as the ideal one. Many security proofs follow this line of argumentation.

Anyway, a cryptographic system is *secure* if a well-def ned adversary is not able to break it, meaning that he or she is not able to solve a well-def ned task. This def nition gives room for several notions of security. In principle, there is a distinct notion for every possible adversary combined with every possible task. As a general rule of thumb, we can say that strong security def nitions assume an adversary that is as powerful as possible and a task to solve that is as simple as possible. If a system can be shown to be secure in this setting, then there is a security margin. In reality, the adversary is likely less powerful and the task he or she must solve is likely more diff cult, and this, in turn, means that it is very unlikely that the security of the system gets broken. In practice, one usually distinguishes between the following two notions of security:

Unconditional security: If an adversary with inf nite computing power is not able to solve the task within a f nite amount of time, then we are talking about *unconditional* or *information-theoretic security*. The mathematical theories behind this notion of security are probability theory and information theory.

Conditional security: If an adversary is theoretically able to solve the task within a f nite amount of time, but the computing power required to do so is beyond his or her capabilities,[7] then we are talking about *conditional* or *computational security*. The mathematical theory behind this notion of security is computational complexity theory.

The distinction between unconditional and conditional security is at the core of contemporary (or modern) cryptography. Interestingly, there are cryptosystems known to be secure in the strong sense (i.e., unconditionally secure), whereas there are no cryptosystems provably known to be secure in the weak sense (i.e., computationally secure). There are many cryptosystems that are assumed to be computationally secure, but no proof is available for any of these systems. In fact, not even the existence of a conditionally or computationally secure cryptosystem has been proven so far. The underlying problem is that it is generally impossible to prove a lower bound for the computational complexity of a problem, such as solving a particular task. This is an inherent weakness and limitation of complexity theory as it stands today.

7 It is usually assumed that the adversary can run algorithms that have a polynomial running time.

In some literature, the term *provable security* is used to refer to an arguably strong notion of security. It goes back to the early days of public key cryptography, when Whitf eld Diff e and Martin E. Hellman proposed a complexity-based proof for the security of their key exchange protocol [38].[8] The idea is to show that breaking a cryptosystem is computationally equivalent to solving a hard mathematical problem. This means that one must prove the following two directions:

- If the hard problem can be solved, then the cryptosystem can be broken;
- If the cryptosystem can be broken, then the hard problem can be solved.

Diff e and Hellman only proved the f rst direction, and they did not prove the second direction (this was done later on). This is unfortunate, because the second direction is important from a security perspective. If we can prove that an adversary who is able to break a cryptosystem is also able to solve the hard problem, then we can reasonably argue that it is unlikely that such an adversary exists, and hence that the cryptosystem in question is likely to be secure. Michael O. Rabin was the f rst researcher who found and proposed a cryptosystem that can be proven to be computationally equivalent to a hard problem [39].

The notion of (provable) security has fueled a lot of research since the late 1970s. In fact, there are many (public key) cryptosystems proven secure in this sense. It is, however, important to note that a complexity-based proof is not absolute, and that it is only relative to the (assumed) intractability of the underlying mathematical problem(s). To make things even more involved, intractability is a worst-case fact that also depends on the size of the problem(s). The situation is comparable to proving that a problem is $\mathcal{NP}$-complete: This proves that the problem is at least as diff cult as all other $\mathcal{NP}$-complete problems, but it does not provide an absolute proof of its computational diff culty. In the past, we have seen quite a few cryptosystems based on $\mathcal{NP}$-complete problems, such as knapsack-based asymmetric encryption systems. Even though the underlying knapsack problem is known to be $\mathcal{NP}$-hard, the respective encryption systems are relatively easy to break. The bottom line is that one has to be cautious whenever people talk about cryptosystems that are provably secure, and that one has to have a closer look at the respective proofs.

Since the publication of [40], people have been routinely using a new methodology to design cryptographic systems that are provably secure. This methodology consists of the following three steps:

8 This paper is the one that off cially gave birth to public key cryptography. There is a companion paper entitled *Multiuser Cryptographic Techniques* that was presented by the same authors at the National Computer Conference on June 7–10, 1976.

1. Design an ideal system that uses random functions[9] also known as *random oracles*. Note that the terms random function and random oracle are used synonymously and interchangeably in the literature related to contemporary cryptography.
2. Prove the security of this ideal system.
3. Replace the random functions with real ones, most notably cryptographic hash functions.

As a result, one obtains an implementation of the ideal system in the real world (where random functions do not exist).

Due to the use of random oracles, this methodology is known as *random oracle methodology*, and it yields cryptosystems that are provably secure in the so-called *random oracle model*.

Such cryptosystems and their respective security proofs are widely used in the f eld, but they must be taken with a grain of salt. In fact, it has been shown that it is possible to construct cryptographic systems that are provably secure in the random oracle model, but become insecure whenever the cryptographic hash function used in the protocol (to replace the random oracle) is instantiated. This theoretical result is somewhat worrisome, and since its publication many researchers have started to think controversially about the random oracle methodology and the usefulness of the random oracle model.

The bottom line and major takeaway is that formal analyses in the random oracle model are not security proofs in a mathematically strict sense. The problem is the underlying ideal assumptions about the randomness properties of the cryptographic hash functions. This is not at all a legitimate assumption in a mathematically strict proof.

Practical Perspective

So far, we have argued about the security of a cryptosystem from a purely theoretical viewpoint. In practice, however, any (theoretically secure) cryptosystem must be implemented, and there are many things that can go wrong (e.g., [41]). For example, the cryptographic key in use may be kept in memory and extracted from there (e.g.,

9 The notion of a random function is introduced in Section 3.2.2.4.

using a *cold boot attack*[10] [42]), or the user of a cryptosystem may be subject to all kinds of phishing and social engineering attacks.

Historically, the first such attacks tried to exploit the compromising emanations that occur in all information-processing systems. These are unintentional intelligence-bearing signals that, if intercepted and analyzed, may disclose the information transmitted, received, handled, or otherwise processed by an equipment. In the late 1960s and early 1970s, the U.S. National Security Agency (NSA) coined the term *TEMPEST* to refer to this field of study (i.e., to secure electronic communications equipment from potential eavesdroppers), and vice versa, the ability to intercept and interpret those signals from other sources.[11] Hence, the term TEMPEST is a codename (not an acronym[12]) that is used broadly to refer to the entire field of emission security or emanations security (EMSEC). There are several U.S. and NATO standards that basically define three levels of TEMPEST requirements (i.e., NATO SDIP-27 Levels A, B, and C).

In addition to cold boot attacks and exploiting compromising emanations, people have been very innovative in finding possibilities to mount attacks against presumably tamper-resistant hardware devices that employ invasive measuring techniques (e.g., [43, 44]). Most importantly, there are attacks that exploit side channel information an implementation may leak when a computation is performed. Side channel information is neither input nor output, but refers to some other information

10 This attack exploits the fact that many dynamic random access memory (DRAM) chips don't lose their contents when a system is switched off immediately, but rather lose their contents gradually over a period of seconds, even at standard operating temperatures and even if the chips are removed from the motherboard. If kept at low temperatures, the data on these chips persist for minutes or even hours. In fact, the researchers showed that residual data can be recovered using simple techniques that require only temporary physical access to a machine, and that several popular disk encryption software packages, such as Microsoft's BitLocker, Apple's FileVault, and TrueCrypt (the predecessor of VeraCrypt) were susceptible to cold boot attacks. The feasibility of such attacks has challenged the security of many disk encryption software solutions, and some solutions (e.g., VeraCrypt since version 1.24) try to additionally encrypt the keys and passwords that reside in memory.

11 https://www.nsa.gov/news-features/declassified-documents/cryptologic-spectrum/assets/files/tempest.pdf.

12 The U.S. government has stated that the term TEMPEST is not an acronym and does not have any particular meaning (it is therefore not included in this book's list of abbreviations and acronyms). However, in spite of this disclaimer, multiple acronyms have been suggested, such as "Transmitted Electro-Magnetic Pulse / Energy Standards & Testing," "Telecommunications ElectroMagnetic Protection, Equipment, Standards & Techniques," "Transient ElectroMagnetic Pulse Emanation STandard," "Telecommunications Electronics Material Protected from Emanating Spurious Transmissions," and—more jokingly—"Tiny ElectroMagnetic Particles Emitting Secret Things." Because it is not an official acronym, it is not included in the list of abbreviations and acronyms compiled in the appendix.

that may be related to the computation, such as timing information or power consumption. Attacks that try to exploit such information are commonly referred to as *side channel attacks*. Let us start with two mind experiments to illustrate the notion of a side channel attack.[13]

1. Assume somebody has written a secret note on a pad and has torn off the paper sheet. Is there a possibility to reconstruct the note? An obvious possibility is to go for a surveillance camera and examine the respective recordings. A less obvious possibility is to exploit the fact that pressing the pen on the paper sheet may have caused the underlying paper sheet to experience the same pressure, and this, in turn, may have caused the underlying paper sheet to show the same groove-like depressions (representing the actual writing). Equipped with the appropriate tools, an expert may be able to reconstruct the note. Pressing the pen on a paper sheet may have caused a side channel to exist, even if the original paper sheet is destroyed.

2. Consider a house with two rooms. In one room are three light switches and in the other room are three light bulbs, but the wiring of the light switches and bulbs is unknown. In this setting, somebody's task is to f nd out the wiring, but he or she can enter each room only once. From a mathematical viewpoint, one can argue (and maybe even prove) that this task is impossible to solve. But from a physical viewpoint (and taking into account some side channel information), the task can be solved: One can enter the room with the light switches, permanently light on one bulb, and light on another bulb for some time (e.g., a few seconds). One then enters the room with the light bulbs. The bulb that lights is easily identif ed and refers to the switch that has been permanently switched on. But the other two bulbs do not light, and hence one cannot easily assign them to the respective switches. But one can measure the temperature of the light bulbs. The one that is warmer more likely refers to the switch that has been switched on for some time. This information can be used to distinguish the two cases and to solve the task accordingly. Obviously, the trick is to measure the temperature of the light bulbs and to use it as a side channel.

In analogy to these mind experiments, there are many side channel attacks that have been proposed to defeat the security of cryptosystems, some of which have turned out to be very powerful. The side channel attack that f rst opened the eyes and then the f eld in the 1990s was a *timing attack* against a vulnerable implementation of the RSA public key cryptosystem [45]. The attack exploited the correlation between a cryptographic key and the running time of the algorithm that employed

13 The second mind experiment is due to Artur Ekert.

the key. Since then, many implementations of cryptosystems have been shown to be vulnerable against timing attacks and some variants, such as cache timing attacks or branch prediction analysis. In 2003, it was shown that remotely mounting timing attacks over computer networks is feasible [46], and since 2018 we know that almost all modern processors that support speculative and out-of-order command execution are susceptible to sophisticated timing attacks.[14] Other side channel attacks exploit the power consumption of an implementation of an algorithm that is being executed (usually named *power consumption* or *power analysis attacks* [47]), faults that are induced (usually named *differential fault analysis* [48, 49]), protocol failures [50], the sounds that is generated during a computation [51, 52], and many more.

Side channel attacks exploit side channel information. Hence, a reasonable strategy to mitigate a specif c side channel attack is to avoid the respective side channel to exist in the f rst place. If, for example, one wants to protect an implementation against timing attacks, then timing information must not leak. At f rst sight, one may be tempted to add a random delay to every computation, but this simple mechanism does not work (because the effect of random delays can be compensated by having an adversary repeat the measurement multiple times). But there may be other mechanisms that work. If, for example, one ensures that all operations take an equal amount of time (i.e., the timing behavior is independent from the input), then one can mitigate such attacks. Also, it is sometimes possible to blind the input and to prevent the adversary from knowing the true value. Both mechanisms have the disadvantage of slowing down the computations. There are fewer possibilities to protect an implementation against power consumption attacks. For example, dummy registers and gates can be added on which useless operations are performed to balance power consumption into a constant value. Whenever an operation is performed, a complementary operation is also performed on a dummy element to assure that the total power consumption remains balanced according to some higher value. Protection against differential fault analysis is less general and more involved. In [48], for example, the authors suggest a solution that requires a cryptographic computation to be performed twice and to output the result only if they are the same. The main problem with this approach is that it roughly doubles the execution time. Also, the probability that the fault will not occur twice is not suff ciently small (and this makes the attack harder to implement, but not impossible). The bottom line is that the development of adequate and sustainable protection mechanisms to mitigate differential fault analysis attacks remains a timely research topic. The same is true for failure analysis and acoustic cryptanalysis, and it may even be true for many other side channel attacks that will be found and published in the future.

14 The f rst such attacks have been named *Meltdown* and *Spectre*. They are, for example, documented at https://www.spectreattack.com.

The existence and difficulty to mitigate side channel attacks have inspired theoreticians to come up with a model for defining and delivering cryptographic security against an adversary who has access to information leaked from the physical execution of a cryptographic algorithm [53]. The original term used to refer to this type of cryptography is *physically observable cryptography*. More recently, however, researchers have coined the term *leakage-resilient cryptography* to refer to the same idea. Even after many years of research, it is still questionable whether physically observable or leakage-resilient cryptography can be achieved in the first place (e.g., [54]). It is certainly a design goal, but it may not be a realistic one.

3.1.2.3 Design Principles

In the past, we have seen many examples in which people have tried to improve the security of a cryptographic system by keeping secret its design and internal working principles. This approach is sometimes referred to as security through obscurity. Many of these systems do not work and can be broken trivially.[15] This insight has a long tradition in cryptography, and there is a well-known cryptographic principle—the *Kerckhoffs' principle*[16]—that basically states that a cryptographic system should be designed so as to remain secure, even if the adversary knows all the details of the system, except for the values explicitly declared to be secret, such as secret keys [55]. We follow this principle in this book, and hence we only address cryptosystems for which we can assume that the adversary knows the details. This assumption is in line with our requirement that the adversaries should be assumed to be as powerful as possible (to obtain strong security definitions according to Section 3.1.2.2).

In spite of Kerckhoffs' principle, the design of a secure cryptographic system remains a difficult and challenging task. One has to make assumptions, and it is not clear whether these assumptions really hold in reality.[17] For example, one usually assumes a certain set of countermeasures to protect against specific attacks. If the adversary attacks the system in another way, then there is hardly anything that can be done about it. Similarly, one has to assume the system to operate in a "typical" environment. If the adversary can manipulate the environment, then he or she may be able to change the operational behavior of the system, and hence to open new vulnerabilities. The bottom line is that cryptographic systems that are based on make-believe, ad hoc approaches, and heuristics are typically broken. Instead, the

15 Note that security through obscurity may work well outside the realm of cryptography.

16 The principle is named after Auguste Kerckhoffs, who lived from 1835 to 1903.

17 The interested reader is referred to a paper entitled "The Uneasy Relationship Between Mathematics and Cryptography" that was published by Neal Koblitz in 2007. It elaborates on the questionable assumptions needed in some security proofs. The paper has been controversially discussed in the community and is still the target of some overheated discussions.

design of a secure cryptographic system should be based on f rm foundations. It typically consists of the following two steps:

1. In a *def nitional step*, the problem the cryptographic system is intended to solve is identif ed, precisely def ned, and formally specif ed;
2. In a *constructive step*, a cryptographic system that satisf es the def nition distilled in step one, possibly while relying on intractability assumptions, is designed.

Again, it is important to note that most parts of modern cryptography rely on intractability assumptions and that relying on such assumptions seems to be unavoidable. But there is still a huge difference between relying on an explicitly stated intractability assumption or just assuming (or rather hoping) that an ad hoc construction satisf es some unspecif ed or vaguely specif ed goals.

3.1.3 Historical Background Information

Cryptography has a long and thrilling history. In fact, probably since the very beginning of the spoken and—even more importantly—written word, people have tried to transform "data to render its meaning unintelligible (i.e., to hide its semantic content), prevent its undetected alteration, or prevent its unauthorized use" [35]. According to this def nition, these people have always employed cryptography and cryptographic techniques. The mathematics behind these early systems may not have been very advanced, but they still employed cryptography and cryptographic techniques. For example, Gaius Julius Caesar[18] used an encryption system in which every letter in the Latin alphabet was substituted with the letter that is found three positions afterwards in the lexical order (i.e., A is substituted with D, B is substituted with E, and so on). This simple additive cipher is known as *Caesar cipher*. Later on, people employed encryption systems that use more advanced and involved mathematical transformations. Many books on cryptography contain numerous examples of historically relevant encryption systems—they are not repeated here; the encryption systems in use today are simply too different.

Until World War II, cryptography was considered to be an art (rather than a science) and was primarily used in military and diplomacy. The following two developments and scientif c achievements turned cryptography from an art into a science:

- During World War II, Claude E. Shannon[19] developed a mathematical theory of communication [56] and a related communication theory of secrecy

18 Gaius Julius Caesar was a Roman emperor, who lived from 102 BC to 44 BC.

19 Claude E. Shannon was a mathematician, who lived from 1916 to 2001.

systems [57] when he was working at AT&T Laboratories.[20] After their publication, the two theories started a new branch of research that is commonly referred to as *information theory*.

- As mentioned earlier, Diffe and Hellman developed and proposed the idea of public key cryptography at Stanford University in the 1970s [38].[21] Their vision was to employ trapdoor functions to encrypt and digitally sign electronic documents. Informally speaking, a trapdoor function is a function that is easy to compute but hard to invert—unless one knows and has access to some trapdoor information. This information represents the private key held by a particular entity.

Diffe and Hellman's work culminated in a key agreement protocol that allows two parties that share no secret to exchange a few messages over a public channel and to establish a shared (secret) key. This key can, for example, then be used to encrypt and decrypt data. After Diffe and Hellman published their discovery, a number of public key cryptosystems were developed and proposed. Like the Diffe-Hellman key exchange protocol, some of these systems are still in use, such as RSA [59] and Elgamal [60], whereas other systems, such as the ones based on the knapsack problem,[22] have been broken and are not used anymore.

Since the early 1990s, we have seen a wide deployment and massive commercialization of cryptography. Today, many companies develop, market, and sell cryptographic techniques, mechanisms, services, and products (implemented in hardware or software) on a global scale. There are cryptography-related conferences and trade shows[23] one can attend to learn more about products that implement cryptographic techniques, mechanisms, and services.

20 Similar studies were done by Norbert Wiener, who lived from 1894 to 1964.

21 Similar ideas were pursued by Ralph C. Merkle at the University of California at Berkeley [58]. More than a decade ago, the British government revealed that public key cryptography, including the Diffe-Hellman key agreement protocol and the RSA public key cryptosystem, was invented at the Government Communications Headquarters (GCHQ) in Cheltenham in the early 1970s by James H. Ellis, Clifford Cocks, and Malcolm J. Williamson under the name *nonsecret encryption* (NSE). You may refer to the note "The Story of Non-Secret Encryption" written by Ellis in 1997 (available at http://citeseer.ist.psu.edu/ellis97story.html) to get the story. Being part of the world of secret services and intelligence agencies, Ellis, Cocks, and Williamson were not allowed to openly talk about their discovery.

22 The *knapsack problem* is a well-known problem in computational complexity theory and applied mathematics. Given a set of items, each with a cost and a value, determine the number of each item to include in a collection so that the total cost is less than some given cost and the total value is as large as possible. The name derives from the scenario of choosing treasures to stuff into your knapsack when you can only carry so much weight.

23 The most important trade show is the RSA Conference held annually in the United States, Europe, and Asia. Refer to http://www.rsaconference.com for more information.

In spite if the fact that *quantum computing* is a hotly debated topic today, the question whether it is possible to build and operate a suff ciently large and stable quantum computer is still controversially discussed in the community. But if such a computer can be built, then we know that many cryptosystems in use today can be broken eff ciently. This applies to almost all public key cryptosystems (because these systems are typically based on the integer factorization problem or discrete logarithm problem that can both be solved on a quantum computer in polynomial time), but it only partly applies to secret key cryptosystems (it is known how to reduce the steps required to perform an exhaustive key search for an n-bit cipher from 2^n to $2^{n/2}$). Against this background, people have started to look for cryptographic primitives that remain secure even if suff ciently large and stable quantum computers can be built and operated. The resulting area of research is known as *post-quantum cryptography* (PQC). In the last couple of years, PQC has attracted a lot of public interest and funding, and many researchers have come up with proposals for PQC. In the case of secret key cryptography, resistance against quantum computers can be provided by doubling the key length. This is simple and straightforward. In the case of public key cryptography, however, things are more involved and new design paradigms are needed. This is where topics like lattice-based cryptography, multivariate cryptography, hash-based (one-time) signature systems, and code-based cryptography come into play. These topics are currently explored in cryptographic research, and some of the resulting (public key) cryptosystems will certainly be used in the future.

3.2 CRYPTOSYSTEMS

In this section, we brief y introduce, overview, and put into perspective the various cryptosystems that are available and can be used in the f eld. We follow the classif cation given above and distinguish between unkeyed, secret key, and public key cryptosystems.

3.2.1 Unkeyed Cryptosystems

The most important unkeyed cryptosystems are one-way functions, cryptographic hash functions, and random generators.

3.2.1.1 One-Way Functions

As illustrated in Figure 3.1, a function $f : X \to Y$ is *one way*, if it is easy to compute but hard to invert. In accordance with the terminology used in complexity

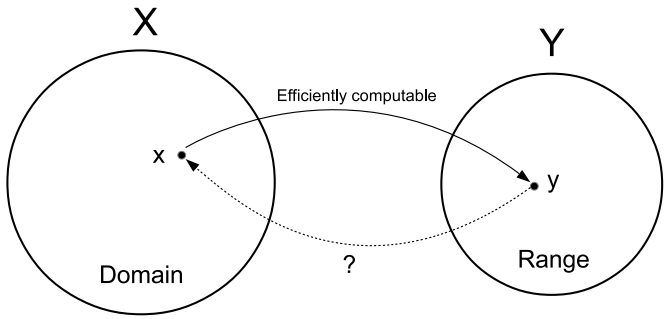


Figure 3.1 A one-way function.

theory, the term *easy* means that the computation can be done eff ciently, whereas the term *hard* means that the computation is not known to be feasible in an eff cient way, that is, no eff cient algorithm to do the computation is known to exist.[24] Consequently, a function f is one way, if $f(x)$ can be computed eff ciently for all $x \in X$, but $f^{-1}(y)$ cannot be computed eff ciently for $y \in_R Y$.[25] Furthermore, a computation is said to be eff cient, if the (expected) running time of the algorithm that does the computation is bounded by a polynomial in the length of the input. Otherwise (i.e., if the expected running time is not bounded by a polynomial in the length of the input), the algorithm requires super-polynomial time and is said to be ineff cient. For example, an algorithm that requires exponential time is clearly super-polynomial. This notion of eff ciency (and the distinction between polynomial and super-polynomial running time algorithms) is yet coarse, but still the best we have to work with.

There are many real-world examples of one-way functions. If, for example, we have a telephone book, then the function that assigns a telephone number to each name is easy to compute (because the names are sorted alphabetically) but hard to invert (because the telephone numbers are not sorted numerically). Also, many

24 Note that it is not impossible that such an algorithm exists; it is just not known.

25 In this def nition, X represents the domain of f, Y represents the range, and the expression $y \in_R Y$ reads as "an element y that is randomly chosen from Y." Consequently, it must be possible to eff ciently compute $f(x)$ for all $x \in X$, whereas it must not—or only with a negligibly small probability—be possible to compute $f^{-1}(y)$ for a y randomly chosen from Y. To be more precise, one must state that it may be possible to compute $f^{-1}(y)$, but that the entity that wants to do the computation does not know how to do it.

physical processes are inherently one way. If, for example, we smash a bottle into pieces, then it is generally infeasible to put the pieces together and reconstruct the bottle. Similarly, if we drop a bottle from a bridge, then it falls down, whereas the reverse process never occurs by itself. Last but not least, the movement of time is one way, and it is (currently) not known how to travel back in time. As a consequence of this fact, we continuously age and have no possibility to make ourselves young again.

In contrast to the real world, there are only a few mathematical functions conjectured to be one way. The most important examples are centered around modular exponentiation: Either $f(x) = g^x \pmod m$, $f(x) = x^e \pmod m$, or $f(x) = x^2 \pmod m$ for a properly chosen modulus m. While the argument x is in the exponent in the f rst function, x represents the base of the exponentiation function in the other two functions. Inverting the f rst function requires computing discrete logarithms, whereas inverting the second (third) function requires computing e^{th} (square) roots. The three functions are used in different public key cryptosystems: The f rst function is, for example, used in the Diff e-Hellman key exchange protocol [38] outlined in Section 3.2.3.3, the second function is used in the RSA public key cryptosystem [59] outlined in Section 3.2.3.1, and the third function is used in the Rabin encryption system [39]. It is important to note that none of these functions has been shown to be one way in a mathematically precise sense, and that it is theoretically not even known whether one-way functions exist at all. This means that the one-way property of these functions is just an assumption that may turn out to be wrong (or illusory) in the future. We don't think so, but it may still be the case.

Assuming the existence of one-way functions, there is a subset of such functions that can be inverted eff ciently, if and—as it is hoped—only if some extra information is known. In fact, a one-way function $f : X \to Y$ is a *trapdoor function* (or a *trapdoor one-way function*, respectively), if there is some extra information (i.e., the *trapdoor*) with which f can be inverted eff ciently (i.e., $f^{-1}(y)$ can be computed eff ciently for $y \in_R Y$). Among the functions mentioned above, $f(x) = x^e \pmod m$ and $f(x) = x^2 \pmod m$ have a trapdoor, namely the prime factorization of n. Somebody who knows the prime factors of m can eff ciently invert the functions. In contrast, the function $f(x) = g^x \pmod m$ for a prime number m is not known to have a trapdoor.

The mechanical analog of a trapdoor (one-way) function is a padlock. It can be closed by everybody (if it is in an unlocked state), but it can be opened only by somebody who holds the proper key. In this analogy, a padlock without a keyhole represents a one-way function with no trapdoor. In the real world, this is not a particularly useful artifact, but in the digital world, as we will see, there are quite a few interesting applications for it.

3.2.1.2 Cryptographic Hash Functions

Hash functions are widely used and have many applications in computer science. Informally speaking, a hash function is an eff ciently computable function that takes an arbitrarily large input and generates an output of a usually much smaller size. More formally, a function $h : X \to Y$ is called a *hash function*, if $|X| \gg |Y|$ and $h(x)$ can be computed eff ciently for all $x \in X$. This idea is illustrated in Figure 3.2.

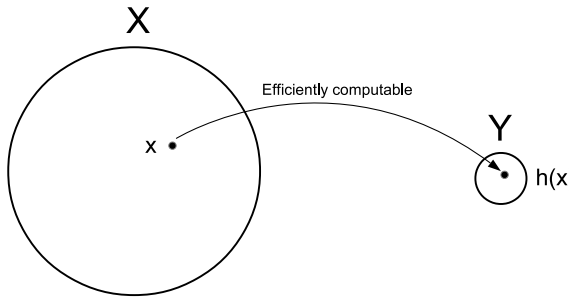


Figure 3.2 A hash function.

The elements of X and Y are strings of characters from a given alphabet. If Σ_{in} is the input alphabet and Σ_{out} is the output alphabet, then a hash function h can be written as $h : \Sigma_{in}^{*} \to \Sigma_{out}^{n}$ or $h : \Sigma_{in}^{n_{max}} \to \Sigma_{out}^{n}$, for an input size restricted to n_{max} for technical reasons[26]. In either case, the output is n characters long. In many practical settings, Σ_{in} and Σ_{out} are identical and refer to the binary alphabet $\Sigma = \{0, 1\}$.[27] In such a setting, the hash function h takes as input arbitrarily long bit strings and generates as output bit strings of f xed size n.

In cryptography, we are talking about bit strings that are a few hundred bits long. Also, we are talking about hash functions that have specif c (security) properties, such as one-wayness (or preimage resistance, respectively) second-preimage resistance, and/or collision resistance. We omit the details here, and only mention brief y that a hash function is second-preimage resistant, if it is computationally infeasible to f nd a second argument $x' \in X$ that happens to have the same hash value as a given argument $x \in X$ (i.e., $x' \neq x$ but $h(x') = h(x)$).

26 One such reason may be that the input length must be encoded in a f xed-length f eld in the padding.

27 Note that this is just a rule of thumb, and that there are quite a few exceptions, such as hashing bit strings onto points on an elliptic curve.

It is called collision resistant, if it is computationally infeasible to find any pair of arguments $x, x' \in X$ that fulfill the same requirements. Note that in this case, x is not fixed and can be arbitrary. The probability of finding such a pair is significantly larger than the probability of finding a second preimage for a fixed x.[28] This, in turn, means that collision resistance is an inherently more difficult to achieve than second-preimage resistance, and hence people sometimes call second-preimage resistance weak collision resistance and collision resistance strong. Anyway, a hash function h is *cryptographic*, if it is either one-way and second-preimage resistant or one-way and collision resistant.

Cryptographic hash functions have many applications in cryptography. Most importantly, a cryptographic hash function h can be used to hash arbitrarily sized messages to binary strings of fixed size. This is illustrated in Figure 3.3, where the ASCII-encoded message "This is a file that includes some important but long statements. Consequently, we may need a short representation of this file." is hashed to `0x492165102a1a9e3179a2139d429e32ce4f7fd837` (in hexadecimal notation).[29] This value represents the *fingerprint* or *digest* of the message and—in some sense—stands for it. The second-preimage resistance property implies that it is difficult—or even computationally infeasible—to find another message that hashes to the same value. It also implies that a minor modification of the message leads to a completely different hash value that looks random. If, for example, a second point were added to the message given above, then the resulting hash value would be `0x2049a3fe86abcb824d9f9bc957f00cfa7c1cae16` (that is completely independent from `0x492165102a1a9e3179a2139d429e32ce4f7fd837`). If the collision resistance property is required, then it is even computationally infeasible to find two arbitrary messages that hash to the same value.

Examples of cryptographic hash functions that are used in the field are MD5, SHA-1 (as depicted in Figure 3.3), the representatives of the SHA-2 family (i.e., SHA-224, SHA-256, SHA-384, and SHA-512, and SHA-3). These functions generate hash values of different sizes.[30] As we will see in the rest of this book, many of these functions are also used in E2EE messaging.

28 This results from the birthday paradox that is well-known in probability theory. It concerns the probability that, in a set of n randomly chosen people, some pair of them will have the same birthday. By the pigeonhole principle, the probability reaches 100% when the number of people reaches 367 (since there are only 366 possible birthdays, including February 29). However, 99.9% probability is reached with already 70 people, and 50% probability is reached with only 23 people. This is much less than one would guess at first sight, and hence we call it paradox.

29 In the hash computation, the message is used without word delimiter.

30 The output of MD5 is 128 bits long. The output of SHA-1 is 160 bits long. Many other hash functions, including SHA-2 and SHA-3, generate an output of variable size. In the case of the representatives of the SHA-2 family, the length of the output is usually appended to the prefix "SHA-" in the respective function name.

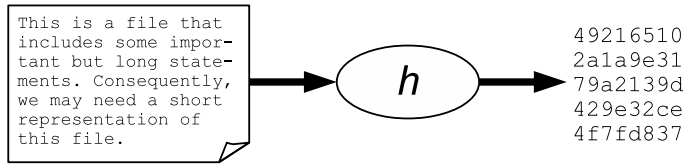


Figure 3.3 A cryptographic hash function.

3.2.1.3 Random Generators

Randomness is the most important ingredient for cryptography and many cryptographic systems depend on some form of randomness. This is certainly true for every key generation and probabilistic encryption algorithm, but it may also be true for many other cryptographic algorithms. This is where the notion of a *random generator* comes into play. It is a device that outputs a sequence of statistically independent and unbiased values. If the output values are bits, then the random generator is also called *random bit generator*.

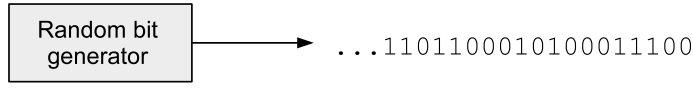


Figure 3.4 A random bit generator.

A random bit generator is depicted in Figure 3.4 as a grey box. It is important to note that such a generator has no input, and that it only generates an output. Also, because the output is a sequence of statistically independent and unbiased bits, all bits must occur with the same probability (i.e., $\Pr[0] = \Pr[1] = 1/2$), or—more generally—all 2^k different k-tuples of bits must occur with the same probability $1/2^k$ for all integers $k \geq 1$. There are many statistical tests that can be used to verify these properties. It is also important to note that a random (bit) generator cannot be implemented in a deterministic way. Instead, it must be inherently nondeterministic, meaning that an implementation must use some physical events or phenomena (for which the outcomes are hard to predict). Alternatively speaking, every (true) random

bit generator requires a naturally occurring source of randomness. The (engineering) task of f nding such a source and exploiting it in a device that may serve as a generator to output binary sequences that are free of biases and correlations is challenging. Also, the device must withstand various types of attacks, and it must still be possible to implement it in a cost-eff cient manner.

One-way functions, cryptographic hash functions, and random generators are unkeyed cryptosystems that are heavily used in cryptography, and serve as building blocks in more sophisticated crytographic systems and applications. The next class of cryptosystems we look at are symmetric or secret key cryptosystems (i.e., cryptosystems that use secret parameters that are shared among different participating entities).

3.2.2 Secret Key Cryptosystems

The most important representatives of secret key cryptosystems are symmetric encryption systems, message authentication systems, pseudorandom generators (PRGs), and—more theoretically important—pseudorandom functions (PRFs).

3.2.2.1 Symmetric Encryption Systems

When people talk about cryptography, they most frequently refer to conf dentiality protection using a *symmetric encryption system* that, in turn, can be used to encrypt and decrypt data. *Encryption* refers to the process that maps a plaintext message to a ciphertext, whereas *decryption* refers to the reverse process (i.e., the process that maps a ciphertext back to the plaintext message). Formally speaking, a *symmetric encryption system* or *cipher* is a 5-tuple $(\mathcal{M}, \mathcal{C}, \mathcal{K}, E, D)$ that consists of a *plaintext message space* $\mathcal{M}$,[31] a *ciphertext space* $\mathcal{C}$, a *key space* $\mathcal{K}$, and two families of eff ciently computable functions:

- A family $E = \{E_k : k \in \mathcal{K}\}$ of *encryption functions* $E_k : \mathcal{M} \to \mathcal{C}$;
- A family $D = \{D_k : k \in \mathcal{K}\}$ of *decryption functions* $D_k : \mathcal{C} \to \mathcal{M}$.

While the decryption functions need to be deterministic, the encryption functions can be deterministic or probabilistic. In the second case, they usually take some random data as additional input (not further explored here). For every message $m \in \mathcal{M}$ and every key $k \in \mathcal{K}$, the functions D_k and E_k must be inverse to each other (i.e., $D_k(E_k(m)) = m$). In a typical setting, $\mathcal{M} = \mathcal{C} = \{0,1\}^*$ refers to the set of all arbitrarily long binary strings, whereas $\mathcal{K} = \{0,1\}^l$ refers to the set of

31 In some literature, the plaintext message space is denoted by $\mathcal{P}$.

all l bits long keys. Hence, l stands for the key length of the symmetric encryption system in use (typically $l = 128$ or 256).

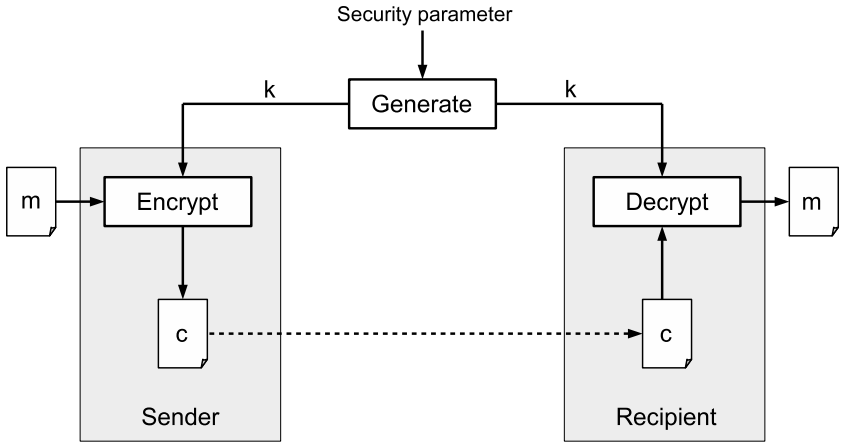


Figure 3.5 The working principle of a symmetric encryption system.

In the description of a symmetric encryption system, we sometimes use an algorithmic notation: Generate then stands for a key generation algorithm (that can also be omitted, due to its simplicity in the symmetric case), Encrypt for an algorithm that implements the encryption function, and Decrypt for an algorithm that implements the decryption function. Using this notation, the working principle of a symmetric encryption system is illustrated in Figure 3.5. First, the Generate algorithm generates a key k by randomly choosing an element from $\mathcal{K}$. This key is then distributed to either side of the communication channel (this is why the encryption system is called symmetric in the first place), namely to the sender (or the sending device, respectively) on the left side and the recipient (or the receiving device, respectively) on the right side.[32] The sender can encrypt a plaintext message $m \in \mathcal{M}$ with its implementation of the encryption function or Encrypt algorithm feeded with k. The resulting ciphertext $E_k(m) = c \in \mathcal{C}$ is sent to the recipient over

32 If there is a way to securely distribute a key to the sender and the recipient, then one may be tempted to use the respective secure channel to send the message (instead of the key). This is not really true, because the key is much smaller than the message and the channel may be short-lived and disappear immediately after the key exchange.

the communication channel that is considered to be insecure (and therefore drawn as a dotted line in Figure 3.5). On the right side, the recipient can decrypt c with its implementation of the decryption function or Decrypt algorithm—again feeded with the same key k. If the decryption is successful, then the recipient is able to retrieve and continue to use the original plaintext message m.

The characteristic feature of a symmetric encryption system is that k is the same on either side of the communication channel, meaning that k is a secret shared by the sender and the recipient. Another characteristic feature of a symmetric encryption system is that it can operate on individual bits and bytes (typically representing a *stream cipher*) or on a larger block (typically representing a *block cipher*). While there are modes of operation that turn a block cipher into a stream cipher, the opposite is not known to be true, meaning that there is no mode of operation that turns a stream cipher into a block cipher. There is actually no need for such a mode of operation: If a block cipher is needed, then one can always use one in the f rst place.

The modes of operation that are relevant for block ciphers are electronic codebook (ECB) and cipher block chaining (CBC), whereas the modes of operation that turn a block cipher into a stream cipher are cipher feedback (CFB), output feedback (OBC), and counter (CTR) modes. CBC is the most important mode of operation in the f rst case, whereas CTR is the most important mode of operation in the second case. When it comes to secure and E2EE messaging, we sometimes have to refer to a particular mode of operation, because some attacks exploit specif c properties of it. In these cases, we provide some background information about the mode on the f y. Also, as discussed in the context of message authentication (see next section), there are modes of operation that not only provide message secrecy but also message authenticity and integrity. These modes of operation are heavily preferred today.

There are many symmetric encryption systems that are either historically important, such as the Data Encryption Standard (DES) and RC4, or practically relevant, such as the block cipher Advanced Encryption Standard (AES) and the stream cipher Salsa20/ChaCha20. These ciphers dominate the f eld, but there are many other ciphers proposed in the literature. Some of these ciphers have been broken while others remain secure. We occasionally come across some of these ciphers throughout the book. But keep in mind that there is hardly any reason to use such a cipher, unless the AES is considered to be insecure (that is clearly not the case yet). As we will see throughout the book, the AES clearly dominates the f eld when it comes to secure and E2EE messaging (and it also applies to most other areas in communication security).

3.2.2.2 Message Authentication Systems

While encryption systems are to protect the conf dentiality of data, there are applications that require rather the authenticity and integrity of data to be protected—either in addition or instead of the conf dentiality. Consider, for example, a f nancial transaction. It is nice to have the conf dentiality of this transaction be protected, but it is somewhat more important to protect its authenticity and integrity. The typical way to achieve this is to have the sender add an *authentication tag* to the message and to have the recipient verify this tag before he or she accepts the message as being genuine. This is conceptually similar to an error correcting code. But in addition to have the code protect the message against transmission errors, the authentication tag is to also protect the message against tampering and deliberate fraud. This means that the tag itself needs to be protected against an adversary who may try to modify the message and/or the tag.

From a bird's eye perspective, there are two possibilities to construct an authentication tag: Either through the use of public key cryptography and a *digital signature system* (DSS)—see Section 3.2.3.2—or through the use of secret key cryptography and a so-called *message authentication code* (MAC).[33] Hence, a MAC is an authentication tag that can be computed and verif ed with a secret parameter (e.g., a secret cryptographic key). In the case of a message sent from one sender to one recipient, the secret parameter must be shared between the two entities. If, however, the message is sent to multiple recipients, then the secret parameter must be shared among the sender and all receiving entities. In this case, the distribution and management of the secret parameter represents a major challenge (and probably one of the Achilles' heels of the entire system). In either case, it is important to note that a MAC has no value in convincing a third party, since from their perspective, either the sender or a recipient could have generated the MAC. This is in sharp contrast to a digital signature that can always be used to convince a third party.

Similar to a symmetric encryption system, one can introduce and formally def ne a system to compute and verify MACs. We use the term *message authentication system* to refer to such a system (contrary to many other terms used in this book, this term is not widely used in the literature). A *message authentication system* is a 5-tuple $(\mathcal{M}, \mathcal{T}, \mathcal{K}, A, V)$ that consists of a *plaintext message space* $\mathcal{M}$, a *tag space* $\mathcal{T}$, a *key space* $\mathcal{K}$, and two families of eff ciently computable functions:

- A family $A = \{A_k : k \in \mathcal{K}\}$ of *authentication functions* $A_k : \mathcal{M} \to \mathcal{T}$;
- A family $V = \{V_k : k \in \mathcal{K}\}$ of *verif cation functions* $V_k : \mathcal{M} \times \mathcal{T} \to \{valid, invalid\}$.

33 In some literature, the term *message integrity code* (MIC) is used synonymously and interchangeably with a MAC. This term, however, is not used in this book.

For every message $m \in \mathcal{M}$ and every key $k \in \mathcal{K}$, $V_k(m,t)$ must yield $valid$ if and only if t is a valid authentication tag for m and k (i.e., $t = A_k(m)$) and hence $V_k(m, A_k(m))$ must yield $valid$. Typically, $\mathcal{M} = \{0,1\}^*$, $\mathcal{T} = \{0,1\}^{l_{tag}}$ for some f xed tag length l_{tag}, and $\mathcal{K} = \{0,1\}^{l_{key}}$ for some f xed key length l_{key}. Very likely $l_{tag} = l_{key} = 128$, and hence the tags and keys are both 128 bits long.

The situation is depicted in Figure 3.6. Similar to a symmetric encryption systems, a Generate algorithm takes as input a security parameter and randomly selects a key k from $\mathcal{K}$ according to this parameter. The key is forwarded to the sender (on the left side) and the recipient (on the right side). The sender uses the authentication function or Authenticate algorithm to compute an authentication tag t from m and k. Both m and t are sent to the recipient. The recipient, in turn, uses the verif cation function or Verify algorithm to check whether t is a valid tag with respect to m and k. The resulting binary value yields the output of the Verify algorithm.

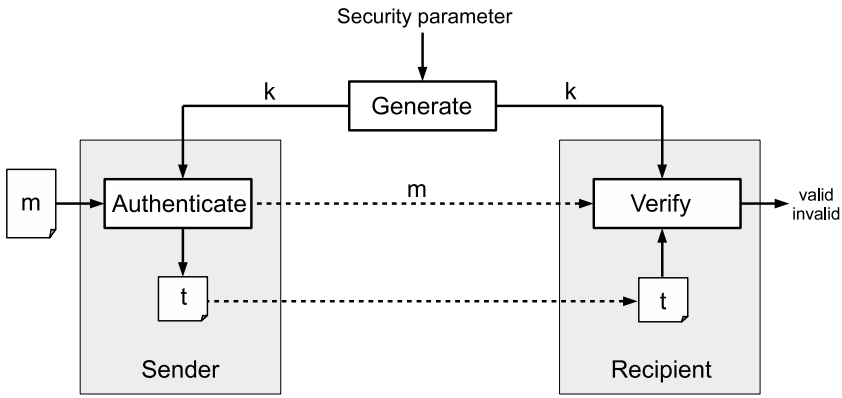


Figure 3.6 The working principle of a message authentication system.

There are several message authentication systems that can used in the f eld. Most of these systems use the *hashed MAC* (HMAC) construction that employs a keyed hash function. In principle, the message and a secret key are hashed with a cryptographic hash function in a very particular way. The resulting value depends on both the message and the key, and hence represents a MAC. Also, there are a few

alternative constructions that are not so widely deployed yet, such as *CBC-MAC*[34] or *Carter-Wegman MACs* (e.g., universal MAC (UMAC), Poly1305, and GMAC).

More recently, cryptographers have come up with modes of operation for block ciphers that simultaneously protect the conf dentiality and the authenticity (and integrity) of messages. They provide *authenticated encryption* or *authenticated encryption with associated data* (AEAD). In the second case, all data is authenticated, but not all data is encrypted (in fact, the associated data is authenticated but not encrypted). Almost all Internet security protocols in use today provide support for AEAD, and the two most widely deployed modes of operation are CCM and GCM. CCM uses CTR mode for encryption and CBC-MAC for authentication, whereas GCM also uses CTR mode for encryption but GMAC for authentication. AEAD is the state of the art in contemporary cryptography when it comes to data encryption and authentication. As we will see in the rest of this book, it is also used in many solutions for E2EE messaging on the Internet today.

3.2.2.3 PRGs

A random bit generator is a device that requires some source of randomness. In many situations, there is no such source available or the source is able to generate only a few values. In this case, the few values may need to be stretched into a long sequence of values that appear to be random. This is where the notion of a PRG comes into play: A *PRG* is an eff ciently computable function that takes as input a relatively short value of length n, called the *seed*, and generates as output a value of length $l(n)$ (where $l(n) \gg n$) that appears to be random (and is therefore called *pseudorandom*). If the input and output values are bit sequences, then the PRG yields a *pseudorandom bit generator* (PRBG). Such a PRBG is illustrated in Figure 3.7.[35]

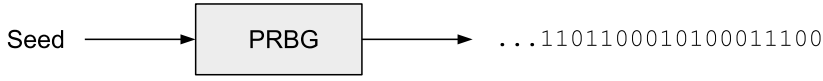


Figure 3.7 A PRBG.

34 This construction uses a symmetric encryption system in the CBC mode of operation. This is a chaining mode, meaning that all ciphertext blocks depend on all previous blocks. Hence, the last ciphertext block depends on all previous blocks and may serve as a MAC. In reality, the CBC-MAC construction is a little bit more involved and specif ed in a standardized mode of operation denoted as *CMAC*.

35 Note the subtle difference between Figures 3.4 and 3.7. Both generators output bit sequences. But while the random bit generator has no input, the PRBG has a seed that represents the input.

The input to the PRBG is a seed that can also be seen as an n-key. This, in turn, means that the key space is the set of all bit sequences of length n (i.e., $\mathcal{K} = \{0,1\}^n$) and hence that a PRBG G def nes a mapping from $\mathcal{K}$ to $\{0,1\}^{l(n)}$, where $l(n)$ represents a stretch function, i.e., it stretches an n-bit input value into a longer $l(n)$-bit output value with $n < l(n) \leq \infty$. Formally, this can be expressed as follows:

$$G : \mathcal{K} \longrightarrow \{0,1\}^{l(n)}$$

Note, however, that this def nition is not precise in a mathematically strong sense, because we have not yet def ned what we mean by saying that a bit sequence appears to be random. Unlike a true random generator, a PRG operates deterministically, and this, in turn, means that a PRG always outputs the same bit sequence if seeded with the same input value. A PRG thus represents a f nite state machine, and hence the sequence of generated bits is going to be cyclic (with a potentially very large cycle). This is why we cannot require that the output of a PRG is truly random; we can only require that it appears to be so (for some computationally bounded adversary). More specif cally, if the adversary only gets some output values, then he or she cannot tell (with a success probability that is signif cantly better than guessing) whether these values have been generated randomly or pseudorandomly. From his or her perspective, the values look as if they were generated randomly.

As in the case of random (bit) generators, there are many statistical tests that can be used to verify the randomness properties of the output of a PR(B)G; the tests are essentially the same. Passing these tests is a necessary but usually not suff cient condition for the output of a PRG to be used for cryptographic purposes. In addition to these tests, one must ensure that the output of a PRG is unpredictable, or—following the line of argumentation introduced above—that no statistical test is able to distinguish the output of a PRG from the output of a true random generator.

PRGs have many applications in cryptography, and the title of [17] suggests that the notion of pseudorandomness and cryptography are closely related and deeply intertwined. Most importantly, every (additive) stream cipher yields a PRG. Examples include LFSR[36]-based stream ciphers, such as the content scrambling system (CSS) that uses two LFSRs for DVD encryption, A5/1 that uses three LFSRs for voice encryption on mobile devices, and E0 that uses four LFSRs for Bluetooth encryption, as well as RC4 and Salsa20 or ChaCha20 (already mentioned above). In addition, there are specif c PRG constructions, like ANSI X9.17 and Yarrow, and constructions that are based on some computational intractability assumptions, but for which it can be proven mathematically that their output is unpredictable.

36 The term acronym LFSR stands for linear feedback shift register. It is a hardware-oriented technology to generate pseudorandom bit sequences.

The most important example of such a construction is the BBS generator that is also known as the squaring generator, because it is based on the modular square function that is supposedly one-way. When it comes to discussing the security of a cryptographic system or application, then one of the key questions to ask (and possibly answer) is how the PRG works that is used internally. If this is done ad hoc in some barely documented way, then the odds are good to mount a successful attack that exploits this fact.

3.2.2.4 PRFs

To properly understand the notion of a PRF, it is necessary to first introduce the notion of a *random function*—also known as *random oracle*. In essence, a random function $f : X \to Y$ is a function that is chosen randomly from the set of all possible functions that map elements of X to elements of Y (i.e., $\mathrm{Funcs}[X, Y]$). Hence, the attribute random in the term random function does not characterize the output of the function, but rather the way it is chosen, i.e., it is chosen randomly from the set of all possible functions. Note that there are $|Y|^{|X|}$ functions in $\mathrm{Funcs}[X, Y]$, and this number is incredibly large (even for moderately sized X and Y). If, for example, X refers to the set of all 2-bit strings and Y refers to the set of all 3-bit strings, then $|X| = 2^2$ and $|Y| = 2^3$, and hence $\mathrm{Funcs}[X, Y]$ comprises $(2^3)^{2^2} = (2^3)^4 = 2^{12} = 4,096$ functions. If X and Y both consist of 128-bit strings, then $\mathrm{Funcs}[X, Y]$ even comprises

$$(2^{128})^{2^{128}} = 2^{128 \cdot 2^{128}} = 2^{2^7 \cdot 2^{128}} = 2^{2^{135}}$$

functions. This number is so incredibly large that one cannot really work with it. Nevertheless, the characteristic feature of a random function is that it can output any value $y \in Y$ for an input value $x \in X$. The only requirement is that the same x is always mapped to the same y. Everything else is possible and does not really matter (for the function to be random).

Having this notion of a random function in mind, a PRF is to simulate it in the sense that it looks like a random function without being one (this is why the attribute pseudorandom is used in the first place). An alternative way of saying that a PRF "looks like" a random function is that it is computationally indistinguishable from a random function, meaning that somebody interacting with either a PRF or a random function cannot tell whether he or she is interacting with a PRF or a true random function. If the two functions are indistinguishable, then—for all practical purposes—they behave similarly and can be used interchangeably. This, in turn, means that one can use a PRF instead of a random function and still achieve the same unpredictability (and hence security) a random function provides.

To formalize this idea, we consider a family of PRFs, denoted as F, where every function from the family depends on a particular key k from a key space $\mathcal{K}$ (i.e., $k \in \mathcal{K}$). A particular function F_k from F can then be def ned as follows:

$$F_k : X \to Y$$

This function looks like a random function but is not (as it is predetermined by the key k). Note that there is one function def ned for every key, and hence the PRF family consists of only $|\mathcal{K}|$ functions, whereas $\mathrm{Funcs}[X, Y]$ comprises $|Y|^{|X|}$ functions. This means that we can use a relatively small key to determine a particular function in the PRF family, and this function still behaves like a random function.

If $X = Y$, then $\mathrm{Funcs}[X, Y] = \mathrm{Funcs}[X, X]$—sometimes denoted as $\mathrm{Perms}[X]$—refers to the set of all possible permutations of X, and a *random permutation* is randomly chosen from this set. For the sake of completeness, we note that a permutation is a bijective (i.e., injective and surjective) mapping from X to itself. Similar to the notion of a PRF or PRF family, we use the notion of a *pseudorandom permutation* (PRP) or a PRP family to refer to permutations that look random. While it is reasonable to talk about PRF and PRP families, people sometimes take a shortcut and use the term PRF to refer to a PRF family and PRP to refer to a PRP family. This is mathematically unprecise, but often more intuitive and hence simpler to understand.

The notions of a PRF or PRP are important in cryptography, mainly because many cryptographic primitives that are practically relevant can be seen this way: A cryptographic hash function can be seen as a PRF, a block cipher can be seen as a PRP, and a PRG can even be built from a PRF and vice-versa. We brief y mentioned before that in many security proofs one starts with the assumption that a cryptographic primitive (typically a cryptographic hash function) is a random function and one then formally proves that the respective system is secure in the random oracle model. Hence, the notions of random functions and PRFs are particularly important in security proofs. From a purely practical viewpoint, they are less important, and this book can also be understood without having fully captured these notions.

The next class of cryptosystems we look at are public key cryptosystems. These are cryptosystems that use secret parameters that are not shared among all participating entities. Alternatively speaking, there are some secret parameters that are privately held. In contrast to random functions and PRFs, public key cryptosystems are very important in practice and need to be understood for almost all cryptographic applications.

3.2.3 Public Key Cryptosystems

Instead of sharing all secret parameters, the entities that participate in an *asymmetric* or public key cryptosystem hold two distinct sets of parameters: One that is private (collectively referred to as the *private* or *secret key*[37] and abbreviated sk), and one that is published (collectively referred to as the *public key* and abbreviated pk).[38] A necessary but usually not sufficient prerequisite for a public key cryptosystem to be secure is that both keys—the private key and the public key—are yet mathematically related, but it is still computationally infeasible to compute one from the other. Another prerequisite that is important in practice is that the public keys are published in a form that provides authenticity and integrity. If somebody is able to introduce faked public keys, then he or she is usually able to mount very powerful attacks. This is why we usually require public keys to be published in certified form, and hence the notions of (public key) certificates and public key infrastructures (PKI) immediately pop up (see Section 3.3).

The fact that public key cryptosystems use secret parameters that are not shared among all participating entities suggests that the respective algorithms are executed by different entities, and hence that such cryptosystems can be defined as sets of algorithms (that are then executed by these different entities). We adopt this viewpoint and define public key cryptosystems as sets of algorithms. In the case of an asymmetric encryption system, for example, there is a key generation algorithm Generate, an encryption algorithm Encrypt, and a decryption algorithm Decrypt. The Generate and Encrypt algorithms are typically executed by the sender of a message, whereas the Decrypt algorithm is typically executed by the recipient. As discussed later, other public key cryptosystems may employ other sets of algorithms.

We now briefly introduce and put into perspective the most important public key cryptosystems used in the field (i.e., asymmetric encryption systems, DSSs, and protocols for key agreement), and we only provide examples where needed and appropriate for the purpose of this book.

3.2.3.1 Asymmetric Encryption Systems

Similar to a symmetric encryption system, an asymmetric encryption system can be used to encrypt and decrypt plaintext messages. The major difference between a symmetric and an asymmetric encryption system is that the former employs secret

37 The preferred choice to name this key is private key, but this leads to the same acronym pk. People therefore more often use the term secret key to refer to the private key of a public key pair. Unfortunately, this risks confusion with a secret key from a secret key cryptosystem, and one has to keep this ambiguity in mind when using the term.

38 It depends on the cryptosystem whether it matters which set of parameters is used to represent the private key and which set of parameters is used to represent the public key.

key cryptography and respective techniques, whereas the latter employs public key cryptography and respective techniques.

As mentioned above, an asymmetric encryption system can be built from a trapdoor function—or, more specif cally—from a family of trapdoor functions. Each public key pair comprises a public key pk that represents a one-way function and a private key sk that represents a respective trapdoor.[39] To send a secret message to the recipient, the sender must look up the recipient's public key, apply the respective one-way function to the plaintext message, and send the resulting ciphertext to the recipient. The recipient, in turn, is the only entity that supposedly holds the trapdoor (information) needed to invert the one-way function and to decrypt the ciphertext accordingly.

Formally speaking, an *asymmetric encryption system* consists of the following three eff ciently computable algorithms:

- $\mathsf{Generate}(1^k)$ is a probabilistic key generation algorithm that takes as input a security parameter 1^k (in unary notation), and generates as output a public key pair (pk, sk) that is in line with the security parameter;
- $\mathsf{Encrypt}(pk, m)$ is a deterministic or probabilistic encryption algorithm that takes as input a public key pk and a plaintext message m, and generates as output a ciphertext $c = \mathsf{Encrypt}(pk, m)$;
- $\mathsf{Decrypt}(sk, c)$ is a deterministic decryption algorithm that takes as input a private key sk and a ciphertext c, and generates as output a plaintext message $m = \mathsf{Decrypt}(sk, c)$.

For every plaintext message m and every public key pair (pk, sk), the Encrypt and Decrypt algorithms must be inverse to each other (i.e., $\mathsf{Decrypt}(sk, \mathsf{Encrypt}(pk, m)) = m$).

The working principle of an asymmetric encryption system is illustrated in Figure 3.8. At the top of the f gure, the Generate algorithm is to generate a public key pair for entity A that is going to act as the recipient of a message. In preparation to the encryption, A's public key pk_A is provided to the sender on the left side. The sender then subjects the message m to the one-way function represented by pk_A, and sends the respective ciphertext $c = \mathsf{Encrypt}_{pk_A}(m)$ to A. On the right side, A knows its secret key sk_A that represents a trapdoor to the one-way function. This trapdoor can then be used to decrypt c and retrieve the original plaintext message $m = \mathsf{Decrypt}_{sk_A}(c)$. Hence, the output of the Decrypt algorithm is the originally sent message m.

39 In essence, the trapdoor is needed to eff ciently compute the inverse of the one-way function.

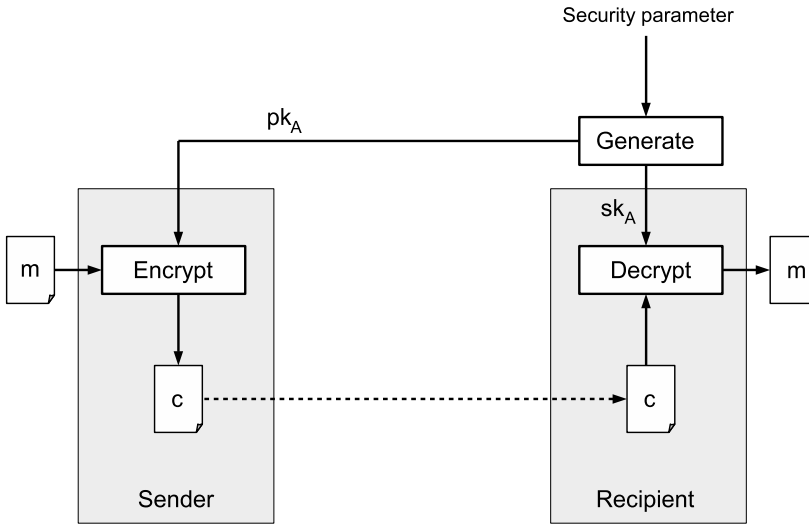


Figure 3.8 The working principle of an asymmetric encryption system.

There are many asymmetric encryption systems that have been proposed in the literature, such as Elgamal, RSA, and Rabin. These systems are based on the three exemplary one-way functions mentioned in Section 3.2.1.1 (in this order). Because it is computationally infeasible to invert these one-way functions, the systems provide a reasonable level of security—even in their basic forms (that are sometimes called textbook versions). But when it comes to more sophisticated attacks, such as chosen ciphertext attacks, or stronger notions of security, some variations of the basic systems are needed. The strongest notion of security that can be achieved by an asymmetric encryption system is def ned in a game-theoretical setting: An adversary can select two equally long plaintext messages and has one of them be encrypted. If he or she cannot tell whether the respective ciphertext is the encryption of the f rst or the second plaintext message with a probability that is signif cantly better than guessing, then the asymmetric encryption system arguably leaks no information and can therefore assumed to be secure. This may hold even if the adversary has access to a decryption oracle, meaning that he or she can have any ciphertext of his or her

choice be decrypted—except, of course, the ciphertext the adversary is challenged with.

For the topic of this book, it is important to understand RSA, and hence we want to brief y summarize this public key cryptosystem here. It looks as follows:

Generate

(1^k)
$p, q \xleftarrow{r} \mathbf{P_{k/2}}$
$n \leftarrow p \cdot q$
$e \xleftarrow{r} (1, \phi(n))$
with $gcd(e, \phi(n)) = 1$
compute $1 < d < \phi(n)$
with $de \equiv 1 \pmod{\phi(n)}$
$((n, e), d)$

Encrypt

$((n, e), m)$
$c \leftarrow m^e \pmod{n}$
(c)

Decrypt

(d, c)
$m \leftarrow c^d \pmod{n}$
(m)

The RSA Generate(1^k) algorithm takes as input a security parameter 1^k (in unary notation), and it generates as output a public key pair (where (n, e) is the public key and d is the private key.). It f rst randomly selects two $k/2$-bit primes[40] p and q, and multiplies the two values to compute the RSA modulus n. This value is part of the public key, whereas p and q are kept secret. The algorithm randomly selects an integer e with $1 < e < \phi(n)$ and $gcd(e, \phi(n)) = 1$,[41] and it then computes the multiplicative inverse d of e modulo $\phi(n)$, using, for example, the extended Euclid algorithm. Finally, the algorithm outputs the public key pair. The prime factors p and q can be used to speed up decryption, and in this case they also represent part of private key. The RSA Encrypt and Decrypt algorithms are even simpler:

- The Encrypt algorithm is deterministic. It takes as input the public key (n, e) and a plaintext message m (interpreted as a number smaller than n), and it generates as output the ciphertext c that is equal to m raised to the power of e modulo n.
- The Decrypt algorithm is structurally the same. It takes as input the private key d and the ciphertext c, and it generates as output the original plaintext message m that is equal to c raised to the power of d modulo n.

Due to Euler's theorem (that is a generalization of Fermat's little theorem), one can show that the RSA public key cryptosystem works and is correct. It can be

40 The set of all $k/2$-bit primes is denoted as $\mathbf{P_{k/2}}$ here.

41 In this notation, ϕ refers to Euler's totient function that counts the numbers that are smaller than n and have no other common divisor with n other than 1 (i.e., they are coprime with n).

used to asymmetrically encrypt and decrypt messages, as well as to generate digital signatures (using the private key) and verify the signatures respectively (using the public key). Discussing the security of RSA is subtle, and there are many details that must be addressed here. With regard to using RSA for encryption, we just note that there is a secure version that employs optimal asymmetric encryption padding (OAEP), abbreviated as RSA-OAEP. It is used in many E2EE messaging protocols that still employ RSA for encryption, such as S/MIME, iMessage, and a few more. Note, however, that many modern E2EE messaging protocols and respective messengers try to avoid RSA for encryption in favor of the Diff e-Hellman key exchange.

3.2.3.2 DSSs

Digital signatures can be used to protect the authenticity and integrity of messages, or—more generally—data objects. According to [35], a *digital signature* refers to "a value computed with a cryptographic algorithm and appended to a data object in such a way that any recipient of the data can use the signature to verify the data's origin and integrity." Similarly, the term digital signature is def ned as "data appended to, or a cryptographic transformation of, a data unit that allows a recipient of the data unit to prove the source and integrity of the data unit and protect against forgery, e.g. by the recipient" in ISO/IEC 7498-2 [61]. Following the second def nition, there are two classes of digital signatures that are sometimes distinguished in the literature:

- If data representing the digital signature is appended to a data unit (or message), then one refers to a *digital signature with appendix*.
- If a data unit is cryptographically transformed in a way that it represents both the data unit (or message) that is signed and the digital signature, then one refers to a *digital signature giving message recovery*. In this case, the data unit is recovered if and only if the signature is successfully verif ed.

In either case, the entity that digitally signs a data unit or message is called the *signer* or *signatory*, whereas the entity that verif es the digital signature is called the *verif er*. Both the signer and the verif er are usually computing devices with respective software (that may operate on a user's behalf).

More formally, a *DSS with appendix* consists of the following three eff ciently computable algorithms:

- $\mathsf{Generate}(1^k)$ is a probabilistic key generation algorithm that takes as input a security parameter 1^k, and generates as output a public key pair (pk, sk) that is in line with the security parameter.

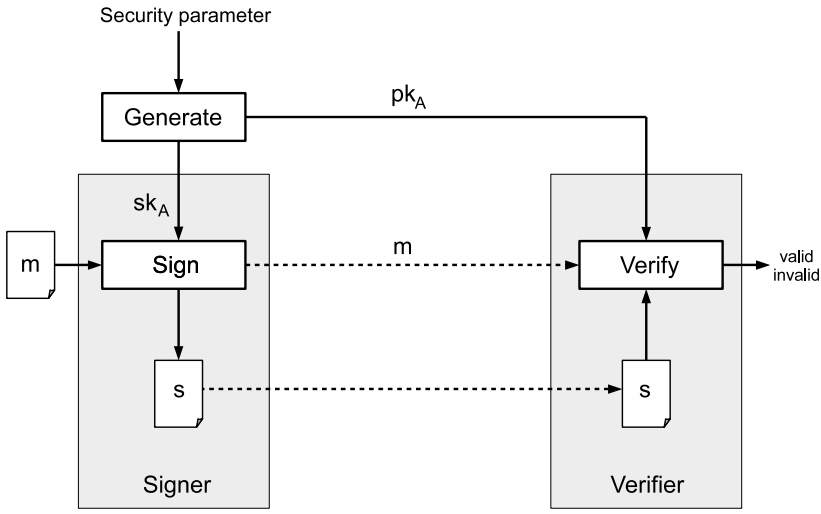


Figure 3.9 The working principle of a DSS with appendix.

- $\mathsf{Sign}(sk, m)$ is a deterministic or probabilistic signature generation algorithm that takes as input a signing key sk and a message m, and generates as output a digital signature s for m. In some literature, a signature s is also denoted as σ, and we sometimes use this notation in this book, too.
- $\mathsf{Verify}(pk, m, s)$ is a deterministic signature verif cation algorithm that takes as input a verif cation key pk, a message m, and a purported digital signature s for m, and generates as output a binary decision whether the signature is valid.

$\mathsf{Verify}(pk, m, s)$ must yield $valid$ if and only if s is a valid digital signature for m and pk. This means that for every message m and every public key pair (pk, sk), $\mathsf{Verify}(pk, m, \mathsf{Sign}(sk, m))$ must yield $valid$. Otherwise, the DSS is not particularly useful.

Similarly, a *DSS giving message recovery* consists of the following three eff ciently computable algorithms:

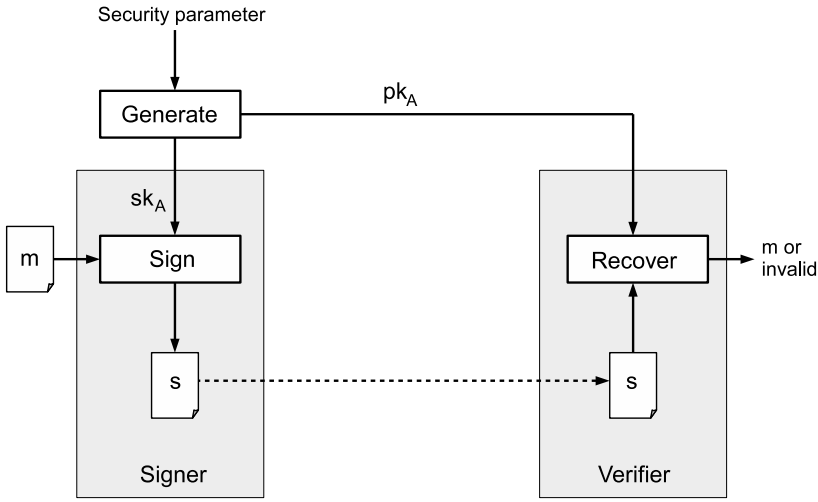


Figure 3.10 The working principle of a DSS giving message recovery.

- $\mathsf{Generate}(1^k)$ is a probabilistic key generation algorithm that takes as input a security parameter 1^k, and generates as output a public key pair (pk, sk) that is in line with the security parameter;
- $\mathsf{Sign}(sk, m)$ is a deterministic or probabilistic signature generation algorithm that takes as input a signing key sk and a message m, and generates as output a digital signature s giving message recovery;
- $\mathsf{Recover}(pk, s)$ is a deterministic message recovery algorithm that takes as input a verif cation key pk and a digital signature s, and generates as output either the message that is digitally signed or a notif cation indicating that the digital signature is invalid.

$\mathsf{Recover}(pk, s)$ must yield m if and only if s is a valid digital signature for m and pk. This means that for every message m and every public key pair (pk, sk), $\mathsf{Recover}(pk, \mathsf{Sign}(sk, m))$ must yield m.

Note that the Generate and Sign algorithms are identical in this algorithmic notation, and that the only difference refers to the Verify and Recover algorithms.

While the Verify algorithm takes the message m as input, this value is not needed by the Recover algorithm. Instead, the message m is automatically recovered, if the signature turns out to be valid.

The working principle of a DSS with appendix is illustrated in Figure 3.9. This time, the Generate algorithm is applied on the left side (i.e., the signer' side). The signer uses the secret key sk_A (representing the trapdoor) to sign message m (i.e., $s = \mathsf{Sign}(sk_A, m)$). This message m and the respective signature s are then sent to the verif er. This verif er, in turn, uses the Verify algorithm to verify s. More specif cally, it takes m, s, and pk_A as input values and ouputs either $valid$ or $invalid$ (depending on the validity of the signature).

If the DSS is giving message recovery, then the situation is slightly different. As illustrated in Figure 3.10, the beginning is the same. But instead of sending m and s to the recipient, the signatory only sends s. The signature encodes the message. So when the recipient subjects s to the Recover algorithm, the output is either m (if the signature is valid) or $invalid$ else.

With the proliferation of the Internet in general and Internet-based electronic commerce in particular, digital signatures and the legislation thereof have become important and timely topics. In fact, many DSSs with specif c and unique properties have been developed, proposed, and published in the literature. The most important basic systems are RSA, Elgamal, and some variations thereof. Most importantly, the digital signature algorithm (DSA) is an optimized version of Elgamal, and—as its name suggests—elliptic curve DSA (ECDSA) employs elliptic curve cryptography (ECC) to implement the DSA.[42] There are several elliptic curves to choose from, when one has to implement ECDSA. The off cial specif cation FIPS 186-4 enumerates 15 curves, including the widely used curves P-256, P-384, and P-521 (see Appendix D of FIPS 186-4). Some people argue that specif cally crafted curves—so-called Edwards curves—provide better properties when it comes to cryptographic applications. Hence, they prefer curves like Curve25519 or Curve448 (both specif ed in RFC 7748) to come up with an Edwards-curve DSA (EdDSA). Ed25519, for example, refers to the EdDSA that employs SHA-512 and Curve25519.[43] Finally, there are several elliptic curves known as Brainpool curves specif ed in RFC 5639.

Similar to asymmetric encryption systems, discussing the security of a DSS is nontrivial and subtle, and there are several notions of security discussed in the literature. The general theme is that is must be computationally infeasible for an

42 One of the major security concerns related to DSA and ECDSA is that an adversary may know a value that needs to be randomly chosen in the signature generation process. If the adversary can learn this value, then he or she can also compute the private signing key. In an attempt to mitigate this threat, people have specif ed a way to generate the value deterministically. It is specif ed in RFC 6979.

43 http://ed25519.cr.yp.to.

adversary to generate a new valid-looking signature, even if he or she has access to an oracle that provides him or her arbitrarily many signatures. In technical parlance, this means that the DSS must resist existential forgery, even if the adversary can mount an adaptive chosen-message attack. The respective DSSs are sophisticated and mathematically involved. We use them as black boxes, and we don't address the details in this book.

3.2.3.3 Key Agreement

If two or more entities want to employ and make use of secret key cryptography, then they must share a secret parameter (that represents a cryptographic key). Consequently, in a large system many secret keys must be generated, stored, managed, used, and destroyed (at the end of their life cycle) in a secure way. If, for example, n entities want to securely communicate with each other, then there are

$$\left(\begin{array}{c} n \\ 2 \end{array} \right) = \frac{n(n-1)}{1 \cdot 2} = \frac{n^2 - n}{2}$$

such keys. This number grows in the order of n^2, and hence the establishment of secret keys is a major practical problem—sometimes called the n^2-problem—and probably the Achilles' heel for the large-scale deployment of secret key cryptography. For example, if $n = 1,000$ entities want to securely communicate with each other, then there are

$$\left(\begin{array}{c} 1,000 \\ 2 \end{array} \right) = \frac{1,000^2 - 1,000}{2} = 499,500$$

keys. Even for moderately large n, the generation, storage, management, usage, and destruction of $(n^2 - n)/2$ keys is prohibitively expensive and the antecedent distribution of all keys is next to impossible. Things even get worse in dynamic systems, where entities may join and leave at will. In such a system, the antecedent distribution of all keys is obviously impossible, because it is not even known in advance whoever may want to join. This means that one has to establish keys when needed, and there are basically two approaches to do so:

- The use of a *key distribution center* (KDC) that provides the entities with the keys needed to securely communicate with each other;
- The use of a *key establishment protocol* that allows the entities to establish the keys themselves.

A prominent and widely deployed example of a KDC is the Kerberos authentication and key distribution system [62]. KDCs in general and Kerberos in particular

have many disadvantages. The most important disadvantage is that each entity must unconditionally trust the KDC and share a master key with it. There are situations in which this level of trust is neither justif ed nor can it be accepted by the participating entities. Consequently, the use of a key establishment protocol that employs public key cryptography yields a viable alternative in many situations and settings.

In a simple key establishment protocol, an entity randomly generates a key and uses a secure channel to transmit it to the peer entity (or peer entities, respectively). The secure channel can be implemented with an asymmetric encryption system: The entity that randomly generates the key encrypts the key with the public key of the peer entity. This protocol is simple and straightforward. It is basically what a Web browser does when it establishes a cryptographic key to be shared with a secure Web server.[44] From a security viewpoint, however, one may face the problem that the security of the secret key cryptographic system that is used with the cryptographic key is bound by the quality and the security of the key generation process (which is typically a PRG). Consequently, it is advantageous to have a mechanism in place in which two or more entities can establish and agree on a commonly shared key. This is where the notion of a key agreement or key exchange protocol comes into play (as opposed to a key distribution protocol).

The single most important key agreement protocol for two entities was originally proposed by Diff e and Hellman [38]. Their protocol—that is called *Diff e-Hellman key exchange* or *exponential key exchange*—solves a problem that sounds like impossible to solve: How can two entities that have no prior relationship and do not share a secret use a public channel to agree on a shared secret? Imagine a room in which people can shout messages to each other. How can two persons (by shouting messages to each other that can be heard by everybody in the room) agree on a shared secret? The Diff e-Hellman key exchange protocol solves this problem in a simple and ingenious way.

The Diff e-Hellman key exchange protocol can be implemented in a cyclic group G in which the discrete logarithm problem is assumed to be intractable, such as the multiplicative group of a f nite f eld $\mathbb{Z}_p$ (i.e., $\mathbb{Z}_p^*$) or some elliptic curve group. If G is an order q subgroup of such a group and g is a generator, then the Diff e-Hellman key exchange protocol can be formally expressed as illustrated in Protocol 3.1. A and B both know G and q, and they want to use the Diff e-Hellman key exchange protocol to agree on a shared secret key k. A therefore randomly selects an (ephemeral) secret exponent x_a from $\mathbb{Z}_q^*$, computes the public exponent $y_a = g^{x_a}$, and sends y_a to B. B does the same: He or she randomly selects a secret exponent

44 A *secure Web server* is a server that implements the secure sockets layer (SSL) or transport layer security (TLS) protocol.

Protocol 3.1 The Diff e-Hellman key exchange protocol.

A		B
(G, g)		(G, g)
$x_a \xleftarrow{r} \mathbb{Z}_q^*$		$x_b \xleftarrow{r} \mathbb{Z}_q^*$
$y_a \longleftarrow g^{x_a}$		$y_b \longleftarrow g^{x_b}$
	$\xrightarrow{y_a}$	
	$\xleftarrow{y_b}$	
$k_{ab} \longleftarrow y_b^{x_a}$		$k_{ba} \longleftarrow y_a^{x_b}$
(k_{ab})		(k_{ba})

x_b from $\mathbb{Z}_q^*$, computes $y_b = g^{x_b}$, and sends y_b to A. A now computes

$$k_{ab} \equiv y_b^{x_a} \equiv g^{x_b x_a}$$

and B computes

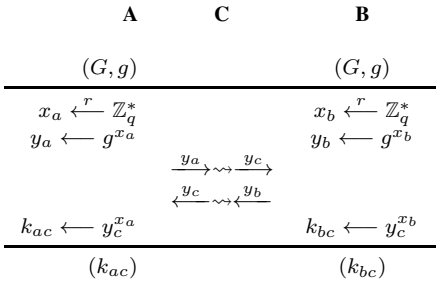
$$k_{ba} \equiv y_a^{x_b} \equiv g^{x_a x_b}$$

According to the laws of exponentiation, the order of the exponents do not matter, and hence k_{ab} is equal to k_{ba}. It is the output of the Diff e-Hellman key exchange protocol and can be used as a secret key k.

Let us consider a toy example to illustrate the working principles of the Diff e-Hellman key exchange protocol: For prime $p = 17$, $\mathbb{Z}_{17}^* = \{1, \ldots, 16\}$ is a cyclic group with $q = 16$ elements and generator $g = 3$ (i.e., 3 generates all elements of $\mathbb{Z}_{17}^*$). A randomly selects $x_a = 7$, computes $y_a = 3^7 \pmod{17} = 11$, and sends the resulting value 11 to B. B, in turn, randomly selects $x_b = 4$, computes $y_b = 3^4 \pmod{17} = 13$, and sends 13 to A. A now computes $y_b^{x_a} \equiv 13^7 \pmod{17} = 4$, and B computes $y_a^{x_b} \equiv 11^4 \pmod{17} = 4$. Consequently, $k = 4$ is the shared secret that may serve as a session key.

Note that an adversary eavesdropping on the communication channel between A and B knows p, g, y_a, and y_b, but neither knows x_a or x_b. The problem of determining $k \equiv g^{x_a x_b} \pmod{p}$ from y_a and y_b (without knowing x_a or x_b) is known as the Diff e-Hellman problem (DHP). Also note that the Diff e-Hellman key exchange protocol can be transformed into a (probabilistic) asymmetric encryption system. For a plaintext message m (that represents an element of the cyclic group in use), A randomly selects x_a, computes the common key k_{ab} (using B's public key y_b and following the Diff e-Hellman key exchange protocol), and combines m with k_{ab} to obtain the ciphertext c. The special case where $c = m \cdot k_{ab}$ refers to the Elgamal asymmetric encryption system mentioned above.

Protocol 3.2 A MITM attack against the Diff e-Hellman key exchange protocol.



If the Diff e-Hellman key exchange protocol is used natively (as outlined in Protocol 3.1), then there is a problem that is rooted in the fact that the values exchanged (i.e., y_a and y_b) are not authenticated, meaning that the values may be replaced by some other values by a properly placed adversary. Assume an adversary C who is located between A and B, and who is able to modify messages as they are sent back and forth. As already brief y mentioned in Section 2.2.3.2, such an adversary is conventionally called a MITM and the attack he or she is able to mount is called a MITM attack.[45] As sketched in Protocol 3.2, the Diff e-Hellman key exchange protocol is susceptible to such an attack: While observing the communication between A and B, C replaces y_a by y_c and y_b by y_c (it would even be possible to use two different keys y_c and $y_{c'}$ on either side of the communication channel, but this makes the attack more complex). When A receives y_c (instead of y_b), he or she computes $k_{ac} = y_c^{x_a}$. On the other side, when B receives y_c (instead of y_a), he or she computes $k_{bc} = y_c^{x_b}$. Contrary to a normal Diff e-Hellman key exchange, the two keys k_{ac} and k_{bc} are not the same, but A and B think they are. C is able to compute all keys, and to decrypt all encrypted messages accordingly. The bottom line is that A shares a key with C (i.e., k_{ac}), but thinks that he or she shares it with B, whereas—on the other side of the communication channel—B shares a key with C (i.e., k_{bc}), but thinks that he or she shares it with A. This allows C to decrypt all messages with one key and reencrypt them with the other key, making the fact that he or she is able to read the messages invisible and unrecognizable to A and B.

Again, the problem is rooted in the fact that the values exchanged (i.e., y_a and y_b) are not authenticated. This means that the most obvious way to mitigate

45 Remember that the acronym MITM stands for man-in-the-middle, but this term is somewhat diff cult, given today's gender-awareness debates. Alternatively, one can try to avoid it or use a more neutral term like malware-in-the-middle, monkey-in-the-middle, or something similar along these lines.

such MITM attacks is to authenticate these values. So in practice, people use an *authenticated Diff e-Hellman key exchange protocol* instead on an unauthenticated (native) one. In the literature, there are many proposals to authenticate the Diff e-Hellman key exchange protocol, using some complementary techniques, like passwords, secret keys, or digital signatures and (public key) certif cates. Many authenticated Diff e-Hellman key exchange protocols support multiple ways to provide the authentication part.

The susceptibility of the Diff e-Hellman key exchange protocol to MITM attacks has been known since the original publication, and many researchers have proposed other mitigation techniques (than to combine the Diff e-Hellman key exchange with some form of authentication). Examples include the *interlock protocol*, the *forced-latency protocol*,[46] Chaum's protocol, and—maybe most importantly—the *encrypted key exchange* (EKE) protocol. The original EKE protocol was later improved and gave birth to an entire family of *authenticated key exchange* (AKE) and *password authenticated key exchange* (PAKE) protocols. Many of these protocols have been widely used and some of them have even been standardized by several standardization organizations.

The Diff e-Hellman key exchange and related protocols can be used in any cyclic group (other than $\mathbb{Z}_p^*$ or an order q subgroup thereof), in which the DLP is intractable, and there are basically two reasons for doing so: Either there may be groups in which the Diff e-Hellman key exchange protocol (or the modular exponentiation function) can be implemented more eff ciently in hardware or software, or there may be groups in which the DLP is more diff cult to solve. The two reasons are not independent from each other: If, for example, one has a group in which the DLP is more diff cult to solve, then one can work with smaller key sizes and still achieve the same level of security. This is the major advantage of ECC. In fact, the ECDLP is more diff cult to solve (than the DLP in $\mathbb{Z}_p^*$ or an order q subgroup thereof), and hence one can usually work with smaller key sizes (unless one uses pairings in the realm of pairing-based cryptography).

There are elliptic curve variants of many key agreement protocols based on the ECDLP, including the elliptic curve Diff e-Hellman (ECDH) and elliptic curve Diff e-Hellman ephemeral (ECDHE) that are used in Signal and many other E2EE messaging protocols.[47] Again, there are several elliptic curves to choose from, including the ones mentioned in Section 3.2.3.2 and Curve448 (also known as Curve448-Goldilocks). ECDH(E) with Curve25519 (Curve448) is sometimes

46 The forced-latency protocol was originally proposed by Zooko Wilcox-O'Hearn in a 2003 blog entry.

47 In most literature, a Diff e-Hellman key exchange that uses non-static keys is called *ephemeral* and the respective acronym uses the additional letter E. Consequently, DHE refers to Diff e-Hellman ephemeral.

also denoted X25519 (X448). As we will see, they are frequently used in E2EE messaging protocols.

3.3 CERTIFICATE MANAGEMENT

Like many Internet security technologies and protocols in use today, E2EE messaging employs public key cryptography and public key certif cates. The management of these certif cates is an involved topic that is brief y addressed here. We introduce the topic in Section 3.3.1, elaborate on X.509 certif cates and OpenPGP certif cates in Sections 3.3.2 and 3.3.3, and elaborate on the state of the art in Section 3.3.4. This chapter is intentionally kept short, and readers who may want to get more information about the topic are referred to the many books that are available [63–66].[48]

3.3.1 Introduction

According to [35], the term *certif cate* refers to "a document that attests to the truth of something or the ownership of something." This def nition is fairly broad and applies to many subject areas, not necessarily related to cryptography or even public key cryptography. In this particular area, the term certif cate was coined and f rst used by Loren M. Kohnfelder in his Bachelor thesis [67] to refer to a digitally signed record holding a name and a public key. As such, it was positioned as a replacement for a public f le[49] that had been used before. A respective certif cate is to attest to the legitimate ownership of a public key and to attribute the key to a principal, such as a person, a hardware device, or any other entity. Quite naturally, such a certif cate is called a *public key certif cate*. Such public key certif cates are used by many cryptographic security technologies and protocols in use today in one way or another. Again referring to [35], a public key certif cate is a special case of a certif cate, namely one "that binds a system entity's identity to a public key value, and possibly to additional data items." As such, it is a digitally signed data structure that attests to the true ownership of a particular public key.

More generally (but still in accordance with [35]), a certif cate can not only be used to attest to the legitimate ownership of a public key (as in the case of a public key certif cate), but also to attest to the truth of some arbitrary property that could be attributed to the certif cate owner. This more general class of certif cates

48 Note that PKIs were hyped in the late 1990s and early 2000s; hence, most books were written in this period of time (with [66] being an exception here).

49 A public f le was just a f at f le that included the public keys and names of the key owners in any particular order (e.g., sorted alphabetically with regard to the names of the key owners). The entire f le could be digitally signed if needed.

is commonly referred to as *attribute certif cates*. The major difference between a public key and an attribute certif cate is that the former includes a public key (i.e., the public key that is certif ed) whereas the latter includes a list of attributes (i.e., the attributes that are certif ed). In either case, the certif cates are issued (and possibly revoked) by authorities that are recognized and trusted by a community of users.

- In the case of public key certif cates, the authorities in charge are called *certif cation authorities* (CAs[50]) or—more related to digital signature legislation—*certif cation service providers* (CSPs);
- In the case of attribute certif cates, the authorities in charge are called *attribute authorities* (AAs).

It goes without saying that a CA and an AA may be the same organization. As soon as attribute certif cates start to take off, it is possible and very likely that CAs will also try to establish themselves as AAs. It also goes without saying that a CA can have one or several *registration authorities* (RAs)—sometimes also called *local registration authorities* or *local registration agents* (LRAs). The functions an RA carries out vary from case to case, but they typically include the registration and authentication of the entities (typically human users) that want to become certif cate owners. In addition, the RA may also be involved in tasks like token distribution, certif cate revocation reporting, key generation, and key archival. In fact, a CA can delegate some of its tasks (apart from certif cate signing) to an RA. Consequently, RAs are optional components that are transparent to the users. Also, the certif cates that are generated by the CAs may be made available in online directories and certif cate repositories.

While the notion of a CA is well def ned and suff ciently precise, the notion of a *public key infrastructure* (PKI) is more vague. According to [35], a PKI is "a system of CAs that perform some set of certif cate management, archive management, key management, and token management functions for a community of users," that employ public key cryptography (as one may tempted to add here). Another way to look at a PKI is as an infrastructure that can be used to issue, validate, and revoke public keys and public key certif cates. Hence, a PKI comprises a set of agreed-upon standards, CAs, structures among multiple CAs, methods to discover and validate certif cation paths, operational and management protocols, interoperable tools, and supporting legislation.

In the past, PKIs have experienced a great deal of hype, and many companies and organizations have started to provide certif cation services on a commercial basis. Unfortunately (and for the reasons discussed in [68]), most of these service

50 In the past, CAs were often called trusted third parties (TTPs). This is particularly true for CAs that are operated by government bodies.

providers have failed to become commercially successful. In fact, the PKI business has turned out to be particularly diff cult to make a living from, and there are only a few CAs that are self-feeding. Most CAs that are still in business also have other sources of revenue.

Many standardization bodies are working in the f eld of public key certif cates and the management thereof. Most importantly, the Telecommunication Standardization Sector of the International Telecommunication Union (ITU-T) has released and is periodically updating a recommendation that is commonly referred to as ITU-T X.509 [69], or X.509 in short. The respective certif cates are addressed in Section 3.3.2. Meanwhile, ITU-T X.509 has also been adopted by many other standardization bodies, including the International Organization for Standardization (ISO) and the International Electrotechnical Committee (IEC) Joint Technical Committee 1 (JTC1) [70]. Furthermore, a few other standardization bodies also work in the f eld of prof ling ITU-T X.509 for specif c application environments.[51] In 1995, for example, the IETF recognized the importance of public key certif cates for Internet security, and chartered an IETF Public-Key Infrastructure X.509 (PKIX[52]) WG to develop Internet standards for an X.509-based PKI. The PKIX WG initiated and stimulated a lot of standardization and prof ling activities within the IETF, and was closely aligned with the activities of the ITU-T. In spite of the practical importance of the specif cations of the IETF PKIX WG, we do not delve deeper into the details in this book (as this is a topic for a book on its own). Feel free to browse through the IETF PKIX WG's Web site and the respective RFC documents and Internet-Drafts; they provide a rich f ora and fauna on the topic. The IETF PKIX WG was concluded in 2013, almost 20 years after it was chartered.[53]

As mentioned before and illustrated in Figure 3.11, a public key certif cate comprises at least the following three main pieces of information:

- A public key;
- Some naming information;
- One or more digital signatures.

The *public key* is the raison d'être for the public key certif cate, meaning that the certif cate only exists to certify the public key in the f rst place. The public

51 To prof le ITU-T X.509—or any general standard or recommendation—basically means to f x the details with regard to a specif c application environment. The result is a prof le that elaborates on how to use and deploy ITU-T X.509 in the environment.

52 http://www.ietf.org/html.charters/pkix-charter.html.

53 To be precise, the IETF PKIX WG was chartered on October 26, 1995, and it was concluded on October 31, 2013. It was therefore active for slightly more than 18 years.

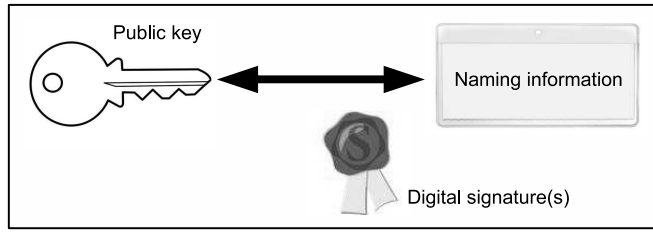


Figure 3.11 A public key certif cate comprises three main pieces of information.

key, in turn, can be from any public key cryptosystem, like RSA, Elgamal, Diff e-Hellman, DSA, or anything else. The format (and hence also the size) of the public key depends on the system in use.

The *naming information* is used to identify the owner of the public key and public key certif cate. If the owner is a user, then the naming information typically consists of at least the user's f rst name and surname—also known as the family name. In the past, there has been some discussions about the namespace that can be used here. For example, the ITU-T recommendation X.500 introduced the notion of a *distinguished name* (DN) that can be used to identify entities, such as public key certif cate owners, in a globally unique namespace. However, since then, X.500 DNs have not really taken off, at least not in the realm of naming persons. In this realm, the availability and appropriateness of globally unique namespaces have been challenged in the research community (e.g., [71]). In fact, the Simple Distributed Security Infrastructure (SDSI) initiative and architecture [72] has started from the argument that a globally unique namespace is not appropriate for the global Internet, and that logically linked local namespaces are simpler and therefore more likely to be deployed (this point is further explored in [73]). As such, work on SDSI inspired the establishment of a Simple Public Key Infrastructure (SPKI) WG within the IETF Security Area. The WG was chartered in 1997 to produce a certif cate infrastructure and operating procedure to meet the needs of the Internet community for trust management in a way as easy, simple, and extensible as possible. This was partly in contrast (and in competition) to the IETF PKIX WG. The IETF SPKI WG published a pair of experimental RFCs [74, 75], before its activities were abandoned in 2001.[54] Consequently, the SDSI and SPKI initiatives have turned out to be dead

54 The WG was formally concluded in February 2001, only four years after it was chartered.

ends for the Internet as a whole. They barely play a role in today's discussions about the management of public key certif cates. But the underlying argument that globally unique namespaces are not easily available remains valid.

Last but not least, the *digital signature(s)* is (are) used to attest to the fact that the other two pieces of information (i.e., the public key and the naming information) belong together. In Figure 3.11, this is illustrated by the two arrowheads that bind the two pieces together. The digital signature(s) turn(s) the public key certif cate into a data structure that is useful in practice, mainly because it can be verif ed by anybody who knows the signatory's (i.e., CA's) public key. These keys are normally distributed with particular software, be it at the operating system or application software level.

As of this writing, there are two types of public key certif cates that are practically relevant and in use: *X.509* and *OpenPGP certif cates*. While their aims and scope are somewhat similar, they use different certif cate formats and trust models. A *trust model*, in turn, refers to the set of rules that a system or application uses to decide whether a certif cate is valid. In the direct trust model, for example, a user trusts a public key certif cate only because he or she knows where it came from and considers this entity to be trustworthy. In addition to the direct trust model, there is a hierarchical trust model, as employed, for example, by ITU-T X.509, and a cumulative trust model, as employed, for example, by OpenPGP. These trust models can also be called *centralized* and *distributed*. It then becomes clear that there is hardly anything in between. Hence, coming up with alternatives to the direct, hierarchical, and cumulative trust models is somewhat challenging.

3.3.2 X.509 Certif cates

As mentioned before (and as their name suggests), X.509 certif cates conform to the ITU-T recommendation X.509 [69] f rst published in 1988 as part of the X.500 directory series of recommendations. It specif es both a certif cate format and a certif cate distribution scheme (while the specif cation language used was ASN.1). The original X.509 certif cate format has gone through two major revisions:

- In 1993, the X.509 version 1 (X.509 v1) format was extended to incorporate two new f elds, resulting in the X.509 version 2 (X.509 v2) format.
- In 1996, the X.509 v2 format was revised to allow for additional extension f elds. This was in response to the attempt to deploy certif cates on the global Internet. The resulting X.509 version 3 (X.509 v3) specif cation has since then been reaff rmed every couple of years.

When people today refer to X.509 certif cates, they essentially refer to X.509 v3 certif cates (and the version denominator is often left aside in the acronym). Let us now have a closer look at the X.509 certif cate format and the hierarchical trust model it is based on.

3.3.2.1 Certif cate Format

With regard to the use of X.509 certif cates, the prof ling activities within the IETF PKIX WG are particularly important. Among the many RFC documents produced by this WG, RFC 5280 [76] is the most relevant one (with some RFC documents that yield some updates on particular topics (e.g., RFC 6818, RFC 8398, and RFC 8399). Without delving into the details of the respective ASN.1 specif cation for X.509 certif cates, we note that an X.509 certif cate is a data structure that basically consists of the following f elds (remember that any additional extension f elds are possible):[55]

- *Version:* This f eld is used to specify the X.509 version in use (i.e., version 1, 2, or 3).
- *Serial number:* This f eld is used to specify a serial number for the certif cate. The serial number is a unique integer value assigned by the (certif cate) issuer. The pair consisting of the issuer and the serial number must be unique—otherwise, it would not be possible to uniquely identify an X.509 certif cate.
- *Algorithm ID:* This f eld is used to specify the object identif er (OID) of the algorithm that is used to digitally sign the certif cate. For example, the OID 1.2.840.113549.1.1.5 refers to `sha1RSA`, which stands for the combined use of SHA-1 with RSA encryption. We list many other OIDs in the chapter of S/MIME.
- *Issuer:* This f eld is used to name the issuer. As such, it comprises the DN of the CA that issues (and digitally signs) the certif cate.
- *Validity:* This f eld is used to specify a validity period for the certif cate. The period, in turn, is def ned by two dates, namely a start date (i.e., Not Before) and an expiration date (i.e., Not After).

55 From an educational viewpoint, it is best to compare the f eld descriptions with the contents of real certif cates. If you run a Windows operating system, then you may look at some certif cates by running the certif cate snap-in for the management console (just enter certmgr on a command line interpreter). The window that pops up summarizes all certif cates that are available at the operating system level.

- *Subject:* This field is used to name the subject (i.e., the owner of the certificate, typically using a DN).
- *Subject Public Key Info:* This field is used to specify the public key (together with the algorithm) that is certified.
- *Issuer Unique Identifier:* This field can be used to specify some optional information related to the issuer of the certificate (only in X.509 versions 2 and 3).
- *Subject Unique Identifier:* This field can be used to specify some optional information related to the subject (only in X.509 versions 2 and 3). This field typically comprises some alternative naming information, such as an e-mail address or a DNS entry.
- *Extensions:* This field can be used to specify some optional extensions that may be critical or not (only in X.509 version 3). While critical extensions need to be considered by all applications that employ the certificate, noncritical extensions are truly optional and can be considered at will. With regard to secure messaging on the Internet, the most important extensions are "Key Usage" and "Basic Constraints."

  – The *key usage extension* uses a bit mask to define the purpose of the certificate (i.e., whether it is used for normal digital signatures (0), legally binding signatures providing nonrepudiation (1), key encryption (2), data encryption (3), key agreement (4), digital signatures for certificates (5) or certificate revocation lists (CRLs) addressed below (6), encryption only (7) or decryption only (8)). The numbers in parentheses refer to the respective bit positions in the mask.

  – The *basic constraints extension* identifies whether the subject of the certificate is a CA and the maximum depth of valid certification paths that include this certificate. This extension should not appear in a leaf (or end entity) certificate.

  Furthermore, there is an *Extended Key Usage* extension that can be used to indicate one or more purposes for which the certified public key may be used, in addition to or in place of the basic purposes indicated in the key usage extension field.

The last three fields make X.509v3 certificates very flexible, but also very difficult to deploy in an interoperable manner. Anyway, the certificate must come along with a digital signature that conforms to the digital signature algorithm specified in the Algorithm ID field.

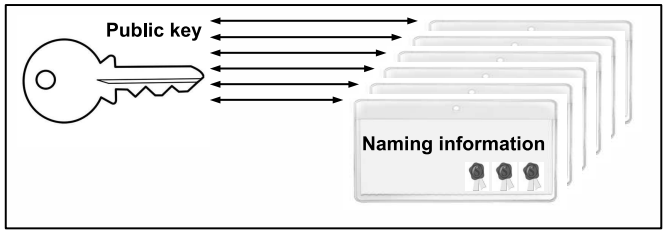


Figure 3.12 The general format of an OpenPGP public key certif cate.

A distinguishing feature of an X.509 certif cate is that there is one single piece of naming information, namely the content of the subject f eld, that is bound to a public key, and that there is one single signature that vouches for this binding. This is different in the case of an OpenPGP certif cate. In such a certif cate, there can be multiple pieces of naming information bound to a particular public key, and there can even be multiple signatures that vouch for this binding. The resulting and more general format of an OpenPGP public key certif cate is illustrated in Figure 3.12. We revisit this format when we address OpenPGP certif cates. Here, we only want to point out the structural differences in the certif cate formats.

3.3.2.2 Hierarchical Trust Model

X.509 certif cates are based on the hierarchical trust model that is built on a hierarchy of (commonly) trusted CAs. As illustrated in Figure 3.13, such a hierarchy consists of a set of *root CAs* that form up the top level and that must be trusted by default. The respective certif cates are self-signed, meaning that the issuer and subject f elds refer to the same entity (typically an organization). Note that, from a theoretical point of view, a self-signed certif cate is not particularly useful. Anybody can claim something and issue a certif cate for this claim. Consequently, a self-signed certif cate basically says: "Here is my public key, trust me." There is no argument that speaks in favor of this claim. However, to bootstrap hierarchical trust, one or several root CAs with self-signed certif cates are unavoidable (because the hierarchy is f nite and must have a top level).

In Figure 3.13, the set of root CAs consists of only three CAs (the three shadowed CAs at the top of the f gure). In reality, we are talking about several dozens of root CAs that come preconf gured in a client software—be it an operating system

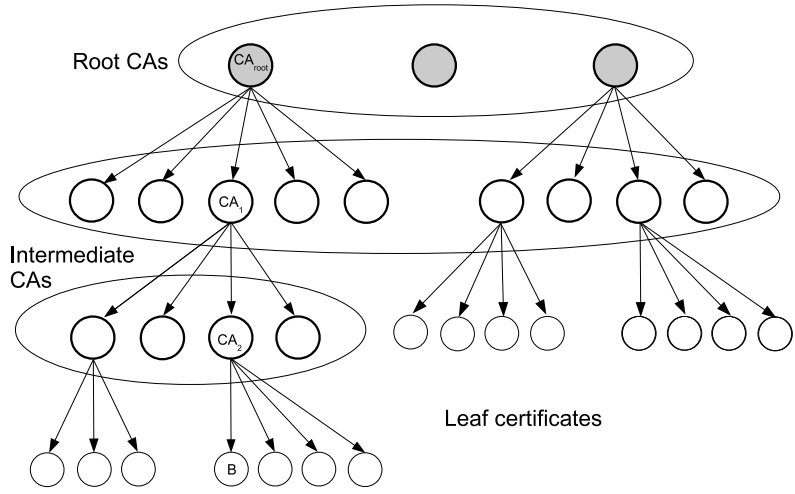


Figure 3.13 A hierarchy of trusted root and intermediate CAs that issue leaf certif cates.

or application software. Each root CA may issue certif cates for other CAs that are called *intermediate CAs*. The intermediate CAs may form up multiple layers in the hierarchy. At the bottom of the hierarchy, the intermediate CAs may issue certif cates for end users or other entities, such as Web servers. These certif cates are called *leaf certif cates* and they cannot be used to issue other certif cates. This, by the way, is controlled by the basic constraints extension mentioned earlier. In a typical setting, a commercial CSP operates a CA that represents a trusted root CA, and several subordinate CAs that may represent intermediate CAs. Note, however, that it is up to the client software to make a distinction between these types of CAs—either type is considered to be trustworthy.

Equipped with one or several root CAs and respective root certif cates, a user may try to f nd a *certif cation path*—or *certif cation chain*—from one of the root certif cates to a leaf certif cate. Formally speaking, a certif cation path or chain is def ned in a tree or wood of CAs (root CAs and intermediate CAs), and refers to a sequence of one or more certif cates that leads from a trusted root certif cate to a leaf certif cate. Each certif cate certif es the public key of its successor. Finally, the leaf certif cate is typically issued for a person or end system. Let us assume that CA_{root}

is a root certif cate and B is an entity for which a certif cate must be verif ed. In this case, a certif cation path or chain with n intermediate CAs (i.e., $CA_1, CA_2, \ldots, CA_n$) may look as follows:

$$
\begin{aligned}
&CA_{root} \ll CA_1 \gg \\
&CA_1 \ll CA_2 \gg \\
&CA_2 \ll CA_3 \gg \\
&\ldots \\
&CA_{n-1} \ll CA_n \gg \\
&CA_n \ll B \gg
\end{aligned}
$$

In Figure 3.13, a certif cation path with 2 intermediate CAs is illustrated. The path consists of $CA_{root} \ll CA_1 \gg$, $CA_1 \ll CA_2 \gg$, and $CA_2 \ll B \gg$. If a client supports intermediate CAs, then it may be suff cient to f nd a sequence of certif cates that lead from a trusted intermediate CA's certif cate to the leaf certif cate. This may shorten certif cation chains considerably. In our example, it may be the case that CA_2 represents a (trusted) intermediate CA. In this case, the leaf certif cate $CA_2 \ll B \gg$ would be suff cient to verify the legitimacy of B's public key.

The simplest model one may think of is a certif cation hierarchy representing a tree with a single root CA. In practice, however, more general structures are possible, using multiple root CAs, intermediate CAs, and CAs that issue cross certif cates. In such a general structure, a certif cation path may not be unique and multiple certif cation paths may exist. In such a situation, it is required to have authentication metrics in place that allow one to handle multiple certif cation paths. The design and analysis of such metrics is an interesting and challenging research topic not further addressed in this book (you may refer to [77] for a respective introduction and overview).

As mentioned above, each X.509 certif cate has a validity period, meaning that it is well-def ned when the certif cate is supposedly valid. However, in spite of this information, it may still be possible that a certif cate needs to be revoked ahead of time. For example, it may be the case that a user's private key gets compromised or a CA goes out of business. For situations like these, it is necessary to address certif cate revocation in one way or another. The simplest way is to have the CA periodically issue a *certif cate revocation list* (CRL). A CRL is basically a blacklist that enumerates all certif cates (by their serial numbers) that have been revoked so far or since the issuance of the last CRL in the case of a delta CRL. In either case, CRLs can be tremendously large and impractical to handle. Due to the CRLs' practical disadvantages, the trend goes to retrieving online status information about the validity of a certif cate. The protocol of choice to retrieve

this information is the Online Certif cate Status Protocol (OCSP) [78] that has problems of its own. There are a few alternative or complementary technologies, such as Google's *Certif cate Transparency*[56] or technologies that employ DNS, such as DNS Certif cation Authority Authorization (CAA) or DNS-based Authentication of Named Entities (DANE). The bottom line is that certif cate revocation remains a challenging issue (e.g., [79]), and that many application clients that employ public key certif cates either do not care about it or handle it incompletely or even improperly. This is especially true for many MUAs used on the Internet. This is why many E2EE messaging solutions try to avoid the use of certif cates in the f rst place.

In spite of the fact that we characterize the trust model employed by ITU-T X.509 as being hierarchical, it is not so in a strict sense. The possibility to def ne cross-certif cates, as well as forward and reverse certif cates, enables the construction of a mesh (rather than a hierarchy). This means that something similar to PGP's web of trust can also be established using X.509. The misunderstanding partly occurs because the X.509 trust model is mapped to the directory information tree (DIT), which is hierarchical in nature (each DN represents a leaf in the DIT). Hence, the hierarchical structure is a result of the naming scheme rather than the certif cate format. This should be kept in mind when arguing about trust models.

3.3.3 OpenPGP Certif cates

We already mentioned that an OpenPGP certif cate is similar to an X.509 certif cate, but that it uses a different format. The most important difference is that an OpenPGP certif cate may have multiple pieces of naming information (user IDs) and multiple signatures that vouch for them. This point is illustrated in Figure 3.12. Hence, an OpenPGP certif cate is inherently more general and f exible than an X.509 certif cate. Also, OpenPGP employs e-mail addresses (instead of DNs) as primary naming information.

Let us f rst look at the OpenPGP certif cate format before we more thoroughly address the cumulative trust model that is used in the realm of OpenPGP and OpenPGP certif cates.

3.3.3.1 Certif cate Format

Like an X.509 certif cate, an OpenPGP certif cate is a data structure that binds some naming information to a public key.

- The naming information consists of one or several user IDs, where each user ID includes a user name and an e-mail address put in angle brackets (< and >).

56 https://www.certif cate-transparency.org.

The e-mail address basically makes the user ID unique. An exemplary user ID is `Rolf Oppliger <rolf.oppliger@esecurity.ch>`.

- The public key is the key that is certified by the certificate. It is a binary string that is complemented by a fingerprint, a key identifier (key ID), an algorithm name (i.e., RSA, Diffie-Hellman, or DSA), and a respective key length. The notion of a fingerprint and key ID in the realm of OpenPGP is introduced in Section 5.2.2. The fingerprint basically represents an SHA-1 hash value of the public key (and some auxiliary data), whereas the key ID refers to the least significant 64 (or 32) bits of the fingerprint.

In addition to the naming information and public key, an OpenPGP certificate may also comprise many other fields (depending on the implementation). The following fields are commonly used.

- *Version number:* This field is used to identify the version of OpenPGP. The current version is 4. Version 3 is deprecated.
- *Creation and expiration dates:* These fields determine the validity period (or lifetime) of the public key and certificate. In fact, it is valid from the creation date to the expiration date. In many cases, the expiration date is not specified, meaning that the respective certificate does not expire by default. Again, this is a difference between X.509 and OpenPGP certificates. While X.509 certificates typically expire after a few years, OpenPGP certificates typically don't expire at all (unless an expiration date is specified).
- *Self-signature:* This field is used to hold a self-signature for the certificate. As its name suggests, a self-signature is generated by the certificate owner using the private key that corresponds to the public key associated with the certificate. Note that X.509 certificates normally do not include self-signatures—except for root CA certificates.
- *Preferred encryption algorithm:* This field is used to identify the encryption algorithm of choice for the certificate owner.

One may think of an OpenPGP certificate as a public key with one or more labels attached to it. For example, several user IDs may be attached to it. Also, one or several photographs may be attached to an OpenPGP certificate to simplify visual authentication. Note that this is a feature that is not known to exist in the realm of X.509 certificates. Also note that the use of photographs in certificates is controversially discussed within the security community. While some people argue that it simplifies user authentication, others argue that it is dangerous because certificates that come along with a photograph only look trustworthy (whereas in

fact they may not be trustworthy at all, or at least not more trustworthy than any certif cate without a photograph). Hence, there are implementations that support the attachment of photographs, and there are implementations that don't. In either case, it is possible to bring in arguments that speak in favor of the respective choice. Therefore, it is a matter of taste whether one wants to use photographs or not.

3.3.3.2 Cumulative Trust Model

The hierarchical trust model of X.509 starts from central CAs that are assumed to be commonly trusted. Contrary to that, the cumulative trust model negates the existence of such CAs, and starts from the assumption that there is no central CA that is trusted by everybody. Instead, every user must decide for himself or herself who he or she is going to trust. If a user trusts another user, then this other user may act as an *introducer* to him or her, meaning that any PGP certif cate signed by him or her will be accepted by the user. It goes without saying that different users may have different introducers they trust and start from.

In practice, things are more involved, mainly because there is no unique notion of trust and trust can come in different f avors (or degrees, respectively). PGP, for example, originally distinguished between *marginal* and *full* trust, and this distinction has been adapted by most OpenPGP implementations. The resulting trust model is cumulative in the sense that more than one introducer can vouch for the validity and trustworthiness of a particular certif cate. The respective signatures are accumulated in the certif cate, and the more people that sign a certif cate, the more likely it is going to be trusted (and hence accepted) by a third party. The resulting certif cation and trust infrastructure is distributed and called a *web of trust*. We more thoroughly elaborate the web of trust employed by OpenPGP in Section 5.3. This includes, among other things, the diff culties one faces when revoking keys in the web of trust. Note that there are many possible ways to implement a cumulative trust model, and that the way such a model is implemented by PGP and most versions of OpenPGP is just one possibility. Also note that the cumulative trust model and the web of trust are seldom used in the f eld and have turned out be dead ends.

3.3.4 State of the Art

Since public key certif cates represent the Achilles' heel of public key cryptography, the management of these certif cates represents an important and practically relevant topic. This also applies to E2EE messaging on the Internet. A user who wants to send a conf dential and cryptographically protected message to a recipient must have access to this recipient's public key. A valid certif cate is one way to achieve this. Similarly, the recipient must have access to a valid certif cate for the sender's public

key if he or she wants to verify the signature of that message. If certif cates can be faked, then any form of active attack becomes feasible and diff cult to mitigate.

While the PKI industry has been partly successful in deploying server-side certif cates, the client-side deployment of certif cates has remained poor. This is equally true for hardware and software certif cates.

- Hardware certif cates refer to hardware devices or tokens that comprise public key pairs. Examples include smartcards or USB tokens. The relevant standards are PKCS #11 and PKCS #15. The question of whether the public key pairs should be generated inside or outside the hardware device or token is controversially discussed within the community.
  - In the f rst case, it can be ensured that no private key can leak the device or token, but the quality of the random number generator may be poor;
  - In the second case, the quality of the random number generator can be controlled, but it may be possible to export the keying material from the device or token (because the respective import function must be supported by default).
- Software certif cates do not require hardware. Instead, the public key pairs are entirely stored in memory—hopefully in some encrypted form (while not being used).

It goes without saying that software certif cates are generally more vulnerable and simpler to attack than hardware certif cates. Using hardware certif cates, one can reasonably argue that extracting private keying material is technically diff cult. This is not true for software certif cates. Here, the respecting commands (to extract private keys) can be disabled by default, but it is very diff cult to technically avoid an adversary who may f nd a way to extract a private key anyway. The bottom line is that for high-secure environments, hardware certif cates are advantageous and should be the preferred choice (this applies to X.509 and OpenPGP certif cates). However, the deployment of hardware certif cates is more involved and expensive, and we hardly see any hardware certif cates for E2EE messaging deployed and used in the f eld.

Another problem that appears in the f eld (at least in the realm of e-mail) is that there are not many publicly available directories that can be used to retrieve user certif cates. The main reason for this lack of directories is that organizations hesitate to make their information publicly available, mainly because they are afraid of people misusing it for spam and targeted headhunting. Hence, they keep this information internal, and this severely restricts its usefulness. Inside an organization, the situation is simpler, because there are usually possibilities to roll out user certif cates at moderate costs. However, these certif cates only allow it to secure

the transfer of internal messages. This is certainly something to consider, but the real threats refer to mail that is transferred across the Internet (i.e., sent from one organization to another). If all mail traff c were internal, then the secure messaging problem would be a minor concern. As of this writing, people use key servers instead of directories and directory services for certif cates (Section 5.3.4), or they use native public keys from trustworthy sources. This trend continues with the increased use and prevalence of E2EE messengers and respective service providers.

Sometimes, people argue that *identity-based encryption* (IBE) yields an appropriate technology to solve the certif cate management problem (e.g., [80]). In IBE, the name—or e-mail address—of an entity basically represents his or her public key, and hence there is no need to come up with public key certif cates and PKIs. The use of IBE in practice, however, has other disadvantages that do not make it clear what approach best serves the needs of the Internet community. For example, in IBE, users cannot generate their own public key pairs. Instead, these key pairs must be generated by some trustworthy authority, and it is not clear what organization could represent this authority. Also, IBE does not provide a solution for digital signatures and key revocation is particularly challenging, because there is no obvious way to refresh an identity. The bottom line is that IBE is controversially discussed, and that the future of IBE remains unclear.

3.4 FINAL REMARKS

In this chapter, we have provided a brief summary of the major cryptographic techniques used in the f eld. Most of these techniques can also be used for secure and E2EE messaging. They refer to three main classes of cryptosystems (i.e., unkeyed cryptosystems, secret key cryptosystems, and public key cryptosystems). People sometimes argue that cryptosystems from one class are superior to the systems of another class, like "let us use a public key cryptosystem, because we want to have better security." This line of argumentation is arguably wrong, and the different classes have different properties but not necessarily different security properties. For example, public key cryptography tends to be computationally expensive (and hence not so eff cient) but well suited for authentication and key management, whereas secret key cryptography tends to be eff cient and well suited for bulk data encryption. It therefore often makes a lot of sense to combine secret and public key cryptography in so-called *hybrid cryptosystems*. In such a cryptosystem, public key cryptography is typically used for authentication and key management, whereas secret key cryptography is used for everything else—most notably bulk data encryption. Hybrid cryptosystems are used everywhere, and almost every non-trivial application of cryptography employs some form of hybrid cryptography. This also

applies to secure and E2EE messaging on the Internet (as we will see throughout the rest of this book).

References

[1] Blahut, R.E., *Cryptography and Secure Communication*, Cambridge University Press, Cambridge, UK, 2014.

[2] Buchmann, J.A., *Introduction to Cryptography*, 2nd edition, Springer-Verlag, New York, 2004.

[3] Delfs, H., and H. Knebl, *Introduction to Cryptography: Principles and Applications*, 3rd edition. Springer-Verlag, New York, 2015.

[4] Dent, A.W., and C.J. Mitchell, *User's Guide to Cryptography and Standards*, Artech House, Norwood, MA, 2004.

[5] Easttom, C., *Modern Cryptography: Applied Mathematics for Encryption and Information Security*, McGraw-Hill Education, 2015.

[6] Ferguson, N., and B. Schneier, *Practical Cryptography*, John Wiley & Sons, New York, 2003.

[7] Ferguson, N., B. Schneier, and T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*, John Wiley & Sons, New York, 2010.

[8] Garrett, P.B., *Making, Breaking Codes: Introduction to Cryptology*, Prentice Hall PTR, Upper Saddle River, NJ, 2001.

[9] Goldreich, O., *Foundations of Cryptography: Volume 1, Basic Tools*, Cambridge University Press, Cambridge, UK, 2007.

[10] Goldreich, O., *Foundations of Cryptography: Volume 2, Basic Applications*, Cambridge University Press, Cambridge, UK, 2009.

[11] Hoffstein, J., J. Pipher, and J.H. Silverman, *An Introduction to Mathematical Cryptography*, Springer-Verlag, New York, 2008

[12] Kahn, D., *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*, Scribner, 1996.

[13] Katz, J., and Y. Lindell, *Introduction to Modern Cryptography*, 2nd edition, Chapman & Hall/CRC, Boca Raton, FL, 2014.

[14] Klein, P.N., *A Cryptography Primer: Secrets and Promises*, Wiley-Interscience, 2007.

[15] Koblitz, N.I., *A Course in Number Theory and Cryptography*, 2nd edition, Springer-Verlag, New York, 1994.

[16] Konheim, A.G., *Computer Security and Cryptography*, 2nd edition, Springer-Verlag, New York, 1994.

[17] Luby, M., *Pseudorandomness and Cryptographic Applications*, Princeton Computer Science Notes, Princeton, NJ, 1996.

[18] Mao, W., *Modern Cryptography: Theory and Practice*, Prentice Hall PTR, Upper Saddle River, NJ, 2003.

[19] Martin, K.M., *Everyday Cryptography: Fundamental Principles & Applications*, Oxford University Press, New York, 2012.

[20] Menezes, A., P. van Oorschot, and S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, Boca Raton, FL, 1996.

[21] Mollin, R.A., *RSA and Public-Key Cryptography*, Chapman & Hall/CRC, Boca Raton, FL, 2002.

[22] Mollin, R.A., *Codes: The Guide to Secrecy From Ancient to Modern Times*, Chapman & Hall/CRC, Boca Raton, FL, 2005.

[23] Mollin, R.A., *An Introduction to Cryptography*, 2nd edition, Chapman & Hall/CRC, Boca Raton, FL, 2006.

[24] Oppliger, R., *Contemporary Cryptography*, 2nd edition, Artech House, Norwood, MA, 2011.

[25] Paar, C., and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer-Verlag, New York, 2009

[26] Schneier, B., *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, 20th Anniversary Edition. John Wiley & Sons, New York, 2015.

[27] Smart, N., *Cryptography Made Simple*, Springer-Verlag, New York, 2015.

[28] Stanoyevitch, A., *Introduction to Cryptography with Mathematical Foundations and Computer Implementations*, Chapman & Hall/CRC, Boca Raton, FL, 2010.

[29] Stinson, D., and M. Paterson, *Cryptography: Theory and Practice*, 4th edition, Chapman & Hall/CRC, Boca Raton, FL, 2018.

[30] Talbot, J., and D. Welsh, *Complexity and Cryptography: An Introduction*, Cambridge University Press, Cambridge, UK, 2006.

[31] Vaudenay, S., *A Classical Introduction to Cryptography: Applications for Communications Security*, Springer-Verlag, New York, 2005.

[32] Von zur Gathen, J., *CryptoSchool*, Springer-Verlag, New York, 2015.

[33] Wang, et al., *Mathematical Foundations of Public Key Cryptography*, CRC Press, Boca Raton, FL, 2015.

[34] Yan, S.Y., *Computational Number Theory and Modern Cryptography*, John Wiley & Sons, New York, 2013.

[35] Shirey, R., *Internet Security Glossary, Version 2*, Informational RFC 4949 (FYI 36), August 2007.

[36] Kelsey, J., B. Schneier, and D. Wagner, "Protocol Interactions and the Chosen Protocol Attack," *Proceedings of the 5th International Workshop on Security Protocols*, Springer-Verlag, 1997, pp. 91–104.

[37] Oppliger, R., "Disillusioning Alice and Bob," *IEEE Security & Privacy*, Vol. 15, No. 5, September/October 2017, pp. 82–84.

[38] Diff e, W., and M.E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, IT-22(6), 1976, pp. 644–654.

[39] Rabin, M.O., "Digitalized Signatures and Public-Key Functions as Intractable as Factorization," MIT Laboratory for Computer Science, MIT/LCS/TR-212, 1979.

[40] Bellare, M., and P. Rogaway, "Random Oracles are Practical: A Paradigm for Designing Eff cient Protocols," *Proceedings of the 1st ACM Conference on Computer and Communications Security*, 1993, pp. 62-73.

[41] Anderson, R., "Why Cryptosystems Fail," *Communications of the ACM*, Vol. 37, No. 11, November 1994, pp. 32–40.

[42] Halderman, J.A., et al., "Lest We Remember: Cold Boot Attacks on Encryption Keys," *Communications of the ACM*, Vol. 52, No. 5, May 2009, pp. 91–98.

[43] Anderson, R., and M. Kuhn, "Tamper Resistance—A Cautionary Note," *Proceedings of the 2nd USENIX Workshop on Electronic Commerce*, November 1996, pp. 1–11.

[44] Anderson, R., and M. Kuhn, "Low Cost Attacks on Tamper Resistant Devices," *Proceedings of the 5th International Workshop on Security Protocols*, Springer-Verlag, LNCS 1361, 1997, pp. 125–136.

[45] Kocher, P., "Timing Attacks on Implementations of Diff e-Hellman, RSA, DSS, and Other Systems," *Proceedings of CRYPTO '96*, Springer-Verlag, LNCS 1109, 1996, pp. 104–113.

[46] Brumley, D., and D. Boneh, "Remote timing attacks are practical," *Proceedings of the 12th Usenix Security Symposium*, USENIX Association, 2003.

[47] Kocher, P., J. Jaffe, and B. Jun, "Differential Power Analysis," *Proceedings of CRYPTO '99*, Springer-Verlag, LNCS 1666, 1999, pp. 388–397.

[48] Boneh, D., R. DeMillo, and R. Lipton, "On the Importance of Checking Cryptographic Protocols for Faults," *Proceedings of EUROCRYPT '97*, Springer-Verlag, LNCS 1233, 1997, pp. 37–51.

[49] Biham, E., and A. Shamir, "Differential Fault Analysis of Secret Key Cryptosystems," *Proceedings of CRYPTO '97*, Springer-Verlag, LNCS 1294, 1997, pp. 513–525.

[50] Bleichenbacher, D., "Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS #1," *Proceedings of CRYPTO '98*, Springer-Verlag, LNCS 1462, 1998, pp. 1–12.

[51] Asonov, D., and R. Agrawal, "Keyboard Acoustic Emanations," *Proceedings of IEEE Symposium on Security and Privacy*, 2004, pp. 3–11.

[52] Zhuang, L., Zhou, F., and J.D. Tygar, "Keyboard Acoustic Emanations Revisited," *Proceedings of ACM Conference on Computer and Communications Security*, November 2005, pp. 373–382.

[53] Micali, S., and L. Reyzin, "Physically Observable Cryptography," *Proceedings of Theory of Cryptography Conference (TCC 2004)*, Springer-Verlag, LNCS 2951, 2004, pp. 278–296.

[54] Renauld, M., et al., "A Formal Study of Power Variability Issues and Side-Channel Attacks for Nanoscale Devices," *Proceedings of EUROCRYPT 2011*, Springer-Verlag, LNCS 6632, 2011, pp. 109–128.

[55] Kerckhoffs, A., "La Cryptographie Militaire," *Journal des Sciences Militaires*, Vol. IX, January 1883, pp. 5–38, February 1883, pp. 161-191.

[56] Shannon, C.E., "A Mathematical Theory of Communication," *Bell System Technical Journal*, Vol. 27, No. 3/4, July/October 1948, pp. 379–423/623–656.

[57] Shannon, C.E., "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, Vol. 28, No. 4, October 1949, pp. 656–715.

[58] Merkle, R.C., "Secure Communication over Insecure Channels," *Communications of the ACM*, Vol. 21, No. 4, April 1978, pp. 294–299.

[59] Rivest, R.L., A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, Vol. 21, No. 2, February 1978, pp. 120–126.

[60] Elgamal, T., "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithm," *IEEE Transactions on Information Theory*, Vol. 31, No. 4, 1985, pp. 469–472.

[61] ISO/IEC 7498-2, *Information Processing Systems—Open Systems Interconnection Reference Model—Part 2: Security Architecture*, 1989.

[62] Oppliger, R., *Authentication Systems for Secure Networks*, Artech House, Norwood, MA, 1996.

[63] Feghhi, J., Feghhi, J., and P. Williams, *Digital Certif cates: Applied Internet Security*, Addison-Wesley, Reading, MA, 1998.

[64] Adams, C., and S. Lloyd, *Understanding PKI: Concepts, Standards, and Deployment Considerations*, 2nd edition, Addison-Wesley, Reading, MA, 2002.

[65] Vacca, J.R., *Public Key Infrastructure: Building Trusted Applications and Web Services*, Auerbach Publications, 2004.

[66] Buchmann, J.A., Karatsiolis, E., and A. Wiesmaier, *Introduction to Public Key Infrastructures*, Springer, 2013.

[67] Kohnfelder, L.M., "Towards a Practical Public-Key Cryptosystem," Massachusetts Institute of Technology (MIT), Cambridge, MA, May 1978, http://groups.csail.mit.edu/cis/theses/kohnfelder-bs.pdf.

[68] Lopez, J., Oppliger, R., and G. Pernul, "Why Have Public Key Infrastructures Failed So Far?" *Internet Research*, Vol. 15, No. 5, 2005, pp. 544–556.

[69] ITU-T X.509, *Information technology–Open systems interconnection–The Directory: Public-key and attribute certif cate frameworks*, 2012.

[70] ISO/IEC 9594-8, *Information technology–Open systems interconnection–The Directory: Public-key and attribute certif cate frameworks*, 2001.

[71] Ellison, C., "Establishing Identity Without Certif cation Authorities," *Proceedings of the 6th USENIX Security Symposium*, 1996, pp. 67–76, http://static.usenix.org/publications/library/proceedings/sec96/ellison.html.

[72] Rivest, R.L., and B. Lampson, "SDSI—A Simple Distributed Security Infrastructure," September 1996, http://people.csail.mit.edu/rivest/sdsi10.html.

[73] Abadi, M., "On SDSI's Linked Local Name Spaces," *Journal of Computer Security*, Vol. 6, No. 1–2, September 1998, pp. 3–21.

[74] Ellison, C., "SPKI Requirements," RFC 2692, September 1999.

[75] Ellison, C., et al., "SPKI Certif cate Theory," RFC 2693, September 1999.

[76] Cooper, D., et al., "Internet X.509 Public Key Infrastructure Certif cate and Certif cate Revocation List (CRL) Prof le," RFC 5280, May 2008.

[77] Reiter, M.K., and S.G. Stubblebine, "Authentication Metric Analysis and Design," *ACM Transactions on Information and System Security*, Vol. 2, No. 2, May 1999, pp. 138–158.

[78] Myers, M., et al., "X.509 Internet Public Key Infrastructure Online Certif cate Status Protocol—OCSP," RFC 2560, June 1999.

[79] Oppliger, R., "Certif cation Authorities under Attack: A Plea for Certif cate Legitimation," *IEEE Internet Computing*, Vol. 18, No. 1, January/February 2014, pp. 40–47.

[80] Martin, L., *Introduction to Identity-Based Encryption*, Artech House, Norwood, MA, 2008.

Chapter 4

Secure Messaging

In this chapter, we introduce and start with the broader topic of the book (i.e., secure messaging). We outline some threats and attacks in Section 4.1, elaborate on various aspects and notions of security—as far as they are relevant for Internet messaging—in Section 4.2, and conclude with some final remarks in Section 4.3. The aim is to examine from a bird's eye perspective secure messaging.

4.1 THREATS AND ATTACKS

As mentioned before, security was not a top priority when people designed, implemented, and put in place the first messaging or e-mail systems.[1] The assumption was that these systems would be used to exchange messages among peers, and that these peers would be nice and well-disposed. So there was no need to design and come up with sophisticated and fancy security mechanisms. Since these early days, however, the situation has changed fundamentally, and current e-mail systems—and even more so messaging systems—are used to exchange messages among people who don't necessarily know each other in environments that may be hostile. This is particularly true for Internet-based messaging systems that—as their name suggests—are operated on the Internet. Here, it is usually simple for an adversary to read messages as they are sent back and forth, modify or delete messages, or even generate dummy messages to flood a recipient (or the recipient's message store, respectively). Internet messaging is wide open to all types of attacks, and we cannot even try to be comprehensive here.

1 There were some e-mail systems with sophisticated security features, such as X.400-based MHSs and the DMS, but these systems failed to become commercially successful, and hence they were never used in the field.

In the network security literature, it is common to distinguish between passive and active attacks. We adopt this distinction and brief y elaborate on some possibilities to attack Internet-based e-mail systems either passively or actively. Note, however, that in a real-world setting, various types of attacks are usually combined to come up with attacks that are as powerful and devastating as possible. This has been demonstrated, for example, in some sensational reports on EFAIL attacks [1]. These attacks exploit vulnerabilities in OpenPGP and S/MIME to reveal the plaintext of encrypted messages. In a nutshell, they abuse active content in HTML messages (e.g., externally loaded images or styles) to exf ltrate plaintext through requested URLs. To empower such exf ltration channels, the adversary needs to have access to the encrypted messages in the f rst place (e.g., by eavesdropping on network traff c, compromising accounts, e-mail servers, backup systems, or even client computers). The messages could even have been collected years ago. The adversary changes an encrypted message in a particular way and sends the result to the victim. The victim's MUA, in turn, may automatically decrypt the message and load some external content, effectively exf ltrating the plaintext message to the adversary.

More recently, similar attacks have been presented that target digital signatures instead of encrypted messages [2]. The general theme of all these attacks is that they exploit the huge sets of functionalities provided by currently deployed MUAs. In addition to implement messaging protocols, like SMTP, POP3, and IMAP4, these MUAs can render HTML and execute code, such as code written in JavaScript. Like in every new technology, these capabilities can be used for good purposes, but they can also be misused for bad purposes. The EFAIL attacks and similar attacks that target digital signatures have clearly demonstrated this point. A similar problem, by the way, is shared by commonly deployed Web browsers that have also become the major source of insecurity on the Web today. Functionality and security often bite each other.

4.1.1 Passive Attacks

In a *passive attack*, an adversary has read (but no write) access to the data being transferred. This data encodes information that may or may not be accessible and visible to the adversary. It may not be visible, for example, if it is encrypted in some not easily breakable way. Consequently, there are two types of passive attacks that can be distinguished:

- If the information is accessible and visible to the adversary, then one is in the realm of *passive wiretapping* or *eavesdropping attacks*. In this case, the adversary is able to retrieve and interpret the information encoded in the data. This clearly represents a problem in almost all application settings.

- If the information is not accessible and visible to the adversary, then one is in the realm of *traff c analysis attacks*. In this case, the adversary is not able to retrieve and interpret the information encoded in the data. More generally speaking, traff c analysis refers to the inference of information from the observation of external traff c characteristics. For example, if an attacker observes that two companies—one f nancially strong and the other f nancially weak—begin to exchange a large number of messages, then he or she may infer that they are discussing a merger. Other examples appear in military environments. Whether traff c analysis attacks represent a problem depends on the application setting, but usually people do not care much about it.

Passive wiretapping attacks are much more powerful (and hence worrisome) than traff c analysis attacks, but traff c analysis attacks are usually more diff cult to mitigate—since encryption does not help. The bottom line is that traff c analysis attacks are almost always feasible in the realm of Internet messaging, even if cryptography or E2EE messaging is put in place (see below).

There are many factors that determine how diff cult it is to mount a passive attack. Most importantly, the diff culty depends on the physical transmission media in use and their accessibility to the adversary. For example, mobile communication—due to its broadcast nature—is usually easy to attack passively, whereas metallic transmission media require at least some form of physical access. This also applies to lightwave conductors, but these are usually very diff cult to tap. As a general rule of thumb one can say that the more complex networking technologies are put in place, the more diff cult and expensive it is to mount a passive attack. It goes without saying that this also applies to network concentrators and multiplexers.

In practice, it is often the case that a passive adversary is not able to tap a physical communications line, but that he or she is able to control the interface that is used to connect a computer system to the network. Normally, such an interface only captures the data that is destined to the respective system. But sometimes, such an interface may be operated in a special mode (i.e., a so-called promiscuous mode) in which it captures all data transmitted on a particular network segment. Such a computer system with a network interface operating in promiscuous mode can then be used to passively attack the segment it is connected to. Such a capability has useful purposes for network analysis, testing, and debugging, but it can also be misused to mount a passive attack. There are several such tools available for monitoring network traff c, primarily for the purpose of network management—the most prominent being Wireshark.[2] Many of these tools can also be used to mount passive attacks. While the use of switching technologies in local area networks has

2 http://www.wireshark.org.

improved the situation considerably, passive attacks still remain a problem in many network environments today.

As mentioned above, neither data encryption nor any other (simple) technology is able to mitigate traff c analysis attacks. In fact, protection against such attacks in packet-switched networks, such as the Internet, is a diff cult research problem. There are some attempts to combine anonymizing proxies and other cryptographic techniques to come up with a networking infrastructure that mitigates traff c analysis attacks. However, the resulting solutions tend to be involved and expensive to operate. One such attempt is *onion routing* [3] that is based on David Chaum's notion of a *mix network* [4]. More specif cally, onion routing refers to a technology for anonymous communication over a computer network that employs messages that are repeatedly encrypted and sent through several mixes called onion routers. Like someone peeling off an onion, each onion router removes one layer of encryption to uncover routing information and the (still encrypted) message that needs to be sent to the next onion router, where the procedure is repeated. The f nal onion router decrypts the message and delivers it to the recipient. All onion routers only see where the message comes from and where it is sent to, and no intermediary router learns the origin, destination, and content of the message. Most importantly, onion routing is employed in *The Onion Router* (TOR) project and network.[3] Hence, someone observing the TOR network is not able to learn who is communicating with whom—unless he or she is able to observe the exit nodes and no end-to-end encryption is put in place. Except onion routing and TOR, there are only a few technologies and techniques that provide protection against traff c analysis for Internet messaging. As mentioned in Chapter 13, two such examples are Bitmessage and Elixxir.

4.1.2 Active Attacks

The distinguishing feature of a passive attack is that the adversary has read but no write access to the data being transmitted. This is different in the case of an *active attack*. Here, the adversary has read and write access, meaning that he or she can do anything with the data. In particular, he or she refers to what is known as a *man-in-the-middle* (MITM), and can modify, extend, delete, or replay data at will. In fact, the adversary has full control over the data that is being sent back and forth.

In the realm of Internet messaging, a particularly worrisome active attack is spoof ng, where an adversary may try to spoof messages on another—maybe nonexistent—user's behalf. There are usually many ways to mount such an attack in a traditional e-mail environment.

3 https://www.torproject.org.

- In the simplest case, the adversary can conf gure his or her MUA with the name and e-mail address of the spoofed user. When a message is sent out, the MUA automatically puts this information in the header section of the message.
- Similarly, it is sometimes possible to conf gure and use wrong display names, such as `Administrator <rolf.oppliger@esecurity.ch>`. Because there are MUAs that only show the display name (if one is available), this may lead to wrong assumptions about who has actually sent out a message. In this example, the display name is `Administrator`, and this suggests that the message was sent by an administrator, and this, in turn, may lead to misbehavior on the user side.
- A technically more sophisticated attack is to establish a TCP connection to an SMTP server (usually running at port 25), and to directly launch STMP commands to compile a spoofed message from scratch (we have brief y sketched the respective SMTP commands in Section 2.2.2.1).

There are even more possible ways to spoof e-mail messages, and the bottom line is that one should never trust the name and e-mail address of a message originator—unless the message is digitally signed. As the originator address is not used to route the message through the Internet, it can literally be anything.

In addition to message spoof ng attacks, there are many other attacks that can be mounted against Internet messaging systems and messages sent back and forth. For example, another popular attack is a denial-of-service (DoS). Generally speaking, a DoS refers to the prevention of authorized access to resources or the delaying of time-critical operations—therefore, a DoS attack prevents resources from functioning properly (i.e., according to their intended purposes). It may range from a lack of available memory or disk space to a partial shutdown of an entire network segment. If the attack is mounted from multiple systems simultaneously, then it is usually called a distributed denial-of-service (DDoS). It goes without saying that DoS attacks (in general) and DDoS attacks (in particular) are simple to mount, but very diff cult to mitigate. For example, e-mail bombing refers to a simple (D)DoS attack against an e-mail account, and there are several ways to mount such an attack:

- An adversary can employ an (anonymous) e-mail account to constantly bombard the victim's e-mail account with arbitrary messages that may contain very long attachments.
- If an adversary controls an MTA, then he or she can write and execute a script that automates the generation and transmission of such messages.

- An adversary can post a controversial or offensive statement to a large audience (e.g., a popular social network) using the victim's return e-mail address. The responses to this posting are then likely to f ood the victim's e-mail account.
- Similarly, an adversary can subscribe the victim's e-mail address to as many listservers as possible. The generated messages are then sent to the victim, unless he or she unsubscribes the listservers.

Also, there are many other possibilities (and sometimes even readily available tools) to mount e-mail bombing attacks on a large scale. The bottom line is that mitigating these attacks is diff cult and technically challenging, and that this is similar to the real world: How can you protect, for example, your mailbox against someone f lling it up with useless paper or other physical material? It seems to be diff cult if not impossible, because the mailbox is to receive arbitrary deliveries. In the digital world, the situation is comparable if not identical—anybody can send you e-mail and use this capability to f ood your account. Directly related to the impossibility of effectively protecting an e-mail account against e-mail bombing is the problem related to spam (i.e., the act of sending junk e-mail messages to advertise a product or service that sometimes thwarts the legitimate use of e-mail). In fact, spam can be seen as a lightweight (and commercially motivated) form of e-mail bombing. We have overviewed some technologies and techniques to protect against spam in Section 2.2.3.1.

4.2 ASPECTS AND NOTIONS OF SECURITY

Having discussed some threats and attacks related to Internet messaging (mainly in the realm of e-mail), it seems appropriate to address secure Internet messaging from a bird's eye perspective, and to discuss respective aspects and notions of security. The following questions pop up in any security discussion:[4]

1. What does "secure messaging" mean?
2. How can "secure messaging" be implemented?

With regard to the f rst question, we claim that there is no mathematically precise def nition for the term secure messaging, and that it is therefore reasonable to follow the line of argumentation pursued in the OSI security architecture [5]. This architecture provides a terminology—in terms of security services and (specif c

4 In either question, we leave aside the word Internet, because we only address Internet-based messaging here.

and pervasive) security mechanisms—that can be used to argue about the security a messaging system. The details can be found in the standard document or the secondary literature that is available (e.g., [6, 7]). All security services are relevant for secure messaging, where the connection-oriented services are better suited for synchronous (instant) messaging and the connectionless services are better suited for asynchronous messaging (e-mail). The security services most relevant for the topic of this book refer to authentication, confidentiality, and integrity, whereas the specific security mechanisms most relevant to provide these services are data encryption and digital signatures (as outlined in Chapter 3). Preferably, these mechanisms are applied on an end-to-end basis, meaning that the security mechanisms are invoked by the end users and their respective systems.

With regard to the second question, there are two distinct approaches to either build-in or add-on mechanisms to provide the security services in a given messaging infrastructure.

- The first approach is to build the security mechanisms into the messaging infrastructure (this approach may be called built-in security). In this case, the message formats and messaging protocols must be modified to incorporate the security mechanisms, and to provide the security services accordingly.
- The second approach is to leave the messaging infrastructure as it is, and to only modify the message formats to incorporate the security mechanisms and to provide the security services accordingly (this approach may be called add-on security). In this case, the messaging protocols must not be touched.

From a theoretical viewpoint, built-in security is the certainly the preferred choice, mainly because it allows security to be provided more integrally and holistically. But from a practical viewpoint, there are several shortcomings and limitations that must be considered with care. First of all, built-in security requires a redesign of all (or most) message formats and messaging protocols (to incorporate the appropriate security mechanisms). Furthermore, the redesigned message formats and messaging protocols must be implemented, deployed, and supported by all participants on the message delivery paths (e.g., MTAs and MUAs in the case of e-mail). This makes built-in security generally hard to achieve and expensive. Contrary to that, add-on security has advantages, since it does not require large modifications on the messaging infrastructures. Instead, all (or at least most) modifications can be done in the end systems, and this makes add-on security simple and comparably inexpensive to implement and deploy.

As discussed in the Preface, the first approach (i.e., built-in security) was followed when strong security features were designed for X.400-based MHSs and the MSP of the DMS. Meanwhile, the trend is to pursue the second approach (i.e.,

add-on security), and to provide security services in a way that is transparent for the underlying messaging infrastructure. This approach has also been followed by all major schemes for secure e-mail on the Internet, including PEM, MOSS, OpenPGP, and S/MIME. When it comes to synchronous (instant) messaging, built-in security seems to be feasible at least to some extent. In fact, most message formats and messaging protocols have been designed or redesigned to provide secure and E2EE messaging by default.

A topic that is ultimately important to understand the current discussions is related to different notions of secrecy. Assume some long-term keying material being compromised. What is the impact on the secrecy of the cryptographically protected (i.e., encrypted) data? Is the secrecy of the data still protected? Is there a difference for data sent in the past and data to be sent in the future? Questions like these have led to different notions of secrecy that are sometimes referred to using different and sometimes even confusing terms.

Since the early 1990s, people have been using the term *perfect forward secrecy* (PFS) to refer to the property of a cryptographic system using a particular key agreement protocol that ensures that session keys don't get compromised even if a long-term (typically private) key gets compromised. This def nition is informal and not mathematically precise, but it is still intuitively clear what it means and what it is standing for. Because the word perfect misleads people to believe that the notion of PFS is somewhat related to Claude Shannon's notion of perfect secrecy, people sometimes leave aside the word perfect and use the term *forward secrecy* instead, synonymously and interchangeably with PFS.

From a technical viewpoint, the provision of PFS or forward secrecy requires an ephemeral Diff e-Hellman key exchange for every session key needed, and the long-term private key to be used only to authenticate the respective key exchange. If this (authentication) key gets compromised, then there is still no way to recompute the session key. Such a key can only get compromised while it (or any of the Diff e-Hellman parameters used to generate it) is stored in memory or is in actual use. Once it is deleted, there is no way to recompute it—and this, in turn, provides PFS or forward secrecy.

Things get more involved if one considers alternative approaches to achieve PFS or forward secrecy (than always performing an ephemeral Diff e-Hellman key exchange). Look, for example, what happens if one generates a new session key simply by hashing the old one. In this case, if a session key gets compromised, it is not possible to compute any previously used session key (because this would require computing the inverse of the cryptographic hash function in use), but it is still feasible to compute all subsequently used session keys (because the session key can simply be subjected to the hash function). Hence, this simple key update mechanism provides some sort of PFS or forward secrecy, namely the one that is

backward-oriented in time: Any previously used session key remains protected, but any session key to be used in the future gets compromised trivially.

This insight has led to a more subtle use of the terms PFS and forward secrecy. In fact, the terms are still used, but they are used in the sense that the respective key agreement protocol protects against a key compromise that may occur in the future. In contrast, if the key agreement protocol protects data secrecy against a key compromise that may have occurred in the past, then people often use the complementary terms *post-compromise security* (PCS[5]) or *future secrecy*. In some sense, PCS and future secrecy is a self-healing property, meaning that a system can recover and turn itself from a compromised and insecure state into a secure state. The above-mentioned scheme to generate a new session key by hashing the old one provides PFS and forward secrecy in the new and more narrow sense, but it does not provide PCS or future secrecy. So when discussing the level of secrecy a key agreement protocol provides, one usually has to discuss the two cases. The question to ask is what happens if some keying material gets compromised? Is the secrecy of past data still protected or not, and vice-versa, is the secrecy of future data still protected or not? The first question leads to the notions of PFS and forward secrecy, whereas the second question leads to the notions of PCS and future secrecy. In the ideal case, both notions of secrecy apply.

The notions of secrecy and respective terminology are summarized in Figure 4.1. Along the time axis t, it shows the direction of the protection a term refers to. Forward secrecy protects data in the past against a key compromise, meaning that a key compromise that occurs today does not affect data that have been transmitted in the past. Similarly, PCS protects data in the future, meaning that the same key compromise (that occurs today) does not affect data that will be transmitted in the future, meaning that the system can heal itself. The terminology is confusing, because the two notions of secrecy could be referred to as *precompromise security* and PCS (but in this case, both acronyms would be the same) or *backward secrecy* and future secrecy (but in this case, we would have to use the term backward secrecy as a synonym to forward secrecy, and this is not very intuitive). For lack of better terminology, we use the terms forward secrecy and PCS in this book (these are the terms that are written in bold face in Figure 4.1). This terminology is neither elegant nor intuitively clear, but it is in line with the literature in the field. Forward secrecy and PCS are going to be important criteria when it comes to discussing secure and E2EE messaging schemes and protocols. While OpenPGP and S/MIME provide neither of the two properties, modern approaches and solutions (like OTR and Signal) typically do. In fact, the provision of forward secrecy and PCS is one of the distinguishing features of modern and state of the art E2EE messaging protocols and respective messengers and messaging apps.

5 The term PCS was first introduced and formalized in [8].



Figure 4.1 Notions of secrecy and respective terminology.

4.3 FINAL REMARKS

In this chapter, we began with a short discussion of some threats and attacks that are relevant for Internet messaging (mostly in the realm of e-mail), before we have elaborated on various aspects and notions of security. Most passive eavesdropping and several active attacks can be mitigated. There are, however, also attacks that cannot be mitigated, and hence the respective systems remain vulnerable and exploitable. Most importantly, almost all secure and E2EE messaging schemes do not protect against traff c analysis. This means that—in spite of all fancy cryptography that is put in place and used—an adversary can still determine who is sending messages to whom. In environments in which this type of information is critical, additional countermeasures can be invoked, such as the use of the TOR network. The most important point to make (and remember) is that no cryptographic protocol provides a silver bullet for all security problems. They provide a viable solution for the provision of some basic message protection services, but they are not a panacea that magically solves all security problems. The EFAIL and related attacks have clearly demonstrated this point. Also, the use of any secure (even E2EE) messaging scheme must still be complemented by mechanisms that ensure that it is securely implemented, put in place, and used. The last point is particularly important and asks for organizational and personnel security measures that can only be addressed on a case-to-case basis. As is usually the case in security, the users and the details matter a lot.

References

[1] Poddebniak, D., et al., "Efail: Breaking S/MIME and OpenPGP Email Encryption using Exf l-tration Channels," *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*, USENIX Association, 2018, pp. 549–566.

[2] Müller, J., et al., "Johnny, you are f red! Spoof ng OpenPGP and S/MIME Signatures in Email," *Proceedings of the 28th USENIX Security Symposium (USENIX Security 19)*, USENIX Association, 2019.

[3] Reed, M.G., Syverson, P.F., and D.M. Goldschlag, "Anonymous connections and onion routing," *IEEE Journal on Selected Areas in Communications*, Vol. 16 (1998), pp. 482–494.

[4] Chaum, D.L., "Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms," *Communications of the ACM*, Vol. 24, No. 2, February 1981, pp. 84–88.

[5] ISO/IEC 7498-2, Information Processing Systems — Open Systems Interconnection Reference Model — Part 2: Security Architecture, 1989.

[6] Pf eeger, C.P., and S.L. Pf eeger, *Analyzing Computer Security: A Threat / Vulnerability / Countermeasure Approach*, Prentice Hall, Upper Saddle River, NJ, 2011.

[7] Pf eeger, C.P., and S.L. Pf eeger, *Security in Computing*, 5th Edition, Prentice Hall, Upper Saddle River, NJ, 2015.

[8] Cohn-Gordon, K., Cremers, C., and L. Garratt, "On Post-Compromise Security." *Proceedings of the 29th IEEE Computer Security Foundations Symposium (CSF 2016)*, 2016, pp. 164–178.

Chapter 5

OpenPGP

PGP was historically the f rst technology to provide secure and E2EE messaging on the Internet—at least for e-mail. This chapter provides a comprehensive introduction and outline of PGP and OpenPGP. Unlike PEM, MOSS, and S/MIME, the terms PGP and OpenPGP do not only refer to protocol specif cations, but also to software packages that are used on the Internet to end-to-end encrypt messages. Since the differences between PGP and OpenPGP are negligible and evolutionary, the terms PGP and OpenPGP are used synonymously and interchangeably in this book. We more frequently use the term OpenPGP, also because the terms Pretty Good Privacy, Pretty Good, and PGP are registered trademarks (currently owned by Symantec). This chapter starts with the origins and history in Section 5.1, elaborates on the technology in Section 5.2, discusses the web of trust in Section 5.3, provides a security analysis in Section 5.4, and concludes with some f nal remarks in Section 5.5. This chapter can stand for itself and can be used as a comprehensive introduction and outline of PGP and OpenPGP.

5.1 ORIGINS AND HISTORY

The original PGP software was developed by Philip R. Zimmermann[1] in the early 1990s [1, 2]. He selected some of the best available cryptographic algorithms of this time (i.e., MD5, IDEA[2], and RSA), integrated them into a platform-independent

1 http://www.philzimmermann.com.

2 The International Data Encryption Algorithm (IDEA) is a block cipher that was developed by James L. Massey und Xueija Lai in 1990. It was designed to be resistant against differential cryptanalysis and was generally considered to be a secure cipher, with only a few minor problems and shortcomings. But it was patented and therefore not a valid contender for the AES competition. After the competition it disappeared and silently sank into oblivion.

software that was based on a small set of easy-to-use commands, and made the resulting software and documentation—including the source code written in the C programming language—publicly and freely available on the Internet, at least for citizens within the U.S. and Canada. Zimmermann also entered into a legal agreement with a company named Viacrypt to provide a fully compatible commercial version of PGP that was reasonably priced.[3] The commercial version of PGP was to satisfy the requirements of users who wanted to have a product with professional support by its vendor.

There were at least two legal problems related to these first versions of the PGP software:

- First, the PGP software employed the RSA algorithm that was patented at this time;[4]
- Second and maybe more worrisome, the U.S. government held that export controls for cryptographic software were violated when the PGP software spread around the world following its publication as freeware.

The first problem was settled with the patent holders of the RSA public key cryptosystem by having the PGP software include and make use of a cryptographic library distributed by RSA Security.[5] More specifically, beginning with version 2.5, the PGP software included and made use of the RSAREF cryptographic library to perform the RSA computations. The RSAREF library was distributed under a license that allowed noncommercial use within the U.S. The commercial use of RSAREF, however, required the payment of a license fee to RSA Security. Since the commercial version of PGP was sold by Viacrypt, the use of RSAREF in this version was properly licensed from the very beginning.

The second problem was more dramatic for Zimmermann, as it led to a three-year criminal investigation by the U.S. government. Zimmermann was accused of a federal crime because the software had flowed across national borders. The investigation was carefully followed by the trade press and the general public (see, for example, [3] for a summary). The U.S. government finally dropped the case in 1996. Soon after, Zimmermann founded a company called Pretty Good Privacy, Inc. that was acquired by McAfee—or Network Associates, as it was

3 The company Viacrypt no longer exists and the domain name viacrypt.com is nowadays also owned by Symantec.

4 The U.S. Patent 4,405,829 on RSA was filed on December 14, 1977. It was issued in September 1983, and expired 17 years later on September 20, 2000.

5 The former name of the company was RSA Data Security. The company was acquired by EMC in 2006, and EMC was acquired by Dell Technologies in 2016. Today, RSA Security LLC is part of the Dell Technologies family of brands.

called at this time—in 1997. PGP was further developed by McAfee only in a half-hearted way, and was f nally abandoned in 2002. A group of McAfee employees around Zimmermann took over the PGP software, founded PGP Corporation, and continued its commercialization. As such, PGP Corporation successfully operated on the market, until it was acquired for 300 million USD by Symantec in 2010. In the subsequent years, the classical PGP products were integrated into the off cial product line of Symantec, and the trademark PGP disappeared and silently sank into oblivion. This is in sharp contrast to the term OpenPGP that has withstood the economic turmoil to this day.

Due to its eventful history and unclear legal situation (with regard to patent infringements and export controls), the IETF became active and chartered a working group to standardize the message format and use of OpenPGP in the 1990s.[6] Before it was concluded in 2008, the WG had come up with three RFC documents[7] [4–6] that have all been updated and become obsolete meanwhile. Today, there is a pair of relevant RFC documents, namely RFC 4880 [7] that specif es the OpenPGP message format and RFC 3156 [8] that specif es the combined use of MIME and OpenPGP. As we will see in later parts of this chapter, there are several complementary RFC documents that address specif c aspects of OpenPGP. They will be introduced when appropriate and needed. More recently, EFAIL and related attacks (Section 4.1) have had people revisit RFC 4880 and improve the cryptographic strength of OpenPGP. As of this writing, the revised version of RFC 4880 is still an Internet-Draft,[8] but is possible and very likely that it will eventually become an off cial RFC document soon. In Section 5.2.5, we brief y outline the changes that are envisioned with this revision. The overall goal is to modernize the cryptographic primitives and mechanisms used in OpenPGP.

Today, there are many software packages that implement OpenPGP. Most of them are integrated into one (or several) MUA(s). An MUA can either natively support OpenPGP, or it can be complemented by some plug-in that provides support for it. Note, however, that OpenPGP does not even need to be integrated into an MUA. A user can always create a message with his or her favorite word processing software (e.g., a text editor or Microsoft Word), digitally sign and/or encrypt the respective f le with an OpenPGP-compliant software, optionally encode it for transport (either using the radix-64 encoding function or any other encoding utility), and f nally use any MUA of his or her choice to send the resulting message to the recipient. The point is that OpenPGP need not be part of the MUA used to

6 http://datatracker.ietf.org/wg/openpgp/charter/.

7 The f rst RFC document is informational, whereas the other two were submitted to the Internet standards track.

8 In September 2019, version 8 of the Internet-Draft was released with the name draft-ietf-openpgp-rfc4880bis-08.

send a message, and that it may reside entirely outside the MUA. This is different from what S/MIME provides—as we will see in the following chapter.

From a user's perspective, it is most convenient to have the functionality of OpenPGP be incorporated into the MUA. In the simplest case, the user has two additional buttons, one for signaling the use of a digital signature (to protect the authenticity and integrity of a message) and one for signaling the use of a digital envelope (to protect the conf dentiality of a message). There are many OpenPGP implementations that work this way. For example, there is a free implementation known as GNU Privacy Guard (GnuPG, or GPG in short), including a Windows version known as Gpg4win. The development of GPG and Gpg4win had originally been funded by a German ministry, but it was later taken over by the GnuPG Project.[9] Due to the high popularity of GPG in the Internet community, the GnuPG Project launched a crowdfunding campaign and raised more than EUR (€) 36,000 for the further development of the software in 2014. Furthermore, there are OpenPGP plug-ins for most widely deployed MUAs, such as GpgOL[10] for Microsoft Outlook, Enigmail[11] for Mozilla Thunderbird,[12] and many more. If an MUA is successful and widely deployed, then it is possible and very likely that some developer(s) will provide an OpenPGP plug-in for it. This also applies to smartphones and tablets. Examples include On-Core SecuMail,[13] iPGMail,[14] and oPenGP for iOS, as well as K-9[15] and OpenKeychain[16] for Android. As the list of OpenPGP implementations is a moving target, we don't even try to be comprehensive here.

5.2 TECHNOLOGY

This section elaborates on the technology employed by OpenPGP. We start with some preliminary remarks and introduce the notion of a key ID, before we delve more deeply into the message format, PGP/MIME, cryptographic algorithms, message processing, and key management. In doing so, we try to be as brief and focused as possible.

9 https://gnupg.com.
10 GpgOL ist part of Gpg4win.
11 https://enigmail.net.
12 The developers of Mozilla Thunderbird have announced that OpenPGP will be natively supported by the software from version 78 on (https://blog.mozilla.org/thunderbird/2019/10/thunderbird-enigmail-and-openpgp/).
13 http://on-core.com/secumail/.
14 http://ipgmail.com.
15 https://k9mail.github.io.
16 https://www.openkeychain.org.

5.2.1 Preliminary Remarks

OpenPGP combines secret and public key cryptography to provide services that are relevant for secure and E2EE messaging. More specif cally, it provides data origin authentication and integrity services through the use of digital signatures, and data conf dentiality services through the use of digital envelopes. Furthermore, OpenPGP is able to compress data, encode messages for transfer (using radix-64 encoding), and manage public keys and certif cates in a unique way. Hence, OpenPGP is multifunctional and provides support for many distinct features.

Unfortunately, there are some terminological problems in many texts about PGP and OpenPGP—including the original PGP documentation [1, 2] and respective manuals. We brief y mention two of these problems to make it easier to read these texts.

- First, the term Diff e-Hellman is misleading. The algorithm to use a modif ed version of the Diff e-Hellman key exchange protocol to encrypt data (e.g., a session key) was proposed by Taher Elgamal [9] a couple of years after the original publication of Diff e and Hellman [10]. Following the name of its inventor, it is known as the Elgamal asymmetric encryption system, and the expression DH/DSS as used in many texts and also in the user interface of many OpenPGP software packages should be replaced with Elgamal/DSS or something similar.[17]
- Second, the term session key is also misleading. E-mail is an asynchronous and hence connectionless application; therefore, there is no session being established between the sender and recipient of an e-mail message (there may only be sessions between pairs of MTAs on the message delivery path). Consequently, the term session key should be replaced with message key, message encryption key, data encryption key, or something similar. It's basically a one-time key that is used to encrypt and decrypt a message—rather than a session.

To make it easier to read the original PGP documentation and manuals, we sometimes use the above-mentioned and arguably wrong terms in this chapter. In particular, we sometimes use the term Diff e-Hellman to refer to the Elgamal encryption or digital signature system, and we sometimes even use the term session key to refer to a message encryption key.

17 Note that Elgamal encryption and the digital signature standard (DSS) are conceptually similar and based on the same mathematical problem, namely the discrete logarithm problem (DLP). While Elgamal encryption is used to encrypt data, the DSS is used to digitally sign data. Also note that the DSS is sometimes referred to the digital signature algorithm (DSA).

5.2.2 Key ID

A digitally enveloped message uses hybrid encryption (i.e., it is symmetrically encrypted with a *session key* that is asymmetrically encrypted with the recipient's public key). If each user employed a single public key pair, then the recipient would immediately know which key to use to decrypt the session key. But a user may have multiple key pairs, and hence there need not be a one-to-one relationship between the user and his or her public key pairs (this is true for any system that makes use of public key cryptography, not just OpenPGP). How, then, does the recipient of an encrypted message know which of his or her public keys was used to encrypt the session key? How does he or she know which private key to use to decrypt it? There are three approaches:

1. The recipient can try all his or her private keys to decrypt the session key (and the message, respectively).
2. The sender can transmit the public key he or she used to encrypt the session key together with the encrypted message. The recipient can then verify that the transmitted public key matches one of his or her public keys, and proceed accordingly.
3. A key identif er (key ID) can be assigned to each public key. This ID must be unique, at least for a particular user identif er (user ID). In this case, a pair consisting of a user ID and a key ID is suff cient to uniquely identify the public key in use, and hence only the much shorter key ID needs to be transmitted to the recipient.

The f rst approach is rather clumsy and ineff cient with regard to the computational overhead required on the recipient's side (remember that the use of public key cryptography requires a lot of computational resources). The second approach is ineff cient with regard to bandwidth consumption. Note that a public key is typically a few thousand bits long (at least in the case of RSA), so every transmitted public key would occupy and consume a considerably large amount of bandwidth. Consequently, the third approach seems to provide a more eff cient way to solve the problem. This approach, however, raises a key management problem, namely how to assign, store, and manage key IDs so that both the sender and the recipient can map a key ID to a particular key pair. OpenPGP employs a simple solution for this problem: It assigns a key ID to each public key that is, with a high probability, unique for a given user ID. The key ID consists of the least signif cant 64 bits of the SHA-1 hash value of the public key. That is, the key ID of A's public key pk_A refers to the mathematical result of computing $h(pk_A)$ modulo 2^{64} (i.e., $h(pk_A) \mod 2^{64}$), where h stands for SHA-1. This is suff cient so that the key ID is unique for all

practical purposes, and that the probability of two keys having the same key ID (for the same user ID) is negligible.[18] For example, the key ID of a formerly used public key is `8E50 BDB3 0AC2 9A5B` (written in hexadecimal notation). This refers to the following binary value:

```
1000 1110 0101 0000   1011 1101 1011 0011
0000 1010 1100 0010   1001 1010 0101 1011
```

Furthermore, if key IDs are displayed, sometimes only the lower 32 bits are shown for further brevity. These 32 bits then refer to the mathematical result of computing pk_A modulo 2^{32} (i.e., $pk_A \mod 2^{32}$). Consequently, the key ID of the public key mentioned above can also be shown as `0AC2 9A5B`—again written in hexadecimal notation. This refers to the following binary value:

```
0000 1010 1100 0010   1001 1010 0101 1011
```

Sometimes, the 32-bit key ID is called the *short key ID*, whereas the 64-bit key ID is called the *long key ID*. In either case, the notion of a key ID is very important for the proper operation of OpenPGP.

5.2.3 Message Format

There is an outdated PGP message format specif ed in [4] and a new OpenPGP message format specif ed in [7].[19] The exact message format is beyond the scope of this book and can be found in either [4] or [7]. This book takes a high-level perspective and does not delve into the details and differences of these formats. In either case, the OpenPGP message format is based on the notion of a record that has traditionally been called packet in OpenPGP parlance. All OpenPGP objects—like messages, keyrings, certif cates, and so on—consist of packets, where each packet may comprise other packets. This means that the OpenPGP packeting scheme is recursive.

As is usually the case in a packeting scheme, an OpenPGP packet has a header and a body. The header consists of the following two f elds:

- A one-byte *tag* f eld that determines the format of the header and the packet content;

18 Note that, in general, a coincidental match of keys or system parameters can have dramatic consequences for the security of a cryptographic algorithm or system. For example, a coincidental match of RSA primes lead to eff cient factorization, and a coincidental match of random values destroy the security of the Elgamal encryption system. In this case, however, the situation is different, because two keys having the same key ID (for the same user ID) is not particularly worrisome.

19 Sometimes the message format of [4] is attributed to PGP version 2, whereas the message format of [7] (and [6]) is attributed to PGP version 5. This is also the format used in OpenPGP.

Table 5.1
The Packet Tag Values

0	Reserved
1	Public-key encrypted session key packet
2	Signature packet
3	Symmetric-key encrypted session key packet
4	One-pass signature packet
5	Secret-key packet
6	Public-key packet
7	Secret-subkey packet
8	Compressed data packet
9	Symmetrically encrypted data packet
10	Marker packet
11	Literal data packet
12	Trust packet
13	User ID packet
14	Public-subkey packet
17	User attribute packet
18	Symmetrically encrypted and integrity protected data packet
19	Modif cation detection code packet
60 to 63	Private or experimental values

- A *length* f eld that has itself a variable length and denotes the length of the entire packet (in number of bytes). The length encoding scheme is relatively complex and not addressed here.[20]

The new OpenPGP message format uses six (out of eight) bits to refer to the packet tag.[21] This means that there are $2^6 = 64$ possible values (the outdated PGP message format used only four bits, meaning that there were $2^4 = 16$ possible values). The valid packet tag values (according to [7]) are summarized in Table 5.1. Most tags stand for themselves. A tag value of one, for example, stands for a public-key encrypted session key packet. Such a packet holds the session key that has been used to encrypt a message. It goes without saying that the session key can only be decrypted with the appropriate private key. In the Internet-Draft that is to replace [7], it is intended to add tag 20 referring to an AEAD encrypted data packet. This is in line with the general trend of adding authenticated encryption to OpenPGP. The tag values 60 to 63 are reserved for private or experimental use, meaning that a

20 It is outlined in [7]; Section 4.2.1 for the outdated PGP message format and Section 4.2.2 for the new OpenPGP message format. Furthermore, Section 4.2.3 provides several encoding examples.

21 If the most signif cant bit is the leftmost bit, meaning that the 8 bits 1,. . . ,8 are written as 87654321, then the seventh bit is set if the new message format is used, and the eighth bit is always set.

packet tagged with such a value refers to some unoff cial use and should be handled accordingly.

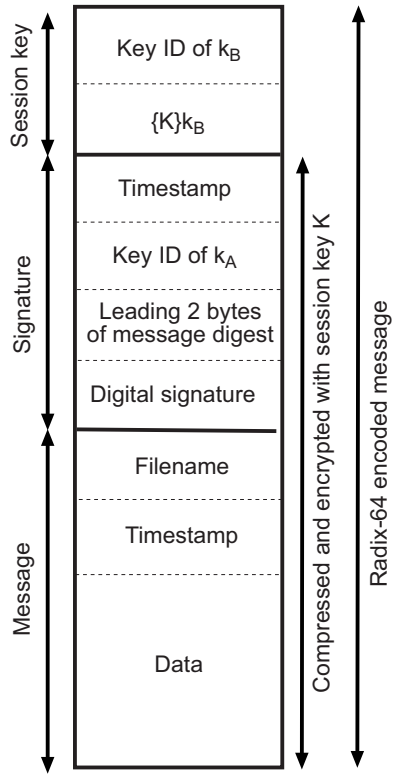


Figure 5.1 The general format of an OpenPGP message or f le.

Having the notion of an OpenPGP packet in mind, one can outline the general structure and format of an *OpenPGP message* or *f le*, as illustrated in Figure 5.1. Note that the f gure is simplif ed considerably, and that it only includes the most important f elds. From a bird's eye perspective, any OpenPGP message or f le may consist of three parts:

- First, an OpenPGP message or f le always includes a *message part* that has the following components:
  - A f lename that specif es the name of the OpenPGP message or f le;

  - A timestamp that specifies the time at which the OpenPGP message or file was created;
  - The data of the file (the data that is stored or transmitted).

- Second, if an OpenPGP message or file is digitally signed, then it includes a *signature part* that consists of the following components:
  - A timestamp that specifies the time at which the signature was created.
  - The key ID[22] of the sender's public key pk_A. This key ID is used to identify the public key that should be used to verify the digital signature.
  - The leading two bytes of the message digest (where the message digest is computed with the cryptographic hash algorithm in use). The aim of this value is to enable the recipient to determine if the correct public key was used to verify the signature.
  - The digital signature for the message. It basically consists of the message digest encrypted with the sender's private key sk_A. The message digest, in turn, is computed over the timestamp of the signature part (to mitigate replay attacks) concatenated with the data of the message part. The filename and timestamp of the message part are not included to ensure that any detached signature is exactly the same as an attached signature prefixed to the message. Note that detached signatures are calculated on a separate file that has none of the message part fields, such as filenames or timestamps.

- Third, if an OpenPGP message or file is digitally enveloped, then it includes a *session key part* for each recipient B_i $(i = 1, \ldots, n)$. This part, in turn, includes the following components:
  - The key ID for the recipient's public key pk_{B_i} that was used by the sender to encrypt the session key;
  - The encrypted session key $\{K\}pk_{B_i}$, standing for $\mathsf{Encrypt}(pk_{B_i}, K)$ here, that is part of the digital envelope for the message.

22 For obvious reasons, a key ID is also required for digital signatures. Because a sender may have multiple private keys to encrypt a message digest (and digitally sign the message accordingly), the recipient must know which public key he should use. Consequently, the digital signature component of an OpenPGP message must include the 64-bit key ID of the required public key. When the message is received, the recipient must verify that the key ID is for a public key that is known for that sender and then proceed to verify the signature.

For the sake of simplicity, Figure 6.1 illustrates only the session key part for a single recipient B. If an OpenPGP message or file has several recipients, then a session key part must be included for every recipient. This also applies to an *additional decryption key* (ADK) that may be configured in some versions of PGP or OpenPGP. The aim of the ADK is to provide a simple message recovery mechanism, as the holder of the private key part of the ADK can always decrypt any encrypted and digitally enveloped message at will. Note that the introduction and use of the ADK and the respective message recovery mechanism has been controversially discussed within the Internet community. As data transmitted is available at either end of the transmission channel, it can also be retrieved there (if needed). The bottom line is that key recovery (or key escrow, as it is sometimes called) remains an emotional topic, even after many years of public discussion.

Both the signature and session key parts of an OpenPGP message or file are optional, meaning that their existence depends on whether a digital signature or digital envelope is used.

In the beginning of this chapter, we mentioned that there are different possibilities to send an OpenPGP message to one (or several) recipient(s). In the simplest case, it is simply sent in the message body part of an RFC 5322-compliant message. There are many MUAs and extensions that support OpenPGP this way (if such an MUA is not available, then it is still possible to perform the OpenPGP transformations outside the MUA). An encrypted and digitally enveloped OpenPGP message sent in the message body part of an RFC 5322-compliant message may look as follows:

```
-----BEGIN PGP MESSAGE-----
Version: PGP Personal Privacy 6.5.3

qANQR1DDDQQDAwJQ3AjP29XbWWDJwB1hZRimoQ1QLBAw55tpRRqs9BY27sQabaVA
/UmaQa6RRZXfe5MiNt+Qdm4MZ+R8oxLE8yaCz/WvBxumU5jynb5Lg4YCJoFeiqLJ
rbETqrj4nClQ8VtXmNXyp637UkCvJxViJbPqa1fKffZnLHi/JHelDnDhHCKbmqGJ
h3tkEpNStuw8OozALt0YCdKyY4E0zLRAYX2utSVk66VQAucgibpX3O8+lAFwqXFr
rPr4cVIHPDvL+f3tjO8dVjR+pC/i3+WZPATR2//aADKpkX95zTa56TI8u3RDzF7D
iClpnA==
=s4eF
-----END PGP MESSAGE-----
```

Similarly, the body part of a digitally signed OpenPGP message may look like this:

```
-----BEGIN PGP SIGNED MESSAGE-----
```

```
Hash: SHA1

This is a digitally signed test message.

-----BEGIN PGP SIGNATURE-----
Version: PGP Personal Privacy 6.5.3

iQA/AwUBORJRro5QvbMKwppbEQI0cwCg0g6+cbxnZH8gyVD/deWCrbA6desAoKdg
5flmAMSqcKLHV10QBh5OtpmP
=CN7I
-----END PGP SIGNATURE-----
```

In the more luxurious case, the OpenPGP functionality is part of the MUA, and the graphical user interface (GUI) of such an MUA provides two additional buttons: One for the generation or verif cation of digital signatures (to protect the authenticity and integrity of the messages), and one for the encryption and decryption of messages (to protect their conf dentiality). The look and feel of such a GUI depends on the MUA in use. We don't provide any screenshot here, mainly because every MUA employs a distinct GUI that is unique for this particular software version.

The transmission of an OpenPGP message in the body part of a normal RFC 5322-compliant message is simple and straightforward. As such, it has specif c advantages and disadvantages. The most important advantage is related to the fact that the receiving MUA need not provide support for OpenPGP. Instead, the recipient can extract the digitally signed and/or digitally enveloped message part and use OpenPGP software outside the MUA to verify the signature and/or decrypt the message. Hence, if the recipient is not known to use an MUA that supports OpenPGP (either natively or through the use of a plug-in), then it may be best to transmit the OpenPGP message in the body part of a normal RFC 5322-compliant message. Contrary to that, the most important disadvantage is related to the fact that MIME is not supported (and hence the f exibility of MIME cannot be used directly). This disadvantage can be remedied by the use of PGP/MIME.

5.2.4 PGP/MIME

Since the mid-1990s, people have been working on combining PGP with MIME. The f rst step was to introduce some security multipart formats for MIME. In particular, RFC 1847 [11] specif ed two MIME `multipart` subtypes (i.e., `multipart/encrypted` and `multipart/signed`) to be employed by MOSS [12]. Work on MOSS culminated in RFC 2015 [5] for PGP, and RFC 3156 [8] for OpenPGP, respectively. This document is still the basis of combining OpenPGP and MIME.

In addition to the MIME subtypes mentioned above, it specifies three content types (or protocol parameters) for making use of OpenPGP. Their names speak for themselves:

- The content type `application/pgp-encrypted` is used to refer to encrypted data;
- The content type `application/pgp-signature` is used to refer to digitally signed data;
- The content type `application/pgp-keys` is used to refer to keying material.

To illustrate the combined use of OpenPGP and MIME, we digitally sign and envelope the message "This is a test message." The skeleton of the resulting message may look as follows:

```
Return-Path: ...
Received: ...
From: ...
To: ...
Subject:
Content-Type: multipart/encrypted;
   protocol="application/pgp-encrypted";
   boundary="=-=dwh+Lqq+2fjNia=-="
MIME-Version: 1.0

--=-=dwh+Lqq+2fjNia=-=
Content-Type: application/pgp-encrypted

Version: 1

--=-=dwh+Lqq+2fjNia=-=
Content-Type: application/octet-stream

-----BEGIN PGP MESSAGE-----
Version: GnuPG v2.0.19 (MingW32)

hQIOA5k1UIH81NrQEAgAuIOja/Lt1PP04lfKBhuLV6zjjZdFkeEWtbnVY6cyvPs8
J2yqfqNVrZEemnNsnRqd6bqAtJohFiZVbG0xBm/X4S8HMiEcakHrxLxts9K1o2WS
I3tCyfAe3EWoanZENuTdlJl9IwT/UJ/fXTlZyJyMLmidGjn1vklnJ+8HAepuMz20
jDeZaWElRcD6Zlq/VXrDojijS+GfCyrFgpuN/mH90OcGkf7jUFMS3HUDEjZ/1GR6
wTLH4SeXhrtd7nDZLN1YWhZtCWh7ZKfwMVkR+XjftbUVgiUXnvLlrvpxD3Slu5ht
gRD1cQZwOFP3cTIH/NDDvBYvsGlsUV2IrksZ8VQ1dAf/a4/XNSFDAcHB2Sno5hpB
RM1QX3gZARCzrsYrSzr4R/wuKqvQn4ydODq6gw9AP8MuX8vpH0flSzYtOv5bD9UB
```

```
muJabB6NSq5OvNVOLrP/UeB4Rsjk8nw+PkuMy/8EPk1aoVc9XEAgwYYk0T7ug8Lj
8pvvdEJ4Vhi4ja2i/VY4V8ZPmyD72mHDpxVzEDW/9eaRtKDw7Q3oLAWKuk/wqfmO
9jJZXZEfvOLrzwrpLJ9yoI+8GPHG38lY6qYBZxK04gBWY+yc1DStbv99f/ZK4Dny
9KD/elSe5NOCQAgacBHbiFbjxVy9a3gQ0PLMbWZFB4vrxuJmY7uSWZp+RJUWL2N+
YcnAwBNWL5OAEAa8giehbu7CpG/VMVRQfeJ3TWR4p3sno1kiSGV4eFqYkFQ6wTL7
SJXraXg+QMMU0z0qdUySY3SnsysZZkbbFoBFl3LJJr8yWvZi9EDD/F2dyRdfYOcZ
auhsMd4YxzW7QUnxqfJ6UJkc+OVWV3ALY6kn7MScNDKhv8vjRtf+FN3zqtGla0g0
Hqny/VaqLic6MojIuih7BaEd1/UdhVvcwZIsJvjcSuszPW7TMmiBLi5qNI3BkKkC
eIkmmxxMrCadEsat3N4QZUjVE1+JO9S+rrIeJQoRZAGjsLQKnkNvVgvAPerNrI6c
vE6OpKTYZgCZ6I1Qfid5AOfK5aS8Org8DTTe5NqYn8TBBEva4Yhpsh68V2G7I3aL
OyKs/uZAbPPsrUMl7Gwoyi+EXVGQGBRx8eYBqxp+ouwAK5/A68mM2C2oM2oj1mUE
pmH6r3qw8k4KbAq+jWMg43s0Aiuw/n3KP0fMU152La5J+ZNJFu1PpIQnUhNrFRgw
czfU/A==
=AF6W
-----END PGP MESSAGE-----
```

The body of the MIME `multipart/encrypted` entity consists of two parts (separated with the boundary string `=-=dwh+Lqq+2fjNia=-=`):

- The first part has a content type `application/pgp-encrypted` and comprises a `Version: 1` field. Since the PGP packet format contains all other information necessary for decrypting, no other information is required here.
- The second part has a content type `application/octet-stream` and comprises an OpenPGP message or file that is digitally signed and enveloped.

The second part must be decrypted with the appropriate private key (the respective key ID can be found at the beginning of the OpenPGP message or file). After decryption, the second part reads like this:

```
MIME-Version: 1.0
Content-Type: multipart/signed;
protocol="application/pgp-signature";
micalg=pgp-sha1       ;
boundary="=-=m4TBlc8/BNg13g=-="

--=-=m4TBlc8/BNg13g=-=
Content-Type: text/plain;
charset="us-ascii"
Content-Transfer-Encoding: 7bit

This is a test message.
```

```
--=-=m4TBlc8/BNg13g=-=
Content-Type: application/pgp-signature

-----BEGIN PGP SIGNATURE-----
Version: GnuPG v2.0.19 (MingW32)

iEYEABECAAYFAlDIQX4ACgkQjlC9swrCmlvcbgCgo3p8WESAA5aQPH2dXbQyZRwh
Ho8AnAkJfENZXDbEyS6BBc5sOONv7v8l
=m1eJ
-----END PGP SIGNATURE-----


--=-=m4TBlc8/BNg13g=-=--
```

Again, the body of the MIME `multipart/signed` entity consists of two parts (now separated with the boundary string `=-=m4TBlc8/BNg13g=-=`):

- The first part has a content type `text/plain` and comprises the message that is digitally signed ("This is a test message.");
- The second part has a content type of `application/pgp-signature` and comprises a digital signature for the message included in the first part.

In addition to the protocol parameter `application/pgp-signature`, the content type `multipart/signed` comes with a boundary parameter that specifies the string with which the message parts are separated, and a micalg parameter that specifies the cryptographic hash algorithm in use (SHA-1 in this example). If more than one cryptographic hash algorithm is used, then a comma-separated list of hash-symbols that identify the algorithms can also be provided.

5.2.5 Cryptographic Algorithms

As mentioned earlier, the first versions of PGP only employed MD5, IDEA, and RSA. This has changed, and the current specification of OpenPGP (i.e., [7] and some complementary RFC documents) support more advanced cryptographic hash, symmetric encryption, public key, and compression algorithms. Furthermore, the ongoing revision of [7] (in the aftermath of EFAIL and related attacks) will further strengthen these algorithms.

5.2.5.1 Cryptographic Hash Algorithms

The cryptographic hash algorithms and respective IDs specified in [7] are summarized in Table 5.2. As mentioned above, the historically most important algorithm

was MD5, but it is nowadays known to be insecure and has therefore been deprecated. Instead, OpenPGP implementations must now support SHA-1, and they may also support algorithms from the SHA-2 family. The Internet-Draft that is going to replace [7] even mandates SHA-256 and additionally provides ID values for 256-bit SHA-3 (ID value 12) and 512-bit SHA-3 (ID value 14). RIPEMD and RIPEMD-160 are European analogs of MD5 and SHA-1 that are not widely used in the f eld—except RIPEMD-160 that is routinely used in Bitcoin.

Table 5.2
Cryptographic Hash Algorithms and IDs (According to [7])

ID	Algorithm
1	MD5
2	SHA-1
3	RIPEMD/RIPEMD-160
4 to 7	Reserved
8	SHA-256
9	SHA-384
10	SHA-512
11	SHA-224
100 to 110	Private/experimental algorithm

5.2.5.2 Symmetric Encryption Algorithms

As of this writing, there are many supposedly secure symmetric encryption algorithms to choose from. The algorithms and respective IDs specif ed in [7] are itemized in Table 5.3. Historically, the most important algorithm was IDEA, but this has changed and OpenPGP implementations must now implement 3DES and they should implement AES-128 and CAST5—also known as CAST-128 and specif ed in [13]. Needless to say, OpenPGP implementations are free to implement any other algorithm at will. Examples include Blowf sh (with algorithm ID 4) andTwof sh (with algorithm ID 10), as well as Camellia [14] with key lengths 128, 192, and 256 bits [15]. According to this RFC document, the IDs reserved for these algorithms are 11, 12, and 13. These IDs are not included in Table 5.3, but they are def ned in the Internet-Draft that is going to replace [7]. This will make [15] obsolete.

All encryption algorithms currently specif ed for OpenPGP are block ciphers that operate in a special variant of cipher feedback (CFB) mode. In particular, the variant provides a feature known as quick check. It yields a possibility to determine at the beginning of a (possibly lengthy) decryption operation whether the key in use is correct—otherwise valuable computing cycles may be wasted. Normal CFB does not provide such a possibility, and hence the developers of some early PGP versions

Table 5.3
Symmetric Encryption Algorithms and IDs (According to [7])

ID	Algorithm	Key length
0	Plaintext or unencrypted data	—
1	IDEA	128 bits
2	3DES	168 bits
3	CAST5	128 bits
4	Blowf sh	128 bits
5	Reserved	—
6	Reserved	—
7	AES-128	128 bits
8	AES-192	192 bits
9	AES-256	256 bits
10	Twof sh	256 bits
100 to 110	Private/experimental algorithm	—

came up with this variant that is also known as PGP CFB or OpenPGP CFB mode. OpenPGP CFB mode works with any block cipher of block length b (counted in bytes) and a CFB shift of the same size. Typically, b is either 8 as in the case of IDEA, 3DES, and CAST5, or 16 as in the case of Blowf sh, Twof sh, and the three off cial versions of AES.

Before we can outline OpenPGP CFB mode, we have to brief y introduce normal CFB mode. This mode of operation turns a block cipher into a stream cipher (i.e., it uses the block cipher to generate a sequence of pseudorandom bits, and these bits are then added modulo 2 to the plaintext message bits to generate the ciphertext). The resulting stream cipher is self-synchronizing, meaning that the receiver can automatically synchronize itself with the keystream generator after having received a certain number of bits.[23] The working principle of normal CFB mode is illustrated in Figure 5.2.

The encrypting and decrypting devices employ two feedback registers (i.e., an input register I and an output register O). The input registers are initialized with an initialization vector (IV) on either side of the communication channel (i.e., $I_0 = IV$). In each step i ($1 \leq i \leq t$), the encrypting device encrypts the input register I_i with the key k using the underlying block cipher, and the result is written to the output register O_i. The r leftmost and most signif cant bits of O_i are then

23 From a bird's eye perspective, there are two types of stream ciphers: In a *synchronous* stream cipher a stream of pseudorandom bits is generated independently from the plaintext message and ciphertext, whereas in an *asynchronous* or *self-synchronizing* stream cipher several of the previously generated ciphertext bits are reused to compute the keystream. This has the advantage that the receiver can automatically synchronize itself with the keystream generator after having received a certain number of bits, making it easier to recover if bits are dropped or added to the message stream.

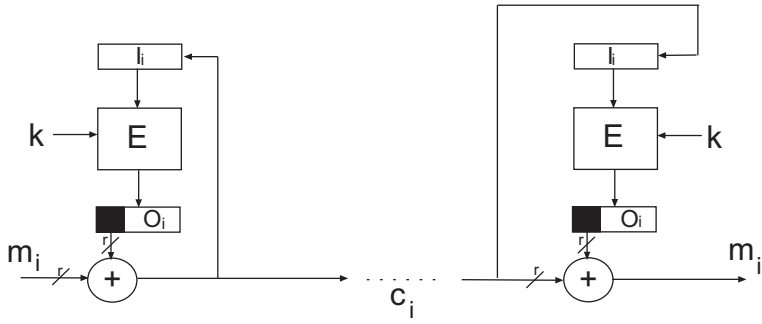


Figure 5.2 The working principle of normal CFB mode.

added modulo 2 to the next r-bit plaintext message block m_i. In theory, the shift r can be arbitrary, but in practice, it is usually set to 1 bit, 1 byte (8 bits), or b bytes ($b \cdot 8$ bits). In the case of OpenPGP CFB, we already said that r comprises all $b \cdot 8$ bits of a block (i.e., the CFB shift is the same as the block length) and hence all bits from the output register are used for encryption. The resulting construction is sometimes called b-byte CFB mode or CFB-$(8b)$ in short. Mathematically expressed, I_i refers to c_{i-1} (in CFB-$(8b)$) and O_i refers to $E_k(I_i) = E_k(c_{i-1})$, and this, in turn, means that CFB-$(8b)$ encryption can be formally expressed as follows:

$$\begin{aligned} c_0 &= IV \\ c_i &= m_i \oplus E_k(c_{i-1}) \text{ for } i > 0 \end{aligned}$$

Similarly, CFB-$(8b)$ decryption can be formally expressed as follows:

$$\begin{aligned} c_0 &= IV \\ m_i &= c_i \oplus E_k(c_{i-1}) \text{ for } i > 0 \end{aligned}$$

Note that m_i and c_i refer to b-byte blocks (where $i > 0$). Also note that OpenPGP CFB mode is very similar to normal CFB mode, and that there are only two subtle differences:

- First, the input register is initialized with an IV that consists of all zeros (instead of a random IV).
- Second, the OpenPGP CFB mode additionally employs a $(b+2)$-byte random string r. The f rst b bytes of r form a block and are randomly selected, whereas

the next 2 bytes are just copies of the last two bytes. So, if $b = 8$ and the first 8 bytes refer to $r_1 r_2 r_3 r_4 r_5 r_6 r_7 r_8$, then r equals $r_1 r_2 r_3 r_4 r_5 r_6 r_7 r_8 r_7 r_8$. Similarly, if $b = 16$ and the first 16 bytes refer to $r_1 \ldots r_{16}$, then r equals $r_1 \ldots r_{15} r_{16} r_{15} r_{16}$. In the general case, the first b bytes refer to $r_1 \ldots r_b$ and r equals $r_1 \ldots r_{b-1} r_b r_{b-1} r_b$. In either case, the string r is prepended to original plaintext message before encryption. If $m = m_1 \ldots m_{nb}$ refers to the original plaintext message that consists of n blocks and nb bytes, then the message that is going to be encrypted is the concatenation of r and m:

$$r \| m = \underbrace{r_1 \ldots r_{b-1} r_b r_{b-1} r_b}_{r} \underbrace{m_1 \ldots m_{nb}}_{m}$$

In this notation, each r_i ($1 \leq i \leq b$) and each m_j ($1 \leq j \leq nb$) refers to a single byte (not a block).

Using r and $c_0 = IV$, OpenPGP CFB mode encrypts m in a sequence of $n+2$ blocks $c_1, \ldots c_{n+2}$ as follows:

$$\begin{aligned}
c_1 &= E_k(c_0) \oplus r \\
c_2 &= E_k(c_1)|_{1,2} \oplus r_{b-1,b} \\
c_3 &= E_k(E_k(c_1)|^{b-2} \| c_2) \oplus m_1 \\
c_4 &= E_k(c_3) \oplus m_2 \\
&\ldots \\
c_i &= E_k(c_{i-1}) \oplus m_{i-2} \\
&\ldots \\
c_{n+2} &= E_k(c_{n+1}) \oplus m_n
\end{aligned}$$

In the first step, c_1 is computed as the CFB encryption of the first block of r. The second step is a little bit special, because only the two leftmost bytes 1 and 2 of $E_k(c_1)$, denoted as $E_k(c_1)|_{1,2}$, are used to encrypt r_{b-1} and r_b. The result is c_2, and this block is only 2 bytes long (instead of b bytes). The remaining (rightmost) $b-2$ bytes of $E_k(c_1)$, denoted as $E_k(c_1)|^{b-2}$, are used in the third step to compute c_3: $E_k(c_1)|^{b-2}$ is concatenated with the two-byte block c_2 before encryption, and the result is used to encrypt m_1. For all subsequent blocks, the encryption follows normal CFB mode, and the final ciphertext block is c_{n+2}. On the recipient's side, decryption works similarly, and the recipient can verify whether the two bytes r_{b-1} and r_b repeat. If they do, then it is likely that the key in use is correct. Otherwise, the

key is not correct and something has gone wrong. This is what the quick check is all about: It is to verify the correctness of the key in use.

As is usually the case if a technology deviates from a standard, the usefulness and security of the OpenPGP CFB mode and its quick check has been discussed controversially within the community. In 2005, for example, Serge Mister and Robert Zuccherato published a research paper in which they described an adaptive chosen-ciphertext attack (CCA2) against OpenPGP CFB mode that, in most circumstance, allows an adversary to determine 2 bytes of a plaintext message block with about 2^{15} oracle queries [16]. This is not something that can be done interactively, so the attack may threaten some backend servers only. Also, the resulting determination of 2 bytes is not a decryption of the entire message, so the practical usefulness and severity of the attack remains vague. In the end, the IETF OpenPGP WG decided that the advantages of the OpenPGP CFB mode overweight its disadvantages and that one does not need to ban the quick check and fall back to normal CFB in future releases of the OpenPGP specif cation. Depending on the security stance, this can be seen as a wise decision or not. In either case, the attack of Mister and Zuccherato must be taken into account.

Until [6], OpenPGP encryption was entirely unauthenticated, meaning that it was impossible for the recipient of an encrypted message to verify its authenticity and integrity. This was changed in [7], when a simple *modif cation detection code* (MDC) mechanism was added to message encryption. In fact, a simple SHA-1 hash value would be computed from the message before it is encrypted. This simple MDC mechanism does not meet the requirements of today's cryptography, but it is at least a f rst try—it is sometimes called weakly authenticated encryption. Needless to say, modern AEAD ciphers do a better job in combining message encryption and authentication.

EFAIL and related attacks have clearly shown that unauthenticated or weakly authenticated encryption is dangerous, and that AEAD algorithms are advantageous. In the Internet-Draft that is to replace [7], there are several AEAD algorithms to choose from. Instead of CCM and GCM that are used in many other Internet security protocols, the Internet-Draft requires implementations to support EAX[24] and optionally OCB [17]. While EAX is patent-free, the current situation with OCB is less clear. But from a cryptographic viewpoint, EAX and OCB are certainly very good choices.

24 https://csrc.nist.gov/csrc/media/projects/block-cipher-techniques/documents/bcm/proposed-modes/eax/eax-spec.pdf

5.2.5.3 Public Key Algorithms

In the realm of OpenPGP, public key algorithms are used for asymmetric encryption and digital signatures. The respective algorithm IDs are itemized in Table 5.4. Historically, the most important algorithm was RSA, but this has changed meanwhile.

Table 5.4
Public Key Algorithms and IDs (According to [7])

ID	Algorithm
1	RSA (Encrypt or Sign)
2	RSA Encrypt-Only
3	RSA Sign-Only
16	ElGamal (Encrypt-Only)
17	DSA
18	Reserved for Elliptic Curve
19	Reserved for ECDSA
20	Reserved
21	Reserved for Diff e-Hellman (X9.42, as def ned for IETF-S/MIME)
100 to 110	Private/experimental algorithm

Today, OpenPGP implementations must implement Elgamal for asymmetric encryption and DSA for digital signatures (IDs 16 and 17), and they should continue to support RSA for the sake of backwards-compatibility. Also, OpenPGP implementations may implement any other public key algorithm at will, such as, for example, the ECC algorithms provided in RFC 6637 [18] that are not included in Table 6.4. Again, the content of [18] is included in the Internet-Draft that is to replace [7], and this, in turn, is going to make [18] obsolete. Hence, the elliptic curves that are going to be supported comprise the NIST curves P-256, P-384, and P-521, two Brainpool curves, and Curve25519 (as used in Ed25519). Hence, ECC is going to play a more important role in future implementations of OpenPGP, and this is in line with the general trend in industry.

5.2.5.4 Compression Algorithms

The compression algorithms and respective IDs specif ed in [7] are summarized in Table 5.5. OpenPGP implementations must always implement uncompressed data and should implement an algorithm specif ed in [19]. This algorithm is usually called DEFLATE, but it is sometimes also called ZIP. It uses a combination of the Lempel-Ziv 77 (LZ77) algorithm (that was proposed by Abraham Lempel and Jacob Ziv in

1977 [20]) and Huffman coding. In addition to ZIP, OpenPGP implementations are free to implement any other algorithm, like ZLIB[25] [21] or BZip2.[26]

Table 5.5
Compression Algorithm and IDs (According to [7])

ID	Algorithm
0	Uncompressed
1	ZIP
2	ZLIB
3	BZip2
100 to 110	Private/experimental algorithm

5.2.6 Message Processing

We now have a closer look at the procedures that are used to digitally sign, compress, encrypt, and transfer encode OpenPGP messages. Figure 5.3 illustrates the situation: The message at the top is subject to the respective procedures to create a message that can be transmitted (these procedures are summarized on the left side). On the receiving side, the transmitted message is decoded, decrypted, and decompressed, and the digital signature is f nally verif ed (the respective procedures are summarized on the right side). Note that on either side of the transmission, the order in which the procedures are applied matters.

In what follows, we use the term sender to refer to the software that is used on the sending side, and the term recipient to refer to the software that is used on the receiving side. Note that neither the sender nor the recipient are human beings.

5.2.6.1 Digital Signatures

In general, the use of digital signatures requires at least one cryptographic hash algorithm and one asymmetric encryption algorithm (that can be used to digitally sign and verify messages). Possible algorithms are summarized in Tables 5.2 and 5.4. Prior versions of PGP mandated the use of MD5 and RSA, whereas current versions prefer SHA-1 and DSA.

On the sender's side, the procedure to digitally sign a message (in canonical form) includes the following three steps:

25 https://zlib.net.
26 http://www.bzip.org.

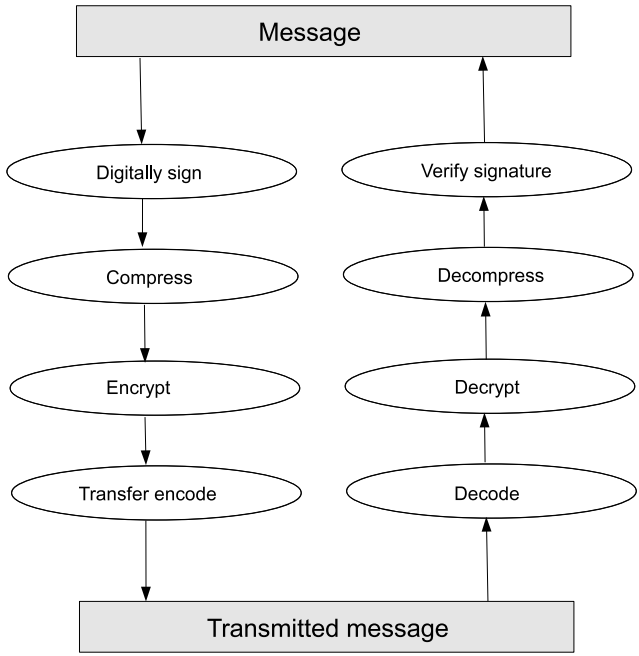


Figure 5.3 OpenPGP message processing.

- First, the sender applies a cryptographic hash algorithm (e.g., SHA-1) to the message to generate a message digest;
- Second, the sender applies an asymmetric encryption algorithm suitable for digital signatures (e.g., DSA) to the message digest to generate a digital signature;
- Third, the sender prepends the digital signature to the message.

The resulting message comprises a signature and a message part (again, you may refer to Figure 5.1 for a graphical representation of this construction). As such, it is transmitted to the recipient(s). Each recipient can, in turn, verify the digital signature using the sender's public key. It may already have this key at hand (i.e., within its public key ring), or it may be able to retrieve it from a key server. We revisit the notion of a key server towards the end of the chapter.

Although digital signatures are usually prepended to the message, this is not always the case. In fact, OpenPGP supports the notion of a detached signature. Such a signature may be stored, processed, and transmitted separately from the message that is signed. There are many applications for detached signatures, such as having multiple parties sign the same document,[27] storing signatures in a log, or using signatures to ensure the integrity of program code before it is being executed.

5.2.6.2 Data Compression

By default, OpenPGP compresses a message after prepending a digital signature but before encryption. There are at least two reasons why a signature should be generated before the message is compressed.

- First, it is preferable to sign an uncompressed message so that one can store only the uncompressed message together with the digital signature for verif cation and future use. Contrary to that, if one signed a compressed message, then it would be necessary to recompress the original message for signature verif cation (or to additionally store the compressed message, as well).
- Second, it is preferable to let the user choose among several compression algorithms. This can only be done if compression is done after signature generation. If one wanted to generate the signature after compression, then one would have to require that all implementations use the same compression algorithm. This is usually hard to achieve in the f eld.

It therefore makes a lot of sense to compress a message after prepending a digital signature. On the other hand, it is important to apply compression before encryption, since encrypted data cannot be compressed anymore—at least if the encryption algorithm is cryptographically strong. Furthermore, people believe that applying compression before encryption strengthens the security of the encryption because the compressed message has less redundancy than the uncompressed message, and

27 If detached signatures were not possible, then the signatures would have to be nested, with the second signer signing both the original document and the signature of the f rst signer (in a setting with two signers). This is not the same as having two signatures that are equally valid and do not depend on each other.

hence cryptanalysis is made more diff cult. Note, however, that some recent attacks against SSL/TLS have put this folklore wisdom into question.[28]

5.2.6.3 Data Encryption

OpenPGP supports two data encryption methods to provide data conf dentiality: public key encryption and secret key encryption. A user can decide on a case-by-case basis which method he or she wants to use.

Public Key Encryption

With public key encryption, a message is encrypted in a digital envelope. This means that the sender performs the following steps:

- First, the sender—randomly or pseudorandomly—generates a session key for the message.
- Second, the message is encrypted with this session key. As mentioned earlier, various versions of OpenPGP use different symmetric encryption algorithms (see Table 5.3). In former times, message encryption was done with a block cipher in OpenPGP CFB mode. This is about to change, and AEAD modes are replacing OpenPGP CFB more and more.
- Third, the session key is encrypted with each recipient's public key.[29] The resulting encrypted session keys are prepended to the (now encrypted) message.

Consequently, the resulting message comprises a session key part (for each recipient) and a message part. Again, you may refer to Figure 5.1 for a respective illustration. The resulting ciphertext represents the message that is being transmitted to the recipient(s).

On the recipient's side, the procedure to open the digital envelope and decrypt the message includes two steps:

- First, the recipient extracts and decrypts—with its own private key—the encrypted session key from the session key part of the message;

28 The attacks are based on theoretical work that was published in 2002 [22]. The attacks themselves, however, are more recent: The Compression Ratio Infoleak Made Easy (CRIME) attack was published in 2012, and the Timing Info-leak Made Easy (TIME) and Browser Reconnaissance and Exf ltration via Adaptive Compression of Hypertext (BREACH) attacks were both published in 2013. The attacks are explained, for example, in [23].

29 The session key may also be encrypted with an ADK, if such a key is conf gured for message recovery.

- Second, the recipient decrypts the message with the now-decrypted session key.

Needless to say, this procedure must be performed by every recipient individually.

Secret Key Encryption

With secret key encryption, there are two possibilities to—either directly or indirectly—encrypt a message:

- The message may be encrypted with a secret key derived from a passphrase or any other shared secret (direct encryption).
- The message may be encrypted in a two-stage procedure similar to public key encryption described above. In this case, the randomly or pseudorandomly generated session key is symmetrically encrypted with a key derived from a passphrase or another shared secret (indirect encryption).

In either case, there must be a way to derive a key from a passphrase. OpenPGP supports several hand-crafted and nonstandard string-to-key specifiers and respective types that serve this purpose.

A practical problem occurs if the recipient of an encrypted message is not able to decrypt the message, simply because he or she does not have an OpenPGP-compliant software at hand. For this situation, some implementations support the notion of a *self-decrypting archive* (SDA). As its name suggests, an SDA is an OpenPGP archive (possibly comprising multiple files) that includes and carries with it the program code needed to decrypt it. If a user double-clicks on such an SDA, then the decryption code is invoked and the user is asked to enter the passphrase (that needs to be exchanged out-of-band between the sender and the recipient). The passphrase is turned into a secret key, and the secret key is then used to decrypt the SDA. There are many situations in which the use of an SDA is advantageous.

5.2.6.4 Transfer Encoding

OpenPGP uses cryptography and generates arbitrary data that may be represented as a sequence of 8-bit bytes. Because many message transfer systems are able to only transfer 7-bit data (e.g., ASCII characters) one has to encode 8-bit data for transfer. This is usually done by converting 8-bit data into a set of universally transferable characters, as provided, for example, by the *base-64* encoding scheme. The term base-64 originates from a specific MIME content transfer encoding, where each

digit represents exactly 6 bits of data. Three 8-bit bytes (i.e., a total of 24 bits) can therefore be represented by four 6-bit base-64 digits.

OpenPGP employs a variant of base-64 known as *radix-64*. It is identical to base-64, with the addition of an optional 24-bit cyclic redundancy check (CRC). The CRC sum is calculated on the data before encoding; it is then encoded with the same base-64 encoding scheme, pref xed by the = symbol as a separator, and appended to the encoded data. In OpenPGP parlance, the result is known as an ASCII armor. Radix-64 encoding and ASCII armors are unique for OpenPGP.

When OpenPGP encodes data into an ASCII armor, it puts specif c headers around the data, so that it can be reconstructed at some later point in time. In essence, an ASCII armor contains the following items (in concatenated form):

- An ASCII armor headerline (appropriate for the data type);
- ASCII armor headers;
- A blank line;
- The ASCII-armored data;
- An ASCII armor checksum (representing a 24-bit CRC as mentioned above);
- An ASCII armor trail (depending on the headerline).

An ASCII armor headerline consists of the appropriate headerline text and f ve dashes on either side of the headerline text. The headerline text, in turn, is chosen based on the type of data that is being armored and how it is being armored. Headerline texts include the following strings:

- `BEGIN PGP MESSAGE` is used for digitally signed, encrypted, or compressed f les;
- `BEGIN PGP MESSAGE, PART X/Y` is used for multipart messages, where the ASCII armor is split into Y f les, and the current part comprises the Xth f le out of Y;
- `BEGIN PGP PUBLIC KEY BLOCK` is used for transferring public keys.

The second option—namely to split a message into multiple pieces—is required because some e-mail systems or system components are restricted to a maximum message length (e.g., 50 KB). Any message longer than that must be broken up into smaller pieces, each of which is sent separately and independently. To accommodate this restriction, OpenPGP may automatically subdivide a message that is too large into segments that are small enough to be delivered by the e-mail system. In fact, the segmentation is done after all other processing, including the radix-64

encoding. Consequently, the encrypted session key(s) and digital signature(s) appear only once, at the beginning of the f rst segment. It is up to the recipient to strip off all header information and to reassemble the entire block before performing all other operations.

Similar to normal message headers, the ASCII armor headers are pairs of strings that can give the recipient information about how to decode or use the message. The headers are a part of the armor, not a part of the message. Consequently, they should not be used to convey any important information, since they can change in transit. We saw `Version` and `Comment` ASCII armor headers as examples earlier in this chapter.

The ASCII armor trail is composed in the same manner as the ASCII armor headerline, except that `BEGIN` is replaced with `END`.

5.2.7 Key Management

OpenPGP employs many cryptographic keys, such as (one-time) session keys, passphrase-based encryption keys, and public key pairs. All of these keys have specif c requirements with regard to their proper generation and management.

5.2.7.1 Session Keys

As mentioned in the beginning of this chapter, the term *session key* refers to a secret key (i.e., a key from a secret key cryptosystem) that can be used to encrypt and digitally envelope a message. The most important requirement for such a key is that it is generated in a way that is unpredictable for an outsider, meaning that it looks like being randomly generated—whereas in reality it is only pseudorandomly generated. So, the generation of session keys depends on a particular implementation and respective PRG. For example, many OpenPGP implementations measure the content and relative timing of user keystrokes to generate a random number that is to seed a PRG based on X9.17 [24]—typically using CAST-128 instead of 3DES. The output of the respective PRG yields as many session keys as required by the application. Needless to say, there are other sources of randomness (to generate a seed) and other PRGs that can be used instead of X9.17.

5.2.7.2 Passphrase-Based Encryption Keys

Like (one-time) session keys, passphrase-based encryption keys are secret keys (i.e., keys from a secret key cryptosystem) that are used to encrypt data. However, unlike session keys, passphrase-based encryption keys are not used to encrypt and digitally

envelope messages, but rather to encrypt and hence protect the private key of a particular user.

The cryptographic strength of a passphrase-based encryption key depends on the quality of the passphrase from which it is derived. If the passphrase is easy to guess, then the cryptographic strength of the passphrase is poor. Otherwise (i.e., if the passphrase is not easy to guess), then the cryptographic strength of the passphrase may be good. So, from a user's perspective, the security requirements for a passphrase are very similar to the requirements for a password: the respective value (passphrase or password) should be as involved as possible—so that it cannot be guessed or found in a dictionary attack), but not too involved (because the user has to type it in repeatedly). The popular and often asked question of whether a password or a passphrase is better from a security viewpoint is highly irrelevant, as there are good and bad choices for passwords, as well as good and bad choices for passphrases. In either case, the security depends on the actual choice and there is no general rule of thumb that applies here.

In the past, security professionals have often recommended that users should choose distinct passwords or passphrases for all purposes, and that they should never write them down. However, this recommendation is wishful thinking and mostly illusory, as users have to select and remember too many passwords and passphrases that they are not able to memorize all of them. So, it is certainly better and more realistic to enable users to write them down, but to equip them with tools that allow them to transparently encrypt and decrypt the respective values. There are many such tools available in the f eld, such as KeePass.[30] In a professional setting, it is certainly better to distribute and encourage the use of KeePass (or a similar tool) than it is to prohibit and outlaw the practice of writing down passwords and passphrases.

5.2.7.3 Public and Private Keys

Since OpenPGP makes use of public key cryptography, there are public and private keys that need to be generated, stored, and managed in a secure way. According to [1], the

> "... whole business of protecting public keys from tampering is the single most diff cult problem in practical public key applications. It is the Achilles' heel of public key cryptography, and a lot of software complexity is tied up in solving this one problem."

This quote hits the point and there is not much to add. The security of an OpenPGP implementation mainly depends on the implementation, as well as the way the public

30 http://keepass.info.

key pairs are generated and the private keys are stored and managed (hopefully in a secure way).

OpenPGP provides a pair of data structures for each user: one to store his public key pairs and one to store the public keys of other users. In OpenPGP parlance, these data structures are called *private keyring* and *public keyring*. From a security perspective, the private keyring is the one that needs to be protected as strongly as possible. If an adversary manages to either read or modify the private keyring, then the security of OpenPGP is compromised. This point will be revisited in Section 5.4.

5.3 WEB OF TRUST

We mentioned previously that OpenPGP uses a unique way to manage public keys and public key certif cates. Remember that a trust model refers to the set of rules that a system or application uses to decide whether a public key certif cate is valid, and that the trust model employed by OpenPGP has historically been called web of trust. It is addressed in this section. We start the discussion with keyrings, before we delve more deeply into trust establishment, key revocation, and key servers.

5.3.1 Keyrings

As mentioned above, OpenPGP employs a *private keyring* to store the public key pairs of a user (including the respective private keys) and a *public keyring* to store the public keys of all other users. In a typical setting, the public keyring is called `pubring.pkr` and the private keyring is called `secring.skr`. In either case, keyring entries are indexed with user IDs or key IDs.

There are many tools and utilities that can be used to manage keyrings. In Gpg4win, for example, there is a certif cate management tool called Kleopatra that can be used to manage keyrings. Each tool or utility has its own GUI, but there is no need to explain and discuss any particular GUI in this book. You may refer to the respective user manual to get more information about this issue.

An open issue in the design of a certif cate management GUI is whether photographs are useful or not. In fact (and as mentioned earlier), the usefulness of including photographs in OpenPGP certif cates is discussed controversially in the community. The developers of Kleopatra, for example, have opted to exclude photographs and not support them, mainly for the following two reasons:

- First, photographs give a false feeling of security, as anybody can include a photograph of his or her choice in a certif cate;

- Second, photographs unnecessarily increase the size of a certif cate.

Both reasons are meaningful, but the second reason heavily depends on the application environment in which OpenPGP is used. If, for example, OpenPGP is used for secure e-mail, then the certif cate size hardly matters. But there are other environments in which the size of a certif cate not only matters but is key to the successful deployment of an application.

As the private keyring holds private keying material, it needs to be protected as strongly as possible. Typically, it is symmetrically encrypted with a key that is derived from a passphrase. Each time the user wants to access his or her private keyring and employ one of his or her private keys, he or she must type in or otherwise provide the correct passphrase. In many implementations, it is possible to cache the passphrase for a conf gurable amount of time (e.g., a few seconds or minutes). Again, it is questionable and controversally discussed in the community whether passphrase caching is a good idea, and if it is, for how long.

To provide a higher level of security, some OpenPGP implementations provide support for secret sharing [25, 26]. Using such a scheme, a private key can be split into multiple parts or shares such that the reconstruction of the private key (to decrypt or digitally sign data) requires at least a certain number of shares. Typically, the user can specify an arbitrary number of shareholders and def ne a threshold on how many shares must be provided to reconstruct the private key. The use of a secret sharing scheme to recover secret or private keys is useful and highly recommended for any system that employs cryptographic keys. This also applies to OpenPGP, and many implementations support it.

In addition to the information mentioned so far, each entry in a keyring is assigned a key legitimacy (KEYLEGIT) f eld, a signature trust (SIGTRUST) f eld, and an owner trust (OWNERTRUST) f eld. These f elds are internally used to determine the trustworthiness of signatures attached to user IDs, and hence to determine the legitimacy of public keys and OpenPGP certif cates. The respective mechanisms to establish trust are outlined next.

5.3.2 Trust Establishment

If A wants to securely communicate with B, then A must have an authentic copy of B's public pk_B—otherwise, many types of MITM attacks become feasible: If an adversary C manages to make A believe that its public key pk_C belongs to B, then C can decrypt all messages sent from A to B (or forge valid-looking signatures for B). So, when using B's public key, A must be sure that pk_B is indeed authentic.

Traditionally, there have been three approaches and respective trust models to achieve this:[31]

1. A can get a copy of pk_B directly from B. In the terminology introduced in Section 3.3.1, this approach uses a direct trust model.
2. A can get a digitally signed copy of k_B from a commonly trusted party that represents a CA. In the terminology introduced in Section 3.3.1, this approach uses a hierarchical trust model.
3. A can get a digitally signed copy of k_B from any party it trusts. In the terminology of OpenPGP, such a party is called an *introducer*, and hence A must get k_B from one or several of its introducers. In the terminology introduced in Section 3.3.1, this approach uses a cumulative trust model.

The first approach has scalability problems, whereas the second approach has to deal with the problem that there is generally no commonly trusted party to start with. Following the third approach, OpenPGP originally employed a cumulative trust model called a web of trust to establish trust without CAs. This has changed, and many OpenPGP implementations nowadays also provide support for X.509 certificates, CAs, and respective PKIs.

To better understand the cumulative trust model and the web of trust, it is important to note that trust is not transitive, and hence may not always be transferable. What this basically means is that if A trusts B and B trusts C, then this does not necessarily mean that A also trusts C. This also applies to user authentication, and it means that you may trust a friend to reliably authenticate the owners of public keys, but you may not necessarily trust the ones that have been authenticated by your friend to be comparably reliable. Put in other words: your friend's friends are not necessarily your own friends. In reality, we are accustomed to the limited transferability of trust, and the cumulative trust model adheres to this limitation in the digital world.

In OpenPGP, a public key is validated by answering the following two questions in the affirmative:

1. Is the public key properly signed (and hence certified)?

31 As we will see later in the book, there is even a fourth approach in frequent use today. This approach is a variant of the first approach and the direct trust model: Instead of getting pk_B directly from B, A can get pk_B from anybody. When using pk_B, however, A has to make sure that the key really belongs to B. In the simplest case, A may call B and have him or her spell pk_B or a hash value therefore. If A is able to recognize B's voice, then he or she may also be sure that the key is authentic. Instead of spelling keys or hash values, modern messengers and messenger apps may also encode the information in a QR code and have the devices compare the codes.

2. Can the user who signed (and hence certif ed) the public key be trusted to certify other people's public keys? Alternatively speaking, is this user a valid introducer?

While the f rst question can be answered automatically (if enough information is available), there is no means to automatically answer the second question. This question involves trust and must be decided by each user individually. The use of off cial CAs seemingly solves the problem, but it only moves the problem to the question of how to decide whether a given CA can be trusted in the f rst place. Again, we come to the situation in which the user must decide whether a source of certif cates is trustworthy from his or her individual viewpoint.[32] To do so, an OpenPGP user can designate a key holder as *unknown*, *untrusted*, *marginally trusted*, or *completely trusted* with regard to the certif cation of other users' public keys (we will answer the question of how to designate a trust level to a particular key holder further below). Having assigned these trust levels to key holders, an OpenPGP certif cate is typically considered to be valid if at least one of the following two conditions hold:

- The certif cate is digitally signed by at least one completely trusted key holder whose certif cate is valid;
- The certif cate is digitally signed by at least two marginally trusted key holders whose certif cates are valid.

Consequently, if a certif cate is digitally signed by an unknown or untrusted key holder, then it is not considered to be valid at all. This makes perfect sense, since—if we do not know or do not trust a key holder—we cannot say anything about the trustworthiness of the certif cates he or she digitally signs and issues. As a result of this trust assignment procedure and certif cate validation scheme, each user establishes his or her own web of trust, and there is no notion of a globally trusted party here. As mentioned above, this approach contrasts sharply with other standards-based public key management schemes, such as the ones employed by S/MIME, which are based on a centralized or hierarchical notion of trust. In fact, the standards-based public key management schemes all rely on CAs that collectively decide who users should trust.

The implementation of the cumulative trust model or web of trust is typically done by requiring three additional f elds that are associated with the entries in the keyrings (we have introduced the names of the f elds earlier).

32 Note that many Internet software packages (e.g., Web browsers) are distributed with lists of preconf gured CAs that are considered to be trustworthy by default. In this case, the user does not have to decide whether a CA is trustworthy from his or her point of view, because the software vendor has already decided on his or her behalf.

- First, each key is associated with a key holder that represents the owner of the key and a respective *owner trust* (OWNERTRUST) f eld. The value of this f eld refers to the degree to which the owner—and hence the key—is trusted by the user to sign other users' public keys (and hence to serve as an introducer). There are usually three possible values:
  - Complete trust (i.e., the owner and hence the key is completely trusted);
  - Marginal trust (i.e., the owner and hence the key is marginally trusted);
  - No trust (i.e., the owner and hence the key is not trusted).

  In addition to these values, the owner trust f eld of a key not included in a keyring is set to unknown (rather than untrusted). On the other side, if a user generates a public key pair (and his or her private keyring holds the respective private key), then the public key (pair) is completely trustworthy and the respective owner trust f eld value is set to complete trust.
- Second, each key is associated with zero or more signatures that the owner of the keyring has collected so far. Each signature, in turn, has associated with it a *signature trust* (SIGTRUST) f eld. The value of this f eld indicates the degree to which the user trusts the creator of the signature to certify public keys. This value is inherited from the owner trust f eld of the respective signer (e.g., complete trust, marginal trust, no trust, or unknown). So the signature trust f eld can also be thought of as cached copies of the owner trust f elds of the relevant keys.
- Third and most importantly, each key is associated with a *key legitimacy* (KEYLEGIT) f eld that indicates to what extent the user trusts that this key is valid and belongs to its claimed owner. This f eld is also known as the *validity f eld*, and there are usually three possible levels:
  - Valid;
  - Marginally valid;
  - Invalid.

  The value of the key legitimacy f eld is computed (and periodically recomputed) on the basis of the signature trust f eld values that have been collected for a particular key.

If user A introduces a new (public) key into his or her public keyring, then OpenPGP must assign a value for the respective owner trust f eld. If A generated the public key pair and owned the corresponding private key (meaning that the

private key is included in the private keyring), then a value of complete trust would automatically be assigned to the owner trust f eld. Otherwise, OpenPGP must ask the user for his assessment regarding the trust level of the owner of the key, and the user must select a desired value (i.e., untrusted, marginally trusted, or completely trusted). Also, one or more signatures may be attached to the public key (more signatures may be added later). For each of these signatures, OpenPGP searches through its public keyring to see whether the signer is among the known key holders.

- If the signer is among the known key holders, then the value of the signature trust f eld is set to the value of the respective owner's trust f eld;
- Contrary to that, if the signer is not among the known key holders, then the value of the signature trust f eld is set to unknown.

Finally, the value of the new key's legitimacy f eld is computed by OpenPGP on the basis of the signatures that are attached to it (or the values of the signature trust f elds, respectively). If at least one signature attached to the key is completely trusted (because the owner trust f eld of the corresponding key holder is completely trusted), then the value of the legitimacy f eld is set to valid. Otherwise, OpenPGP computes a weighted sum of the signature trust values. A weight of $1/X$ is given to signatures that are completely trusted and $1/Y$ to signatures that are marginally trusted, where X and Y are system parameters. In most implementations, $X = 1$ and $Y = 2$, but it should be noted that other parameters are possible, as well. When the total of weights of the public key reaches 1, the key is considered to be trustworthy, and hence the key legitimacy value is set to valid. So X signatures that are completely trusted or Y signatures that are marginally trusted or some combination thereof is needed to declare a key as valid. Most OpenPGP implementations periodically recompute the key legitimacy f eld for all keys found in the public keyring to achieve consistency.

There are many possibilities to visualize the OpenPGP trust model and the process of establishing trust in the resulting web of trust. For example, [27] introduces a graphical notation to illustrate the content of an OpenPGP public keyring and the way in which signature trust and key legitimacy are related.[33] Similarly, PathServer was an experimental Web-based service for authenticating OpenPGP public keys [28]. PathServer allowed a user to f nd certif cate paths from a key he or she trusts to a key he or she wants to learn about. The technical challenge was to allow the user to specify properties about the paths that are acceptable and desirable, such as independence and length properties. The problem of f nding paths that are in line with these properties is computationally hard. If OpenPGP (or OpenPGP's trust model, respectively) were deployed on a large scale, tools like PathServer would be very important for the usability, as they would allow users to visualize and better

33 The notation is credited to Philip R. Zimmermann.

understand the notion of trust with regard to the public keys and certif cates in current use. On the theoretical side, we already mentioned that the system parameters $X = 1$ and $Y = 2$ are somewhat arbitrary and that other values are equally f ne. So there is f exibility in OpenPGP's trust model and this f exibility has been explored in research (e.g., [29]). Also, some researchers have provided an abstraction of OpenPGP's trust model (e.g., [30, 31]). Both topics are not further addressed here. The same is true for the impact that social media have on the web of trust and the way trust is established therein. It goes without saying that the emerging use and deployment of social media offers new possibilities and challenges.

A f nal word is due to the relationship between OpenPGP's notion of an introducer (i.e., a completely trusted key holder) and a CA in a X.509-style PKI. In OpenPGP parlance, an introducer who is commonly trusted (i.e., trusted by all employees within an organization) is called a *trusted introducer*. The trusted introducer concept, in turn, can be used to model a hierarchical two-level X.509-style PKI. In this case, a trusted introducer acts as a CA for a large number of individual key holders. People trust the trusted introducer or CA to establish the validity for all certif cates. This means that everyone relies upon the trusted introducer or CA to go through the whole manual validation process for them. This is f ne up to a certain number of users or number of sites. Beyond that number, however, it is generally required to add other validators in order to maintain the same level of quality. This is where the concept of a *meta-introducer* comes into play. Similar to a king who hands his seal to his trusted advisors so they can act on his authority, the meta-introducer enables others to act as trusted introducers. These trusted introducers can validate keys to the same effect as that of the meta-introducer. They cannot, however, nominate and create new trusted introducers. The meta-introducer concept can be used to model a hierarchical three-level X.509-style PKI. In this case, the meta-introducers are located on the top, trusted introducers are located in the middle, and individual key holders are located at the bottom. Both concepts—trusted introducers and meta-introducers—are particularly helpful if OpenPGP-like webs of trust and X.509-like PKIs must be conf gured in a way to interoperate and complement each another. In reality, there is hardly any situation that requires more than three levels in a PKI hierarchy. Consequently, trusted introducers and meta-introducers seem to provide enough f exibility to model any practically relevant PKI structure.

5.3.3 Key Revocation

In theory, OpenPGP certif cates are created with a specif c validity period and lifetime (def ned by a start date and time and an optional expiration date and time), and each certif cate is expected to be usable only during its lifetime. In practice, however, this feature is seldom used and OpenPGP certif cates typically don't expire.

In either case, there may be situations in which it is necessary to invalidate a certif cate (prior to its expiration date, if such a date is specif ed in the f rst place). Most importantly, if a private key is compromised, then the respective certif cate needs to be invalidated as soon as possible. The process of invalidating a certif cate prior to its expiration date is known as *certif cate* or *key revocation* (we already addressed this issue in the realm of X.509 certif cates in Section 3.3.2.2). Note that a revoked certif cate is more dangerous than an expired certif cate because the fact that it has been revoked is not visible from the certif cate itself (the fact that a certif cate has expired is detectable by simply looking at its expiration date and time). It is commonly agreed that certif cate or key revocation is a particularly hard problem when it comes to the large-scale deployment of public key cryptography. For example, it has been argued that much of the implied cost savings of public key cryptography over secret key cryptography is nothing more than an illusion [32]. To further clarify this point, it is argued (in [32]) that the sum of the cost for cryptographic key issuance and the cost for cryptographic key revocation is almost no different for both public key cryptography and secret key cryptography. There seems to be some truth in this insight, especially when we consider the diff culty and pain we experience today when we try to establish fully operational PKIs that provide support for certif cate revocation. Most important and worrisome, any viable solution to address the certif cate or key revocation problem makes it mandatory to (re)introduce some online components for otherwise off ine CAs (in the realm of X.509 certif cates, these online components are the OCSP servers that are needed for the replacement of CRLs). These components have originally been thought to become obsolete due to the use of public key cryptography. This has not turned out to be true.

To make things worse, OpenPGP follows a decentralized and fully distributed approach with regard to certif cate and trust management, and this approach makes the key revocation problem even more diff cult to address. This is because there is no single authority that keeps track and may hold a list of recently revoked keys and corresponding certif cates (unless, of course, one uses and restricts oneself to a single key server). Also, there is a fundamental difference between revoking signatures in the X.509 world and revoking signatures in the OpenPGP world:

- With X.509 certif cates, a revoked signature is practically the same as a revoked certif cate, given the fact that the only signature on the certif cate is the one that made it valid in the f rst place (i.e., the signature of the CA). Consequently, only the issuer of an X.509 certif cate should be able to revoke it.
- Contrary to that, OpenPGP certif cates can be signed multiple times, and anyone who has signed a certif cate can also revoke his or her signature. A

revoked signature, in turn, indicates that the signer no longer believes the public key and user ID belong together, or believes that the certif cate's public key or the corresponding private key has been compromised. However, it is not an absolute statement about the validity of the certif cate.

In addition to the possibility of revoking single signatures, OpenPGP also provides a feature that allows a user to revoke his or her entire certif cate (not just the signatures that are assigned to it) if he or she feels that the certif cate has been compromised. Note, however, that only the certif cate's owner (the holder of the private key) or someone who the certif cate's owner has designated as a *revoker* can actually revoke an OpenPGP certif cate. There is no centralized party that can revoke certif cates for all users. This is again in sharp contrast to X.509, where a CA can revoke all the certif cates that it has issued. In OpenPGP, a user can revoke a certif cate by issuing a *key revocation certif cate* and disseminating it as widely as possible. A key revocation certif cate, in turn, is similar to a normal certif cate but includes an indicator that the purpose of this certif cate is to revoke the use of the public key. As such, it is digitally signed either by the certif cate owner or any of its designated revokers. Note that the ability to designate revokers is a very useful feature practice, as it is often the loss of the passphrase for the certif cate's private key that leads an OpenPGP user to revoke his or her certif cate. If the passphrase is not available, then the user can no longer issue a revocation certif cate himself or herself. In a corporate setting, it is therefore very important to organize who is the revoker for whom. In the simplest case, it is recommended to designate a revoker for all users. This can, for example, be an employee in the human resources department.

When a certif cate is revoked (by issuing a key revocation certif cate), it is important to make potential users of the certif cate aware of this fact (i.e., that the certif cate is no longer valid). With OpenPGP certif cates, the most common way to communicate that a certif cate has been revoked is to post it on a certif cate or key server so others may be warned not to use that particular public key anymore. The notion of a key server that can also be used to post key revocation certif cates is discussed next.

5.3.4 Key Servers

In public key cryptography, certif cates are typically issued by CAs and distributed by directory services. This works perfectly f ne in a hierarchical trust model. But in a cumulative trust model as used by OpenPGP, things are slightly more involved. Here, certif cates are issued by other users (instead of CAs) and distributed by so-called *key servers* (instead of directory services). So the aim of a key server is to make OpenPGP certif cates publicly available.

There is a long history in developing and providing OpenPGP key servers to the public, starting in the mid-1990s with a thesis of Marc Horowitz.[34] Since then, many individuals and companies have developed software to run similar services on the Web. Most importantly, Symantec operates such a service as the PGP Global Directory,[35] but there are many alternatives running similar services on the Web. Most of these alternative services are running the Synchronizing Key Server (SKS) software[36] that implements a federated key server system. This applies, for example, to the MIT PGP Public Key Server[37] and the many servers in the SKS OpenPGP Key server pool.[38] The services all employ the *HTTP keyserver protocol* (HKP) that uses port 11371 and is specif ed in an Internet-Draft.[39]

A f nal word is due to the fact that the use of PGP key servers is not without controversy in the Internet community. For many individuals, the purpose of using cryptography is to obtain a higher level of privacy in personal interactions and relationships. It has been pointed out that allowing a public key to be uploaded to a key server when using a decentralized web of trust, like OpenPGP, may reveal information that an individual otherwise wishes to keep private. Since OpenPGP relies on signatures on an individual's public key to determine the authenticity of that key, potential relationships can be revealed simply by analyzing the signers of a given public key. This information can be turned into a social graph that may indicate who interacts with whom and who trusts whom. Such graphs may be very interesting and valuable for the intelligence community as a whole.

To make things worse, people have also exploited the federated nature of the SKS system to mount large-scale spam and DoS attacks. In fact, they have started to upload public keys with thousands of (useless) signatures that make the respective data sets incredibly large. The problem is that there is no upper limit for the number of signatures that may be attached to a public key, and that the keys and respective signatures are constantly synchronized among all servers. As an alternative, some developers have come up with (nonfederated) key servers, such as `keys.openpgp.org`, that do not support signatures attached to public keys (except for some self-signatures). This keeps the data sets reasonably sized, and hence partly mitigates the attacks. But it somewhat also restricts the usefulness of the trust establishment mechanism employed in OpenPGP's Web of Trust.

34 http://www.mit.edu/afs/net.mit.edu/project/pks/thesis/paper/thesis.html.
35 https://keyserver.pgp.com.
36 https://bitbucket.org/skskeyserver/sks-keyserver/wiki/Home.
37 http://pgp.mit.edu:11371.
38 http://sks-keyservers.net.
39 http://tools.ietf.org/html/draft-shaw-openpgp-hkp-00.

5.4 SECURITY ANALYSIS

When talking about the security of OpenPGP, one has to distinguish between the security of the OpenPGP specif cation and the security of specif c implementations. Furthermore, the cryptographic algorithms employed by OpenPGP can themselves also be attacked. In 2012, for example, a group of researchers mounted a large-scale attack against RSA by collecting huge amounts of public keys and computing pairwise greatest common divisors to f nd coincidentally common primes in the moduli.[40] Against all odds, the attack turned out to very successful and some of the RSA public keys also came from OpenPGP.

5.4.1 Specif cation

Since the early 1990s, people have been looking into the security of the PGP and—more recently—OpenPGP specif cation. In spite of this effort, people have found only a few shortcomings and vulnerabilities. Most of them are theoretically interesting, but not practically relevant (because they are not easily exploitable in practice). Also, OpenPGP is about cryptographically protecting messages. It is not about hiding the existence of messages; hence, OpenPGP does not care and does not protect against traff c analysis. This is a topic that falls within the realm of anonymous messaging that is not the main focus of this book.

There are basically two attacks (or classes of attacks) that have been found to work against the OpenPGP specif cation.

- In March 2001, Czech cryptographers Vlastimil Klíma and Tomáš Rosa published a paper in which they reported a problem in the OpenPGP private keyring format [33]. If an adversary is able to modify a private key (stored in the private keyring) in a specif c way and subsequently capture a message that is digitally signed with this key, then he or she is able to determine and compromise the key. This attack is sometimes also referred to as the Klíma-Rosa or ICZ attack.[41] Its impacts are devastating (as the private key is compromised), but it is not particularly easy to mount. Note that the adversary needs to have write access to the private keyring, and that he or she must then be able to modify the private key in an undetectable way. If the adversary has access to the private keyring, then it is generally much simpler to copy the keyring and install a keylogger to retrieve the user passphrase (that is additionally required to decrypt and unlock the keyring).

40 https://eprint.iacr.org/2012/064.pdf.

41 The term ICZ attack stems from the company ICZ (http://www.i.cz), which both cryptographers were aff liated with at the time of the publication.

- In June 2000, Jonathan Katz and Bruce Schneier published a paper in which they proposed a chosen ciphertext attack against several secure messaging protocols, such as PGP and S/MIME [34]. In 2002, Kahil Jallad joined Katz and Schneier to delve more deeply into the topic and implement a chosen ciphertext attack against some OpenPGP implementations [35]. In such an attack, the adversary modif es a ciphertext and sends it back to its sender. If the sender then returns the erroneously decrypted message to the adversary, then he or she acts as a decryption oracle that can be (mis)used to decrypt the original message. There are some subtleties that need to be considered when encryption and compression are combined, but the basic outline of the attack remains the same. The bottom line is that any OpenPGP implementation should be careful when it returns erroneously decrypted messages. In case of doubt, it should return an error message that signals a security problem.

The publication of the Klíma-Rosa and Katz-Schneier-Jallad attacks received a lot of attention in the international media. Since then, no other attack against the security of the OpenPGP specif cation has been published. All other publications refer to implementation issues and specif c implementation details.

5.4.2 Implementations

As mentioned earlier, the security of the OpenPGP specif cation is a necessary, but usually not suff cient, requirement for the security of a specif c implementation. This means that there may be security problems that don't exist in the OpenPGP specif cation but that occur in a specif c implementation. This also means that a product that implements the OpenPGP specif cation is not automatically secure only because it implements this specif cation. There are, for example, issues related to physical security that cannot be addressed by the OpenPGP specif cation (note, for example, that the Klíma-Rosa attack mentioned earlier also requires physical access to the private keyring of the victim). Physical security is generally harder to achieve in the multi-user environment that we live in today. So, the underlying operating system (or hypervisor, in a virtualized environment) has to ensure that a user cannot read or tamper with the f les of another user. The same is true if the keyrings are stored in a cloud storage service such as Dropbox.[42] In the realm of physical security, things like electromagnetic radiation must also be addressed on an implementation-by-implementation basis. Some newer versions of PGP have, for example, been able to display decrypted messages using a specially designed font to minimize the physical strength of the electromagnetic signals produced by

42 http://www.dropbox.com.

the screen. This software-based Tempest feature is known as Secure Viewer. It is designed to make it more diff cult to remotely detect the signals.

In addition to physical security, there are issues related to key management that are equally critical for the security of a given implementation. If, for example, the keys in use leak, then there is nothing that the PGP specif cation can do about it. This is clearly an implementation issue that can only be addressed by the implementation. Similarly, if a pseudorandom number generator is weak, then this is also an implementation issue that does not affect the PGP specif cation. This occurred in May 2000, when a f aw was found in the process by which the Linux and OpenBSD command-line versions of PGP 5.0 generated pseudorandom numbers. The generated numbers were predictable, and therefore the respective cryptographic keys potentially insecure (see CERT Advisory CA-2000-09[43]). In the current version of the OpenPGP specif cation [7], it is recommended to adhere to following the recommendations of RFC 4086 [36] when it comes to generating pseudorandom numbers.

In July 2001, Sieuwert van Otterloo announced a vulnerability in the graphical user interface of PGP 5.0 and above. After a patch was released on September 4, 2001, a paper entitled "A Security Analysis of Pretty Good Privacy" was published.[44] The paper explained how to exploit the vulnerability in a *multiple user ID attack*. Due to the fact that the vulnerability had been patched before the paper was released, the multiple user ID attack could not be mounted in the f eld.

Like any password-based system, OpenPGP is susceptible to password or passphrase guessing attacks. If a user selected a bad passphrase, then the OpenPGP implementation could be attacked and broken, no matter how secure it is otherwise implemented. The security of the user's passphrase then represents an upper bound for the overall security of the implementation. This is bad news and basically means that any passphrase grabbing attack can be used to compromise the security of an OpenPGP implementation. If, for example, an adversary has read access to a user's private keyring and is able to install a key logger, then he or she has access to the user's passphrase and private key; therefore, he or she can act as if he or she were the user. If the adversary has no physical access to the user's machine, then he or she can still mount a malicious software (malware) attack. In 1998, for example, a group called the Codebreakers released the *Caligula Word 97 macro virus* that tried to copy the user's private keyring, upload it to an FTP site, and run an off ine password guessing attack against the passphrase. While similar attacks are feasible today, the low number of OpenPGP users somewhat limits the usefulness of such attacks. But there are also vulnerabilities in some OpenPGP implementations that allow malware to be executed locally (e.g., CVE-2001-0265).

43 http://www.cert.org/advisories/CA-2000-09.html.

44 http://www.bluering.nl/pgp/pgp.pdf.

The bottom line is that most OpenPGP implementations are susceptible to password grabbing attacks. It is rumored that most secret services and agencies routinely break OpenPGP implementations by using malware that acts as a key logger to grab user passphrases. Equipped with these passphrases and the respective private keyrings, any OpenPGP-protected message can be decrypted. OpenPGP is missing forward secrecy and PCS. If a user's long-term private key is compromised, then essentially all correspondence of this user (protected with this key) is at risk. This is unnecessary and modern approaches to secure and E2EE messaging try to mitigate the respective risks.

5.5 FINAL REMARKS

In this chapter, we have elaborated on OpenPGP as one of the conventional approachs for secure messaging on the Internet. The OpenPGP technology is mature, well understood, and established. Hence, it is possible and very likely that we will see some OpenPGP implementations in the future, and that applications outside the scope of secure messaging will also make use of OpenPGP and its features. We have seen, for example, OpenPGP-based extensions for Kerberos, OpenPGP-based solutions to secure Internet transactions [37], and OpenPGP-based extensions for the authentication in the TLS protocol [38].

A final word is due to the usability of OpenPGP. No security mechanism or feature will ever be widely deployed in the field if it is not usable. Hence, good usability is a prerequisite for any security mechanism or feature to be deployed and used in the first place. Against this background, Alma Whitten and Doug Tygar empirically investigated the usability of PGP version 5.0 in the late 1990s [39]. Their results were pessimistic and showed that the software was far too complicated to be used in the field. For example, most participants of the empirical tests were unable to sign and encrypt a message within 90 minutes. Similar studies [40, 41] have shown that the results and usability concerns still apply, and that usability remains a major concern when it comes to using OpenPGP in the field. Again, modern E2EE messengers try to improve the situation and come up with alternative ways to authenticate users and their respective public keys in so-called *authentication ceremonies*.

References

[1] Zimmermann, P.R. *The Official PGP User's Guide*, MIT Press, Cambridge, MA, 1995.

[2] Zimmermann, P.R. *PGP Source Code and Internals*, MIT Press, Cambridge, MA, 1995.

[3] Garf nkel, S., *PGP: Pretty Good Privacy*, O'Reilly & Associates, Sebastopol, CA, 1995.

[4] Atkins, D., Stallings, W., and P.R. Zimmermann, "PGP Message Exchange Formats," Request for Comments 1991, August 1996.

[5] Elkins, M., "MIME Security with Pretty Good Privacy (PGP)," Request for Comments 2015, October 1996.

[6] Callas, J., Donnerhacke, L., Finney, H., and R. Thayer, "OpenPGP Message Format," Request for Comments 2440, November 1998.

[7] Callas, J., et al., "OpenPGP Message Format," RFC 4880, November 2007.

[8] Elkins, M., Del Torto, D., Levien, R., and T. Roessler, "MIME Security with OpenPGP," RFC 3156, August 2001.

[9] Elgamal, T., "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithm," *IEEE Transactions on Information Theory*, IT-31(4), 1985, pp. 469–472.

[10] Diff e, W., and M.E. Hellman, "New Directions in Cryptography," *IEEE Transactions on Information Theory*, IT-22(6), 1976, pp. 644–654.

[11] Galvin, J., Murphy, S., Crocker, S., and N. Freed, "Security Multiparts for MIME: Multipart/Signed and Multipart/Encrypted," RFC 1847, October 1995.

[12] Galvin, J., and M.S. Feldman, "MIME object security services: Issues in a multi-user environment," *Proceedings of USENIX UNIX Security V Symposium*, June 1995.

[13] Adams, C., "The CAST-128 Encryption Algorithm," RFC 2144, May 1997.

[14] Matsui, M., Nakajima, J., and S. Moriai, "A Description of the Camellia Encryption Algorithm," RFC 3713, April 2004.

[15] Shaw, D., "The Camellia Cipher in OpenPGP," RFC 5581, June 2009.

[16] Mister, S., and R. Zuccherato, "An Attack on CFB Mode Encryption As Used By OpenPGP," Cryptology ePrint Archive: Report 2005/033, 2005.

[17] Krovetz, T., and P. Rogaway, "The OCB Authenticated-Encryption Algorithm," RFC 7253, May 2014.

[18] Jivsov, A., "Elliptic Curve Cryptography (ECC) in OpenPGP," RFC 6637, June 2012.

[19] Deutsch, P., "DEFLATE Compressed Data Format Specif cation version 1.3," RFC 1951, May 1996.

[20] Liv, J., and A. Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, IT-23(3), 1977, pp. 337–343.

[21] Deutsch, P., and J-L. Gailly, "ZLIB Compressed Data Format Specif cation version 3.3," RFC 1950, May 1996.

[22] Kelsey, J., "Compression and Information Leakage of Plaintext," *Proceedings of the 9th International Fast Software Encryption (FSE) Workshop*, Springer-Verlag, LNCS 2365, 2002, pp 263–276.

[23] Oppliger, R., *SSL and TLS: Theory and Practice*, 2nd Edition, Artech House, Norwood, MA, 2016.

[24] American National Standards Institute, *American National Standard X9.17: Financial Institution Key Management*, Washington, DC, 1985.

[25] Shamir, A., "How to share a secret," *Communications of the ACM*, 22(11), November 1979, pp. 612–613.

[26] Blakley, G.R., "Safeguarding cryptographic keys," *Proceedings of the AFIPS National Computer Conference*, 1979, pp. 313–317.

[27] Stallings, W., *Cryptography and Network Security: Principles and Practice*, 2nd Edition, Prentice-Hall, Upper Saddle River, NJ, 1998.

[28] Reiter, M.K., and S.G. Stubblebine, "Path Independence for Authentication in Large-Scale Systems," *Proceedings of the 4th ACM Conference on Computer and Communications Security*, 1997, pp. 57–66.

[29] Hänni, R., "Using probabilistic argumentation for key validation in public-key cryptography," *International Journal of Approximate Reasoning*, 38(3), March 2005, pp. 355–376.

[30] Maurer, U.M., "Modelling a Public-Key Infrastructure," *Proceedings of the European Symposium on Research in Computer Security (ESORICS 96)*, Springer-Verlag, LNCS 1146, 1996, pp. 325–350.

[31] Maurer, U.M., and R. Kohlas, "Conf dence Valuation in a Public-key Infrastructure Based on Uncertain Evidence", *Proceedings of Public Key Cryptography 2000*, Springer-Verlag, LNCS 1751, 2000, pp. 93–112.

[32] Rubin, A.D., Geer, D., and M.J. Ranum, *Web Security Sourcebook*, John Wiley & Sons, Inc., New York, NY, 1997.

[33] Klíma, V., and T. Rosa, "Attack on Private Signature Keys of the OpenPGP format, PGP programs and other applications compatible with OpenPGP," IACR ePrint Archive, March 2002, http://eprint.iacr.org/2002/076.pdf.

[34] Katz, J., and B. Schneier, "A Chosen Ciphertext Attack against Several E-Mail Encryption Protocols," *Proceedings of 9th USENIX Security Symposium*, 2000, pp. 241–246.

[35] Jallad, K., Katz, J., and B. Schneier, "Implementation of Chosen-Ciphertext Attacks against PGP and GnuPG," *Proceedings of 5th International Information Security Conference (ISC 2002)*, Springer-Verlag, LNCS 2433, 2002, pp. 90–101.

[36] Eastlake, D., Schiller, J., and S. Crocker, "Randomness Requirements for Security," RFC 4086, June 2005.

[37] Weeks, J.D., Cain, A., and B. Sanderson, "CCI-Based Web Security—A Design Using PGP," *Proceedings of 4th International World Wide Web Conference*, December 1995, pp. 381–395.

[38] Mavrogiannopoulos, N., and D. Gillmor, "Using OpenPGP Keys for Transport Layer Security (TLS) Authentication," RFC 6091, February 2011.

[39] Whitten, A., and J.D. Tygar, "Why Johnny Can't Encrypt: A Usability Evaluation of PGP 5.0," *Proceedings of the 8th USENIX Security Symposium*, August 1999, pp. 169-184.

[40] Garf nkel, S.L., and R.C. Miller, "Johnny 2: A User Test of Key Continuity Management with S/MIME and Outlook Express," *Proceedings of the 2005 Symposium on Usable Privacy and Security (SOUPS '05)*, ACM, 2005, pp. 13-24.

[41] Ruoti, S., et al., "Why Johnny Still, Still Can't Encrypt: Evaluating the Usability of a Modern PGP Client," arXiv:1510.08555v2, 2015.

Chapter 6

S/MIME

In this chapter, we focus solely on S/MIME. We start with the origins and history of S/MIME in Section 6.1, elaborate on the technology in Section 6.2, overview and discuss the use of certif cates in Section 6.3, provide a brief security analysis in Section 6.4, and conclude with some f nal remarks in Section 6.5. Similar to the previous chapter, this chapter stands for itself and can be used as a comprehensive introduction and outline of S/MIME—together with the complementary material referenced throughout the text.

6.1 ORIGINS AND HISTORY

In the Introduction, we said that PEM was an early standardization effort for secure messaging on the Internet that suffered from two major limitations and shortcomings,[1] that MOSS was an attempt to overcome them, but that it failed to become commercially successful. In parallel with the development of PGP and MOSS, an industry working group led by RSA Security started to develop another specif cation for conveying digitally signed and digitally enveloped messages in accordance with MIME and some early versions of the *public key cryptography standards* (PKCS). The protocol specif cation that was developed by this working group was named S/MIME, an acronym standing for *secure MIME* or *Secure/Multipurpose Internet Mail Extensions*, respectively. Similar to PEM and MOSS, S/MIME refers only to a protocol specif cation (and not also to an implementation like PGP). Also similar to MOSS, S/MIME was specif cally designed to add security to MIME messages.

As a reminder, the structure of a MIME message is illustrated in Figure 6.1. It consists of a header (with several MIME headers) and a body part that may comprise

1 The PEM specif cation was limited to 7-bit ASCII messages and a three-layer hierarchy of CAs.

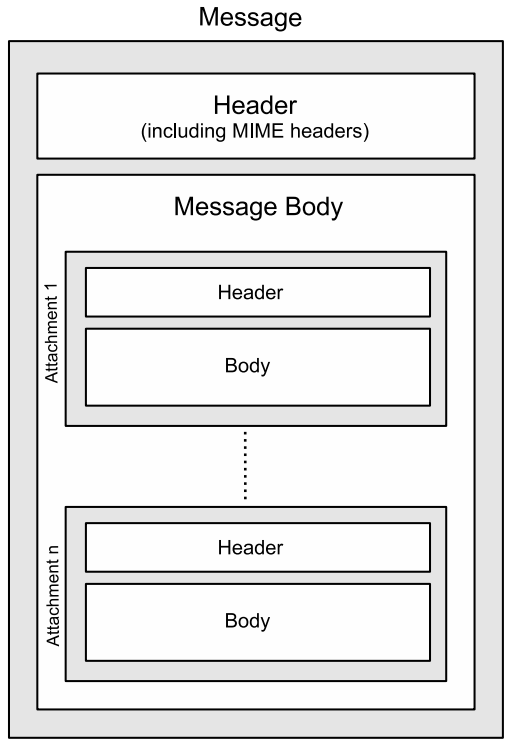


Figure 6.1 The structure of a MIME message.

multiple attachments, where each attachment may recursively consist of a header and body part of its own (Figure 6.1 illustrates n such attachments). Note that the recursive nature of a MIME message can also be represented as a tree, where the message refers to the root, each attachment refers to a node, and each MIME entity that is going to be processed individually refers to a leaf in the tree.

S/MIME goes hand in hand with MIME, and this basically means that it cannot reasonably be used with an MUA that does not support MIME. Unlike PEM and MOSS, S/MIME has been moderately successful and is widely implemented and deployed in many products for secure messaging on the Internet. This includes, for example, the products of all leading software companies. Microsoft, for example,

has been supporting S/MIME in Outlook since the early days of the standard, and it continues to support S/MIME in Office 365 and its cloud-based version of Outlook.

While PGP, OpenPGP, PEM, and MOSS are based on some hand-crafted algorithms and protocols for message encoding and delivery, S/MIME is based on standards that are well-established in the field. In particular, S/MIME is based on MIME and PKCS, and this has the big advantage that one does not have to start from scratch when analyzing the security of the respective algorithms and protocols. We revisit this point later in this chapter.

There are four versions of S/MIME. Versions 2 and 3 are mostly used in the field, whereas version 4 is relatively new and will hopefully become the preferred choice.

- S/MIME version 1 was specified and officially published by RSA Security in 1995 [1].
- S/MIME version 2 was specified by the IETF S/MIME Mail Security (SMIME) WG in a pair of RFC documents [2, 3] in 1998.[2]
- The work continued within the IETF SMIME WG and finally culminated in S/MIME version 3 that was officially released in 1999. S/MIME version 3 is specified in a set of five RFC documents [4–8]. Except for the supported algorithms, the changes between version 2 and version 3 are not particularly significant, and hence it is recommended that S/MIME version 3 implementations should attempt to have the greatest possible interoperability with version 2 implementations. Later on, the S/MIME version 3 certificate handling—as specified in RFC 2632 [6]—was modified in RFC 3850 [9] for version 3.1 and RFC 5750 [11] for version 3.2, and the S/MIME version 3 message format—as specified in RFC 2633 [7]—was modified in RFC 3851 [10] for version 3.1 and RFC 5751 [12] for version 3.2. The bottom line is that [9, 10] refer to S/MIME version 3.1, whereas [11, 12] refer to S/MIME version 3.2. Again, the changes are relatively moderate and not very important here.
- In the aftermath of EFAIL and related attacks (Section 4.1), the IETF Limited Additional Mechanisms for PKIX and SMIME (LAMPS) WG has taken up the task of updating the cryptography used in S/MIME in a new version 4.0.[3] The aim was to include new and more timely cryptographic primitives and techniques, such as authenticated encryption and ECC in S/MIME. The resulting RFC documents 8550 [13] and 8551 [14] were officially released

2 The pair was complemented by three RFC documents that specified early versions of PKCS #1 (RFC 2313), PKCS #10 (RFC 2314), and PKCS #7 (RFC 2315). The latter RFC document is also referenced in [18].

3 This was because the IETF SMIME WG was officially concluded in 2010.

in April 2019—they specify an updated certif cate handling mechanism and message format for S/MIME.

The detailed changes from S/MIME version 3 to version 3.1, version 3.1 to version 3.2, and version 3.2 to 4.0 are summarized in Section 1.5 of [14]. They are not repeated here.

In the past, S/MIME had had some diff culties receiving consideration as an Internet standards track protocol due to its extensive use of patented technologies and algorithms. This is because all standards approved by the IETF must use only public domain technologies and algorithms, so anyone can implement them without paying royalties to the respective patent holders. This situation has improved, because newer versions of S/MIME provide more f exibility with regard to the cryptographic algorithms that can be used, and many public key patents have expired meanwhile.[4]

The bottom line is that the history of S/MIME is more down-to-earth and less exciting than the history of PGP. It started with an industry working group and is rooted in established standards that were available in the 1990s. Similar to PGP/MIME (Section 5.2.4), S/MIME has parts of its roots in RFC 1847 [15] and the respective MIME `multipart` subtypes (i.e., `multipart/encrypted` and `multipart/signed`). Furthermore, it is based on the *Cryptographic Message Syntax* (CMS) that is specif ed in RFC 5652[5] [16] and—in the case of multiple signatures—also in RFC 5752 [17].[6] The CMS itself has its roots in PKCS #7 [18] and later evolved in several RFC documents (not referenced here).[7] The evolution of the CMS is likely to continue and future RFC documents will probably make [16] and [17] obsolete one day.

6.2 TECHNOLOGY

Like OpenPGP or any other secure messaging scheme, S/MIME employs cryptographic techniques and mechanisms to provide basic message protection services, like data origin authentication, connectionless conf dentiality, connectionless integrity, and nonrepudiation services with proof of origin. However, in spite of this conceptual similarity, there are (at least) two fundamental differences between OpenPGP and S/MIME:

4 For example, the RSA patent expired in 2000.

5 As mentioned in the Introduction, RFC 5652 became an Internet Standard (STD 70) in June 2013.

6 In addition to these RFCs that are submitted to the Internet Standards Track, informational RFC 6268 provides a summary about other RFC that are also related to the CMS.

7 You may think of the CMS as being a ref ned version of PKCS #7 that is particularly crafted for secure messaging. PKCS #7, in turn, is independent from an application setting and use case.

- OpenPGP and S/MIME use different message formats;
- OpenPGP and S/MIME handle public keys and certif cates in fundamentally different ways (we elaborated on OpenPGP's web of trust in Section 5.3 and we will brief y address the use of X.509 certif cates later in this chapter).

Both differences lead to a situation in which OpenPGP and S/MIME implementations do not interoperate automatically. There are some MUAs that—natively or through the use of a plug-in—provide support for both technologies, but this need not be the case. Also, there are some OpenPGP implementations that provide support for X.509 certif cates, mainly because the commercial world is using them (while OpenPGP certif cates still have a shadowy existence). However, from an Internet standardization viewpoint, we have to live with the uncomfortable situation that there are two competing solutions and standards to solve essentially the same problem, namely to provide (cryptographic) security for asynchronous messaging in the form of e-mail.

In the remaining part of this section, we overview and discuss the message formats (according to the CMS), cryptographic algorithms, signer attributes, and enhanced security services (ESS) as far as they are relevant for S/MIME and its actual use in the f eld.

6.2.1 Message Formats

S/MIME is based on the CMS, and this means that S/MIME entities are formatted accordingly. The CMS provides an encapsulation syntax for data protection that can be applied recursively. It is thus possible to digitally envelope a previously signed MIME entity, or to digitally sign a previously enveloped entity. S/MIME is not specif c about how to apply protection—anything that makes sense from an application viewpoint can be expressed in the CMS (and then be implemented using S/MIME). This is in sharp contrast to OpenPGP that always requires a particular order in message processing. Furthermore, the CMS allows arbitrary attributes, such as, for example, timestamps, to be signed along with a MIME entity (Section 6.2.3). Again, this is not something that is natively supported in OpenPGP.

In general, CMS values are generated using a combination of the *Abstract Syntax Notation 1* (ASN.1[8]) and the *Basic Encoding Rules* (BER[9]) that used to be popular in the design of networking techniques and protocols in the past. Today, ASN.1 and BER are not so popular anymore, and currently deployed protocols are often specif ed in simpler terms. To keep the discussion as simple as possible, we do not delve into the technical details of ASN.1 and BER in this book. If you want

8 ASN.1 is def ned in ITU-T Recommendations X.680–X.683.
9 The BER are def ned in ITU-T Recommendation X.690.

to learn more about these topics, then you may refer to the many resources that are available online.

Table 6.1
Content Types Natively Def ned in the CMS

Description	CMS Content Type	ASN.1 Type
Arbitrary data	`data`	`Data`
Digitally signed data	`signed-data`	`SignedData`
Digitally enveloped data	`enveloped-data`	`EnvelopedData`
Digested data	`digested-data`	`DigestedData`
Encrypted data	`encrypted-data`	`EncryptedData`
Authenticated data	`authenticated-data`	`AuthenticatedData`

As shown in Table 6.1, the CMS natively def nes six content types that can be used recursively (i.e., to encapsulate any other content type). Each content type is identif ed by a unique object identif er (OID).[10] The names of the content types speak for themselves.

- The content type `data` (represented by the ASN.1 type `Data` and OID 1.2.840.113549.1.7.1 in the dot notation[11]) is to contain arbitrary data, such as ASCII text, which may or may not have an internal structure. The interpretation of the data is left to the application. For cryptographic protection, content of this type is usually encapsulated in some other type, such as `signed-data`, `enveloped-data`, `digested-data`, `encrypted-data`, or `authenticated-data`.
- The content type `signed-data` (represented by the ASN.1 type `SignedData` and OID 1.2.840.113549.1.7.2) is to contain digitally signed data, i.e., data of any type that comes along with one or several digital signatures. For each signature, the verif er must know what (message digest and signature verif cation) algorithms and public key to use. This information is provided in a (per-signer) data structure of ASN.1 type `SignerInfo` (Section 6.2.3).
- The content type `enveloped-data` (represented by the ASN.1 type `EnvelopedData` and OID 1.2.840.113549.1.7.3) is to contain digitally enveloped

10 The OID values can be found in a public repository, such as http://www.oid-info.com.

11 This notation was introduced by the IETF. ASN.1 originally used another notation is uses spaces and braces, with optional text labels. 1.2.840.113549.1.7.1 would become something like { iso(1) member-body(2) us(840) rsadsi(113549) pkcs(1) pkcs-7(7) data(1) }. The representatives within the IETF thought that this was inconvenient, and decided to use a space-free notation. This is, among other things, spelled out in RFC 1778 (Section 2.15), but was in use long before that time.

data (i.e., data of any type that is encrypted with a content-encryption key, where the content-encryption key and some other information is encrypted for each recipient in a per-recipient data structure of ASN.1 type `RecipientInfo`). The recipient opens the digital envelope by decrypting one of the encrypted content-encryption keys and then decrypting the encrypted content with it.

- The content type `digested-data` (represented by the ASN.1 type `DigestedData` and OID 1.2.840.113549.1.7.5) is to contain data of any type that comes along with a hash value (also known as message digest). This is used, for example, to digest data before it is encapsulated with the `enveloped-data` content type to provide an integrity tag.
- Like `enveloped-data`, the content type `encrypted-data` (represented by the ASN.1 type `EncryptedData` and OID 1.2.840.113549.1.7.6) is to contain encrypted data. However, unlike `enveloped-data`, it includes neither recipient information nor encrypted keying material. Instead, the keys required for decryption must be distributed and managed out-of-band.
- Last but not least, the content type `authenticated-data` (represented by the ASN.1 type `AuthenticatedData` and OID 1.2.840.113549.1.9.16.1.2) is to contain data of any type that comes along with a MAC and the keying information that is required to verify it (i.e., the encrypted authentication keys for one or more recipients).

An implementation that conforms to the CMS must at least implement the `data`, `signed-data`, and `enveloped-data` content types. The other content types are optional and may be implemented at will. In the realm of S/MIME, two complementary content types are also relevant: `AuthEnvelopedData` (OID 1.2.840.113549.1.9.16.1.23) that refers to `EnvelopedData` with integrity protection (basically providing authenticated encryption in S/MIME version 4) and `CompressedData` (OID 1.2.840.113549.1.9.16.1.9) that—as its name suggests—refers to data that is compressed (mainly to reduce its size).

The S/MIME message specification defines how to cryptographically protect a MIME entity and turn it into an S/MIME entity or CMS object. It also explains the use of the MIME `multipart/signed` content type, as well as several new subtypes of the MIME `application` content type. These types and subtypes are summarized in Table 6.2. According to this table, there are multiple ways of declaring a digital signature in S/MIME. In addition to the MIME content type `application/pkcs7-mime` and the smime-type parameter `signed-data`, one can also employ the content types `multipart/signed` or `application/pkcs7-signature` without smime-type parameter. Even the

Table 6.2
MIME Content Types and Subtypes Employed by S/MIME

Type	Subtype	Smime-type Parameter
multipart	signed	
application	pkcs7-mime	signed-data
	pkcs7-mime	certs-only
	pkcs7-mime	enveloped-data
	pkcs7-mime	authEnveloped-data
	pkcs7-mime	compressed-data
	pkcs7-signature	
	pkcs10-mime	

content type pkcs10-mime that refers to a certif cate request message that conforms to PKCS #10 may comprise a digital signature.

Application/pkcs7-mime is by far the most important MIME content type employed by S/MIME.[12] It is used to carry CMS objects of several types, including, for example, data that is digitally signed or digitally enveloped. The application/pkcs7-mime content type comes along with the optional smime-type parameter that is aimed at conveying details about the security applied along with information about the content. The possible values for the smime-type parameter are summarized in Table 6.3. If a CMS object encapsulates data that is digitally signed, then the respective smime-type parameter is signed-data (for normally digitally signed data) or certs-only (for public key certif cates). If a CMS object encapsulates and digitally envelopes data, then the smime-type parameter is enveloped-data; if it is digitally signed and enveloped, then it is authEnveloped-data. Finally, if a CMS object encapsulates data that is compressed, then the smime-type parameter is compressed-data.

Table 6.3
MIME Types and File Extensions

MIME Type	Smime-type Parameter	File Extension
application/pkcs-7-mime	signed-data	.p7m
	enveloped-data	.p7m
	authEnveloped-data	.p7m
	certs-only	.p7c
	compressed-data	.p7z
application/pkcs-7-signature		.p7s

12 Note that some MUAs use application/x-pkcs7-mime instead of application/pkcs7-mime. This is only for historical reasons, and the two MIME content types are equivalent.

In addition to the smime-type parameter, the `application/pkcs7-mime` S/MIME content type has a few optional parameters, such as name and f lename to specify a f lename (limited to eight characters) with an appropriate extension (limited to three characters).[13] Again, refer to Table 6.3 for an overview of the relevant MIME types, smime-type parameters, and the respective f le extensions. The f lename base `smime` is often used to indicate that the MIME entity is associated with S/MIME. According to this convention, the f lename `smime.p7m` is used to refer to a MIME entity that carries a CMS object that is digitally signed, enveloped, or subject to authenticated encryption.

As mentioned several times so far, S/MIME is just a syntax that can be used in many ways to cryptographically protect messages. The procedure to cryptographically protect a message comprises the following four steps:

1. The message is prepared according to the normal rules for MIME processing. This means that the message is turned into a data structure that is in line with Figure 6.1.
2. Each MIME entity is converted to a canonical form. The details of this canonicalization depend on the media type and subtype in use. For example, canonicalization of type `text/plain` is different from canonicalization of type `audio/basic`. Other than text types, most types have only one representation regardless of the underlying computing platform. If the media type is `text`, then the canonicalization involves converting the line endings to <CRLF> and choosing a registered character set. Anyway, the details of the canonicalization are beyond the scope of this book and not discussed here.
3. The (now converted) MIME entity, together with some security-related information, such as algorithm identif ers or public key certif cates, is processed to generate a CMS object.
4. This object is wrapped—possibly together with some other CMS objects—in a message that can be sent through the Internet. This also means that some additional MIME headers may be prepended to the message.

Steps 1 and 2 must be performed by the nonsecurity part of an MUA, whereas S/MIME only addresses steps 3 and 4. In either case, the resulting message is sent to the intended recipient(s).

In the rest of this section, we delve more deeply into the details of creating MIME entities that are compressed-only, enveloped-only, signed-only, signed and enveloped, and certif cates-only (and we sketch some examples). We emphasize that all of these operations may be nested in any order, and that MIME entities may

13 The f lename parameter is sent together with the Content-Disposition MIME header.

encapsulate other MIME entities at will. In particular, it is possible to digitally sign an enveloped entity, or—vice versa—to digitally envelope a signed entity. S/MIME supports either possibility and leaves the decision to the MUA developer or user. Note that either possibility has specif c properties and characteristics:

- When an entity is f rst digitally signed and then enveloped, then the signature(s) is (are) obscured by the digital envelope, meaning that the signature(s) can no longer be verif ed by everybody, and hence that the signer(s) may stay anonymous.[14]
- Contrary to that, when a message is f rst digitally enveloped and then signed, then the signature(s) is (are) visible by everybody and can be verif ed by everybody—without removing the envelope. This defeats the possibility of providing anonymity services, but it may be useful in situations where automatic signature verif cation should take place and appropriate actions should be performed before a message reaches its recipient(s).

Which possibility is advantageous in a particular situation depends on the use case. Sometimes it is advantageous to f rst digitally sign and then envelope a message; sometimes it is advantageous to do the opposite. S/MIME supports either possibility and does not care. In either case, however, it is important to perform the compression operation f rst. This is because the output of a cryptographic operation is usually indistinguishable from random data, and this means that this data cannot be compressed anymore.

6.2.1.1 Compressed-Only Entities

As its name suggests, a compressed-only MIME entity is only compressed (i.e., it is neither digitally signed nor encrypted and enveloped). This possibility was introduced in [19] and soon after incorporated into S/MIME version 3.1 [10]. The skeleton of a compressed-only MIME entity looks as follows:

```
MIME-Version: 1.0
Content-Type: application/pkcs7-mime;
     smime-type=compressed-data;
     name="smime.p7z"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
```

14 If a message has only one signature, then the signer is often also the sender of the message. This means that one can also determine the likely signer from the From f eld in the message header. It is possible and likely that this yields the signer. But it need no be the case, because the sender may not be equal to the signer and the content of the From f eld may not even be correct.

```
    filename="smime.p7z"

...
```

As its name suggests, the `MIME-Version` header refers to the version of MIME used to compose the message (usually version 1.0). Strictly speaking, this header does not belong to S/MIME (it rather belongs to MIME), but it is nevertheless shown here. The same is true for the `Content-Transfer-Encoding` header that specifies how the MIME entity is transfer encoded (i.e., base-64 in this case). The two headers that actually belong to S/MIME are `Content-Type` and `Content-Disposition`.

- As its name suggests, the `Content-Type` header specifies the content type. According to what has been said earlier, the content type here is `application/pkcs7-mime` with the smime-type parameter is set to `compressed-data` and the name parameter is set to `smime.p7z` (according to Table 6.3).
- The `Content-Disposition` header specifies that the attachment included in the message body is aimed at representing a file with the name `smime.p7z` (note that the name and filename parameters are often the same).

Finally, the three dots at the bottom refer to the base-64 encoded data that is compressed and actually transferred in the message or S/MIME entity, respectively. Note that there is an empty line that separates the header from the body part of the S/MIME entity.

6.2.1.2 Enveloped-Only Entities

An enveloped-only MIME entity is only encrypted, meaning that—similar to the way OpenPGP encrypts a packet—a one-time encryption key for some secret key encryption system is randomly selected, the MIME entity is encrypted with this key, and the key itself is encrypted with the recipient's public key. Both the ciphertext and the encrypted key are then forwarded to the recipient. If there are multiple recipients, then the one-time encryption key needs to be encrypted with the public key of each recipient. Note that such an entity is only protected in terms of data confidentiality, and not in terms of message origin authentication or data integrity. If this were necessary, then the `authEnvelopedData` content type would have to be used instead (instead of `enveloped-data`). The skeleton of an enveloped-only S/MIME entity looks as follows:

```
MIME-Version: 1.0
```

```
Content-Type: application/pkcs7-mime;
     smime-type=enveloped-data;
     name="smime.p7m"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
     filename="smime.p7m"

...
```

The `MIME-Version` and `Content-Transfer-Encoding` headers are the same as above, whereas the `Content-Type` and `Content-Disposition` headers are slightly different. In fact, the only differences refer to the smime-type parameter (set to `enveloped-data`) and the extension of the f lename (set to `.p7m`). Everything else remains the same. This also applies to the three dots at the bottom that refer to the base-64 encoded data that is enveloped in the message or S/MIME entity, respectively.

As mentioned above, the skeleton of an authenticated enveloped-only S/MIME entity would be similar, except that the smime-type parameter would be set to `authEnveloped-data` (instead of `enveloped-data`).

6.2.1.3 Signed-Only Entities

As mentioned earlier, S/MIME provides two different formats for digitally signed MIME entities:

- The f rst format uses the `application/pkcs7-mime` media type[15] with the smime-type parameter set to `signed-data`. The skeleton of such a digitally signed S/MIME entity looks as follows:

  ```
  MIME-Version: 1.0
  Content-Type: application/pkcs7-mime;
       smime-type=signed-data;
       name="smime.p7m"
  Content-Transfer-Encoding: base64
  Content-Disposition: attachment;
       filename="smime.p7m"

  ...
  ```

- The second format uses the `multipart/signed` and `application/pkcs7-signature` media types. This format is typically used to transport

15 Note that this is the same media type that is also used for enveloped-only entities.

detached signatures. The respective S/MIME entity comprises two parts (i.e., the MIME entity that is digitally signed in the clear and the detached digital signature). The skeleton of such a digitally signed S/MIME entity looks as follows:

```
MIME-Version: 1.0
Content-Type: multipart/signed;
     protocol="application/pkcs7-signature";
     micalg=sha1;
     boundary="foo"

--foo
Content-Type: text/plain

This is a test message to demonstrate the clear-
signing format for signed-only messages.

--foo
Content-Type: application/pkcs7-signature;
     name="smime.p7s"
Content-Transfer-Encoding: base64
Content-Disposition: attachment;
     filename="smime.p7s"

...
--foo--
```

Note that the `multipart/signed` MIME type has two parameters: the protocol parameter and the micalg parameter. The protocol parameter must be set to `"application/pkcs7-signature"`,[16] whereas the micalg parameter must be set to the message integrity check (MIC) algorithm in use, such as `md5` for MD5 or—as is the case here—`sha1` for SHA-1. As usual, the three dots at the bottom refer to the base-64 encoded detached signature for the test message.

There are no f xed rules for when a particular format should be used (receiving MUAs must be able to handle either format). In fact, this decision depends on the capabilities of all the recipients and the relative importance of recipients with

16 The quotation marks are required because MIME requires that the slash character in the parameter value be quoted.

S/MIME facilities being able to verify the signature versus the importance of recipients without S/MIME facilities being able to view the message. More specifically, messages signed using the second format (i.e., `multipart/signed` and `application/pkcs7-signature`) can always be viewed by the recipients, whether they have an S/MIME-enabled MUA or not. This format is also sometimes referred to as the clear-signing format. Contrary to that, messages signed with the first format (i.e., `application/pkcs7-mime`) cannot be viewed by a recipient unless he or she has an S/MIME-enabled MUA. Since this may cause problems in some environments, the second format is usually the preferred choice.

6.2.1.4 Certificates-Only Entities

A certificate-only MIME entity comprises digitally signed data that refers to public key certificates or CRLs. This means that such an entity is represented by a CMS object of type `signed-data` that is enclosed in an `application/pkcs7-mime` entity (with the smime-type parameter set to `certs-only` and the name parameter to something with an extension of `.p7c`). The details are omitted here.

6.2.2 Cryptographic Algorithms

The CMS [16] is a syntax that does not mandate the use of specific algorithms. Instead, the cryptographic algorithms used for S/MIME are specified in the referenced RFC documents and [20], as well as a few complementary RFC documents (i.e., [21–26]). Note that a somewhat nonstandard terminology is used in this context. In addition to the normal key words,[17] like MUST, SHOULD, and MAY, three additional key words are used:

- SHOULD+ is the same as SHOULD, but there is reason to expect the algorithm to be promoted to a MUST in future editions of the specification;
- SHOULD- is also the same as SHOULD, but there is reason to expect the algorithm to be demoted to a MAY in future editions of the specification;
- Finally, MUST- is the same as MUST, but it is reasonable to expect the algorithm to be demoted to a SHOULD (or SHOULD-) in future editions of the specification.

Also note that EFAIL and a few related attacks have made it necessary to upgrade the cryptographic algorithms used for S/MIME.

17 The key words are specified in BCP 14 (RFC 2119 and RFC 8174).

- With regard to cryptographic hash algorithms, S/MIME version 4.0 implementations MUST support SHA-256 and SHA-512 [23], and they may optionally support other algorithms from the SHA-2 family, such as SHA-224 and SHA-384. In former versions of S/MIME, the standard algorithms were MD5 and SHA-1, but they are no longer considered secure and have been deprecated.
- With regard to digital signatures, S/MIME version 4.0 implementations MUST support ECC in terms of ECDSA with curve P-256 and SHA-256 and EdDSA with Curve25519 in PureEdDSA mode according to [24]. Receiving MUAs must support both curves, whereas sending MUAs must support either of the two curves. This follows the robustness principle that basically says that one should be conservative in selecting cryptographic algorithms to send messages and liberal to receive messages.[18] Besides ECC, S/MIME version 4.0 implementations MUST- support "normal" RSA PKCS #1 version 1.5 with SHA-256, and they SHOULD also support RSA-PSS[19] with SHA-256 and appropriately sized keys.[20] Note that RSA-PSS is cryptographically more advanced but also less widely deployed than RSA PKCS #1 version 1.5.
- With regard to key exchange and encryption, S/MIME version 4.0 implementations MUST support ECDH ephemeral-static mode for curve P-256 according to [25] and X25519 using HKDF-256 according to [26].[21] Furthermore, they MUST- support RSA encryption according to [20], and they SHOULD+

18 More specifically, conservative means that one should go for algorithms that are strong but still widely supported, whereas "liberal" means that one should also accept algorithms that may not be state of the art (just to make sure that cryptography can be applied in the first place). Hence, a distinction is made between MUAs that receive messages and MUAs that send out messages.

19 The suffix PSS stands for probabilistic signature scheme that is a secure variant of the RSA digital signature system that uses a distinct padding. The official acronym for RSA-PSS is RSASSA-PSS.

20 Since S/MIME version 4.0, appropriately sized means keys that are between 2,048 and 4,096 bits long.

21 Since e-mail is not an interactive application, it is not immediately clear how a key agreement, using, for example, a Diffie-Hellman (DH) key exchange, can take place. Note that a normal DH key exchange requires a message to be transmitted in either direction, but that e-mail employs only one message to be transmitted from a sender to one or multiple recipients. There is no possibility of having a message sent from the recipient to the sender, and at least the recipient must employ a static DH public key (and a respective certificate). Only the sender can employ an ephemeral DH public key. So DH comes in two flavors in the realm of S/MIME: ephemeral-static (E-S) DH (if the sender employs an ephemeral DH public key) and static-static (S-S) DH (if the sender also employs a static DH public key) [27]. In either case, DH can be used to exchange a key, but this key should not be directly used to encrypt a message. Instead, it is better to use it to encrypt a message key that is then used to encrypt the actual message content. So if DH is used, then a key-encryption (or key wrapping) algorithm must also be used [16]. Respective key wrapping algorithms are provided in [20] and [28].

support the cryptographically more advanced version RSA-OAEP according to [21].

- With regard to message encryption, S/MIME version 4.0 implementations MUST support AES-128 GCM and AES-256 GCM [29]. Furthermore, they MUST- support AES-128 in CBC mode [28], and they SHOULD+ support ChaCha20-Poly1305 originally designed for the SSL/TLS protocols [30].
- With regard to compression, S/MIME version 4.0 refers to [19] and—similar to previous versions—mandates the use of DEFLATE [31] and ZLIB [32] as introduced in Section 5.2.5.4.

Note that each algorithm may have an OID assigned to it. Also note that cryptographic algorithms can be broken or weakened over time, and that implementers and users should therefore periodically check that the algorithms that have been deployed continue to provide the expected level of security. To support this, the IETF occasionally issues documents—mainly informational RFCs—that deal with specif c attacks and their implications for Internet security protocols in general, and S/MIME in particular (e.g., [33–35]). These documents should be taken into consideration when implementing a cryptographic algorithm and discussing its security. Some agility is required here.

6.2.3 Signer Attributes

Earlier in this chapter, we said that digitally signed data must come along with an ASN.1 data structure of type `SignerInfo` for each signature, and that this data structure yields information about what algorithms and public key to use to verify the signature. The `SignerInfo` data structure also allows the inclusion of (unsigned and signed) attributes—let's call them signer attributes—along with a signature.

Sending MUAs should always generate one instance of each of the following signed attributes in each S/MIME message, whereas receiving MUAs must be able to handle zero or one instance of each attribute. The f rst two attributes are def ned in [16], whereas the other three attributes are def ned in [14].

- *Content type:* This signed attribute refers to the content type of the signed data.
- *Message digest:* This signed attribute refers to the message digest (or hash value) of the signed data. The hash algorithm is determined by the signer.
- *Signing time:* This signed attribute refers to a timestamp and conveys the time that the signer signed the message. The attribute is created by the signer of a

message, and it is therefore only as trustworthy as the signer and his or her local clock.

- *SMIME capabilities:* This signed attribute refers to the cryptographic capabilities of the signer as far as they are relevant for S/MIME. This includes, for example, digital signature, symmetric encryption (with or without authentication), and key exchange algorithms[22] in order of their preference.
- *Encryption key preference:* This signed attribute allows the signer to unambiguously describe which of the signer's certif cates embodies the signer's preferred encryption key. This is particularly useful if the signer has multiple certif cates or separate keys for encryption and signing. It is up to the receiving MUA to respect the preference(s) expressed in the attribute for the encryption of future messages.

In addition to these attributes, sending MUAs should generate one instance of the signing certif cate attribute (Section 6.2.4.4) in each `SignerInfo` data structure, and receiving MUAs should be able to handle zero or one such instance. Additional attributed and values for these attributes may be def ned in the future.

Anyway, interactive sending MUAs that may want to include signed attributes that are not mentioned here should display those attributes to the user, so that the user is aware of all data being signed. Also, receiving MUAs should handle attributes or values they do not recognized in a graceful manner, meaning that they should soft-fail whenever they come into touch with unknown attributes.

6.2.4 Enhanced Security Services

The enhanced security services for S/MIME are specif ed [8] and partly updated in [36]. They comprise signed receipts, security labels, secure mailing lists, and signing certif cates as introduced and brief y discussed next.

6.2.4.1 Signed Receipts

As its name suggests, the idea of the signed receipts extension is to have the recipient of a message automatically (i.e., without user interaction) return a signed receipt to the originator to serve as a proof of message delivery. The proof allows the originator to argue that the recipient has in fact received the message and has been able to verify the signature. Note that the extension is relevant and applicable only to messages that are digitally signed in the f rst place. A message that is not digitally signed can be changed at will, and hence a receipt for such a message is not particularly useful.

22 As usual, the algorithms are referenced by their respective OID values.

Also note that the recipient of a message may additionally encrypt the (signed) receipt to protect its conf dentiality.

The signed receipts extension works as follows: The originator digitally signs and sends out a message for which he or she wants to get signed receipts. As the message is digitally signed, it comes along with a `SignerInfo` data structure. To request signed receipts, the originator must add a `receiptRequest` attribute to the list of (signed) attributes of the `SignerInfo` data structure. Note that there may be multiple `SignerInfo` data structures that refer to a signed message,[23] and therefore each of the data structures may have a distinct `receiptRequest` attribute. In either case, the recipient (or the recipient's MUA, respectively) should automatically create a signed receipt and return it to the requester in accordance with various options, such as the mailing list expansion, conf guration, and some local security policy options.

The usefulness of the signed receipts extension is controversially discussed in the community. If the parties involved are honest and play by the rules, then the extension seems to work and fulf ll its intended purpose. In this case, however, one can also argue that signed receipts are not required in the f rst place. If, on the other side, the parties involved are not honest and do not play by the rules, then the extension is pointless because a misbehaving recipient can always ignore the `receiptRequest` attribute and not issue a signed receipt in the f rst place (so the originator does not receive a proof of message delivery). The bottom line is that the signed receipts extension only seemingly solves the proof of message delivery problem, and that more involved and sophisticated solutions are needed for certif ed mail (e.g., [37, 38]). This topic is not further addressed here.

6.2.4.2 Security Labels

In general parlance, a security label refers to some security-related information assigned to an object. In the realm of secure messaging, such an object may be (the content of) a message that is cryptographically protected, for example, using S/MIME. Hence, the idea of the security labels extension is to provide a syntax for security labels that can be used for authorization and access control decisions. An MUA receiving a labeled message can then decide whether the recipient is authorized to see the content of the message. Again, the feature is available only for messages that are digitally signed.

The security labels extension works as follows: The originator digitally signs and sends out a message that may carry a security labels attribute (included in the set of signed attributes that come along with the `SignerInfo` data structure).

23 For example, an originator can send a signed message with two SignerInfo data structures, one containing a DSS signature and one containing an RSA signature.

According to ITU-T recommendation X.411, there are six predef ned values for security labels (i.e., *unmarked*, *unclassif ed*, *restricted*, *conf dential*, *secret*, and *top-scret*) but anybody can def ne and add arbitrary values at will—following, for example, [39] to implement a company classif cation policy. In either case, the recipient of the message can examine the attribute and use it to decide whether or not the recipient is allowed to see the content of the message.

Providing support for security labels and def ning a security labels extension for S/MIME is certainly a good idea. The problem, however, is that security labeling works well in theory, but is hard to achieve in practice. In fact, we have been trying to deploy security labels in the f eld for decades—without any meaningful success. This is not going to change, simply because there is now a def ned way for using security labels in the realm of S/MIME. The other problem is similar to the signed receipts extension: We have to assume a recipient who is honest and plays by the rules. If this assumption is wrong, then everything is possible. Again, any misbehaving recipient can simply ignore all security labels and provide unrestricted access to everybody.

6.2.4.3 Secure Mailing Lists

If a message must be sent to a large number of recipients in a secure way, then it might be useful to off oad message encryption to an entity known as *mail list agent* (MLA). The MLA appears to be a normal recipient, but it acts as a message expansion point for a mailing list. This basically means that the MLA receives a message and redistributes it to all members of a mailing list. The secure mailing lists extension is to enable this mechanism. This is relatively simple and straightforward. Special care must only be taken so that the secure redistribution of a message cannot lead to a mail loop.

6.2.4.4 Signing Certif cates

In principle, a public key can have multiple certif cates. In this case, it is not immediately clear what certif cate should be used to verify a signature that is generated with a respective private key. There are a few attacks that may be mounted against signature verif cation by substituting or replacing a certif cate. Three such attacks are, for example, outlined in [8]. To mitigate such attacks, it may be useful to restrict the set of certif cates that may be used to verify a signature. This is where the signing certif cates attribute comes into play. Again, this attribute is part of the signed attributes section of the `SignerInfo` data structure; it allows the signer to specify the certif cate(s) that is (are) appropriate to use.

According to [8], the cryptographic hash function SHA-1 is used to identify a certif cate by its hash value. Due to some recent attacks against SHA-1, people have concluded that it may be appropriate to add some algorithmic agility here, and to make it possible to replace SHA-1 with some other cryptographic hash function if appropriate. This is where the update [36] comes into play: It adds algorithm agility and makes the identif cation mechanism for certif cates more f exible.

6.3 CERTIFICATES

S/MIME largely depends on X.509 certif cates and a hierarchical trust model (as introduced and brief y discussed in Section 3.3). So, from a theoretical viewpoint, there is not much to say here. But from a practical viewpoint, there are still a few questions that must be answered before S/MIME can be deployed on a large scale. If, for example, an MUA is to send out a message that is digitally signed, then it must have access to the appropriate private (signing) key. How can this access be granted to the MUA, but not to anybody else? In the simplest case, this is achieved by software, and the respective solutions are software-based (with all vulnerabilities and security problems that are inherent to software). In the more involved case, it is achieved by some dedicated hardware—ranging from smartcards and USB tokens to hardware security modules (HSM). In this case, availability and usability are major concerns. On the other side, a receiving MUAs must also have access to the originator's public key (or public key certif cate, respectively). This is where the notion of a PKI comes into play. Many companies and organizations have tried to establish and operate a PKI for the Internet—most of them have vanished and disappeared, and we still don't have an Internet PKI that can be used for S/MIME. In fact, there are only a fractional number of users equipped with proper S/MIME certif cates. Typically, these are employees of (large) organizations that have an internal PKI. Other users are seldom willing to spend money to buy a certif cate from any of the commercially operating CAs or CSPs.

In general, the market for S/MIME certif cates is dynamic and constantly changing, and hence it represents a moving target. There are several providers competing for market share, and most of them have different offerings for different customers (with different security requirements). All of them provide high-end certif cates with respective price tags. But some of them also provide free certif cates that can also be used for S/MIME. The rationale behind these offerings is to promote the technology and to boost the market (so that it may become big and prolif c in the future). Sometimes, the free certif cates provided this way have a relatively short lifetime (e.g., a few days up to one month). Similar to Lets Encrypt[24] in the realm

24 https://letsencrypt.org.

of SSL/TLS certif cates, there are also some community-driven initiatives to freely distribute S/MIME certif cates. An example of this type is CAcert.[25] To get a CAcert certif cate, a user must become a member of CAcert.org and agree to the respective community agreement. A major disadvantage of using a CAcert certif cate is that the respective root certif cates are not included in the most widely deployed certif cate stores.

A question that is sometimes discussed controversially in the community is whether a user's public key pair (of which the public key is part of the certif cate) should be generated locally (i.e., on his or her own computer system) or not. In the second case, it can be generated on a specif cally designed key generation server that may have a built-in randomness source. There are advantages and disadvantages on either side: If the key pair is generated locally, then it can be ensured that it exists only locally, by using a software like OpenSSL to generate the key pair and a respective certif cate signing request (CSR) that is sent to the CA or CSP. In this case, the private key never leaves the user's computer system. The f ip side is that this system may have a poor randomness source, meaning that there is no guarantee that the key pair that is generated in cryptographically strong. If, however, the key pair is generated outside the computer system, then the randomness source can be strictly controlled and strengthened, but in this case the component that generates the key pair must have access to it. So there is no guarantee that a copy of the private key is not stored externally (from the user or user's computer system).

Last but not least, instead of using a full-f edged (X.509 or S/MIME) certif - cate, one may also use a self-signed certif cate. In addition to OpenSSL, there are many other tools that can be used to play around with such certif cates and generate them at will. In the general case, the value of a self-signed certif cate is not particularly high (because the identity of the certif cate holder is neither verif ed nor guaranteed), but in some situations, they still provide a reasonable level of security. If all that needs to be guaranteed is that a user is the same as the last time, then self-signed certif cates are reasonable and perfectly f ne. In e-commerce, for example, there are surprisingly many settings in which only this fact needs to be ensured. As an example, you may consider a prepaid service. The true identity of the user does not really matter, but it must be ensured that the user is the same as the one that paid for the service at some point in the past. This can be easily achieved with self-signed certif cates.

25 http://www.cacert.org.

6.4 SECURITY ANALYSIS

Most things we said when we analyzed the security of OpenPGP in Section 5.4 also apply to S/MIME. This is particularly true for the distinction between the security of the S/MIME specif cation and the security of a particular implementation.

- With regard to the specif cation, the fact that S/MIME is based on well-established security standards really pays off. The cryptographic vulnerabilities and subtleties that had enabled EFAIL and related attacks led to a major revision of the standard (and the respective RFC documents). The resulting S/MIME version 4 seems to mitigate most of these attacks—at least as far as they are cryptographic and cryptanalytic in nature. The introduction and use of authenticated encryption is certainly the biggest change and improvement here, but there are also some minor changes and improvements, such as the introduction and use of SHA-2, HMAC, RSA-OAEP, and RSA-PSS (Section 6.2.2), as well as the generally larger key sizes (e.g., between 2048 and 4096 bits for RSA). Note, however, that some problems are not cryptographic but due to the overly large functionality provided by currently deployed S/MIME-enabled MUAs, and hence that these problems cannot be solved cryptographically.
- With regard to a particular implementation, the situation is more subtle and diff cult to assess. There are so many things that can go wrong that it is inherently diff cult to make any statement here. As is usually the case, the devil is in the details.

From a practical viewpoint, there are two major concerns that affect the security of S/MIME and S/MIME-enabled MUAs. The f rst concern refers to the way public key pairs and respective private keys are generated. If this generation process is not fed with enough randomness (entropy[26]), then the resulting keying material may be easy to guess (and hence be insecure). This clearly undermines security, and hence the key generation process is key to the security of the respective MUA. The same is true for the entire key management process. If, for example, a key is stored in memory in some unprotected way, then it is usually simple to f nd and extract it from a memory dump. The second concern refers to the way a user is interfaced to the MUA and his or her cryptographic key(s). This is a general

26 Note that the term *entropy* is not unique, and that there are (at least) three measures: *min-entropy*, *Shannon entropy*, and *max-entropy*. All measures are greatest, for a given number of outcomes, when each outcome occurs with equal probability. In this case, all measures are equal. Otherwise, the min-entropy is less or equal to the Shannon entropy, and the Shannon entropy is less or equal to the max-entropy. In this particular case, the term entropy refers to the min-entropy (and not the Shannon entropy most people think of), but let's not further delve into this topic.

concern for any cryptographic software, but it is a particularly large concern in the realm of secure and E2EE messaging. If an adversary has access to a user's private key, then he or she can do anything the user is authorized to do. He or she can, for example, digitally sign messages and decrypt messages with his or her private key, so properly protecting private keys is key to the security of S/MIME and any implementation thereof. Because of this situation, many vendors of S/MIME-enabled MUAs are looking into possibilities to use hardware-based certif cates (e.g., smartcards or USB tokens) and to adequately protect the users' private keys and the computational processes performed with these keys. This is clearly the Achilles' heel (and the primary point of attack) for any S/MIME implementation—it also applies to many other cryptographic software packages.

6.5 FINAL REMARKS

As stated at the beginning of this chapter, we have focused solely on S/MIME and the way it is specif ed in the relevant RFC documents. People interested in implementing S/MIME may refer to [40] for exemplary S/MIME messages—at least when it comes to a prior version of S/MIME. Keep in mind that S/MIME, by its nature, is not restricted to user-initiated asynchronous messaging on the Internet. Instead, it can also be used in automated MTAs and systems that do not require human interaction at all, such as the signing of software-generated documents, HTTP traff c that refers to MIME entities, or even the encryption of fax messages sent over the Internet. In fact, S/MIME can be used to secure any system that can transport MIME entities, and it is thus possible that we will see many complementary and innovative applications of S/MIME in the future. Secure messaging is just the f rst use case for S/MIME that comes to mind.

References

[1] RSA Data Security, *S/MIME Implementation Guide*, Interoperability Prof le, Version 1, August 1995.

[2] Dusse, S., et al., "S/MIME Version 2 Message Specif cation," RFC 2311, March 1998.

[3] Dusse, S., et al., "S/MIME Version 2 Certif cate Handling," RFC 2312, March 1998.

[4] Housley, R., "Cryptographic Message Syntax," RFC 2630, June 1999.

[5] Rescorla, E. "Diff e-Hellman Key Agreement Method," RFC 2631, June 1999.

[6] Ramsdell, B. (Ed.), "S/MIME Version 3 Certif cate Handling," RFC 2632, June 1999.

[7] Ramsdell, B. (Ed.), "S/MIME Version 3 Message Specif cation," RFC 2633, June 1999.

[8] Hoffman, P. (Ed.), “Enhanced Security Services for S/MIME,” RFC 2634, June 1999.

[9] Ramsdell, B. (Ed.), “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.1 Certif cate Handling,” RFC 3850, July 2004.

[10] Ramsdell, B. (Ed.), “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.1 Message Specif cation,” RFC 3851, July 2004.

[11] Ramsdell, B., and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.2 Certif cate Handling,” RFC 5750, January 2010.

[12] Ramsdell, B., and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 3.2 Message Specif cation,” RFC 5751, January 2010.

[13] Ramsdell, B., and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Certif cate Handling,” RFC 8550, April 2019.

[14] Ramsdell, B., and S. Turner, “Secure/Multipurpose Internet Mail Extensions (S/MIME) Version 4.0 Message Specif cation,” RFC 8551, April 2019.

[15] Galvin, J., et al., “Security Multiparts for MIME: Multipart/Signed and Multipart/Encrypted,” RFC 1847, October 1995.

[16] Housley, R., “Cryptographic Message Syntax (CMS),” RFC 5652, September 2009.

[17] Turner, S., and J. Schaad, “Multiple Signatures in Cryptographic Message Syntax (CMS),” RFC 5752, January 2010.

[18] Kaliski, B., “PKCS #7: Cryptographic Message Syntax Version 1.5,” RFC 2315, March 1998.

[19] Gutmann, P., “Compressed Data Content Type for Cryptographic Message Syntax (CMS),” RFC 3274, June 2002.

[20] Housley, R., “Cryptographic Message Syntax (CMS) Algorithms,” RFC 3370, August 2002.

[21] Housley, R., “Use of the RSAES-OAEP Key Transport Algorithm in the Cryptographic Message Syntax (CMS),” RFC 3560, July 2003.

[22] Schaad, J., “Use of the RSASSA-PSS Signature Algorithm in Cryptographic Message Syntax (CMS),” RFC 4056, June 2005.

[23] Turner, S., “Using SHA2 Algorithms with Cryptographic Message Syntax,” RFC 5754, January 2010.

[24] Housley, R., “Use of Edwards-Curve Digital Signature Algorithm (EdDSA) Signatures in the Cryptographic Message Syntax (CMS),” RFC 8419, August 2018.

[25] Turner, S., and D. Brown, “Use of Elliptic Curve Cryptography (ECC) Algorithms in Cryptographic Message Syntax (CMS),” RFC 5753, January 2010.

[26] Housley, R., “Use of the Elliptic Curve Diff e-Hellman Key Agreement Algorithm with X25519 and X448 in the Cryptographic Message Syntax (CMS),” RFC 8418, August 2018.

[27] NIST Special Publication 800-57, “Recommendation for Key Management, Part 1: General,” Revision 4, January 2016.

[28] Schaad, J., "Use of the Advanced Encryption Standard (AES) Encryption Algorithm in Cryptographic Message Syntax (CMS)," RFC 3565, July 2003.

[29] Housley, R., "Using AES-CCM and AES-GCM Authenticated Encryption in the Cryptographic Message Syntax (CMS)," RFC 5084, November 2007.

[30] Langley, A., et al., "ChaCha20-Poly1305 Cipher Suites for Transport Layer Security (TLS)," RFC 7905, June 2016.

[31] Deutsch, P., "DEFLATE Compressed Data Format Specif cation version 1.3," RFC 1951, May 1996.

[32] Deutsch, P., and J-L. Gailly, "ZLIB Compressed Data Format Specif cation version 3.3," RFC 1950, May 1996.

[33] Zuccherato, R., "Methods for Avoiding the 'Small-Subgroup' Attacks on the Diff e-Hellman Key Agreement Method for S/MIME," RFC 2785, March 2000.

[34] Rescorla, E., "Preventing the Million Message Attack on Cryptographic Message Syntax," RFC 3218, January 2002.

[35] Hoffman, P., and B. Schneier, "Attacks on Cryptographic Hashes in Internet Protocols," RFC 4270, November 2005.

[36] Schaad, J., "Enhanced Security Services (ESS) Update: Adding CertID Algorithm Agility," RFC 5035, August 2007.

[37] Oppliger, R., "Certif ed Mail: The Next Challenge for Secure Messaging," *Communications of the ACM*, Vol. 47, No. 8, August 2004, pp. 75–79.

[38] Oppliger, R., "Providing Certif ed Mail Services on the Internet," *IEEE Security & Privacy*, Vol. 5, No. 1, January/Februrary 2007, pp. 16–22.

[39] Nicolls, W., "Implementing Company Classif cation Policy with the S/MIME Security Label," RFC 3114, May 2002.

[40] Hoffman, P. (Ed.), "Examples of S/MIME Messages," RFC 4134, July 2005.

Chapter 7

Evolutionary Improvements

OpenPGP and S/MIME are conceptually similar and mostly represent conventional approaches and solutions for secure and E2EE messaging. As discussed before, they are diff cult to use and therefore lack wide deployment. Consequently, there have been some attempts to change this. The changes are neither fundamental nor radical, and hence the respective improvements are being called evolutionary here. This is the topic of this chapter. More specif cally, we introduce and discuss WKD and WKS in Section 7.1, the use of the DNS to distribute public keys in Section 7.2, opportunistic encryption in Section 7.3, and Web-based solutions in Section 7.4. We conclude Section 7.5 with some f nal remarks.

7.1 WKD AND WKS

We have brief y discussed the notion and use of PGP public key servers—including SKS—in Section 5.3.4. With regard to their deployment and use in the f eld, there are two major problems and obstacles:

- First, users must manually upload their public keys to these servers. If a user does not upload his or her public key, then it cannot be served to other users.
- Second, there is no guarantee that an e-mail address assigned to a public key is genuine and legitimate. Anybody can assign any e-mail address of his or her choice to a public key. The assignment may not be trusted by anybody (because there is no signature that vouches for it), but it is still technically feasible and may lead to confusion and user misbehavior. Also, a user may upload multiple public keys to a key server, in which case it is not obvious for a sender what key to retrieve and use to encrypt a message for that particular

user. Alternatively speaking, the public key selection process is ambiguous per se.

Both problems can be partly solved, if the PGP public keys are served by the users' e-mail providers (instead of some independent third parties). An e-mail provider has a unique address for each of its users, and it can even automate the upload process of the respective public key. This idea was proposed by Werner Koch—the original creator of GPG—and he coined the term *Web Key Directory* (WKD) to refer to such a PGP public key server (that is hosted by an e-mail provider).[1] Consequently, a WKD is just a Web server operated by an e-mail provider that can be queried with well-formed URLs to serve a particular public key. If, for example, Joe Doe has an e-mail address `Joe.Doe@example.org`, then his PGP public key can be served by the WKD running at `https://openpgpkey.example.org`, and the URL to retrieve this key may look as follows (written in one string):[2]

```
https://openpgpkey.example.org/.well-known/openpgpkey/
example.org/hu/iy9q119eutrkn8s1mk4r39qejnbu3n5q?l=
Joe.Doe
```

In this URL, the string `iy9q119eutrkn8s1mk4r39qejnbu3n5q` refers to the Z-Base-32-enoded[3] SHA-1 hash value of the local part of the e-mail address in lowercase letters. The resulting hash value is 160 bits long, and these 160 bits can be split into 32 5-bit blocks. According to Z-Base-32 encoding, each of these 5-bit blocks can then be encoded with a unique character. In response to this URL, the WKD returns Joe Doe's OpenPGP public key that can then be used by the sender to encrypt a message.

This method of retrieving public keys from a WKD is called *advanced*. It requires a subdomain named `openpgpkey`, and hence an additional DNS entry. If this is not possible or not desirable, then a *direct* method can be used instead. In this case, the URL may look as follows:

```
https://example.org/.well-known/openpgpkey/
hu/iy9q119eutrkn8s1mk4r39qejnbu3n5q?l=Joe.Doe
```

Independent from the method (advanced or direct), a WKD can be conf gured manually to serve the proper public keys. But manual conf guration does not scale

1 This is work in progress and specif ed in an Internet-Draft.

2 The example is taken from the Internet-Draft mentioned in footnote 1.

3 The Z-Base-32 encoding scheme was originally proposed by Bryce Wilcox-OHearn and is specif ed in http://philzimmermann.com/docs/human-oriented-base-32-encoding.txt and documented in [1]. It differs from normal Base-32 [2] to represent bit sequences in a form that is convenient for human users to manipulate with minimal ambiguity. There are some online tools that can be used for Z-Base-32 encoding and decoding, such as the ones provided at https://cryptii.com/pipes/z-base-32.

well, and hence the *Web Key Service* (WKS) refers to a set of protocols and tools that can be used to automatically publish and update OpenPGP public keys in a WKD. For the large-scale deployment of WKDs, the use of a WKS seems to be mandatory.

The future success of WKD and WKS is attached to OpenPGP: If OpenPGP is more widely used in the f eld, then we will see WKD and WKS be more widely deployed as well. But this is not the most likely scenario, and it seems more probable that WKD and WKS will only be deployed in a few situations where OpenPGP plays a role today.

7.2 DNS-BASED DISTRIBUTION OF PUBLIC KEYS

As an alternative to PGP public key servers and WKD/WKS, one may also think about using DNS and DANE (as brief y introduced in Section 3.3.2.2) to distribute public keys. DANE is mainly used on the Web, but it can also be used in the realm of secure and E2EE messaging. Depending on whether OpenPGP or S/MIME public keys need to be distributed, there are two distinct RFCs relevant here. They are both experimental and not (yet) submitted to the Internet standards track.

- RFC 7929 [3] specif es how DNS and DANE can be used to distribute OpenPGP public keys, in particular using a new OPENPGPKEY resource record (with type 61);
- RFC 8162 [4] does the same for S/MIME, using a new SMIMEA resource record (with type 53).

OPENPGPKEY and SMIMEA resource records provide a convenient way to distribute public keys using DNS and DANE. The disadvantage of this approach, however, is that the appropriate and secure use of DANE also requires and depends on DNSSEC (Section 2.2.3.2), and that DNSSEC is not yet widely deployed—at least not as widely as originally anticipated. As soon as this changes, OPENPGPKEY and SMIMEA resource records can certainly be more widely used in the f eld. Today, they still have a shadowy existence.

7.3 OPPORTUNISTIC ENCRYPTION

Informally speaking, *opportunistic encryption* refers to an approach in which one tries to invoke encryption whenever possible, and falling back to no encryption otherwise [5]. The approach was f rst introduced in the realm of messaging with the SMTP STARTTLS extension, but it is now also being explored in context of

OpenPGP (and sometimes S/MIME). The most prominent examples are *Autocrypt*, *Pretty Easy Privacy* (pEp or p≡p), and the *LEAP Platform* that is being developed as part of the LEAP Encryption Access Project[4] (but not further addressed here). In either case, the idea is to have the MUAs handle the key management and invocation of OpenPGP on the user's behalf in a way that is as convenient and transparent as possible. From the user's perspective, the result is similar to a gateway solution, such as the one that was pioneered by PGP Universal Server and that is nowadays being provided by many other (conceptually similar) products.

7.3.1 Autocrypt

Following the principles of opportunistic encryption and trust of f rst use (TOFU), some people has developed and publicly released the *Autocrypt*[5] specif cation that can be implemented and integrated into existing MUAs. It helps the user invoke OpenPGP. The actual message processing is still being done with OpenPGP, meaning that the messages that are sent back and forth are OpenPGP messages, but the user interface is greatly simplif ed. Autocrypt Level 1 was released in 2017 and employs 3072-bit RSA keys to digitally sign and envelope messages. As of this writing, Autocrypt Level 2 is still under development and it is currently unknown when it will be released.

The mode of operation of Autocrypt is relatively simple and straightforward: Each time an Autocrypt-enabled MUA sends out a message, it adds an `Autocrypt:` header that provides the recipient(s) with the originator's e-mail address and public key (in base-64 encoded form), as well as some optional parameters, such as a preference for encryption. If a message is sent to multiple recipients, then the `Autocrypt:` header is replaced with an `Autocrypt-Gossip:` header. To defeat the pssibility of using such headers to spread wrong keys, `Autocrypt:` headers are always preferred to `Autocrypt-Gossip:` headers.

If, for example, I use an Autocrypt-enabled MUA to send you a message, then the `Autocrypt:` header looks as follows:

```
Autocrypt: addr=rolf.oppliger@esecurity.ch;
   prefer-encrypt=mutual; keydata=...
```

The `addr` parameter has your MUA (if it is also Autocrypt-enabled) assign the keying material provided with the `keydata` parameter (indicated with three dots) to my e-mail address. As mentioned above, the keying material is basically a 3072-bit public RSA key that conforms to the OpenPGP specif cation. Furthermore,

4 https://leap.se.
5 https://autocrypt.org.

the `prefer-encrypt` parameter informs your MUA that encryption should always be invoked and take place mutually. The next time your (Autocrypt-enabled) MUA were to send me a message (or to `rolf.oppliger@esecurity.ch`, respectively), it would grab my public RSA key from its local key repository and use it to digitally envelope the message. It would also use your private RSA key to digitally sign the message, so an encrypted message is always digitally signed. Most importantly, your MUA would also include an `Autocrypt:` header that is structured identically to the one indicated above but provide my MUA with your keying material (i.e., your 3072-bit public RSA in OpenPGP format). My MUA would locally store this key and assign it to your e-mail address. Due to the preference for encryption (i.e., `prefer-encrypt=mutual`) my MUA would thereafter digitally envelope and sign all messages sent to you with the proper keying material. In group communication (i.e., messages sent to multiple recipients) the message f ow is similar but invokes the `Autocrypt-Gossip:` header mentioned above.

If a user wants to use Autocrypt on multiple MUAs, then he or she can transfer the Autocrypt settings with a specif cally crafted setup message that is symmetrically encrypted with AES and a randomly chosen transfer key. Obviously, each MUA must know the transfer key to decrypt the setup message and read in the setting. As of this writing, this can only be achieved by having the user manually type in the transfer key encoded in 36 digits. This is not particularly user-friendly, but it works, also because users generally don't use many MUAs simultaneously. So there are usually only a few key transfers that need to take place.

From a security viewpoint, the single most important shortcoming of Autocryp is its lack of authenticity for keydata. Note that this data is only included in the `Autocrypt:` and `Autocrypt-Gossip:` headers, and that it is not authenticated at all. Instead, Autocrypt depends on TOFU, meaning that it trusts and uses the f rst key it receives for a particular user or e-mail address. There are neither certif cates nor other trust assumptions in place. A MUA takes whatever keying material it receives and uses it in an opportunistic way. This is overly simple, but it works reasonably well in many real-world situations. The issue of how to authenticate keydata is not considered in Autocrypt Level 1. It is certainly a research topic that will be addressed in Autocrypt Level 2. One possibility that is currently being discussed is the use of DKIM signatures to at least provide a basic level of authenticity.

There are several Autocrypt Level 1 implementations available today (see, for example, `https://autocrypt.org/dev-status.html` for a respective overview). Some MUAs, like Mailpile,[6] DeltaChat,[7], and K-9 Mail,[8] natively

6 https://www.mailpile.is.
7 https://delta.chat.
8 https://k9mail.github.io.

support Autocrypt,[9] whereas other MUAs can be complemented with Autocrypt-enabled plug-ins, like Autocrypt[10] and Enigmail.[11] Also, there are some tools available that can be used to implement Autocrypt Level 1 compliant MUAs, such as muacrypt,[12] PyAC,[13] and GMime.[14]

7.3.2 p≡p

The rationale behind p≡p is similar to Autocrypt. Both try to implement opportunistic encryption and TOFU in a way that is as simple and transparent to the user as possible. In contrast to Autocrypt, p≡p is not only a specif cation, but also comes with open source software implementations (i.e., a p≡p engine and several adapters that can be used to p≡p-enable any type of communication software—be it an MUA for e-mail or any type of messaging app). The software is owned and further developed by the p≡p Foundation,[15] but it is also marked by a company called p≡p Security[16] located in Switzerland and Luxembourg. This company uses the p≡p engine and adapters to build commercial software for Microsoft Outlook, Android, and iOS. There is also a (freely available) extension of Enigmail for Firefox that supports p≡p.

The mode of operation of p≡p is similar to Autocrypt, but it is slightly more sophisticated. If a MUA is p≡p-enabled, then the p≡p engine generates a key pair and installs it locally. By default, this is a 4,096-bit RSA key pair. The p≡p engine then analyzes all messages that are sent or received from this MUA, and invokes the p≡p functionality automatically (i.e., without user invocation). It therefore supports multiple standards (i.e., OpenPGP, S/MIME, and OTR) and is able to invoke them dynamically.

In contrast to Autocrypt, p≡p supports a simple form of trust management. In particular, it distinguishes different security levels and statuses for a particular communication relationship and respective messages:

- The security level is *grey*—unknown and insecure—if the sender has no keying material to protect a message sent to the recipient. If, for example, a message is sent to a formerly unknown address, then the respective security level is grey. This is usually the case for a new communication partner.

9 As of this writing, DeltaChat is only available for Android and K-9 Mail for Android and MacOS. Mailpile is currently the only MUA that is available for all major platforms.
10 https://addons.thunderbird.net/en-US/thunderbird/addon/autocrypt.
11 https://www.enigmail.net.
12 https://pypi.org/project/muacrypt.
13 https://pyac.readthedocs.io.
14 https://developer.gnome.org/gmime.
15 https://pep.foundation.
16 https://www.pep.security.

- The security level is *yellow*—secure—if a message is encrypted but (the recipient) not yet properly authenticated.
- The security level is *green*—secure and trusted—if a message is encrypted and (the recipient) is trusted, meaning that it is authenticated and conf rmed with a handshake using a separate communication channel. The standard way to establish trust is to use a phone call and have both parties read loudly a trustword that can be verif ed and hence conf rmed by the other party. Here, a trustword is just a sequence of English words, like CAR HORSE BATTERY STAPLE APPLE, and it is conceptually similar to a f ngerprint used in other settings.
- The security level is *red*—mistrusted or under attack—if there are reasons to believe that somebody has successfully mounted an attack, such as a MITM attack.

Note that a handshake needs to be done only once between two communicating parties and the respective (p≡p-enabled) MUAs. If the handshake succeeds, then all messages that are received from or send to this party inherit the green status that, by the way, is visualized in the GUI of the MUA. Also note that the privacy status of a message is always shown, even when composing it. If it is yellow or green, then the user has the option to disable protection, meaning that the MUA will send the message unencrypted. This is certainly not the preferred choice from a security perspective (since it disables protection), but it is still one the user has.

Let us assume that I use a p≡p-enabled MUA and you want to send me a message. Since your MUA does not yet know my key, it has to send an unencrypted message to me. The security level of this message is grey, and it carries your public key. My MUA can decode this key and store it localy for later use. So when I send back a message to you, my MUA grabs this key and digitally envelopes it with this key. The security level of this message thus switches to yellow. The message my MUA sends out additionally carries my public key that your MUA may store. This can continue until we decide to challenge our trustworthiness and do a handshake. We therefore use our phones and compare the trustwords (that are distinct). If everything is f ne, then all future messages have a security level that is green. This is the f nal and most trustworthy status.

By default, a p≡p-enabled MUA attaches the public key of its user to all outgoing messages. But if the user sets p≡p to passive mode, then this behavior changes and the public key in no longer attached. More specif cally, it attaches the key if and only if it detects that the receiving MUA is also p≡p-enabled.

Furthermore, it is often questionable whether originally encrypted messages should be stored in encrypted or decrypted form. With regard to this question, p≡p

does not provide a unique answer but supports either possibility. In a cloud setting, for example, people often prefer to store the messages in encrypted form, whereas they discard this requirement, if a mail server is hosted on premise.[17] Sometimes people want to decide on a per-account basis. In p≡p for Outlook, for example, there is an option called store messages securely for all accounts that causes messages to be stored only in encrypted form. If this option is disabled, then the user can choose for each of his or her accounts individually by checking or unchecking the respective store messages securely setting.

As of this writing, p≡p is still a moving target and comes along with several questions that are under investigation. For example, people are struggling with the muliple-device setting, in which a user may have and simultaneously use multiple p≡p-enabled MUAs. What is the optimal synchronization strategy here? An early attempt was to use IMAP to synchronize the MUAs, but this attempt has turned out to have some shortcomings and problems. Another research question is how to add anonmyity to p≡p, or—more generally—how to protect meta information about the messages that are sent and received. With regard to this question, p≡p cooperates with GNUnet[18] that is basically an overlay network on the existing Internet to provide a new protocol stack for building secure, distributed, and privacy-preserving applications. It is certainly less widely deployed than, for example, TOR.

7.4 WEB-BASED SOLUTIONS

Web-based messaging systems, such as Gmail, Yahoo Mail, and Outlook.com, are very popular and widely used today. Consequently, people have tried to f nd ways to combine these systems with OpenPGP or S/MIME, and to make the user experience as comfortable as possible. There are basically two approaches to achieve these goals, and these approaches have led to two types of Webmail solutions:

- On the one hand, there are many (mostly open source) browser add-ons (for Firefox) and extensions (for Chrome) that support existing Webmail solutions. For example, Google has developed a Chrome extension called *End-To-End* that supports OpenPGP (and OTR).[19] The extension is open source and available on GitHub.[20] Unfortunately, it is not ready for general use, but a

17 One way to decide whether a server is hosted on premise is to look at its IP address: If it is an address from a private (nonroutable) space according to [6] for IPv4 or [7] for IPv6, then it is very likely that a server hosted on premises.

18 https://gnunet.org.

19 https://opensource.google/projects/end-to-end.

20 https://github.com/google/end-to-end.

fork of it has been made available for Yahoo mail.[21] Other examples include FlowCrypt, WebPG, and Mailvelope.[22]

- FlowCrypt[23] is a Firefox add-on and Chrome extension that supports the integration of OpenPGP with Gmail. Other Webmail systems (than Gmail) can be served with a FlowCrypt Android app.
- WebPG[24] is a similar tool that is freely available in all versions and even supports a few more browsers and platforms.
- Mailvelope[25] is a similar tool either, but it has been designed to be compatible with as many Web-based messaging systems as possible (in addition to Webmail).

The greatest common denominator of all these tools is that they try to automate the invocation of OpenPGP or S/MIME as far as possible, so that a user doesn't have to deal with the technical details and subtleties of OpenPGP and S/MIME or the software that implements them. They are hidden and operate under the hood, without having the user necessarily be aware of them. In the most extreme case, the e-mail provider does everything on the user's behalf, such as implemented, for example, in Google's hosted S/MIME offering that has been available for the enterprise use of Gmail since 2017.[26]

With regard to user transparency, there is a caveat to mention here: In 2013, a group of researchers built a tool named Private WebMail (PWM), pronounced poem, that is fully transparent to the user. The evaluation of the tool showed that it is too transparent for the user, and that they were too confused to use it properly [8]. This led to signif cant changes in PWM version 2.0, such as introducing an artif cial delay in the encryption process to enhance user conf dence and providing several inline and context-sensitive instructions and tutorials [9].

- On the other hand, there are a few Webmail solutions that are entirely new, such as Hushmail, ProtonMail, and Tutanota. Hushmail[27] is a commercial mail service provider based in Canada, whereas ProtonMail[28] is based in

21 https://github.com/YahooArchive/end-to-end.

22 There are also a few tools that work similarly in the sense that they can be used to encrypt messages but that neither implement OpenPGP nor S/MIME, such as Encipher.it (https://encipher.it/email-encryption).

23 https://f owcrypt.com.

24 https://webpg.org.

25 https://www.mailvelope.com.

26 https://security.googleblog.com/2017/02/hosted-smime-by-google-provides.html.

27 https://www.hushmail.com.

28 https://protonmail.com.

Switzerland and Tutanota[29] is based in Germany. These services (and probably many more) use SSL/TLS to encrypt all data in transit and OpenPGP to additionally encrypt messages on top of it. Again, the goal is to provide a user experience that lacks the details and subtleties of the cryptographic software in use.

In either case, securely storing the private keys and properly authenticating users are the main challenges and distinguishing features of all these solutions. If the user's private key is stored on the server side, then the overall security and the nonrepudiation properties that can be achieved are questionable. On the other hand, the server has generally more possibilities to invoke special hardware to securely store the key. A similar controversy affects user authentication: Enforcing strong—possibly multifactor—authentication is preferable from a security viewpoint, but it is usually not so convenient to use. Again, there is trade-off that must be discussed very thoroughly in a given application setting. There is no universally optimal solution here, and, as always, the devil is in the details.

7.5 FINAL REMARKS

In this chapter, we have outlined some evolutionary improvements to the conventional approaches and solutions to secure and E2EE messaging (i.e., OpenPGP and S/MIME). This includes WKD and WKS, DNS-based distribution of public keys, opportunistic encryption (with Autocrypt and p≡p), and some Web-based solutions. These approaches and solutions improve the situation a little bit, but they do not provide an ultimate solution. In the medium and long term, they can only be successful if OpenPGP and S/MIME survive the next couple of years. Because this is questionable (to say the least), it is useful to go for more advanced approaches and solutions here. This is what the rest of this book is all about. Starting with OTR addressed in the following chapter, we look at several more modern approaches and solutions to secure and E2EE messaging on the Internet. The ultimate culmination point will be the Signal protocol that is omnipresent today and used in most modern E2EE messengers and messaging apps. Like OpenPGP and S/MIME dominated the secure messaging space in the past, the Signal protocol and its variants clearly dominate the E2EE messaging space today. It is therefore important to understand the protocol and the rationale behind its design.

29 https://tutanota.com.

References

[1] Zimmermann, P., Johnston, A. (Ed.), and J. Callas, "ZRTP: Media Path Key Agreement for Unicast Secure RTP," RFC 6189, April 2011.

[2] Josefsson, S., "The Base16, Base32, and Base64 Data Encodings," RFC 4648, October 2006.

[3] Wouters, P., "DNS-Based Authentication of Named Entities (DANE) Bindings for OpenPGP," RFC 7929, August 2016.

[4] Hoffman, P., and J. Schlyter, "Using Secure DNS to Associate Certif cates with Domain Names for S/MIME," RFC 8162, May 2017.

[5] Dukhovni, V., "Opportunistic Security: Some Protection Most of the Time," RFC 7435, December 2014.

[6] Rekhter, Y., et al., "Address Allocation for Private Internets," RFC 1918, February 1996.

[7] Hinden, R., and B. Haberman, "Unique Local IPv6 Unicast Addresses," RFC 4193, October 2005.

[8] Ruoti, S., et al., "Confused Johnny: When Automatic Encryption Leads to Confusion and Mistakes," *Proceedings of the 9th Symposium on Usable Privacy and Security (SOUPS 2013)*, ACM Press, New York, NY, 2013, Article number 5.

[9] Ruoti, S., et al., "Private Webmail 2.0: Simple and Easy-to-Use Secure Mail," *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (UIST 2016)*, ACM Press, New York, NY, 2016, pp. 461–472.

Chapter 8

OTR

In this chapter, we introduce and discuss OTR and its use in secure and E2EE messaging. We don't do this because OTR is widely used in the f eld, but rather because the Signal protocol has its intellectual roots in it and it has actually paved the way for Signal and many other Signal-based E2EE messengers. If one understands OTR, then it is relatively simple and straightforward to also understand the Signal protocol that is addressed in the next chapter. We start with the origins and history of OTR in Section 8.1, elaborate on the technology employed in Section 8.2, provide a brief security analysis in Section 8.3, and conclude with some f nal remarks in Section 8.4. Note that this chapter is intentionally kept short and that more information is available in the referenced literature and on the OTR homepage.[1]

8.1 ORIGINS AND HISTORY

After the development and standardization of OpenPGP and S/MIME, it was commonly believed that the secure messaging problem was solved, and that public key cryptography provided a viable solution for it: Digital signatures for authentication (and nonrepudiation) and digital envelopes for conf dentiality. It was also believed that the unsuccessfulness and poor adoption of OpenPGP and S/MIME in the f eld was due to a lack of usability, rather than technical inadequacy.

This popular wisdom was challenged by Nikita Borisov, Ian Goldberg, and Eric Brewer in a 2004 paper [1], in which they questioned the adequacy of existing technologies for secure messaging—mostly instant messaging—on the Internet. In particular, they criticized the fact that these technologies neither provide PFS[2] nor

1 https://otr.cypherpunks.ca.

2 Refer to Section 4.2 for the term PFS and related notions of secrecy. In the rest of this chapter, we use the term forward secrecy rather than PFS (as used in the original literature on OTR).

deniable authentication, and that these shortcomings severely limit their usefulness in the field. Note what happens if a long-term private key gets compromised. In this case, all messages that have ever been or will ever be enveloped with this key are compromised, too. The respective damage in terms of confidentiality loss is as large as it can possibly be. Furthermore, many of these messages are digitally signed, and hence carry a cryptographic proof of their origin. This, in turn, means that the originator of such a message cannot legitimately deny having sent it. There are certainly cases in which this undeniability (or nonrepudiation) property does not pose a problem and is in fact desired. But there are also cases in which it poses a huge problem to the originator of a message. Examples include messages sent by whistle-blowers, dissidents, or political activists.

Against this background, the authors of [1] argued that people sometimes want to hold a casual conversation that is private, informal, and unofficial. It is like a conversation held in a back room without any witnesses. In real life, we attribute the term *off-the-record* (OTR) to this type of conversation; it does not leave a trace or record that may prove that the conversation ever took place. The notion of OTR can also be used in the realm of secure messaging, and hence *OTR messaging* refers to this type of messaging.[3] It is as private as possible, and it provides repudiation, meaning that it can be denied by its participants.

For the reasons mentioned above, OTR messaging cannot be implemented with digital envelopes and digital signatures only. Instead, some complementary technologies are needed to provide forward secrecy and deniability—or even plausible deniability.

- Forward secrecy is usually achieved by using an ephemeral Diffie-Hellman key exchange,[4] and it can be further improved by restricting the lifetime of the respective keys. As discussed below, OTR messaging uses a Diffie-Hellman ratcheting mechanism to refresh these keys as often as possible.
- Plausible deniability, in turn, is achieved by using MACs instead of digital signatures. Note that a MAC is symmetric in nature, meaning that a MAC can be generated and verified by either side of a message exchange (i.e., the originator and the recipient of a message). This, in turn, allows the originator to deny having sent a particular message. As discussed below, there are even some complementary techniques that can be used to further improve plausible

3 Note that OTR messaging has nothing to do with the go off the record feature of Google Talk and Gmail. This feature only means that the messages are not stored; it does not mean that OTR messaging is used.

4 In the realm of secure messaging, the use the Diffie-Hellman key exchange to come up with short-lived keys to provide forward secrecy was first proposed in 1997 [2].

deniability, such as malleable encryption and the revelation of authentication keys after their use.

The protocols proposed in [1] refer to OTR messaging version 1. Shortly after their publication, it was pointed out in [3] that an *identity misbinding attack*—as originally suggested in [4]—can be mounted against the initial Diff e-Hellman key exchange used in OTR version 1. The possibility to successfully mount such an attack made it necessary to come up with OTR messaging version 2 in 2005. In theory, there are many possibilities and respective protocol changes that can mitigate the threat. In OTR version 2, however, a variant of the SIGMA authenticated key exchange protocol that had been proposed in the realm of IP security and the Internet Key Exchange protocol (IKE) protocol [5] was used.[5] It yields an AKE (i.e., a Diff e-Hellman key exchange that is authenticated using a long-term public key pair).

In addition to the protocol change to defeat the identity misbinding attack, OTR version 2 also tried to simplify the user interface. Instead of requiring the user to understand concepts, like public keys, certif cates, and f ngerprints (of public keys), a solution to the *socialist millionaires' problem* [6] was adapted for authentication based on a shared secret. The socialist millionaires' problem refers to the question of how two millionaires can f gure out whether they are equally rich or not without revealing any other information. This, in turn, is a variant of the *millionaires' problem* [7, 8], in which two millionaires wish to know who is richer without actually revealing any information about their wealth. Both the millionaires' problem and the socialist millionaires' problem are well known in the theory of cryptography.

Using a solution to the socialist millionaires' problem in OTR allows the participants to authenticate each other with a shared secret (instead of having them verify public keys, certif cates, or f ngerprints). It is sometimes assumed that this is more intuitive and therefore simpler to use. The *socialist millionaires' protocol* (SMP) used in OTR version 2 is a variant of a protocol originally proposed in [9]. It is outlined in [10] and further addressed in Section 8.2.1.

OTR version 3 was introduced in 2012. It came along with a few minor changes that are subtle and less relevant for the technology as a whole. Most importantly, an additional key is derived during the OTR AKE protocol (whereas the protocol itself remains unchanged). This key can then be used to secure communication over a different channel, such as a channel for f le transfer or voice communication. This topic is ignored and not further addressed in this book.

5 The acronym SIGMA is derived from SIGn-and-MAc, meaning that the protocol requires a MAC to be signed.

More recently, people have started to work on OTR version 4. This endeavor is managed on GitHub[6] and is a moving target (and hence subject to change). What can already be seen is that this version of OTR comes along with many improvements and fundamental changes. Most importantly, it adapts techniques from the Signal protocol to also support asynchronous messaging, and it uses more modern cryptographic primitives and building blocks than the ones used before. We brief y itemize some of these changes and cryptographic primitives towards the end of this chapter.

Since its beginning, OTR messaging was designed for a two-party setting, in which an originator sends a message to a single recipient. In some situations this is not adequate, because people want to communicate in groups and send messages to multiple recipients. In fact, group messaging in the form of group chats is certainly one of the more important features and advantages of instant messaging—at least if compared to SMS. In 2007, a team of researchers therefore proposed a simple method for extending OTR messaging to group chats [11]. The basic idea is to designate a party as a virtual server that manages the group conversations on the other group members' behalf. While this approach is feasible and works under certain circumstances, it is not in line with the original intent of OTR, namely to enable private group conversations that are not centrally managed.

As an alternative, a group team of researchers around Goldberg soon after proposed and prototyped a method—or rather a framework—for extending OTR messaging to group messaging that is called multiparty OTR (mpOTR) [12]. It does not require a virtual server or a central authority. Instead, it requires the members of a group to mutually authenticate themselves using some long-term keying material, and to exchange and prove possession of ephemeral (and thus deniable) signature keys. These keys are then used to perform an AKE, and hence to establish a group key. All members can broadcast messages by encrypting them with the group key and signing them with their ephemeral signing key. Note that the encryption always uses the same key and does not provide forward secrecy, and that this also deviates from the original intention of OTR. If the members of a group agree that there are no more messages in transit, then they calculate the hash values for all messages they have authored during the session, sorted in lexicographical order, and send them to all other members. This, in turn, means that all members of the group can then individually verify the hash values and signatures of all messages they have received so far.

The design of mpOTR is simple and straightforward, but there is room for improvement and optimization. In [13], for example, a group of researchers proposed and prototyped a protocol named group OTR (GOTR) that is based on a group key agreement protocol due to Mike Burmester and Yvo Desmedt [14]. The

6 https://github.com/otrv4.

Burmester-Desmedt protocol generalizes the Diff e-Hellman key exchange to more than two parties. It is eff cient, but requires the parties to organize themselves in a circle. This is not trivial to achieve and certainly one of the major challenges for the use of the protocol in the f eld.

Both approaches—mpOTR and GOTR—work in a synchronous setting, but they rarely work in an asynchronous setting. This is why the Signal protocol uses a completely different approach (Section 9.2.4). Also, towards the end of the book we will see that the design of optimal group key agreement protocols and their use in E2EE messaging on the Internet is a current research topic and a work focus of the relevant IETF WG.

Originally, OTR messaging was implemented as a plug-in for the GAIM instant messaging client (that is nowadays called Pidgin[7]). The implementation employed the cryptographic open source library Libgcrypt.[8] Since the public release of GAIM (Pidgin), support for OTR messaging has been incorporated into several instant messaging protocols—be it open (e.g., XMPP) or proprietary (e.g., OSCAR)—and respective clients. These clients either natively support OTR, like Adium[9] for Mac OS X, or come along with a respective plug-in, like Pidgin and Miranda IM.[10] Anyway, from a real-world perspective, OTR messaging has somewhat become obsolete, because most E2EE messengers in use today also support asynchronous messaging (in addition to instant messaging) and therefore employ a protocol like Signal. In order to understand these protocols, however, it is still reasonable and useful to have a look at OTR f rst.

8.2 TECHNOLOGY

In contrast to the secure messaging technologies addressed so far (i.e., OpenPGP and S/MIME) OTR has originally been designed for a synchronous setting, in which the originator and recipient of a (text) message are assumed to be online and operate in real time. This means that they can establish a session, and this simplif es the provision of forward secrecy considerably. The technology employed by OTR is centered around an AKE—in this context known as OTR AKE—used in the initialization phase of a session, a Diff e-Hellman ratchet to derive a series of Diff e-Hellman keys used during the session, and a message processing that employs MACs (instead of digital signatures) and malleable encryption. These topics are now addressed in this particular order.

7 http://pidgin.im.
8 http://www.gnu.org/software/libgcrypt.
9 https://www.adium.im.
10 http://www.miranda-im.org.

8.2.1 OTR AKE

The aim of an AKE protocol is to combine an ephemeral Diff e-Hellman key exchange with some form of authentication that allows the participants of the protocol (i.e., A and B) to authenticate to each other, using, for example, public key cryptography. In OTR version 1, this was achieved by having A and B send to each other a digitally signed version of a Diff e-Hellman parameter together with the public key required to verify the signature. In principle, any digital signature system can be used, where (sk_A, pk_A) refers to A's private and public keys, and (sk_B, pk_B) refers to B's key pair. As usual, the private key sk is used to generate digital signatures and the public key pk is used to verify signatures.

The original OTR implementation uses signatures according to the DSA. If G stands for a cyclic group of prime order q in which the Diff e-Hellman problem is assumed to be intractable and g refers to a generator of that group,[11] then the AKE protocol used in OTR version 1 can be formally expressed as shown in Protocol 8.1. Note that y_a and y_b refer to elements of G.

Protocol 8.1 The AKE protocol used in OTR version 1.

A		**B**
$(G, g, (sk_A, pk_A))$		$(G, g, (sk_B, pk_B))$
$x_a \xleftarrow{r} \mathbb{Z}_q^*$		$x_b \xleftarrow{r} \mathbb{Z}_q^*$
$y_a \longleftarrow g^{x_a}$		$y_b \longleftarrow g^{x_b}$
$\sigma_a \longleftarrow \mathsf{Sign}(sk_A, y_a)$		$\sigma_b \longleftarrow \mathsf{Sign}(sk_B, y_b)$
	$\xrightarrow{pk_A, y_a, \sigma_a}$	
	$\xleftarrow{pk_B, y_b, \sigma_b}$	
$\mathsf{Verify}(pk_B, y_b, \sigma_b)$		$\mathsf{Verify}(pk_A, y_a, \sigma_a)$
$k_{ab} \longleftarrow y_b^{x_a}$		$k_{ba} \longleftarrow y_a^{x_b}$
(k_{ab})		(k_{ba})

While sk_A and pk_A refer to A's signing and verif cation keys that are long-lived, x_a and $y_a = g^{x_a}$ refer to A's Diff e-Hellman parameters that are ephemeral and short-lived (that's why the subscript is put as a lowercase letter). The notation used for B's keys and parameters is the same. The core of Protocol 8.1 is that A and B exchange their public Diff e-Hellman parameters in digitally signed form together with the respective verif cation keys, and that these keys are then used to verify the

11 The group G generally used in OTR messaging is the 1536-bit MODP Group def ned in Section 2 of RFC 3526 [15]. It is also known as the *Diff e-Hellman Group 5*, and it consistently uses the generator 2 (i.e., $g = 2$).

signatures. If the verif cation succeeds, then the Diff e-Hellman keys k_{ab} (on A's side) and k_{ba} (on B's side) are computed independently. According to the math that underlies the Diff e-Hellman key exchange protocol, k_{ab} and k_{ba} refer to the same value that may serve as session key k.

Protocol 8.2 An identity misbinding attack against OTR version 1.

A	**C**	**B**
$(G, g, (sk_A, pk_A))$		$(G, g, (sk_B, pk_B))$
$x_a \xleftarrow{r} \mathbb{Z}_q^*$		$x_b \xleftarrow{r} \mathbb{Z}_q^*$
$y_a \longleftarrow g^{x_a}$		$y_b \longleftarrow g^{x_b}$
$\sigma_a \longleftarrow \mathsf{Sign}(sk_A, y_a)$		$\sigma_b \longleftarrow \mathsf{Sign}(sk_B, y_b)$
	$\xrightarrow{pk_A, y_a, \sigma_a} \rightsquigarrow \xrightarrow{pk_C, y_a, \sigma_c}$	
	$\xleftarrow{pk_B, y_b, \sigma_b}$	
$\mathsf{Verify}(pk_B, y_b, \sigma_b)$		$\mathsf{Verify}(pk_C, y_a, \sigma_c)$
$k_{ab} \longleftarrow y_b^{x_a}$		$k_{ba} \longleftarrow y_a^{x_b}$
(k_{ab})		(k_{ba})

As mentioned above, an identity misbinding attack against OTR version 1 was pointed out in [3]. Such an attack allows an adversary C to interfere with the AKE protocol in such a way that A and B still reach the same key at the end of the protocol, but A thinks he or she is talking to B while B thinks he or she is talking to C. The attack is formally expressed in Protocol 8.2. It requires an active adversary that acts as a MITM. But in contrast to a normal MITM attack, C does not change the Diff e-Hellman parameter y_a as it is passing by. Instead, C removes A's signature σ_a on y_a, and resigns y_a with his or her own private key sk_C. The resulting signature σ_c is then sent along with y_a and C's public key pk_C to B. The bottom line of this change is that the message received by B looks as it originated from C, and this, in turn, means that B thinks he or she is talking to C. The other message that is sent from B back to A is left unchanged. After the attack, A and B still share a Diff e-Hellman key (that is unknown to C), but B thinks he or she shares it with C (instead of A). Whether this poses a problem, depends on the application setting. Anyway, it is not a desired property, and hence we refer to it as an attack.

After the publication of [3], the developers of OTR decided to mitigate the attack by adopting an AKE protocol that is resistant against it. More specif cally, they chose a variant of the SIGMA protocol that deviates from the original protocol by having the Diff e-Hellman parameters be sent in encrypted from (instead of having them be sent in the clear). The protocol works in two phases: In the f rst phase, it performs an anonymous Diff e-Hellman key exchange to establish an

unauthenticated but encrypted channel, and in the second phase, it uses this channel for authentication. The two phases are formally expressed in Protocols 8.3 and 8.4. The resulting protocol is quite sophisticated; it is the same in OTR versions 2 and 3.

Protocol 8.3 Phase 1 of the AKE protocol used in OTR version 2 and 3.

A		**B**
(G, g)		(G, g)
$r \xleftarrow{r} \{0,1\}^{128}$		
$x_a \xleftarrow{r} \{0,1\}^{\geq 320}$		$x_b \xleftarrow{r} \{0,1\}^{\geq 320}$
$y_a \longleftarrow g^{x_a}$		$y_b \longleftarrow g^{x_b}$
$c' \longleftarrow E_r(y_a)$		
$h' \longleftarrow h(y_a)$		
	$\xrightarrow{c', h'}$	
	$\xleftarrow{y_b}$	
$k_{ab} \longleftarrow y_b^{x_a}$		
	$\xrightarrow{r}$	
		$y_a \longleftarrow D_r(c')$
		$h(y_a) \stackrel{?}{=} h'$
		$k_{ba} \longleftarrow y_a^{x_b}$
(k_{ab})		(k_{ba})

In phase 1 of the OTR AKE protocol (i.e., Protocol 8.3) an unauthenticated but encrypted channel is established between A and B. A randomly selects a 128-bit string r (that later serves as encryption key) and a private ephemeral Diff e-Hellman parameter x_a that is at least 320 bits long.[12] As usual, A computes the public ephemeral Diff e-Hellman parameter $y_a = g^{x_a}$. On the recipient' side, B does the same and generates x_b and $y_b = g^{x_b}$. A then encrypts y_a with AES-128 in CTR mode using key r. This yields the ciphertext c'. A also hashes y_a with the hash function h (that is SHA-256 in the current implementation) to form h'. The pair consisting of c' and h' is then transmitted to B, and B returns its public ephemeral Diff e-Hellman parameter y_b to A. A can now compute the Diff e-Hellman key $k_{ab} = y_b^{x_a}$ and send the encryption key r to B. Using this key, B can decrypt c' and retrieve y_a accordingly. If the SHA-256 hash value of y_a is equal to the received h', then B can be sure that it has the correct value. It can then compute the ephemeral

12 The minimal bitlength of 320 bits was originally proposed in [10] and must be seen in the context of the cyclic group in use (i.e., the 1536-bit MODP Group [15]). For other cyclic groups, this value may have to be adapted accordingly.

Diff e-Hellman key $k_{ba} = y_a^{x_b}$ that is equal to k_{ab}, and hence A and B can use it as a session key.

Protocol 8.4 Phase 2 of the AKE protocol used in OTR version 2 and 3.

	A		**B**
	$(y_a, y_b, k_{ab}, (sk_A, pk_A))$		$(y_a, y_b, k_{ba}, (sk_B, pk_B))$
	$k_{a_1}, k_{a_2}, k_{b_1}, k_{b_2}, k_{a_3}, k_{b_3} \leftarrow \text{KDF}(k_{ab})$		
	$keyid_a \leftarrow f(y_a)$		
	$t_a \leftarrow \text{MAC}_{k_{a_1}}(y_a \parallel y_b \parallel pk_A \parallel keyid_a)$		
	$m_a \leftarrow pk_A \parallel keyid_a \parallel \text{Sign}(sk_A, t_a)$		
	$c_1 \leftarrow E_{k_{a_3}}(m_a), h_1 \leftarrow \text{MAC}_{k_{a_2}}(c_1)$		
		$\xrightarrow{c_1, h_1}$	
			$k_{a_1}, k_{a_2}, k_{b_1}, k_{b_2}, k_{a_3}, k_{b_3} \leftarrow \text{KDF}(k_{ba})$
			$\text{MAC}_{k_{a_2}}(c_1) \stackrel{?}{=} h_1$
			$m_a \leftarrow D_{k_{a_3}}(c_1)$
			$pk_A, keyid_a, \text{Sign}(sk_A, M_A) \leftarrow m_a$
			$t_a \leftarrow \text{MAC}_{k_{a_1}}(y_a \parallel y_b \parallel pk_A \parallel keyid_a)$
			$\text{Verify}(pk_A, t_a, \text{Sign}(sk_A, t_a))$
			$keyid_b \leftarrow f(y_b)$
			$t_b \leftarrow \text{MAC}_{k_{b_1}}(y_b \parallel y_a \parallel pk_B \parallel keyid_b)$
			$m_b \leftarrow pk_B \parallel keyid_b \parallel \text{Sign}(sk_B, t_b)$
			$c_2 \leftarrow E_{k_{b_3}}(m_b), h_2 \leftarrow \text{MAC}_{k_{b_2}}(c_2)$
		$\xleftarrow{c_2, h_2}$	
	$\text{MAC}_{k_{b_2}}(c_2) \stackrel{?}{=} h_2$		
	$m_b \leftarrow D_{k_{b_3}}(c_2)$		
	$pk_B, keyid_b, \text{Sign}(sk_B, t_b) \leftarrow m_b$		
	$t_b \leftarrow \text{MAC}_{k_{b_1}}(y_b \parallel y_a \parallel pk_B \parallel keyid_b)$		
	$\text{Verify}(pk_B, t_b, \text{Sign}(sk_B, t_b))$		

Phase 2 of the OTR AKE protocol (i.e., Protocol 8.4) yields a way for A and B to mutually authenticate each other using the channel established in phase 1. Input to the protocol are the public ephemeral Diff e-Hellman parameters y_a and y_b from phase 1, the resulting shared secrets k_{ab} on A's side and k_{ba} on B's side (that is the same), and the respective long-term public key pairs on either side of the channel (i.e., (sk_A, pk_A) on A's side and (sk_B, pk_B) on B's side). A f rst uses k_{ab} and a key derivation function (KDF) to generate four 256-bit MAC keys $k_{a_1}, k_{a_2}, k_{b_1}$, and k_{b_2}, as well as two 128-bit AES encryption keys k_{a_3} and k_{b_3}. Also, a key identif er $keyid_a$ that represents a serial number is derived from y_a using some well-def ned

function f (there are no cryptographic requirements for this function f, in particular, it need not be one-way). The first MAC key k_{a_1} is then used to compute a MAC for the concatenation of y_a, y_b, pk_A, and $keyid_a$. In this and all subsequent MAC computations, the HMAC construction [16] is used. The result is t_a, and this value is signed by A using sk_A. The resulting signature is prefixed with pk_A and $keyid_a$ to form message m_a. This message is AES encrypted in CTR mode using key k_{a_3}. The resulting ciphertext is referred to as c_1. Finally, A uses the second MAC key k_{a_2} to generate another MAC for c_1. The result is h_1, and this value is sent together with c_1 to B.

B does similar computations: It uses k_{ba} to derive the keys $k_{a_1}, k_{a_2}, k_{b_1}$, k_{b_2}, k_{a_3}, and k_{b_3}, uses k_{a_2} to compute a MAC for c_1, and continues if and only if the resulting value equals h_1. In this case, c_1 is AES decrypted using k_{a_3} in CTR mode to retrieve m_a. This string can be decomposed into its three components pk_A, $keyid_a$, and the signature $\mathsf{Sign}(sk_A, t_a)$. B then recomputes t_a and uses pk_A to verify the signature for it. If the signature is valid, then B uses the same function f to compute $keyid_b$ from y_b. Using key k_{b_1}, B computes a MAC on y_b, y_a, pk_B, and $keyid_b$, and assigns the resulting value to t_b. This value, in turn, is digitally signed with sk_B. Concatenated with pk_B and $keyid_b$, this signature is assigned to message m_b. This message is AES encrypted with k_{b_3} in CTR mode to form ciphertext c_2, and this value is authenticated with k_{b_2} to form MAC h_2. B sends c_2 and h_2 back to A.

After having received c_2 and h_2, A uses k_{b_2} to recompute the MAC for c_2 and verifies whether the resulting value matches h_2. If this is the case, then A decrypts c_2 with k_{b_3}. The resulting value m_b is decomposed into its three components pk_B, $keyid_b$, and signature $\mathsf{Sign}(sk_B, t_b)$. Using k_{b_1}, B recomputes the MAC t_b for y_b, y_a, pk_B, and $keyid_b$, and finally verifies the signature $\mathsf{Sign}(sk_B, t_b)$ for it using pk_B. If everything is fine, then either side of the protocol can use the resulting key from phase 1 as a now authenticated session key.

In reality, the two phases of the OTR AKE protocol are not executed one after another, but in an interleaved way. First, phase 1 is executed until the point that is indicated with a dotted line in Protocol 8.3 (i.e., after A has computed k_{ab} and before he or she sends r to B). Second, phase 2 is executed until the point that is again indicated with a dotted line (i.e., before A sends c_1 and h_1 to B). This message is now complemented with r that also needs to be transmitted from A to B. Third, the remaining parts of phases 1 and 2 are executed to complete the protocol.

The OTR AKE protocol allows A and B to establish a secure channel, where authentication is achieved through digital signatures. This, in turn, requires users to ensure the authenticity of public keys, using, for example, public key certificates or fingerprints. As mentioned earlier, this may be asked too much of users, and simpler and more straightforward methods to authenticate public keys and communication

peers are preferred. In the simplest case, A and B simply share a secret, and authentication only verif es whether A's secret and B's secret are the same. This is the purpose of the SMP introduced in OTR version 2 and outlined in Protocol 8.5: It allows A and B to verify whether they hold the same secret without revealing any information other than the fact that they are the same. Let s_a and s_b be the secrets A and B may hold, and let either secret be a SHA-256 hash value of the concatenation of some mutually known values, such as the two parties' f ngerprints,[13] a session ID (that essentially refers to the session key k), and an original secret string shared between A and B. Ideally, s_a and s_b are the same, and the aim of the SMP is to either verify or reject this fact—without leaking any other information.

Protocol 8.5 The Socialist Millionaires' Protocol used in OTR messaging (since version 2).

A		**B**
(G, g, s_a)		(G, g, s_b)
$a_1, a_2 \xleftarrow{r} \mathbb{Z}_q^*$		$b_1, b_2 \xleftarrow{r} \mathbb{Z}_q^*$
$g_{a_1} \leftarrow g^{a_1}, g_{a_2} \leftarrow g^{a_2}$		$g_{b_1} \leftarrow g^{b_1}, g_{b_2} \leftarrow g^{b_2}$
	$\xrightarrow{g_{a_1}, g_{a_2}}$	
		$g_1 \leftarrow g_{a_1}^{b_1}, g_2 \leftarrow g_{a_2}^{b_2}$
		$b \xleftarrow{r} \mathbb{Z}_q^*$
		$P_b \leftarrow g_2^b, Q_b \leftarrow g^b g_1^{s_b}$
	$\xleftarrow{g_{b_1}, g_{b_2}, P_b, Q_b}$	
$g_1 \leftarrow g_{b_1}^{a_1}, g_2 \leftarrow g_{b_2}^{a_2}$		
$a \xleftarrow{r} \mathbb{Z}_q^*$		
$P_a \leftarrow g_2^a, Q_a \leftarrow g^a g_1^{s_a}$		
$R_a \leftarrow (Q_a/Q_b)^{a_2}$		
	$\xrightarrow{P_a, Q_a, R_a}$	
		$R_b \leftarrow (Q_a/Q_b)^{b_2}$
		$R_{ab} \leftarrow R_a^{b_2}$
		$R_{ab} \stackrel{?}{=} P_a/P_b$
	$\xleftarrow{R_b}$	
$R_{ab} \leftarrow R_b^{a_2}$		
$R_{ab} \stackrel{?}{=} P_a/P_b$		

We use mathematics that is similar to the OTR AKE protocol. G is again a cyclic group with generator g. Input to the SMP is G, g, and the respective secrets (i.e., s_a on A' side and s_b on B' side). Either side randomly selects two elements

13 Here, a f ngerprint refers to the SHA-1 value of the respective party's public key (i.e., SHA-1(pk_A) for A and SHA-1(pk_B) for B).

from $\mathbb{Z}_q^*$ and computes respective Diff e-Hellman parameters: A randomly selects a_1 and a_2, and computes $g_{a_1} = g^{a_1}$ and $g_{a_2} = g^{a_2}$, whereas B randomly selects b_1 and b_2, and computes $g_{b_1} = g^{b_1}$ and $g_{b_2} = g^{b_2}$. A sends its values to B, and B uses them to compute the new generators g_1 and g_2 as ephemeral Diff e-Hellman values. B then randomly selects another element b from $\mathbb{Z}_q^*$, and uses this value together with the generators g, g_1, and g_2, as well as s_b to compute P_b and Q_b that are sent—together with g_{b_1} and g_{b_2}—back to A. A, in turn, can use g_{b_1} and g_{b_2} to also compute g_1 and g_2. It then randomly selects an element a from $\mathbb{Z}_q^*$, and uses it together with g, g_1, g_2, and s_a to compute P_a and Q_a, as well as (Q_a/Q_b) to the power of a_2. The result of this computation yields R_a. A sends P_a, Q_a, and R_a to B, and B computes (Q_a/Q_b) to the power of b_2. The result yields R_b. B computes R_{ab} as R_a to the power of b_2, and verif es whether this value equals P_a/P_b. If this is the case, then B returns R_b to A. A raises this value to the power of a_2 to compute R_{ab}, and also verif es whether this value is equal to P_a/P_b. If either check succeeds, then A and B can be assured that they actually hold the same secret. Otherwise, nothing can be said and no information about either s_a or s_b leaks through.

To verify the correctness of the SMP, we start from the end where A computes

$$R_{ab} = R_b^{a_2} = (Q_a/Q_b)^{b_2 a_2}$$

and B computes

$$R_{ab} = R_a^{b_2} = (Q_a/Q_b)^{a_2 b_2}.$$

It is obvious that both values are the same, and that the only difference is within the order of the exponents (that is irrelevant). On the one hand, R_{ab} can be written as

$$R_{ab} = (Q_a/Q_b)^{a_2 b_2} = \left(\frac{g^a g_1^{s_a}}{g^b g_1^{s_b}}\right)^{a_2 b_2} = \frac{g^{a a_2 b_2} g_1^{s_a a_2 b_2}}{g^{b a_2 b_2} g_1^{s_b a_2 b_2}} \tag{8.1}$$

On the other hand, $(P_a/P_b) = g_2^a/g_2^b$ can be written as $\frac{g^{b_2 a_2 a}}{g^{a_2 b_2 b}}$. This last term can be used to rewrite the rightmost side of (8.1):

$$\frac{g^{a a_2 b_2} g_1^{s_a a_2 b_2}}{g^{b a_2 b_2} g_1^{s_b a_2 b_2}} = \frac{P_a (g_1^{a_2 b_2 s_a})}{P_b (g_1^{a_2 b_2 s_b})} = \frac{P_a}{P_b} \left(g_1^{a_2 b_2}\right)^{(s_a - s_b)}$$

This means that the following equation must hold:

$$R_{ab} = \frac{P_a}{P_b} \left(g_1^{a_2 b_2}\right)^{(s_a - s_b)}$$

If s_a is equal to s_b, then the exponent of $g_1^{a_2 b_2}$ is equal to zero, and hence $R_{ab} = P_a / P_b$. This yields the f nal check performed by A and B in the SMP. If the checks (on either side) succeed, then the protocol successfully terminates, and A and B can be sure that they hold the same secret (i.e., $s_a = s_b$). Because this secret also takes into account the session ID and the respective session k, they can now be sure that they are authentic and that they are using a secure channel for all subsequent communication.

8.2.2 Diff e-Hellman Ratchet

In cryptography, it is usually good practice to limit the lifetime of a key, and to restrict the number of messages that are encrypted and decrypted with it. This also applies to a session key that results from a Diff e-Hellman key exchange as used, for example, in the OTR AKE. In the most extreme case, the session key is changed after every message sent or received.

There are several ways to achieve a per-message key change. In the simplest case, for example, a starting key k_0 can be generated using a Diff e-Hellman key exchange, and this key is then expanded in a series of keys, simply by recursively hashing k_i to derive k_{i+1} (i.e., $k_{i+1} = h(k_i)$ for any cryptographic hash function h and integer $i > 0$). This simple mechanism is known as a *secret key ratchet*: Knowing a key allows one to compute the next key, but—due to the one-way property of h—does not allow one to compute the previous key. More specif cally, k_{i+1} can be computed from k_i by applying h, but k_i cannot be computed from k_{i+1}, because this would require the computation of the inverse of k_{i+1} with regard to h (and this computation is assumed to be computationally infeasible for a cryptographic hash function). This property is best visualized with a ratchet; it can be turned in the forward direction, but it cannot be turned in the backward direction. As brief y mentioned in the following chapter, such a secret key ratchet was, for example, employed in a no longer used protocol named *Silent Circle Instant Messaging Protocol* (SCIMP).[14] In modern terminology, we would say that a secret key ratchet provides forward secrecy but it does not provide PCS.

If one wants to provide forward secrecy and PCS, then one has to employ a more involved key updating and refreshing mechanism that uses multiple Diff e-Hellman key exchanges executed in an interleaved way. This brings into play the idea (and notion) of a *Diff e-Hellman ratchet* that is used—as we will see—in most

14 As its name suggests, SCIMP was developed and originally proposed by Silent Circle (https://www.silentcircle.com). SCIMP version 1.0 was specif ed in December 2012 by Vinnie Moscaritolo, Gary Belvin, and Phil Zimmermann. The specif cation is not off cially published but can be found on the Internet.

E2EE messaging protocols today (in many cases, a secret key ratchet and a Diff e-Hellman ratchet are combined in a so-called *double ratchet*). In a Diff e-Hellman ratchet, A and B interchangeably exchange Diff e-Hellman parameters and compute a new Diff e-Hellman key after every such exchange. This makes sure that any long-term key compromise does not affect any past encryption key (to provide forward secrecy) or any future encryption key (to provide PCS). At every single point in time, only the currently used encryption keys are at stake—at least if all previously used Diff e-Hellman parameters are properly deleted and no longer available on the compromised system.

Protocol 8.6 The working principle of a Diff e-Hellman ratchet.

A		**B**
(G, g)		(G, g)
		$x_{b_1} \xleftarrow{r} \mathbb{Z}_q^*$
		$y_{b_1} \leftarrow g^{x_{b_1}}$
	$\xleftarrow{y_{b_1}}$	
$x_{a_1} \xleftarrow{r} \mathbb{Z}_q$		
$y_{a_1} \leftarrow g^{x_{a_1}}$		
	$\xrightarrow{y_{a_1}}$	
$k_{ab_1} \leftarrow y_{b_1}^{x_{a_1}}$		$k_{ba_1} \leftarrow y_{a_1}^{x_{b_1}}$
		$x_{b_2} \xleftarrow{r} \mathbb{Z}_q^*$
		$y_{b_2} \leftarrow g^{x_{b_2}}$
	$\xleftarrow{y_{b_2}}$	
$k_{ab_2} \leftarrow y_{b_2}^{x_{a_1}}$		$k_{ba_2} \leftarrow y_{a_1}^{x_{b_2}}$
$x_{a_3} \xleftarrow{r} \mathbb{Z}_q^*$		
$y_{a_3} \leftarrow g^{x_{a_3}}$		
	$\xrightarrow{y_{a_3}}$	
$k_{ab_3} \leftarrow y_{b_2}^{x_{a_3}}$		$k_{ba_3} \leftarrow y_{a_3}^{x_{b_2}}$
		$x_{b_4} \xleftarrow{r} \mathbb{Z}_q^*$
		$y_{b_4} \leftarrow g^{x_{b_4}}$
	$\xleftarrow{y_{b_4}}$	
	$\ldots$	
$(k_{ab_1}, k_{ab_2}, k_{ab_3}, \ldots)$		$(k_{ba_1}, k_{ba_2}, k_{ba_3}, \ldots)$

The working principle of a Diff e-Hellman ratchet is illustrated in Protocol 8.6. It starts with a normal Diff e-Hellman key exchange. B and A both generate

ephemeral Diff e-Hellman parameters (i.e., (x_{b_1}, y_{b_1}) and (x_{a_1}, y_{a_1})) and they exchange the public parameters y_{b_1} and y_{a_1}. A computes its f rst Diff e-Hellman key

$$k_{ab_1} = y_{b_1}^{x_{a_1}}$$

and B does the same:

$$k_{ba_1} = y_{a_1}^{x_{b_1}}$$

As usual, k_{ab_1} and k_{ba_1} refer to the same value that can be used as a session key. In each subsequent exchange, only one party provides a new Diff e-Hellman parameter, whereas both parties compute a new Diff e-Hellman key. In round two, for example, B provides y_{b_2}, and A and B compute

$$k_{ab_2} = y_{b_2}^{x_{a_1}} \text{ and } k_{ba_2} = y_{a_1}^{x_{b_2}}$$

that refer to the same value. Similarly, in round three, A provides y_{a_3}, and A and B compute

$$k_{ab_3} = y_{b_2}^{x_{a_3}} \text{ and } k_{ba_3} = y_{a_3}^{x_{b_2}}$$

This can be continued inf nitely many times or at least as many times as message keys are needed in a conversation.

The Diff e-Hellman ratchet is heavily used in OTR (and almost all E2EE messengers in use today). In the f rst round of OTR, however, the normal Diff e-Hellman key exchange is replaced with the OTR AKE protocol. Also, all subsequent Diff e-Hellman parameters are sent along with the encrypted messages. This means that every encrypted OTR message is typically sent along with the Diff e-Hellman parameter for the next round. This will become clear when we go through message processing next.

8.2.3 Message Processing

OTR messaging as addressed so far comprises an AKE protocol to initialize a session, a Diff e-Hellman ratchet to steadily refresh the keys (to provide PFS, or forward secrecy and PCS, respectively), a MAC system to authenticate messages (to provide plausible deniability), and a symmetric encryption system to encrypt the messages. Let us now delve more deeply into the procedures to perform these tasks and process the messages accordingly.

Using the OTR AKE protocol, A and B can authenticate each other and exchange a shared secret key k. From this key, both an encryption key k_{enc} and an

authentication key k_{auth} (to compute and verify MACs) are consecutively derived as follows:

$$\begin{aligned} k_{enc} &= \text{truncate}(\text{SHA-1}(k), 128) \\ k_{auth} &= \text{SHA-1}(k_{enc}) \end{aligned}$$

First, k is hashed with SHA-1 and the resulting 160-bit hash value is truncated to 128 bits (because AES-128 is used for encryption and AES-128 uses 128-bit keys). The result is k_{enc}. Second, k_{enc} is again subjected to SHA-1 to generate k_{auth}. This key is 160 bits long and doesn't need to be truncated. The bottom line is that both keys—k_{enc} and k_{auth}—are deterministically generated from k, and that knowing the encryption key even means that one also knows the authentication key. This, in turn, means that anybody who can encrypt and decrypt a message can also generate and verify a MAC for it. This helps providing deniable authentication.

When A is to send a message m to B, it determines the two latest Diffe-Hellman parameters $keyid_a$ and $keyid_b$ that refer to y_{a_i} and y_{b_j}, computes the respective Diffe-Hellman key k, derives the two keys mentioned above (i.e., k_{enc} and k_{auth}) and uses k_{enc} to encrypt m and k_{auth} to generate a MAC. As mentioned above, the encryption uses AES-128 in CTR mode to compute the ciphertext c. This can be formally expressed as follows:

$$c = \text{AES-128}_{k_{enc}}(m)$$

As its name suggests, CTR mode requires a counter. In OTR, this counter ctr is 8 bytes long but is not indicated in the formula above.

After having generated the ciphertext c, A compiles a record T that comprises c, $keyid_a$, $keyid_b$, ctr, and $y_{a_{i+1}}$ that refers to A's next parameter for the Diffe-Hellman ratchet. A then uses k_{auth} to compute an authentication tag t (representing a MAC) using the HMAC construction with SHA-256 and truncating the result to 160 bits (or 20 bytes, respectively). This can be formally expressed as follows:

$$\begin{aligned} T &= c \parallel keyid_a \parallel keyid_b \parallel ctr \parallel y_{a_{i+1}} \\ t &= \text{HMAC-SHA256-160}(k_{auth}, T) \end{aligned}$$

In the end, A sends T and t to B, together with the old authentication keys that are no longer needed. The revelation of these keys serves (or rather improves) plausible deniability, because everybody now learns the keys that are needed to generate valid MACs for past messages of his or her choice. Again, due to the one-way property of SHA-1, it is not possible to derive the encryption key k_{enc} from k_{auth}. Hence, the security of the encryption keys remains unaffected by the revelation of the authentication keys.

On the recipient's side, B is to decrypt the ciphertext and verify the authenticity of the message. It therefore retrieves c, $keyid_a$, $keyid_b$, ctr, and $y_{a_{i+1}}$ from T, uses the Diff e-Hellman parameters referenced by $keyid_a$ and $keyid_b$ to compute the Diff e-Hellman key k, and derives k_{enc} and k_{auth} from that key. It then uses k_{enc} and ctr to decrypt c (again using AES-128 in CTR mode), and k_{auth} to authenticate T. If everything is f ne, then it accepts the message and updates the Diff e-Hellman ratchet with $y_{a_{i+1}}$ to be prepared for the next message.

8.3 SECURITY ANALYSIS

In 2011, a group of researchers from Stanford University performed a f nite-state analysis of OTR version 2, in which they identif ed a few subtle f aws and vulnerabilities that are diff cult to exploit.[15] The mitigation techniques proposed in the study are also relevant for OTR version 3, but due to its limited user base and the fact that OTR messaging is currently being revised fundamentally for version 4, the OTR version 3 specif cation has not been changed so far to incorporate these changes. Instead, people think that OTR version 4 is going to be so different from versions 2 and 3 that most of these changes are irrelevant and obsolete.

Except for this 2011 study, there is hardly any security analysis of OTR messaging that has been done independently from its original developers. This is unfortunate, but it is also due to the fact that the research community has moved away from OTR to more sophisticated E2EE messaging protocols like Signal and some variants thereof. These protocols have been analyzed more thoroughly.

The bottom line is that OTR—as it stands today—is assumed to be secure. This assumption is supported by the fact that no serious attack has been published against OTR messaging in the past 15 years. This is a reasonably long time to have good faith in OTR and its use in the f eld. Note, however, that this is just a gut feeling and not a proof. So it may still happen that somebody f nds a serious security f aw or vulnerability in OTR messaging, or that somebody f nds implementation bugs that are easy to exploit.

8.4 FINAL REMARKS

In this chapter, we introduced and discussed OTR and its use in secure and E2EE messaging. We started with the original idea behind its design, namely to improve the existing technology (based on digital envelopes and signatures) with something that provides forward secrecy and deniable authentication. The new ideas are (i)

15 http://www.jbonneau.com/doc/BM06-OTR_v2_analysis.pdf.

to use a Diffe-Hellman ratchet to periodically update and refresh the session key to achieve forward secrecy, and (ii) to use MACs instead of digital signatures to achieve deniable authentication. Furthermore, there are a few complementary technologies and techniques in place to further improve the user-friendliness and deniability of OTR messaging.

OTR messaging was designed for a synchronous communication setting as used for instant messaging. It requires the participants to be online, so that protocols, like OTR AKE and SMP, can be executed in the first place. These protocols cannot be executed in an asynchronous setting, in which the recipient of a message does not need to be online. This is going to be the major improvement of the Signal protocol that can also be executed in an asynchronous setting. This makes the resulting protocol suitable not only for instant messaging, but also for e-mail. It also makes it more suitable for a multiparty setting and group messaging. Hence, the Signal protocol can be seen as a generalization or extension of OTR towards any form of messaging on the Internet—be it synchronous or asynchronous—and potentially more than one recipient for a particular message.

At the beginning of this chapter it was mentioned that work on OTR version 4 is currently under way, and that OTR version 4 is fundamentally different from version 3. In summary, OTR version 4 is to work on top of any messaging protocol, including XMPP, and it is to support both synchronous and asynchronous messaging. To achieve better forward secrecy, OTR version 4 employs a double ratcheting mechanism that is similar to the one employed by the Signal protocol, and it also invokes a new cryptographic primitive known as *deniable AKE* (DAKE) [17]. In fact, there are two variants of DAKE currently used in OTR version 4:

- *DAKE with zero knowledge* (DAKEZ) for normal (i.e., interactive) messaging, where both parties are online;
- *Extended zero knowledge Diffe-Hellman* (XZDH) for messaging, where one party—typically the recipient—is offline.

Furthermore, OTR version 4 uses more modern cryptographic primitives and building blocks, like XSalsa20 (instead of AES in CTR mode),[16] SHAKE-256,[17] 224-bit ECC,[18] and 3072-bit Diffe-Hellman parameters according to [15]. These changes and update are in line with the current state of the art in cryptography, but the success of OTR version 4 still remains questionable (because the Signal protocol is so overwhelmingly successful). This seems to be largely independent from usability

16 XSalsa20 is a variant of Salsa20 that uses a longer nonce.

17 SHAKE-256 is an extendable output function (XOF) that is part of the SHA-3 family of cryptographic hash functions.

18 The elliptic curve is Ed448-Goldilocks specified as Curve448 in [18] and briefly mentioned in Section 3.2.3.2.

issues and respective user studies that have been done in the f eld (e.g., [19]). Such studies are more appropriate for messengers that have a larger user base. But OTR version 4 will still remain a research topic and an area where new cryptographic primitives like DAKE, DAKEZ, and XZDH can be explored.

References

[1] Borisov, N., Goldberg, I., and E. Brewer, "Off-the-Record Communication, or, Why Not To Use PGP," *Proceedings of the ACM Workshop on Privacy in the Electronic Society (WPES 2004)*, ACM Press, New York, NY, 2004, pp. 77–84.

[2] Schneier, B., and C. Hall, "An Improved E-Mail Security Protocol," *Proceedings of the 13th Annual Computer Security Applications Conference (ACSAC 1997)*, 1997, pp. 227–230.

[3] Di Raimondo, M., Gennaro, R., and H. Krawczyk, "Secure Off-the-Record Messaging," *Proceedings of the ACM Workshop on Privacy in the Electronic Society (WPES 2005)*, ACM Press, New York, NY, 2005, pp. 81–89.

[4] Diff e, W., van Oorschot, P.C., and M.J. Wiener, "Authentication and Authenticated Key Exchanges," *Designs, Codes and Cryptography*, Volume 2, Issue 2, 1992, pp. 107–125.

[5] Krawczyk, H., "SIGMA: The SIGn-and-MAc Approach to Authenticated Diff e-Hellman and Its Use in the IKE Protocols," *Proceedings of CRYPTO 2003*, Springer-Verlag, LNCS 2729, 2003, pp. 400–425.

[6] Jakobsson, M., and M. Yung, "Proving Without Knowing: On Oblivious, Agnostic and Blindfolded Provers," *Proceedings of CRYPTO 1996*, Springer-Verlag, LNCS 1109, 1996, pp. 186–200.

[7] Yao, A., "Protocols for Secure Computations," *Proceedings of the 23rd IEEE Symposium on Foundations of Computer Science (FOCS '82)*, IEEE Computer Society, 1982, pp. 160–164.

[8] Yao, A., "How to Generate and Exchange Secrets," *Proceedings of the 27th IEEE Symposium on Foundations of Computer Science (FOCS '86)*, IEEE Computer Society, 1986, pp. 162–167.

[9] Boudot, F., Schoenmakers, B., and J. Traoré, "A Fair and Eff cient Solution to the Socialist Millionaires' Problem," *Discrete Applied Mathematics*, Volume 111 (2001), pp. 23–36.

[10] Alexander, C., and I. Goldberg, "Improved User Authentication in Off-The-Record Messaging," *Proceedings of the ACM Workshop on Privacy in the Electronic Society (WPES 2007)*, ACM Press, New York, NY, 2007, pp. 41–47.

[11] Bian, J., Seker, R., and U. Topaloglu, "Off-the-Record Instant Messaging for Group Conversation," *Proceedings of the IEEE International Conference on Information Reuse and Integration (IRI '07)*, IEEE Computer Society, 2007, pp. 79–84.

[12] Goldberg, I., et al., "Multi-party Off-The-Record Messaging," *Proceedings of the 16th ACM conference on Computer and Communications Security (CCS '09)*, ACM Press, New York, NY, 2009, pp. 358–368.

[13] Liu, H., Vasserman, E.Y., and N. Hopper, "Improved Group Off-the-Record Messaging," *Proceedings of the ACM Workshop on Privacy in the Electronic Society (WPES 2013)*, ACM Press, New York, NY, 2013, pp. 249–254.

[14] Burmester, M., and Y. Desmedt, "A Secure and Eff cient Conference Key Distribution System," *Proceedings of EUROCRYPT '94*, Springer, LNCS 950, 1995, pp. 275–286.

[15] Kivinen, T., and M. Kojo, "More Modular Exponential (MODP) Diff e-Hellman groups for Internet Key Exchange (IKE)," RFC 3526, May 2003.

[16] Krawczyk, H., Bellare, M., and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, February 1997.

[17] Unger, N., and I. Goldberg, "Improved Strongly Deniable Authenticated Key Exchanges for Secure Messaging," *Proceedings on Privacy Enhancing Technologies*, De Gruyter Open, Volume 2018, Issue 1, pp. 21–66.

[18] Langley, A., Hamburg, M., and S. Turner, "Elliptic Curves for Security," RFC 7748, January 2016.

[19] Stedman, R., Yoshida, K., and I. Goldberg, "A User Study of Off-The-Record Messaging," *Proceedings of the 4th Symposium On Usable Privacy and Security (SOUPS 2008)*, ACM Press, New York, NY, 2008, pp. 95–104.

Chapter 9

Signal

In this chapter, we explain in detail the Signal messenger and the protocol of the same name. We start with the origins and history in Section 9.1, and we then make a deep dive into the technology employed in Section 9.2. In many respects, Section 9.2 is the core of this book, and it is in fact the most important part to read. Understanding the technology and protocol used in the Signal messenger is key to understanding the state of the art in E2EE messaging as it stands today. In Section 9.3, we brief y summarize the results that are known with regard to the security of Signal, and in Section 9.4 we overview and summarize the basic properties of a few other implementations of the protocol (i.e., other than the Signal messenger and WhatsApp that is addressed in the next chapter). This includes Viber, Wire, and Riot. In Section 9.5, we conclude with some f nal remarks.

9.1 ORIGINS AND HISTORY

After the launch of OTR in the 2000s, it became evident that the then-available solutions for secure messaging on the Internet (i.e., OpenPGP and S/MIME) were insuff cient and had to be questioned from the bottom up. OTR itself could be used to provide forward secrecy and plausible deniability in a synchronous setting, such as required by instant messaging. But many of the technologies employed by OTR are interactive in nature and cannot be applied in an asynchronous setting, such as those required by e-mail. In such a setting, an authenticated Diff e-Hellman key exchange cannot be performed directly and interactively. So the challenge was to come up with a technology that works similar to OTR but is also suitable for an asynchronous setting (in which interaction may not be possible).

In 2013, Trevor Perrin and Moxie Marlinspike—then with Open Whisper Systems[1]—came up with a solution to the challenge and proposed a protocol named *Axolotl*.[2] The protocol ingeniously combines two complementary ratchet mechanisms in a so-called *double ratchet*.

- On the one hand, it employs a *Diff e-Hellman ratchet* as originally introduced in OTR (Section 8.2.2).
- On the other hand, it also employs a *symmetric key* or *hash ratchet* as originally introduced in SCIMP (Section 8.2.2). Again, the idea is to use a one-way function f, such as a cryptographic hash function, to generate a series of keys by iteratively mapping an old key k_{old} to a new one k_{new} (i.e., $k_{new} = f(k_{old})$). Due to the properties of one-way functions, k_{old} cannot be computed from k_{new}, and hence this ratchet provides forward secrecy. In contrast to the Diff e-Hellman ratchet, however, it does not provide PCS. If somebody knows a key, then he or she can compute all subsequent keys by iteratively applying f to this key. As we will see later in this chapter, the symmetric key or hash ratchet used by Signal is slightly more involved, but it is based on the same (admittedly very simple) idea.

The double ratchet is at the core of the Axolotl protocol that was built into the two major apps developed and distributed by Open Whisper Systems, namely the text messaging app *TextSecure* and the voice calling app *RedPhone*. After a major revision of the Axolotl protocol, TextSecure and RedPhone were merged into a single and unif ed messenger app called *Signal*,[3] and the Axolotl protocol was renamed to become the Signal protocol. Hence, the terms Axolotl, Signal, and double ratchet are sometimes used synonymously and interchangeably in the literature and in this book. But keep in mind that Axolotl refers to a protocol, Signal refers to both a protocol and a respective messenger, and double ratchet refers to the cryptographic key update mechanism that is used in either case.

The Signal messenger is available for both major mobile platforms (i.e., iOS and Android) whereas the desktop version is available for Windows, MacOS, and

1 Whisper Systems was founded in 2010 by Marlinspike and Stuart Anderson. In 2011, the company was acquired by Twitter. Some of the Whisper Systems software was made available by Twitter under an open source license, and Marlinspike established an organization called Open Whisper Systems to serve this purpose. The organization no longer exists.

2 The protocol is named after the Mexican walking f sh that has a distinct ability to heal itself. This self-healing property is related to the provision of PCS in the realm of secure and E2EE messaging: After the compromise of a key, the Diff e-Hellman key exchange can be executed again to provide some new keying material.

3 Consequently, Signal is an E2EE messenger app that supports text messaging (like TextSecure) and voice calling (like RedPhone).

Linux. What sets Signal apart from many other E2EE messengers is the fact that it is completely open source.[4] This reassures people that it does what its developers claim, since everybody can review the cryptographic algorithms and protocols and audit the respective source code.

After the successful launch of Signal, Open Whisper Systems teamed up with Facebook to incorporate the Signal protocol in WhatsApp and the secret conversations mode of the Facebook Messenger.[5] Microsoft has also adopted the protocol to implement secret conversations in Skype, and Google to implement incognito mode in Allo (before the Allo messenger was f nally abandoned in 2019). In contrast to Signal, all of these implementations are closed source, and hence it is not always possible to inspect and properly verify them.

In addition to the original Open Whisper Systems implementations and respective libraries, there are also a few independent (open source) implementations of the Signal protocol, such as *Proteus* (used in Wire) and *Olm* (used in Matrix and Riot).[6] Both implementations are further addressed in Section 9.4. The Signal protocol and the Olm implementation have also been the basis for an E2EE XEP[7] *OMEMO*—recursively standing for OMEMO Multi-End Message and Object Encryption—that is used by a few other E2EE messengers, such as *Conversations*,[8] *Cryptocat*,[9] and *ChatSecure*.[10] Due to the lack of space, these messengers are not further addressed in this book.

As of this writing, the Signal protocol represents the state of the art in secure and E2EE messaging on the Internet, and this is not likely to change anytime soon. In February 2018, Marlinspike and WhatsApp cofounder Brian Acton announced the formation of the *Signal Foundation*[11] as a nonprof t organization whose mission is "to support, accelerate, and broaden Signals mission of making private communication accessible and ubiquitous." The foundation was started with an initial 50 million USD in funding from Acton, who had left WhatsApp's parent company Facebook in September 2017. Acton serves as the foundation's executive chairman, whereas Marlinspike is the CEO of a limited liability company named

4 https://github.com/signalapp.

5 As of this writing, the question whether Facebook should generalize the secret conversations mode and make it the default mode is controversially discussed in politics.

6 There are also a few experimental implementations that are not ready to use, such as Molch (https://github.com/1984not-GmbH/molch).

7 https://xmpp.org/extensions/xep-0384.html.

8 https://conversations.im.

9 In 2019, the experiment that led to the development of Cryptocat was discontinued. The Cryptocat source code is still available on GitHub under the GPL version 3 license, but it is not further developed. Instead, the fomer homepage of Cryptocat (i.e., https://crypto.cat) recommends to switch to another E2EE messenger.

10 https://chatsecure.org.

11 https://signal.org/blog/signal-foundation.

Signal Messenger.[12] Backed with this organization and funding, Signal can be further developed professionally, and one may hope that the future of Signal is going to be less turbulent than the history of PGP has been in the past.

9.2 TECHNOLOGY

The technology employed by Signal is relatively involved and sophisticated. It is explained in a series of documents [1–4] that are available online.[13] The documents are well written and overview the technology in quite some detail, but they don't provide the background information needed to look behind the scenes and understand the details and some of the design decisions that have been made. This is where this section comes into play: It tries to provide some complementary material and information that may help a novice reader to get himself or herself familiar with the technology.

Signal targets the mobile phone as the user's primary device. In the current implementation, the phone registration process is empowered by Twilio[14] that provides an industry standard widely used in the field. After an initial phone registration, a user is identified with the respective (unique) phone number and authenticated with a password. The password, in turn, is transparent to the user, meaning that he or she does not have to enter it each time Signal is used. Instead, the password is randomly chosen by the Signal server during the device's first use. The identifier (phone number) and password are then transmitted in every request sent to the Signal server. Therefore, some transport layer encryption must be put in place to protect the channel between the device and the server. Signal uses the TLS protocol for this purpose (as we will see later on, WhatsApp uses the Noise protocol framework instead of TLS). In some sense, this is an (implementation) detail the user does not need to be aware of. It just ensures that a passive eavesdropper cannot extract a user's password from the network traffic by simply using a tool like Wireshark. The bottom line is that there are two layers of encryption put in place: transport layer encryption for messenger-server communication and application layer encryption for the messages that are sent back and forth. The focus of this book is on the application layer encryption, and we leave the transport layer encryption aside.

In the rest of this section, we address key agreement and session establishment, the double ratchet mechanism, the authentication ceremony, and group messaging.

12 The limited liability company is to exist only while the Signal Foundation's nonprofit status is pending.

13 https://signal.org/docs.

14 https://www.twilio.com.

Again, the aim is to provide a comprehensive overview and explanation of the Signal protocol and the rationale behind its design, and we therefore focus on the Signal protocol only. Complementary technologies used by Signal, such as the Opus audio codec [5, 6], RTP and SRTP (Section 2.3) for voice and video calls,[15] as well as SQLite[16] and SQLCipher[17] for the local storage of data in encrypted form, are not further addressed. They represent standard technologies not directly related to E2EE messaging and are explained in many other places and resources.

9.2.1 Key Agreement and Session Establishment

To solve the challenge mentioned above, the designers of the Signal protocol had to adapt the synchronous and interactive nature of OTR and make it applicable to the asynchronous and noninteractive setting of e-mail. The main problem is the Diff e-Hellman key exchange that requires interaction by default.[18] Instead of allowing two parties to interact directly, Signal uses a key repository that allows one party—typically the recipient of a message—to register and upload its public Diff e-Hellman keying material,[19] and the other party—typically the sender—to download and use it in a Diff e-Hellman key exchange. The output of the key exchange can then be used to encrypt a message, and the encrypted message can be sent (together with the sender's public Diff e-Hellman key) to the recipient. Finally, the recipient can use the sender's public Diff e-Hellman key to also perform the key exchange and use its output to decrypt the message. This outline is simplif ed and the details are more involved, also because the Diff e-Hellman key exchange is actually a set of multiple Diff e-Hellman key exchanges performed simultaneously and concurrently (as explained below). It is also important to note that the key repository can be centralized, but that it is theoretically feasible to replace it with a decentralized or even distributed repository using blockchain or some other distributed ledger technology (DLT). This is a research topic that is not further addressed here, meaning that we assume the key repository to be centrally operated by a single company or organization.

15 https://signal.org/blog/signal-video-calls-beta.

16 https://www.sqlite.org.

17 https://www.zetetic.net/sqlcipher.

18 There is a non-interactive version of the Diff e-Hellman key exchange that uses static keys. This version works f ne and has been used in several Internet security protocols, such as the Simple Key-Management for Internet Protocol (SKIP) that was a former candidate for what f nally became the Internet Key Exchange (IKE) protocol for IP security (IPsec) and several cipher suites for the SSL/TLS protocols. Note, however, that Diff e-Hellman with static keys provides neither forward secrecy nor PCS. This is always the case if static keys are used.

19 In the case of Signal, there is not only one public Diff e-Hellman key but multiple such keys. That's why we are talking about material instead of a key. The off cial term used in Signal is a key bundle, but this term is introduced later in this chapter.

The use of a key repository solves the interaction problem, but it must still be ensured that the public Diffie-Hellman keys—called *prekeys* in the terminology of Signal—are protected in terms of authenticity and integrity. This is usually achieved with digital signatures issued by some trusted entity, such as a CA. As discussed before, the dependence on CAs is critical, and hence the designers of Signal tried to avoid it. They chose a mechanism that allows a participant to self-sign his or her prekeys, and to postpone the authentication of the respective (signature) keys to some later point in time. More specifically, an *identity key* pair is assigned to a participant,[20] and the participant can use its private identity key to digitally sign its prekey(s). The level of forward secrecy and PCS provided then depends on the frequency of the prekey and signature change: the more frequently they are changed, the better the forward secrecy and PCS properties. In the ideal case, one may use one prekey per message, and the respective prekeys are then called *one-time prekeys*. In this case, however, the key repository has to be fed with sufficiently many prekeys, and one faces a problem when the repository runs out of prekeys—either because the participant has not uploaded sufficiently many prekeys or somebody mounts a (D)DoS attack. To overcome this problem, the Signal protocol follows a compromise: It uses long-term identity keys to digitally sign medium-term prekeys, and it mixes in some one-time prekeys whenever possible.

Following this line of argumentation, the Signal protocol employs the following three classes of public key pairs assigned to participant A:

- A long-term *identity key pair* $(pk_A^{\text{ID}}, sk_A^{\text{ID}})$ that uniquely identifies A on a particular installation of Signal.[21]
- A medium-term *signed prekey pair* $(pk_a^{\text{PK}}, sk_a^{\text{PK}})$ that is changed on a regular basis (e.g., once a week, month, or so), and of which the public key pk_a^{PK} is digitally signed with sk_A^{ID}—denoted as $(pk_a^{\text{PK}}, \mathsf{Sign}(sk_A^{\text{ID}}, pk_a^{\text{PK}}))$ here.
- A pool of n ephemeral *one-time prekey pairs*

$$(pk_{a,1}^{\text{OT}}, sk_{a,1}^{\text{OT}}), (pk_{a,2}^{\text{OT}}, sk_{a,2}^{\text{OT}}), \ldots, (pk_{a,n}^{\text{OT}}, sk_{a,n}^{\text{OT}})$$

  that are each used only for one Diffie-Hellman key exchange (meaning that they are short-lived). Again, the aim of these keys is to provide forward

20 As mentioned before, a participant is identified with a globally unique phone number in Signal. This is just one possibility to uniquely identify participants, and there are other possibilities one may think of, such as identifying participants with e-mail addresses. The Signal protocol is largely independent from how participants are identified, but it requires one possibility to do so.

21 This key pair could also be denoted as (pk_A, sk_A), because the capital letter A already refers to the fact that the key is assigned to A and expected to never change. The superscript ID is only to emphasize the fact that the key is indeed an identity key. This notation is not used in other parts of the book.

secrecy and PCS. The pool may get exhausted, in which case the protocol still works but it no longer provides these properties. In contrast to A's public signed prekey, its public one-time prekeys are not digitally signed.

Taken into account the current state of the art in public key cryptography, the Signal protocol employs and makes use of ECC. While [2] allows for the two elliptic curves specif ed in [7] (i.e., Curve25519 and Curve448) most implementations in use today—including Signal and WhatsApp—only use Curve25519. This means that all public key pairs mentioned above refer to this curve, the respective ECDH key exchange[22] refers to X25519, and the digital signatures (used to sign the prekeys) are EdDSA or—more specif cally—Ed25519 signatures.

For the sake of completeness, we note that in the realm of secret key cryptography Signal uses AES-256 in an AEAD mode of operation, SHA-256 and SHA-512 for hashing, the HMAC [8] construction for message authentication, and both the HMAC and the HMAC-based extract-and-expand key derivation function (HKDF) [9] constructions for key derivation. There are two AEAD modes provided by Signal: AES-256 using a synthetic initialization vector [10] and AES-256 in CBC mode and PKCS #7 padding with a subsequent HMAC computation. Because message authentication is done after encryption in the second case, the CBC mode can be used without being vulnerable to the same padding oracle attacks that can sometimes be mounted against some earlier versions of the SSL/TLS protocols. First encrypting and then authenticating a message is in fact the preferred choice and the more secure way of composing the two operations.

When participant A installs the software on a new device, the software randomly selects the public key pairs mentioned above (at least a f rst set of such keys) and generates a respective *key bundle*—sometimes also called *prekey bundle*—that consists of the following $n + 2$ public keys:

- The public identity key pk_A^{ID};
- The signed public prekey $(pk_a^{\mathrm{PK}}, \mathsf{Sign}(sk_A^{\mathrm{ID}}, pk_a^{\mathrm{PK}}))$;
- A batch of n (unsigned) public one-time prekeys $pk_{a,1}^{\mathrm{OT}}, pk_{a,2}^{\mathrm{OT}}, \ldots, pk_{a,n}^{\mathrm{OT}}$.

The software then registers A with the Signal server. In doing so, it uploads A's key bundle to the key repository. This is done for all participants (users), and hence the repository has and makes available a unique key bundle for each user. From time to time, the prekey pair $(sk_a^{\mathrm{PK}}, pk_a^{\mathrm{PK}})$ needs to be changed, and hence $(pk_a^{\mathrm{PK}}, \mathsf{Sign}(sk_A^{\mathrm{ID}}, pk_a^{\mathrm{PK}}))$ must be updated accordingly. Also, A must regularly

22 In much of the literature, a Diff e-Hellman key exchange that uses nonstatic keys is called *ephemeral* and the respective acronym uses the additional letter E. Consequently, DHE refers to Diff e-Hellman ephemeral. In this sense, the Signal protocol also uses ECDHE instead of ECDH, but to be consistent with the original literature, we also use the acronym ECDH here.

provide some new and fresh one-time public prekeys to make sure that there enough of such keys available. It is important to note that the repository never stores any private keying material, and hence the repository provider(s) has (have) no access to private keys—always assuming an honest and faithful implementation on the client side.

We don't look into the details of the user registration process, mainly because its details are independent from the Signal protocol and each messenger may handle them differently. This also applies to the user authentication mechanisms that may be put in place. As we said earlier, the Signal messenger employs server-selected passwords assigned to users that are resubmitted from the messenger to the server for every single request in a way that is transparent to the user. Following this approach binds the security of the messenger to the security of the device and its operating system. This is reasonable and provides a basic level of security. In a more advanced setting, however, it is possible to plug in more sophisticated user authentication mechanisms, such as requiring a user to type in a PIN for every message sent or received. This is inconvenient for the user, and the Signal protocol is largely independent from any such mechanism (and can be ignored here).

More related to the Signal protocol is the question what happens if A wants to send a message to B. Signal is session-oriented, and this means that a session must be established f rst. Such a session is typically long-lived (e.g., in the range of months and even years) and hence it can be used to send a huge quantity of messages back and forth. A simple form of user authentication takes place during session establishment. If a user wants to more reliably authenticate his or her peer, then a more sophisticated authentication ceremony can be performed at some later point in time. In the meantime, messages are authenticated with some keying material that is derived from the initial user authentication (as explained in detail below).

If A is to establish a session with B, then A acts as an *initiator* and B acts as a *responder*. First, A—or rather the client software acting on A's behalf—downloads some keys of B's bundle from the repository. From a security viewpoint, this is certainly a critical step, mainly because A has no possibility to verify the authenticity and integrity of the respective public keys. This means that somebody being able to feed a faked key bundle into the repository can have A establish a session to whatever user he or she likes (including himself or herself). As mentioned above, the authenticity of the peer can be verif ed after the session is established (Section 9.2.3), but at this point in time A has to accept whatever the repository provides. Again, this trust model is known as TOFU, and we have already seen it in the realm of opportunistic encryption in Section 7.3.

When A downloads some keys of B's bundle, it actually downloads pk_B^{ID}, $(pk_b^{\mathrm{PK}}, \mathsf{Sign}(sk_B^{\mathrm{ID}}, pk_b^{\mathrm{PK}}$, and—if a one-time prekey is available—a $pk_{b,j}^{\mathrm{OT}}$ for some $1 \leq j \leq n$ from $pk_{b,1}^{\mathrm{OT}}, pk_{b,2}^{\mathrm{OT}}, \ldots, pk_{b,n}^{\mathrm{OT}}$. A can use pk_B^{ID} to verify the signature of

pk_b^{PK}, and it continues if and only if the signature is valid. A then randomly selects an ephemeral public key pair (pk_a, sk_a) and uses it together with its own identity key pair $(pk_A^{\text{ID}}, sk_A^{\text{ID}})$, pk_B^{ID}, pk_b^{PK}, and $pk_{b,j}^{\text{OT}}$ to execute a key agreement protocol known as *eXtended Triple Diff e-Hellman* (X3DH).[23]

The X3DH protocol is distinct and characteristic for Signal; it combines multiple (i.e., three or four) Diff e-Hellman key exchanges in a single key agreement. Remember that the OTR AKE protocol uses signatures to authenticate users and a single Diff e-Hellman key exchange. The X3DH protocol is different here: it uses Diff e-Hellman key exchanges (with different keys) also for authentication—at least as far as the TOFU trust model allows. The reason for this is that Diff e-Hellman key exchanges—especially when performed on elliptic curves—are computationally more eff cient than signatures and provide better deniability properties. Note that anybody can execute the X3DH protocol with B, and that no interaction with B is actually required for this purpose. This also means that B can afterwards deny having participated in a particular protocol execution. This would be more diff cult, if digital signatures were used.

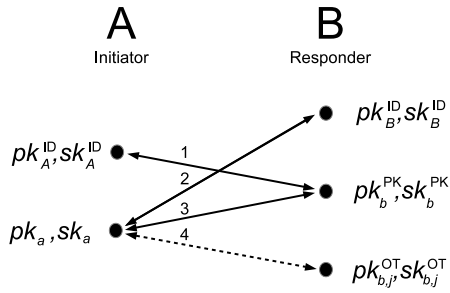


Figure 9.1 The X3DH protocol.

The X3DH protocol is illustrated in Figure 9.1. In addition to some keys of B's bundle as mentioned above and illustrated on the right side of Figure 9.1, A also needs its private identity key sk_A^{ID} and an ephemeral key pair (pk_a, sk_a) that it must particularly generate for this session. These keys are illustrated on the left side of Figure 9.1. Equipped with all these keys, A can execute the X3DH protocol

23 In Section 9.4, we will see that there are a few implementations of the Signal protocol that only use identity keys and some ephemeral keys (called prekeys). The resulting key agreement protocol is a simplif ed version of X3DH. We use the term *Triple Diff e-Hellman* (3DH) to refer to it.

and compute a respective master secret s accordingly. More specif cally, s consists of three or four outputs of properly keyed ECDH key exchanges concatenated in the end. This can be formally expressed as follows:

$$\begin{aligned} s \quad = \quad & \text{ECDH}(sk_A^{\text{ID}}, pk_b^{\text{PK}}) \parallel \text{ECDH}(sk_a, pk_B^{\text{ID}}) \parallel \\ & \text{ECDH}(sk_a, pk_b^{\text{PK}}) \left[\parallel \text{ECDH}(sk_a, pk_{b,j}^{\text{OT}}) \right] \end{aligned}$$

The f rst invocation of ECDH combines A's private identity key sk_A^{ID} and B's public prekey pk_b^{PK}, the second invocation A's private ephemeral key sk_a and B's public identity key pk_B^{ID}, the third invocation again sk_a and pk_b^{PK}, and—last but not least—the fourth invocation sk_a and B's j^{th} public one-time prekey $pk_{b,j}^{\text{OT}}$. Note that the last invocation only applies, if there is a one-time prekey available that has not been used yet. It is optional and therefore drawn with a dotted line in Figure 9.1 and written in square brackets in the formula given above.

Each output of an ECDH key exchange is 32 bytes long, meaning that the resulting value of s is 96 or 128 bits long in total—depending on whether three or four exchanges take place. A can then use s to derive the keying material that is going to be used in the Signal protocol (as explained below). To provide forward secrecy and PCS, A must delete its ephemeral private key sk_a and the ouputs of each ECDH key exchange after use.

As mentioned in Section 3.2.2.2, the current trend in cryptography is to invoke AEAD to protect messages whenever possible. This means that the actual message is protected in terms of conf dentiality and authenticity, but some *assoiated data* (AD) may only be protected in terms of authenticity (it cannot be protected in terms of conf dentiality, because it must be available in the clear). In the case of the Signal protocol, the AD contains at least some identity information about A and B, such as the concatenation of some encoding of the public identity keys:

$$\text{AD} = \text{Encode}(pk_A^{\text{ID}}) \parallel \text{Encode}(pk_B^{\text{ID}})$$

A may optionally append other information to the AD, such as A and B's usernames, certif cates, or anything else. Different implementations of the Signal protocol may use different constructions here.

In the f rst message sent to B (after having executed the X3DH protocol), A provides the following pieces of information to B:

- A's public identity key pk_A^{ID};
- A's public ephemeral key pk_a;
- An identif er j stating which of B's one-time prekeys A actually used;

- A ciphertext encrypted with an AEAD encryption scheme (where AD is constructed as specif ed above).

After having received this message, B can extract A's public identity key pk_A^{ID} and public ephemeral key pk_a, and use them—together with its private identity key sk_B^{ID}, private prekey sk_b^{PK}, and j^{th} private one-time prekey $sk_{b,j}^{\text{OT}}$—to recompute s according to the following formula:

$$\begin{aligned} s \;\; = \;\; & \text{ECDH}(pk_A^{\text{ID}}, sk_b^{\text{PK}}) \parallel \text{ECDH}(pk_a, sk_B^{\text{ID}}) \parallel \\ & \text{ECDH}(pk_a, sk_b^{\text{PK}}) \left[\parallel \text{ECDH}(pk_a, sk_{b,j}^{\text{OT}}) \right] \end{aligned}$$

As is usually the case in an (elliptic curve) Diff e-Hellman key exchange, essentially the same computation is done on either side—with the roles of public and private keys simply being swapped. The f rst invocation of ECDH combines A's public identity key pk_A^{ID} and B's private prekey sk_b^{PK}, the second invocation A's public ephemeral key pk_a and B's private identity key sk_B^{ID}, the third invocation again pk_a and sk_b^{PK}, and the fourth invocation—if available—pk_a and B's secret one-time prekey $sk_{b,j}^{\text{OT}}$.

After having recomputed s, B must also delete all outputs of the ECDH key exchanges to provide PCS. B can then construct the AD with pk_A^{ID} and pk_B^{ID}, and decrypt the ciphertext embedded in the message with s and the AD. The use of an AEAD mode suggests that B must abort the session and delete s, if the ciphertext fails to decrypt correctly. If, however, the ciphertext decrypts successfully, then the session can be established and B must remove the now used j^{th} one-time prekey $pk_{b,j}^{\text{OT}}$ from its batch. Also, B may continue using s and the keys that can be derived from it within any post-X3DH protocol to securely communicate with A. Most importantly, it can be used as a starting value for the double ratchet mechanism addressed next. The aim is to refresh the keying material as often as possible.

9.2.2 Double Ratchet Mechanism

To properly understand the double ratchet mechanism, it is necessary to f rst introduce the notion of a *KDF chain*—mainly because multiple such chains are generated by a symmetric key or hash ratchet and used simultaneously in the Signal protocol. A *key derivation function* (KDF) is a (cryptographic) function that takes some input data and returns some output data.

- The input data comprise a key—called a KDF key—and some other input value;
- The output data comprise an updated version of the KDF key and some other output value.

In the case of the Signal protocol, the length of the KDF key is 32 bytes. If the key is unknown, then the output data must be indistinguishable from random data, meaning that the KDF is one-way and represents a PRF (in cryptographic parlance). According to what has been said above, you may think of using the HMAC [8] or—more specif cally—the HKDF construction [9] to serve as a KDF, typically with a 256-bit or 512-bit hash function, such SHA-256 or SHA-512. The HMAC construction is well known and widely used in the f eld. It takes as input a key k and a message m, and it generates and outputs a respective MAC for m (that also depends on k). Similarly, the HKDF construction takes as input a salt s, a key k, an arbitrary string str, and an output length l, and it generates an l-byte output string $output$ from which the required keying material can then be taken. More formally, this can be expressed as follows:

$$\text{HKDF}(s, k, str, l) = output \text{ with } |output| = l$$

Note that the KDF takes a KDF key as input and may output another KDF key. This means that the KDF can be iterated multiple times to implement some form of ratcheting.

The result is a KDF chain as illustrated in Figure 9.2 (with only three iterations). The KDF key is used as a chaining value, and in each iteration an input value is mapped to an output value. If the input values are constant (i.e., the same input value is used in each iteration), then the resulting KDF chain is degenerated and can be used to implement a SCIMP-like symmetric key or hash ratchet. The KDF is then just used to iteratively hash a chain key. In each iteration, the KDF updates the chain key and outputs some additional data that yields a message key.[24] The message keys represents the (cryptographic) workhorses in the Signal protocol, meaning that they are used for message encryption and authentication. This way of using a KDF chain to implement a symmetric key or hash ratchet is illustrated in Figure 9.3. We say that a normal KDF chain (where a new input value is fed into the KDF in each iteration) is of type I, whereas a degenerated KDF chain (where the input value is always the same constant) is of type II. The Signal protocol employs either type.

The Signal protocol employs three KDF chains: A KDF chain of type I that represents a *root chain*, and two KDF chains of type II that represent a *sending chain* and a *receiving chain*. As their names suggest, the sending chain is used in a symmetric key or hash ratchet to generate the encryption keys (i.e., the keys that are used to encrypt and send out messages), whereas the receiving chain is used to generate the decryption keys (i.e., the keys that are used to decrypt the received messages). It goes without saying that the keys generated in the sending chain of

24 The rationale behind the separation of the chain key and the message key is explained later in this section.

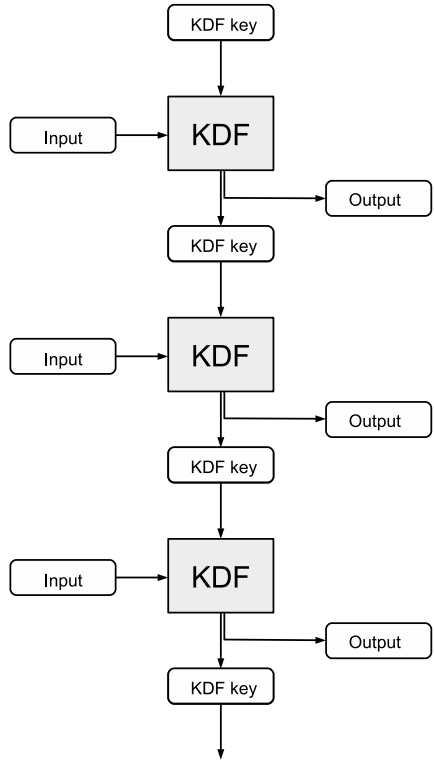


Figure 9.2 A KDF chain with variable input (Type I).

one user must match the keys generated in the receiving chain of the other user, if the two users want to communicate and exchange (encrypted) messages with each other. An encrypted message must always be decrypted with the same key.

All KDF chains work in concert to generate and update the keys required in the Signal protocol. We have already seen the master secret s that results from the execution of the X3DH protocol. This value is the starting point to derive different keys that serve different purposes. In fact, the following types of keys are used:

- A *root key* is derived from the master secret and is ratcheted forward in the root chain. In each iteration, the root key is updated and an additional output—the

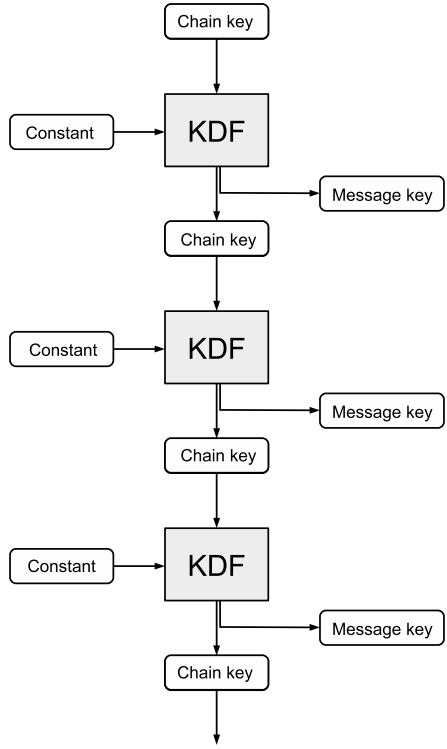


Figure 9.3 A KDF chain with constant input (Type II).

chain key—is to start a new type II KDF chain (i.e., a new sending chain or a new receiving chain).

- As mentioned above, a *chain key* is the starting value of a type II KDF chain—either a sending chain or a receiving chain. The chain key is then ratcheted forward in the KDF chain. In each iteration, the chain key is updated and an additional output—the message key—is generated.
- Finally, a *message key* is the working horse of the Signal protocol, and it is used to cryptographically protect (i.e., encrypt and authenticate) a message.

The use of message keys (and the way they are def ned here) suggests that a new key is used for each and every message. This is as far as one can go in terms of key refreshment and update cycles.

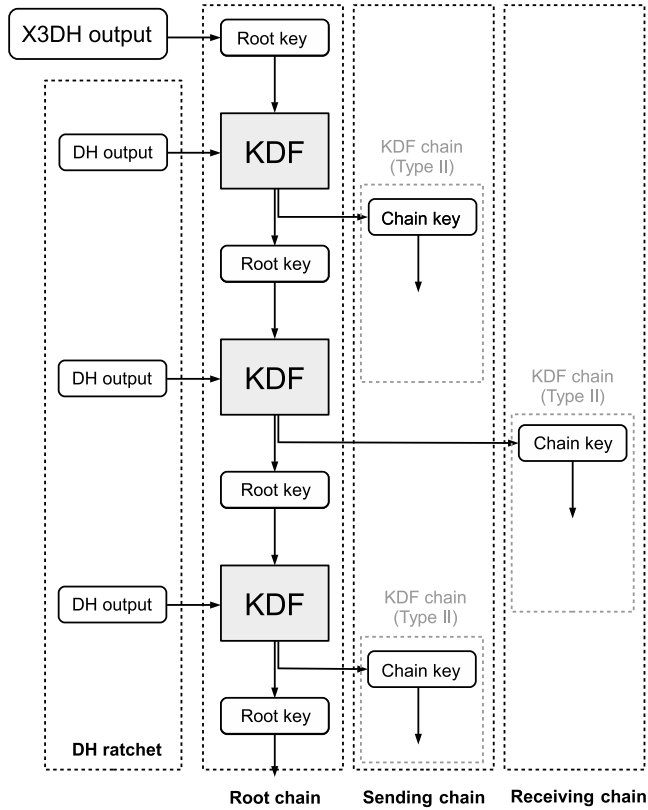


Figure 9.4 The double ratchet mechanism employed by Signal (schematic representation).

With regard to Figure 9.3 and the description given above, one may wonder why the chain key is not used directly as the message key. Why are there two distinct outputs from the KDF: the chain key and the message key? To understand the rationale behind this design, it is useful to have a look at the way messages are transmitted on the Internet. It may happen that messages get lost or are received out of order. In this case, it is important to forward the ratchet and cache the respective

message keys until they are used (note that message keys can be stored without affecting the security of any other message key). This simplif es the management of the ratchet considerably.

Having prepared all ingredients, we are now ready to outline the double ratchet mechanism employed by the Signal protocol as schematically represented in Figure 9.4. From a bird's eye perspective, there is a Diff e-Hellman ratchet that provides the input values to the root chain, and—as mentioned above—two type II KDF chains that represent the sending and receiving chains. The root chain is a type I KDF chain initialized with the output of the X3DH protocol (i.e., the master secret s) and each output of the root chain provides a starting point (and hence the initial chain key) for a new sending or receiving chain. These two chains each output the message keys that are used to either encrypt or decrypt the messages. The message keys from the sending chain are used for encryption and the message keys from the receiving chain are used for decryption. The message keys themselves are not illustrated in Figure 9.4, because they would only complicate things without adding much value here.

At the beginning, the Diff e-Hellman ratchet is initialized with A's ephemeral private key sk_a and B's public prekey pk_b^{PK}. The resulting output $\text{ECDH}(sk_a, pk_b^{\text{PK}})$ is the input to A's root chain, from which a chain key for the sending chain is derived. The sending chain is ratcheted forward to yield a message key, and A can use this key to encrypt the message. The ephemeral public pair pk_a is sent together with the encrypted message to B, so that B can compute $\text{ECDH}(pk_a, sk_b^{\text{PK}})$ and initialize its root chain accordingly. Again, B can derive the same chain key for the receiving chain. This means that A and B are in now sync (i.e., they have synchronized root chains and A's sending chain is synchronized with B's receiving chain). When B sends an encrypted message back to A, a new ephemeral public key pk_b is provided. This allows A and B to ratchet forward their Diff e-Hellman ratchets and root chains. With the output of the root chains, B's sending chain and A's receiving chain can be initialized.

The frequency of the Diff e-Hellman ratchet determines the frequency of the root chain, and this frequency, in turn, determines the lengths of the sending and receiving chains. The more frequently the Diff e-Hellman ratchet outputs a new value, the more frequently the root chain is forwarded, and hence the more frequently a new (sending or receiving) chain is instantiated. In the most preferred case, each message comes along with a new Diff e-Hellman parameter that triggers a new Diff e-Hellman key exchange and also ratchets forward the root chain.

Last but not least, we note that the Signal protocol employs message headers that contain ratchet public keys and values to determine the proper ordering of the messages within a session, and that there is a variant of the Signal protocol that supports header encryption. This may be desirable so that a passive adversary can't tell which messages belong to which sessions, or the ordering of messages within

a particular session. Header encryption is relatively sophisticated and beyond the scope of this book. You may refer to Section 4 of [3] for a respective outline.

9.2.3 Authentication Ceremony

Earlier in this chapter we said that A downloads some of B's public keys from the repository and uses them to execute the X3DH protocol, but that proper authentication in terms of key verif cation and trust establishment is postponed to an authentication ceremony that may be performed after session establishment. What this basically means is that once the session is established, A and B can mutually authenticate themselves without disturbing the message f ow. Peer authentication is optional and may happen at any point in time, but it is not required, meaning that the Signal protocol can be executed without ever having authenticated the peer. It can be used to improve the TOFU trust model that is otherwise used by default.

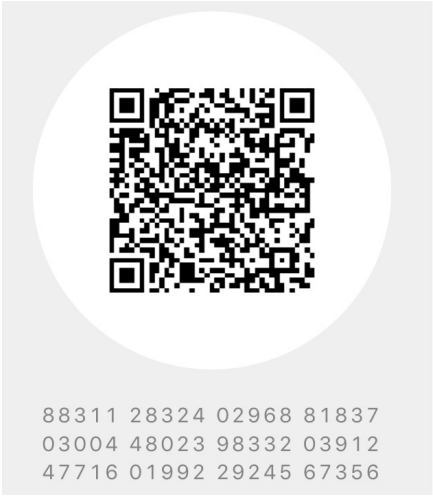


Figure 9.5 A QR code with respective 60-digit security number.

Signal does not require users to manually verify public key certif cates or f ngerprints, or the clients to execute a protocol like the SMP used in OTR. Instead, peer authentication can be done by either scanning a QR code or comparing a 60-digit security number. The respective user interface is illustrated in Figure 9.5 (the QR code is at the top and the security number is at the bottom).

- The QR code contains the following pieces of information:
  - The version number in use;
  - The user identif er for both parties (i.e., A and B);
  - The public identity key pk^{ID} for both parties (i.e., pk_A^{ID} and pk_B^{ID}).

  Either user can scan the other user's QR code as displayed on the screen. It is then verif ed whether the other user's public identity key (as contained in the QR code) matches the key retrieved from the key repository and used in the protocol so far. If this is the case, then everything is f ne and the user is considered to be authenticated.
- The 60-digit security number is generated by concatenating a 30-digit f n-gerprint for either party's public identity key. For user A, for example, this f ngerprint is computed as follows:
  - The public identity key and user identif er pair (pk_A^{ID}, A) is hashed 5200 times with the cryptographic hash function SHA-512:

$$hash = \text{SHA-512}^{5200}(pk_A^{\text{ID}}, A).$$

  - Only the f rst 30 bytes of $hash$ are considered.
  - These 30 bytes are split into 6 chunks of 5 bytes each.
  - Each 5-byte chunk is converted into 5 digits by interpreting it as a big-endian unsigned integer and reducing it modulo 100000.
  - The 6 resulting chunks of 5 digits each are concatenated into a total of 30 digits. The result is A's f ngerprint and B's f ngerprint is computed similarly.

  If the peers have a consistent view on their public identity keys (i.e., the keys used in the session) then they are authentic. This means that each user can verify that his or her security number is the same as the number of the other user. The simplest way to achieve this is to have one user read the number and the other user to verify it.

Either of these possibilities is simple and convenient to use. Hence, one could assume that users frequently employ them. Unfortunately, this is not the case and many users routinely employ the Signal messenger without ever having authenticated any of his or her peers (this also applies to WhatsApp and any other E2EE messenger that employs the Signal protocol). This is unfortunate and something that may be improved by providing a technology that is even simpler to

use and hence more user-friendly. It will be interesting to see what such a technology may look like in the future. In the meantime, people are asked to use this relatively simple peer authentication mechanism.

9.2.4 Group Messaging

In Section 8.1, we brief y explained mpOTR and the way it tries to expand OTR to group messaging. We concluded that mpOTR is not particularly well suited for an asynchronous setting (in which group members may be off ine and not able to participate in an interactive protocol, and in which sessions may be long-lived), and that the designers of the Signal protocol therefore had to follow another approach—originally named private group messaging.[25]

Many traditional (non-E2EE) messengers and messenger apps—especially if they are operated centrally—employ a group messaging mechanism that is known as *server-side fan-out*. What this basically means is that the sender transmits a group message to the server, and the server then fans out the message to all—let's say n—participants of the group (or group members, respectively). This may relieve the sender considerably, especially if the message is very large. Note that the sender only transmits a single message to the server.

In E2EE messaging, the n messages sent to the group members are encrypted with different keys and are therefore distinct. This means that a server-side fan-out is not compatible with E2EE messaging per se, and that one must use a trick to implement it anyway. The trick is to encrypt the message with a so-called *sender key*, and to distribute this key to all group members, using, for example, normal E2EE messaging. What this basically means is that the sender establishes a pairwise (secure) session with every group member and uses this session to securely transmit the sender key to him or her. If a group consists of n members, then there are n sender keys that need to be securely distributed this way. Once the message is encrypted with the sender key, it can be fanned out by the server to all group members, and each group member can then decrypt the message with its copy of the sender key. This approach saves computational power and bandwidth, and is used, for example, in WhatsApp (Section 10.2.4). This variant of the basic Signal protocol is sometimes referred to as Sender Keys. It makes group messaging more eff cient, but it also has disadvantages related to privacy. To perform a server-side fan-out, the server must know or somehow be told what users belong to what groups. This information is sensitive and there are users who prefer not to reveal it to a central server and the operator of it.

In private group messaging, Signal avoids a server-side fan-out and implements another group messaging mechanism that is known as *client-side fan-out*.

25 https://signal.org/blog/private-groups.

This mechanism is very simple and uses normal E2EE messaging to build a group. Instead of a single message that is sent to the server to be fanned out, the sender transmits n E2EE messages to the n members of the group. This also means that the group members must know what other users are members of the group. Hence, the information related to group memberships can be shared among all users (in a decentralized or fully distributed way) or it can be stored on the server side. In either case, the role the server has to play in a client-side fan-out is much smaller than the role it has to play in a server-side fan-out, and hence a client-side fan-out is generally better suited to provide privacy, especially when it comes to group memberships.

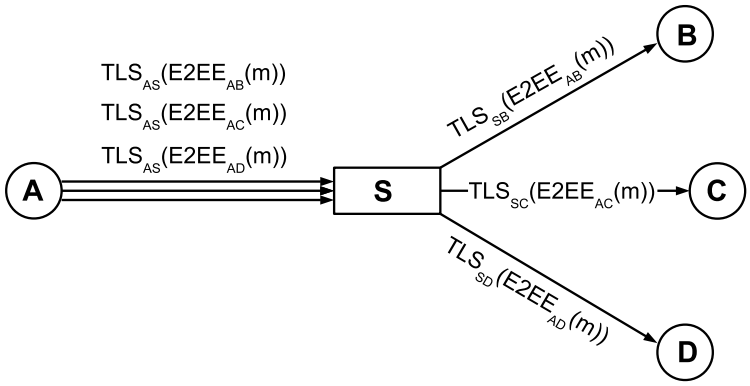


Figure 9.6 The Signal client-side fan-out mechanism.

The way Signal implements a client-side fan-out mechanism for group messaging is schematically represented in Figure 9.6. The sender A on the left side wants to send an E2EE message m to a group of three recipients (i.e., B, C, and D) on the right side, via the server S. A therefore establishes three E2EE sessions to B, C, and D, and composes three distinct messages for them: $\text{E2EE}_{AB}(m)$ encrypted and destined for B, $\text{E2EE}_{AC}(m)$ encrypted and destined for C, and $\text{E2EE}_{AD}(m)$ encrypted and destined for D. All three messages are collectively delivered to S, using a secure channel between A and S. In Signal, this secure channel is provided by the TLS protocol—denoted as TLS_{AS} in Figure 9.6. This channel is used to send all E2EE messages to S. When S receives them, it simply forwards them to B, C, and D, again invoking three separate TLS sessions between S and B (denoted as TLS_{SB}), S and C (denoted as TLS_{SC}), and S and D (denoted as TLS_{SD}). The

messages that are forwarded on these sessions are the same as the ones originally provided by A. This means that B, C, and D can decrypt the messages using the E2EE sessions they have previously established with A. This means that Signal uses normal E2EE messages to simulate group messaging or a group chat in a simple and straightforward way.

To properly implement a client-side fan-out, all clients (of the group members) must share or somehow have access to the group state that comprises items like a group identif er, a name, an image, some information about the group memberships, and many more. As mentioned above, the group state can be stored in either a decentralized or a centralized way. Each possibility has advantages and disadvantages. If the state is stored in a decentralized way (i.e., on the client side) then the server doesn't have to know what users are members of what groups, but it is then diff cult to maintain the consistency of the state. If, on the other hand, the state is stored in a centralized way, then the state can be kept consistent, but one has to live with the fact that the server knows what users are members of what groups.

More recently, the developers of Signal have proposed a technology that may used to store the group state in a centralized way, and hence to prof t from the advantage of making it simple to maintain consistency, without making it necessary for the server to know what users are members of what groups [11]. The technology extends keyed-verif cation anonymous credentials (KVAC) originally proposed in [12] for group messaging in Signal. As of its writing, this is just a proposal. It is, however, possible and very likely that the proposal will be implemented in future releases of the Signal messenger.

In contrast to many other E2EE messengers that support group messaging and group chats (e.g., WhatsApp, Threema, and many more), Signal implements non-administered groups, meaning that all members of a group are equal and can speak for the entire group, meaning that they can administer the group and manipulate the group management information accordingly.

9.3 SECURITY ANALYSIS

A predecessor of the Signal protocol used in TextSecure was analyzed in [13].[26] The researchers found that message content deniability was not as strong as originally anticipated and that some subtle f aws in the protocol could be exploited in an attack known as an *unknown key-share attack* [14]. This attack is conceptually similar to the *identity misbinding attack* against OTR version 1 (Section 8.1): An attacking user (C) can download another user's (B's) key bundle from the repository and register the same keying material for himself or herself. When user A tries to establish a

26 The analyzed protocol is referred to as TextSecure version 2.

session with C, he or she actually establishes a session with B. The session cannot be decrypted by C, because C does not have the required private keys, but A is still misled and believes he or she is sharing the key with C (where in fact it is shared with B). Whether this poses a problem depends on the application setting. To mitigate this (unknown key-share) attack, it is necessary to uniquely bind a registered public key to a particular user. In the Signal protocol, this is done by having the prekeys be signed with the private identity key of the respective user and keeping track of what user provided what prekey. As pointed out in [13], this binding can be improved to mitigate some more subtle forms of the attack.

More recently, the double ratchet mechanism as used in the Signal protocol has become a research topic of its own, and some researchers use the term *ratcheted key exchange* (RKE) to refer to it. If they want to emphasize the fact that an RKE also works in an asynchronous setting, then they add the adjective asynchronous, and if they want to emphasize the fact that messages can be exchanged in either direction, then they even add the adjective bidirectional. Using this terminology, the Signal protocol actually provides a *bidirectional asynchronous RKE*, and this basic cryptographic primitive has been studied in terms of security, formal verif ability (using automated tools), and optimization. The results achieved so far [15–20] look promising and speak for themselves,[27] and even more results are expected to be found and published in the future.

The bottom line is that the Signal protocol is commonly considered to be secure, at least if used in a two-party setting. In a multiparty setting, however, the situation is more involved, and recent research has revealed some subtle vulnerabilities and shortcomings in the way some E2EE messengers, like Signal, WhatsApp, and Threema, handle the management of groups [21]. With regard to Signal, an adversary could exploit the facts that Signal groups are not administered, meaning that anybody can send group management messages to the server, and that the implementation was buggy in the sense that it did not properly check whether the sender of such a message was indeed a member of the group. This allowed an adversary to illegitimately add a new member to a group, and hence to defeat the original purpose of E2EE messaging. Luckily, the attack was more diff cult to mount in practice, mainly because the adversary had to know a random-looking (and hence hard-to-guess) 128-bit ID for the group. Also, the implementation could be easily patched by making sure that a group management message must always come from a legitimate member of the group. However, the mere existence of the attack was controversially discussed in the community, and even today people sometimes have a bad gut feeling when they use group chats in Signal.

27 The results come along with many new acronyms that are not introduced here. It is assumed that many of these acronyms will be relevant only in research circles and not used in public.

9.4 IMPLEMENTATIONS

As the Signal protocol represents the state of the art in E2EE messaging on the Internet today, many messengers use it directly or indirectly (i.e., they use a variant of the Signal protocol). This is true, for example, for Silent Circle' *Silent Phone* that originally started with the SCIMP and later adapted Signal's double ratchet mechanism to provide PCS (in addition to forward secrecy),[28] but it is equally true for many other E2EE messengers and messenger apps. Some of them strictly follow the Signal protocol, whereas others are loose and deviate from it signif cantly. In this section, we have a closer look at Viber, Wire, and Riot as three examples. There are many other messengers and messenger apps that also use the Signal protocol, but they are somewhat more stealthy and less widely used in the f eld. They are not addressed in this book, but you can still f nd a lot of information about them on the Internet.

9.4.1 Viber

The *Viber*[29] messenger was originally developed in 2010 by an Israel company called Viber Media. In 2014, the company was acquired by the Japanese company Rakuten. Today, Rakuten has a subsidiary called Rakuten Viber—currently based in Luxembourg—to commercialize Viber. Besides WhatsApp, Facebook Messenger, and WeChat (in China), Viber is presumably one on the most widely used messengers worldwide with more than one billion users. It is especially popular for voice and video calls, where it has its roots.

Since April 2016 and version 6.0, the Viber messenger supports E2EE messaging by default using a simplif ed variant of the Signal protocol that has been independently developed and implemented from scratch. Unfortunately, the documentation of Viber's E2EE messaging feature is rather short [22] and the implementation is closed source. This means that everything said here must be taken with a grain of salt. Independent from this disclaimer, the cryptographic algorithms employed by Viber are state of the art: X25519 for key agreement, Salsa20 for encryption, SHA-256 for hashing, and HMAC-SHA256 for message authentication. For audio or video calls, the same elliptic curve (i.e., Curve25519) is also used for digital signatures using Ed25519.

In contrast to several other E2EE messengers, Viber supports multiple devices per user, meaning that multiple devices can be associated with one user account. Each device is identif ed with a Viber-specif c *unique device ID* (UDID). It may also have a mobile phone number, but this need not be the case. This means that a device

28 https://www.silentcircle.com.
29 https://www.viber.com.

need not be a mobile phone, but can be anything connected to the Internet, such as a desktop, notebook, iPad, or tablet. Each user has a primary device that is to generate an identity key pair $(sk^{\mathrm{ID}}, pk^{\mathrm{ID}})$ for the user account, and several secondary devices that share this key pair. As we will see below, each device—be it a primary device or a secondary device—additionally holds some unique keying material that is used to establish a secure session with it.

Because the primary device and the secondary devices belong to the same (user) account, there must be a possibility to have the primary device share and securely transmit the private identity key to the secondary devices. Each secondary device therefore generates an ephemeral public key pair and generates a QR code that comprises its UDID and the public ephemeral key. This QR code is displayed on the secondary device, from where the user can scan it with his or her primary device. The primary device then generates another ephemeral public key pair and performs an ECDH computation with its private key and the public key from the QR code. The result is hashed with SHA-256 to create a secret key. The primary device then symmetrically encrypts the private identity key (that it wants to share) with this secret key and sends it together with its own ephemeral public key to the secondary device (that is identif ed with the UDID from the QR code). As a side remark we note that the ciphertext is also authenticated with HMAC-SHA256.[30] If the secondary device receives the ciphertext, it uses the primary device's ephemeral public key to perform the same ECDH computation and hashes the result to obtain the secret key. This key is then used to decrypt the private identity key and verify the HMAC value accordingly. If the verif cation succeeds, then the secondary device can start using the private identity key to subsequently establish sessions on the user's behalf (as discussed below). This procedure must be repeated for all secondary devices of the user.

Having the primary device share the private identity key with all secondary devices allows Viber to treat all devices identically and to keep them in sync. Messages sent or received by any of the (primary or secondary) devices can be displayed on all devices registered for the user account, from the time of their registration and onward. This multi-device support is certainly an added value of Viber with regard to many other E2EE messengers in use today, but it also has its security issues, because the private identity key now resides on multiple devices and must be protected accordingly. This obviously increasing the attack surface.

A session needs to be established between every two devices that wish to communicate securely. Once such a session is established, it can be used to send an

30 Unfortunately, the documentation does not specify what key is used to generate and verify the HMAC value and in what order message encryption and authentication are applied. As we know from the SSL/TLS protocols, this order is important when it comes to specif c attacks, such as padding oracle attacks.

unlimited number of messages in either direction. To send a message, sessions must exist between the sending device and all of the recipient's devices, as well as between the sending device and all of the sender's other devices. Sessions between devices of the same account are established when the devices are registered. However, only one session is required between any two devices, and that session can be used to synchronize any number of conversations with other user accounts (if such conversations exist in the f rst place).

Viber uses a variant of the Signal protocol, meaning that there are some points where the Viber protocol deviates from the original. First and foremost, Viber uses fewer keys and a simpler version of the X3DH protocol—let's call it 3DH protocol. Instead of using identity keys, signed prekeys, and one-time prekeys as in X3DH, Viber only uses identity keys and prekeys that are intended to be used once (and represent one-time prekeys though). This means that there are no signed prekeys. In Viber, every prekey refers to a set of two public key pairs—a handshake key pair $(sk^{\text{HS}}, pk^{\text{HS}})$ and a ratchet key pair $(sk^{\text{R}}, pk^{\text{R}})$. So when user A registers his or her i^{th} device, A's public identity key pk_A^{ID} and n public prekeys $(pk_{a_i,1}^{\text{HS}}, pk_{a_i,1}^{\text{R}}), (pk_{a_i,2}^{\text{HS}}, pk_{a_i,2}^{\text{R}}), \ldots, (pk_{a_i,n}^{\text{HS}}, pk_{a_i,n}^{\text{R}})$ are actually uploaded to Viber's key repository. This sums up to a bundle of $2n + 1$ keys.

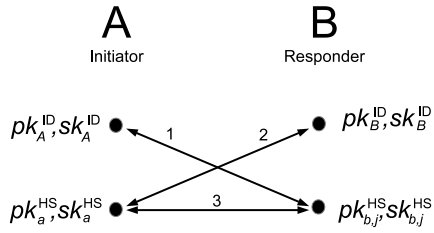


Figure 9.7 The 3DH protocol implemented in Viber.

If user A wants to establish a session to B with one of his or her devices, this device sends a query to the Viber server with the recipient's phone number.[31] The server responds with B's public identity key pk_B^{ID} and a series of public prekeys, one for each device that is currently registered for B. If, for example, B has registered 3 devices, then the series comprises $(pk_{b_1,j}^{\text{HS}}, pk_{b_1,j}^{\text{R}}), (pk_{b_2,k}^{\text{HS}}, pk_{b_2,k}^{\text{R}}), (pk_{b_3,l}^{\text{HS}}, pk_{b_3,l}^{\text{R}})$

31 While UDIDs are used to identify devices, phone numbers are still used to identify users.

for some arbitrarily chosen $1 \leq j, k, l \leq n$.[32] A's device then establishes a session to each of these devices (the sessions to each of A's other devices have already been established during device registration). To simplify the outline and notation, we only consider the session establishment to one of B's devices (so we can leave aside the respective indexes) with the jth prekey—denoted as $(pk_{b,j}^{\text{HS}}, pk_{b,j}^{\text{R}})$. The respective 3DH protocol is illustrated in Figure 9.7 (you may compare this f gure to the full X3DH protocol illustrated in Figure 9.1). A's ephemeral key pair used in the Signal protocol becomes a prekey in the Viber protocol that consists of a handshake key pair $(sk_a^{\text{HS}}, pk_a^{\text{HS}})$ and a ratchet key pair $(sk_a^{\text{R}}, pk_a^{\text{R}})$. To establish a session, only the handshake key pair is used, and the ratchet key pair is later used in the Diff e-Hellman ratchet. Furthermore, $pk_{b,j}^{\text{OT}}$ in the X3DH protocol is replaced with $pk_{b,j}^{\text{HS}}$ here, and $sk_{b,j}^{\text{OT}}$ is replaced with $sk_{b,j}^{\text{HS}}$. A can compute a master secret s as

$$s = \text{ECDH}(sk_A^{\text{ID}}, pk_{b,j}^{\text{HS}}) \parallel \text{ECDH}(sk_a^{\text{HS}}, pk_B^{\text{ID}}) \parallel \text{ECDH}(sk_a^{\text{HS}}, pk_{b,j}^{\text{HS}})$$

and B computes the same value as

$$s = \text{ECDH}(pk_A^{\text{ID}}, sk_{b,j}^{\text{HS}}) \parallel \text{ECDH}(pk_a^{\text{HS}}, sk_B^{\text{ID}}) \parallel \text{ECDH}(pk_a^{\text{HS}}, sk_{b,j}^{\text{HS}})$$

Viber's 3DH protocol is simpler and more straightforward than Signal's X3DH protocol, but it also has the disadvantage that the key repository may run out of prekeys for B. Remember that this was one of the reasons why the X3DH protocol employed both signed prekeys and one-time prekeys in the f rst place. If this happens, then the 3DH protocol can no longer be executed with this device.[33] This is certainly a drawback of Viber, but if it happens, then B may still have other devices he or she can use to continue work.

Another point where Viber differs from Signal is key derivation. Remember that the double ratchet mechanism employed by Signal uses a Diff e-Hellman ratchet and three symmetric key or hash ratchets. In contrast, Viber uses a Diff e-Hellman ratchet and only one type I KDF chain that yields a root chain. Session keys are directly derived from the root chain, and there are no sending and receiving chains in Viber.

- The Diff e-Hellman ratchet consumes the ratchet key pairs that are part of the prekeys.

32 This notation is simplif ed, because n is not a constant and may be different for each user and each of this user's device.

33 In the following subsection we will see that the Wire messenger has a fallback mechanism for this case: The last prekey is special in the sense that it can repeated an unlimited number of times. In the Viber documentation, there is no evidence for this or any other fallback mechanism.

- The root chain starts with an initial root key derived from s. In each iteration, the root key and an output of the Diff e-Hellman ratchet are fed into the KDF, and the KDF updates the root key (using a temporary key as explained below) and outputs a session key.

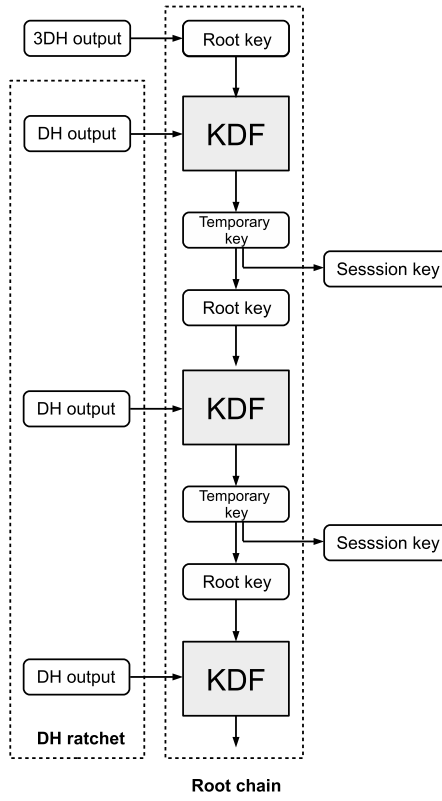


Figure 9.8 The interplay of the Diff e-Hellman ratchet and the root chain in Viber.

The interplay of the Diff e-Hellman ratchet and the root chain is illustrated in Figure 9.8. First, the root key k_{root} of the root chain is initialized with the 32-byte SHA256 hash value of s (i.e., the output of Viber's 3DH protocol):

$$k_0^{root} = \text{SHA256}(s)$$

In each iteration of the Diff e-Hellman ratchet, the latest ratchet key pairs of A and B are used to execute the ECDH key exchange protocol. In iteration $i > 0$, the resulting value v_i is used to compute a temporary key k_i^{temp}. If $(sk_a^{\mathrm{R}}, pk_a^{\mathrm{R}})$ and $(sk_b^{\mathrm{R}}, pk_b^{\mathrm{R}})$ are A and B's latest ratchet key pairs, then A computes v_i as $\mathrm{ECDH}(pk_b^{\mathrm{R}}, sk_a^{\mathrm{R}})$ and B computes v_i as $\mathrm{ECDH}(pk_a^{\mathrm{R}}, sk_b^{\mathrm{R}})$. In accordance with Figure 9.8, a temporary key k_i^{temp} can now be computed as the HMAC-SHA256 value of v_i keyed with k_i^{root}:

$$k_i^{temp} = \text{HMAC-SHA256}(k_i^{root}, v_i)$$

From this value a new root key k_{i+1}^{root} and a session key $k_i^{session}$ are derived using two constant but distinct strings (i.e., root and mesg):[34]

$$\begin{aligned} k_{i+1}^{root} &= \text{HMAC-SHA256}(k_i^{temp}, \text{root}) \\ k_i^{session} &= \text{HMAC-SHA256}(k_i^{temp}, \text{mesg}) \end{aligned}$$

In the end, A is equipped with the next root key and a session key that can be used to secure the session to B (or one of B's devices, respectively). The first message A sends out is a *session start message* that contains A's public identity key pk_A^{ID}, a reference j for B's prekey used by A, and A's own prekey $(pk_a^{\mathrm{HS}}, pk_a^{\mathrm{R}})$—consisting of the public handshake key pk_a^{HS} and the public ratchet key pk_a^{R}. When B's device receives the session start message, it can reconstruct the root key k_0^{root} and initialize the root chain with it. All temporary keys, updated root keys, and session keys can then be derived in the same way. This allows A and B to share the keying material, and hence to secure the session accordingly.

As a consequence of Viber's multi-device support, the sending device has to encrypt a message for every receiving device. To achieve this, Viber uses a mechanism that is conceptually similar to the Sender Keys variant of the Signal protocol: The sending device generates an ephemeral 128-bit symmetric key that is used to encrypt the message with the stream cipher Salsa20. The ephemeral message key is then encrypted for every receiving device (i.e., it is encrypted with every session key the sending device shares with a receiving device). All ciphertexts are collectively sent to the server in a single message, and the server performs a server-side fan-out, meaning that it delivers the encrypted message and the encrypted ephemeral key to each receiving device.

A similar mechanism is used for group messaging: The group creator sends a secret key to all participating devices using normal sessions. The secret key is

34 Figure 9.8 is simplified here. To be complete and technically correct, another KDF (implemented with HMAC-SHA256) would have to be inserted. This KDF would take the temporary key and a constant as input, and would output a new root key and a session key. Due to the cryptographic properties of a KDF, the use of constants is sound here.

then used to encrypt (and decrypt) a group message, and it is ratcheted forward using HMAC-SHA256 after every message sent. Again, this is similar to SCIMP and implements a symmetric key or hash ratchet. Each group message contains a sequence number that refers to the number of times the ratchet has been forwarded so far. This allows messages to be delivered out of order, but recompiled in the correct order. This means that the encryption used for group messaging in Viber is forward secure, but it does not provide PCS.

If Viber is used to encrypt (audio or video) calls, then the procedures for a call setup and encryption are simpler. In this case, each device participating in the call generates an ephemeral public key pair and signs the public key with the device's private identity key. The two public keys with their respective signatures are exchanged between the two devices during the call setup phase. Each device then verif es the signature of its peer, performs the ECDH computation, and derives a session key. This key is valid only for the duration of the call, and it only resides in volatile memory. Finally, the RTP stream of the call is converted to SRTP and encrypted with Salsa20 using the session key.

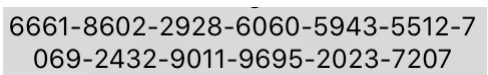


Figure 9.9 An exemplary Viber security number.

In Viber, key verif cation and trust establishment is done in the context of an audio or video call. In such a call, each user can have his or her device display a numerical string (that represents a security number similar to the one used in Signal) and compare it to the one displayed on the other party's device. The string is computed as follows: Both devices perform a Diff e-Hellman computation with their own private identity key and the other party's public identity key used in the call setup phase. The respective output is hashed with SHA-256 and truncated to 160 bits. The resulting 160 bits are then converted to a string of 48 decimal digits (i.e., 0–9) that are grouped into 12 blocks with 4 digits each. An exemplary Viber security number is illustrated in Figure 9.9. This is the string that needs to be compared by the users to mutually authenticate themselves. If the same identity keys are used, then the respective normal sessions used for messaging are also authenticated this way.

In summary, Viber provides support for E2EE messaging, especially when it comes to audio or video calls. The respective protocol is conceptually similar to Signal, but it has a few simplif cations and subtle differences whose security

implications are not fully understood. This and the fact that the software is closed source suggest that the provider of the Viber messenger—the Rakuten company and its Rakuten Viber subsidiary—must be trusted to some degree. Given the information available today, it is questionable whether this level of trust is in fact justif ed. So from a security perspective, Viber is certainly not the top E2EE messenger one may use today. But it still represents a reasonable alternative.

9.4.2 Wire

The development of Wire started in 2012 by some developers who had previously worked for Skype and Microsoft. The f rst version of the messenger was released by Wire Swiss GmbH[35] in 2014. At this point in time, Wire did not yet offer E2EE and was unencrypted. But similar to the developers of many other messengers at this time, the developers of Wire were pressed to add E2EE in a newer version. They did so by incorporating a protocol and open source implementation named Proteus[36] that is an early implementation of the Axolotl protocol based on a cryptographic library called libsodium.[37] Libsodium, in turn, is a fork of a library called NaCl (pronounced salt) that was originally developed by Daniel J. Bernstein, Tanja Lange, and Peter Schwabe.[38] Wire is available on all major platforms (i.e., iOS, Android, Windows, MacOS, and Linux) and it also runs on Web browsers. Again, the documentation is rather short [23], and everything said must be taken with a grain of salt.

Like Signal and Viber, Wire uses state of the art cryptography: X25519 for key agreement, ChaCha20 for encryption (remember that Viber uses Salsa20 here), SHA-256 for hashing, HMAC-SHA256 for message authentication, and HKDF for key derivation. If the user has a password, then he or she can authenticate himself or herself with it. The server does not store the password in the clear, but uses the *scrypt* (pronounced "ess crpyt") password-based KDF [24] to store it in encrypted form [24].[39]

Like Proteus and Signal, the Wire messenger is open source,[40] but—unlike Signal and WhatsApp—the use of Wire requires an account that is liable to pay costs. A user can register with a phone number or e-mail address (remember that Signal and WhatsApp always require a phone number). In either case, he or she

35 https://wire.com.
36 https://github.com/wireapp/proteus.
37 https://libsodium.gitbook.io.
38 https://nacl.cr.yp.to.
39 Scrypt ist a *memory-hard* function, meaning that it requires a lot of memory to execute. This makes it particularly diff cult to execute it on many processors that work in parallel. The use of such functions is sometimes recommended to reduce the dependency on dedicated hardware in applications like Bitcoin mining.
40 https://github.com/wireapp.

receives a verif cation code[41] that needs to be entered manually and returned to the server accordingly. The user has 3 attempts to do so, before the code is automatically invalidated and a new code needs to be requested. Unlike Signal and WhatsApp, Wire does not read the code received in an SMS message automatically, and hence the Wire app need not be conf gured to have access to the SMS inbox. This is certainly an advantage from a security perspective.

Upon successful registration, the user is assigned a Wire *unique user ID* (UUID) and receives an authentication token that is sent as an HTTP cookie in every request. The token is actually a string that is digitally signed[42] by the Wire server. It includes the UUID and the expiration time as a Unix timestamp, and it can be persistent or session-based (depending on the user's preferences). Like Viber, Wire supports multiple devices per user. In the current version, however, the upper bound for the number of devices a user may have is eight.

The use of identity keys and prekeys is similar to Viber (remember that prekeys are used only once, and hence represent one-time prekeys, and that there are no signed prekeys). But as mentioned above, Wire has a fallback mechanism in place for the case that the server runs out of prekeys for a particular user. One prekey[43] is distinct and refers to a *last resort prekey*. This prekey is never removed from the server, meaning that is can be reused again and again, until new prekeys are uploaded to the server.

To send an encrypted message, the sending device must establish a unique session to every receiving device. All E2EE messages are then compiled in a batch that is sent to the server. The server checks the batch and makes sure that there is an encrypted message for every device that is to receive the message. Technically speaking, a client-side fan-out is performed, while the server double-checks the result. So the server is in the position of knowing who is communicating with whom and what devices are used for this purpose. This is quite a lot of metadata the server has access to.

For very large f les—called *assets* in Wire parlance—Wire uses hybrid encryption: The sending device randomly selects a key k and encrypts the asset with this key (using AES-256 in CBC mode and PKCS #7 padding). It also computes a SHA-256 hash value of the encrypted asset, and encrypts k, the hash value, and some metadata related to the asset for each receiving device. The encrypted asset and the receiving device-specif c ciphertexts are then sent to the server for distribution. Each receiving device gets the encrypted asset and its respective ciphertext. It can decrypt the ciphertext and extract k and the hash value. Also, it can recompute the SHA-256

41 If the user registers with a phone number, then the verif cation code is 6 decimal digits long. If he or she registers with an e-mail address, then it is 192 bits long.

42 The signature system that uses Curve25519 is called Ed25519.

43 It is the prekey with ID 65535.

hash value from the ciphertext, and decrypt the encrypted asset with k if and only if the resulting hash value matches the received value.

The multiple-device feature of Wire requires that the sender of a message verifies the authenticity of all receiving devices. In Wire, this is done by having the sender verify the respective fingerprints. Wire displays all devices that are registered for the recipient; a full blue shield indicates a verified device, whereas a half blue shield indicates a device that has not yet been verified. To verify a not-yet-verified device, the sender can either call the recipient over the phone or meet the device holder to manually verify the fingerprint. In either case, this is neither simple nor straightforward, and it is certainly something that does not scale well. Again, alternative approaches to perform an authentication ceremony—especially in a multiple-device setting—are needed here.

In summary, Wire provides support for E2EE messaging using a simplified version of the Signal protocol with some extensions related to the prevention of prekey exhaustion and support for multiple devices per user. In contrast to Viber, the Wire implementation is open source, meaning that anybody can verify it. While this argument is certainly true in theory, it may not be true in practice, because not many people have really looked into the source code of Proteus and Wire—at least not in any documented form. To improve this situation (and the lack of public scrutiny), Wire Swiss GmbH has published the results of some security audits that have looked into implementation issues.[44]

9.4.3 Riot

Olm[45] is an open source implementation of the Signal protocol written entirely from scratch in C++. It is used, for example, in the *Matrix*[46] open source project that is to publish open standards for secure and decentralized communication, as well as respective implementations. One of the flagship applications of the Matrix project is a messenger called *Riot.im*, or *Riot* in short.[47] A fork of Riot is also used, for example, in the *Tchap* messenger launched by the French government as an official alternative to WhatsApp or any other E2EE messenger for internal use.[48]

In contrast to many other messengers, Riot does not require a mobile phone number for user registration, and can be used with an e-mail address only. Also, due to the architecture of Matrix, Olm is device-centric (instead of user-centric), meaning

44 https://wire.com/en/security/#audits/.
45 https://gitlab.matrix.org/matrix-org/olm.
46 https://matrix.org.
47 https://about.riot.im.
48 https://www.tchap.gouv.fr.

that sessions are established between devices. The cryptographic algorithms employed by Olm are standard: X25519 for key agreement, AES-256 in CBC mode with PKCS #7 padding for message encryption, SHA-256 for hashing, HMAC with SHA-256 for message authentication, and HKDF with SHA-256 for key derivation.

Like Viber, Olm implements the 3DH protocol (instead of the full X3DH protocol) and only uses identity keys and (one-time) prekeys. But unlike Viber, a prekey is just a single public key pair here. Remember that a prekey in Viber refers to two public key pairs (i.e., a handshake key pair and a ratchet key pair). This distinction is not made in Olm. Instead, the prekey is just a handshake key pair, and a ratchet key pair is not used. In Olm, A computes a master secret s as

$$s \quad = \quad \text{ECDH}(sk_A^{\text{ID}}, pk_b) \parallel \text{ECDH}(sk_a, pk_B^{\text{ID}}) \parallel \text{ECDH}(sk_a, pk_b)$$

and B computes the same value as

$$s \quad = \quad \text{ECDH}(pk_A^{\text{ID}}, sk_b) \parallel \text{ECDH}(pk_a, sk_B^{\text{ID}}) \parallel \text{ECDH}(pk_a, sk_b)$$

This value can be used by A and B to generate an initial 256-bit root key k_0^{root} and an initial 256-bit chain key $k_{0,0}^{chain}$: HKDF-SHA256(0, s,"OLM_ROOT",64) returns 64 pesudorandom bytes, from which the left 32 bytes refer to k_0^{root} and the right 32 bytes refer to $k_{0,0}^{chain}$.

As with OTR and the Signal protocol, Olm's Diff e-Hellman ratchet can advance whenever one of the parties provides a new ephemeral public key (that is sent together with a message). Whenever this happens, a new root key k_i^{root} and a new chain key $k_{i,0}^{chain}$ can be generated using the HKDF-SHA256 construction: HKDF-SHA256(k_{i-1}^{root},ECDH(A,B),"OLM_RATECHET",64) returns 64 bytes, from which the left 32 bytes refer to k_i^{root} and the right 32 bytes refer to $k_{i,0}^{chain}$. As usual, ECDH(A,B) refers to an elliptic curve Diff e-Hellman key exchange with the latest available Diff e-Hellman parameters provided by A and B.

Having generated the chain key $k_{i,0}^{chain}$ this way, the HMAC-SHA256 construction can be used iteratively to advance it and create a new message key: Starting with $k_{i,j-1}^{chain}$ for $j > 0$, the next chain key $k_{i,j}^{chain}$ is computed as the HMAC-SHA256 value of byte 0x02 with key $k_{i,j-1}^{chain}$:

$$k_{i,j}^{chain} = \text{HAMC-SHA256}(k_{i,j-1}^{chain}, \text{0x02})$$

Similarly, from the current chain key $k_{i,j}^{chain}$ a new message key $k_{i,j}$ can be created as follows:

$$k_{i,j} = \text{HAMC-SHA256}(k_{i,j}^{chain}, \text{0x01})$$

As a side remark, we note that this mechanism of ratcheting forward a chain key and deriving message keys from it is almost identical to the one used in WhatsApp (Chapter 10). The only difference is that WhatsApp applies the two operations in reverse order: It first computes the message key from the chain key, before it updates the chain key. The constant bytes 0x01 and 0x02 are the same in either case.

As mentioned above, Olm (at least in version 1) uses AES-256 in CBC mode with PKCS #7 padding for message encryption and HMAC with SHA-256 for message authentication. This means that a 32-byte AES key, a 16-byte IV (for AES in CBC mode) and another 32-byte key for the HMAC construction are needed. This sums up to 80 bytes. Again, these bytes can be generated from $k_{i,j}$ using the HKDF construction: HKDF-SHA256(0,$k_{i,j}$,"OLM_KEYS",80) returns 80 bytes, from which the first 32 bytes yield the AES key, the next 32 bytes yield the HMAC key, and the remaining 16 bytes yield the IV. Equipped with this keying material, the message can be properly encrypted and authenticated.

From its very beginning, the Matrix project in general and the Riot messenger in particular have been designed with a focus on group messaging. Riot thus provides chat rooms to which users—or rather devices—can join at will and communicate in E2EE form. The chat rooms may be very large, so the protocol for group messaging must scale. Olm therefore comes along with a group ratcheting mechanism called Megolm.[49] The basic idea of Megolm is simple and straightforward: It uses Olm to establish group state on a peer-to-peer basis, and it then uses this state to encrypt and authenticate the messages sent to the group. The group state, in turn, consists of a symmetric key—called *ratchet*—to encrypt messages, and an Ed25519 public key pair to digitally sign them. The sending device encrypts the message with some keying material derived from the ratchet (using the HKDF construction), and it digitally signs the resulting ciphertext with the private Ed25519 key. So far, everything looks fine and standard. But there are also some details of Megolm that deviate from the standards and are pretty unique. For example, the function used to update the ratchet on a per-message basis is highly involved and not intuitive. We don't repeat it here. Instead, we note that Megolm yields a symmetric key or hash ratchet that provides forward secrecy, but it does not provide PCS (there is no Diffie-Hellman ratchet in place to periodically feed in new keying material). This fact was criticized in a 2016 review report published by the NCC Group, [50] but the critique does not seem to be fair.[51]

49 https://gitlab.matrix.org/matrix-org/olm/blob/master/docs/megolm.md.

50 https://www.nccgroup.trust/us/our-research/matrix-olm-cryptographic-review.

51 According to a talk given by Matthew Hodgson at FOSDEM 2017, the Matrix developers had been fully aware of this weakness and intentionally designed the Megolm group ratchet mechanism to trade off PCS against efficiency and usability. A voice or video recording of the talk is available at https://archive.fosdem.org/2017/schedule/event/encrypting_matrix.

In summary, Riot is a viable alternative to Signal, Viber, and Wire. It implements the Signal protocol—including the 3DH protocol—in a fairly strict sense, and the resulting software is open source and can be freely used. This makes Riot particularly popular among open source and free software advocates, but is not widely used in the commercial world.

9.5 FINAL REMARKS

In this chapter, we have outlined and discussed in detail the Signal messenger and—even more importantly—the protocol it employs (that is also used in many other E2EE messengers, including WhatsApp, Viber, Wire, and Riot). The major advantage of Signal is its sophisticated key update mechanism that provides forward secrecy and PCS. Furthermore, it is open source and has received a lot of public scrutiny. This applies to both the protocol and its implementation in the Signal messenger. Consequently, the Signal messenger has a very good reputation in the security community, meaning that it is believed to be secure and does not comprise of any trapdoors. The major disadvantage (at least from a usability viewpoint) is that a Signal account is bound to a particular phone number, and hence a Signal user can be registered to only one device at a time. This, in turn, means that if a user employs the same phone number on another device, then this automatically deactivates the first one. It goes without saying that there is a good reason for doing so, namely to strengthen the security and to keep the private keys on one device only (any mechanism that supports the replication of a private key, such as the one employed by Viber, introduces new vulnerabilities that may be exploited in one way or another). Another point that is sometimes criticized when it comes to using Signal in a business environment is that it is not possible to install Signal on a device without granting access to many of the user's items, such as the calender, location, photos, and contacts (for obvious reasons, access to the phone is required to make phone calls, and access to SMS is required for registration). This is unfortunate, because people sometimes want to avoid revealing private information to some external party—even one that is assumed to be trustworthy.

Signal comes along with a few privacy features that try to minimize the information revealed to the server. The features are transparent to the user, meaning that they are activated by default and automatically invoked (i.e., without user interaction).

Encrypted profile: It is obvious that the server needs to know some information about a user profile, such as his or her phone number that also serves as identifier. This information must be available in the clear, and cannot be encrypted. But there is some complementary information about the user

prof le, such as his or her display name or picture, that can be encrypted in a way that is opaque to the server, meaning that it cannot be decrypted and accessed by the server. More specif cally, this information is encrypted with a key that the user shares with other users whom he or she is willing to trust. The server does not need to know the prof le key, and hence it can neither decrypt nor reveal the respective information to some untrusted party. This clearly improves the privacy with regard to the server.

Private contact discovery: Contact discovery is about f nding out whether other users are employing the same messenger. The standard approach to implement contact discovery is to compute a (truncated) hash value for all phone numbers found in the local contacts on a phone and to transmit these hash values to the server. The server then tries to match them against the hash values of all registered users, and indicates for which users a match is found (by returning a unique user identif er, such as a name, e-mail address, or phone number). This approach is simple and straightforward, but it also requires the server to behave honestly and not log all requests made by a particular user (to afterwards construct a social graph). Signal tries to enforce such honest server behavior by exploiting some features supported by modern microprocessors, such as software guard extensions (SGX) and remote attestation in the case of Intel or TrustZone in the case of ARM. Because the exploitation of these features is tricky, Signal also employs some sophisticated cryptographic techniques, such as oblivious RAM (ORAM).[52] The overall goal is to enforce honest server behavior when it comes to contact discovery, and Signal goes probably as far as one can go here.

Sealed sender: We know from postal mail that no information about the sender is required to route a message to its intended recipient. This also applies to Internet messaging, and hence Signal tries to hide this information in a feature called sealed sender.[53] To achieve this, Signal employs short-lived sender certif cates and delivery tokens that are not further addressed here. The sender certif cates are issued by the server, whereas the delivery tokens are usually part of the encrypted prof le (and hence protected with the prof le key). This means that everybody who has access to the prof le key can also decrypt the token and use it to send messages to the respective user. This is done by default, meaning that the sender of such a message doesn't have to do anything to invoke the feature. Note that the effectiveness of the feature is discussed controversially in the community, mainly because the identity of the sender can also be found out by analyzing IP addresses.

52 https://signal.org/blog/private-contact-discovery.
53 https://signal.org/blog/sealed-sender.

Also due to these (advanced) privacy features, Signal has become a target of choice for censorship in some countries, meaning that these countries try to ban the use of the Signal messenger in their territories. To still enable the use of Signal in these areas, a technology called *domain fronting* [25] has been implemented and is supported by Signal. Domain fronting is a masquerading technique that is used to circumvent Internet censorship by making traff c look like its destined to a server in a domain that is not restricted. Usually, domain fronting relies on a content delivery network (CDN) that hosts multiple domains, such as Akamai, Amazon, Microsoft Azure, or CloudFlare. A TLS connection is established to such a CDN, and the TLS server name indication (SNI) extension then causes the connection to be forwarded to the origin server, which is a Signal server or a proxy for it in this case. In the Signal messenger, the use of domain fronting can be activated in the extended settings whenever a connection to a Signal server cannot be established. Otherwise, it is greyed out and cannot be activated in the f rst place (in most situations this is the default setting).

We end the chapter by noting that the Signal protocol (as outlined and explained in this chapter) represents the state of the art in E2EE messaging on the Internet. This means that any new project is likely to implement this protocol or a variant thereof. Only if nonrepudiation is required and deniability is not wanted by default may another protocol, such as OpenPGP or S/MIME, be better suited. This is seldom the case today and will be even more so in the future.

The specif cation of the Signal protocol is quite comprehensive, and there are only a few details and subtleties in which implementations can differentiate themselves, such as the cryptographic algorithms, authentication ceremony, multi-device support, and group messaging. Any organization that wants to implement the Signal protocol and provide a respective (Signal-based) E2EE messenger must specify and nail down the details, and the result has implications not only on security but also on usability. In fact, there is an increasingly large body of research that addresses the usability of various options available. We already mentioned the usability issue in the realm of OpenPGP, and we will revisit the topic in Chapter 13. With regard to the Signal messenger, a usability study was published in 2016 [26]. The results are somewhat disillusioning, at least when it comes to peer authentication and how seriously authentication ceremonies actually take place. In the next chapter, we look at yet another implementation of the Signal protocol, namely the one provided by one of the most widely used E2EE messengers in the f eld.

References

[1] Perrin, T. (Ed.), "The XEdDSA and VXEdDSA Signature Schemes," Revision 1, October 20, 2016.

[2] Marlinspike, M., and T. Perrin (Eds.), "The X3DH Key Agreement Protocol," Revision 1, November 4, 2016.

[3] Perrin, T., and M. Marlinspike (Eds.), "The Double Ratchet Algorithm," Revision 1, November 20, 2016.

[4] Marlinspike, M., and T. Perrin (Eds.), "The Sesame Algorithm: Session Management for Asynchronous Message Encryption," Revision 2, April 14, 2017.

[5] Valin, JM., Vos, K., and T. Terriberry, "Def nition of the Opus Audio Codec," RFC 6716, September 2012.

[6] Valin, JM., and K. Vos, "Updates to the Opus Audio Codec," RFC 8251, October 2017.

[7] Langley, A., Hamburg, M., and S. Turner, "Elliptic Curves for Security," RFC 7748, January 2016.

[8] Krawczyk, H., Bellare, M., and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, February 1997.

[9] Krawczyk, H., and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, May 2010.

[10] Harkins, D., "Synthetic Initialization Vector (SIV) Authentic Encryption Using the Advanced Encryption Standard (AES)," RFC 5297, October 2008.

[11] Chase, M., Perrin, T., and G. Zaverucha, "The Signal Private Group System and Anonymous Credentials Supporting Eff cient Verif able Encryption," Cryptology ePrint Archive: Report 2019/1416, 2019.

[12] Chase, M., Meiklejohn, S., and G. Zaverucha, , "Algebraic MACs and Keyed-Verif cation Anonymous Credentials," *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (ACM CCS 2014)*, ACM Press, 2014, pp. 1205–1216.

[13] Frosch, T., et al., "How Secure is TextSecure?" *Proceedings of the IEEE European Symposium on Security and Privacy*, 2016, pp. 457–472.

[14] Diff e, W., van Oorschot, P.C., and M.J. Wiener, "Authentication and Authenticated Key Exchanges," *Designs, Codes and Cryptography*, Volume 2, Issue 2, 1992, pp. 107–125.

[15] Cohn-Gordon, K., et al., "A Formal Security Analysis of the Signal Messaging Protocol," *Proceedings of the 2nd IEEE European Symposium on Security and Privacy (Euro S&P 2017)*, 2017, pp. 451–466.

[16] Kobeissi, N., Bhargavan, K., and B. Blanchet, "Automated Verif cation for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach," *Proceedings of the 2nd IEEE European Symposium on Security and Privacy (Euro S&P 2017)*, 2017, pp. 435–450.

[17] Poettering, B., and P. Rösler, "Towards Bidirectional Ratcheted Key Exchange," *Proceedings of CRYPTO 2018*, Springer, LNCS 10991, 2018, pp. 3–32.

[18] Alwen, J., Coretti, S., and Y. Dodis, "The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol," *Proceedings of EUROCRYPT 2019*, Springer, LNCS 11476, 2019, pp. 129–158.

[19] Jost, D., Maurer, U., and M. Mularczyk, "Eff cient Ratcheting: Almost-Optimal Guarantees for Secure Messaging," *Proceedings of EUROCRYPT 2019*, Springer, LNCS 11476, 2019, pp. 159–188.

[20] Betül Durak, F., and S. Vaudeney, "Bidirectional Asynchronous Ratcheted Key Agreement with Linear Complexity," *Proceedings of the International Workshop on Security (IWSEC 2019)*, Springer, LNCS 11689, 2019, pp. 343–362.

[21] Rösler, P., Mainka, C., and J. Schwenk, "More is Less: On the End-to-End Security of Group Chats in Signal, WhatsApp, and Threema," *Proceedings of the 3rd IEEE European Symposium on Security and Privacy (Euro S&P 2018)*, 2018, pp. 415–429.

[22] Rakuten Viber, "Viber Encryption Overview," https://www.viber.com/app/uploads/viber-encryption-overview.pdf.

[23] Wire Swiss GmbH, "Wire Security Whitepaper," August 17, 2018, https://wire-docs.wire.com/download/Wire+Security+Whitepaper.pdf.

[24] Percival, C., and S.Josefsson , "The scrypt Password-Based Key Derivation Function," RFC 7914, August 2016.

[25] Fif eld, D., et al., "Blocking-resistant Communication through Domain Fronting," *Proceedings on Privacy Enhancing Technologies*, De Gruyter Open, Volume 2015, Issue 2, pp. 46–64.

[26] Schröder, S., et al., "When SIGNAL hits the Fan: On the Usability and Security of State-of-the-Art Secure Mobile Messaging," *Proceedings of the 1st European Workshop on Usable Security (EuroUSEC 2016)*, Internet Society, 2016.

Chapter 10

WhatsApp

In this chapter, we elaborate on the way the Signal protocol is implemented and used in WhatsApp.[1] In some sense, this topic could have also been addressed in Section 9.4 as yet another implementation of the Signal protocol. But WhatsApp is so important because it is probably the most widely used E2EE messenger in the f eld (with more than two billion users worldwide) and therefore deserves a chapter of its own. Nevertheless, the chapter can still be kept short, because we have introduced and explained most ingredients of WhatsApp and the way it implements the Signal protocol in the previous chapter. We start with a few comments about the origins and history of WhatsApp in Section 10.1, address some specif c implementation details in Section 10.2, analyze the security in Section 10.3, and provide some f nal remarks in Section 10.4.

10.1 ORIGINS AND HISTORY

The history of WhatsApp is relatively short and brief y told. In 2009, Jan Koum and Brian Acton founded a small company named WhatsApp in Santa Clara, California. The company name was a play on words, and mixes up the terms what's up and app. This ref ects the company's goal, namely to provide an Internet-based messaging service and messenger app that could be used for free and was therefore a strong competitor of commercial SMS/MMS from telecom operators. From its beginning, WhatsApp was very successful and many traditional SMS/MMS customers started to use WhatsApp. Hence, WhatsApp experienced an exponential growth and was highly disruptive for the SMS/MMS market.

1 The focus of the previous two chapters has been the protocol that builds the core of OTR and Signal, whereas the focus of this chapter is a specif c E2EE messenger (i.e., WhatsApp) that implements the Signal protocol.

In 2014, Facebook acquired the startup company for 19 billion USD. At this time, WhatsApp was still a non-secure messenger app, but it was around the time when the Snowden revelations made press headlines, and WhatsApp started to lose market share to other E2EE messengers like Threema. By the end of 2014, WhatsApp therefore announced that they would support E2EE messaging for all users in future releases of the app. As already mentioned in Section 9.1, WhatsApp teamed up with Open Whisper Systems to implement the Signal protocol in WhatsApp and to support E2EE messaging by default. This was a risky announcement, because it was not known whether the Signal protocol would really scale to the number of messages sent and received by WhatsApp on a daily basis. Luckily, the endeavor ended successfully, and in April 2016, the implementation was complete. WhatsApp had migrated from an insecure to a fully secure E2EE messenger. This is where we stand today, and WhatsApp clearly plays in the league of state-of-the-art E2EE messengers.

10.2 IMPLEMENTATION DETAILS

The WhatsApp implementation of the Signal protocol was done in cooperation with Open Whisper Systems and is partly based on some of its open source libraries.[2] But in contrast to the Signal messenger, the WhatsApp implementation is closed source[3] and poorly documented—at least if compared to its large-scale use. Except for a short technical white paper [1] originally published in April 2016 and updated in December 2017, there is hardly any technical material available in public. The poor documentation and closed source nature of WhatsApp make it particularly diff cult to understand what is going on behind the scenes. Once again, everything said in this chapter remains unverif ed and must be taken with a grain of salt.

In the following subsections, we address transport layer security and complementary technologies, cryptographic algorithms and key generation, message attachments, and group messaging. For each of these topics, we mainly focus on the differences and specif cs of WhatsApp (as compared to the Signal protocol and the messenger).

2 https://github.com/signalapp.

3 Some people have tried to write an open source implementation of a messenger app that can interoperate with WhatsApp. For example, Mazapp was an attempt for the Nokia N9 smartphone that was done in 2012 (https://wiki.maemo.org/Wazapp). It uses a Python library called yowsup that is available on GitHub (https://github.com/tgalal/yowsup). So far, WhatsApp has always blocked the use of such third-party implementations in an attempt to force users employing WhatsApp software only.

10.2.1 Transport Layer Security and Complementary Technologies

While the Signal messenger and most other implementations of the Signal protocol employ the TLS protocol for transport layer encryption, WhatsApp is different here: It employs the *Noise protocol framework*[4] that is developed by Perrin—one of the coinventors of the Signal protocol. In contrast to WhatsApp, the Noise protocol framework and the rationale behind its design are well documented and thoroughly analyzed.[5] The general idea is to provide secure channel protocols that—similar to the X3DH protocol—are based on multiple executions of the Diffe-Hellman key exchange protocol (instead of combining it with digital signatures). Besides WhatsApp, Noise pipes are also used in many other application areas, such as virtual private networks (VPNs) based on WireGuard,[6] Lightning inter-node communication in blockchains, or the Invisible Internet Project (I2P[7]) to provide an anonymization infrastructure. WhatsApp uses Noise pipes with Curve25519, AES-GCM, and SHA-256, and hence the transport layer security provided by WhatsApp is state of the art and comparable to TLS.

In Section 9.5, we already mentioned that in a real-world setting the Signal protocol must be complemented with other technologies and protocols, such as the SRTP for voice and video calls. More specif cally, if a WhatsApp user initiates such a call, then the initiator's device A establishes a normal E2EE session to the recipient's device B (if one does not already exist), generates a random 32-byte master secret for the SRTP, and sends an E2EE message to B that signals an incoming call and contains the master secret. If B answers the call, then a fully encrypted call using the SRTP can take place. This basically means that the Signal protocol is used to establish the session state for the call, whereas the SRTP is used to actually encrypt the respective voice and video data.

Similar to Signal (and most other E2EE messengers), WhatsApp users are required to verify the public keys or the respective f ngerprints of the other users with whom they want to communicate (to mitigate MITM attacks). To achieve this, an authentication ceremony is needed, and the authentication ceremony of WhatsApp is more or less the same as the one of Signal (Section 9.2.3). This suggests that the user experience is also more or less the same, and hence that somebody familiar with the authentication ceremony of Signal is very likely also familiar with the one from WhatsApp.

4 http://www.noiseprotocol.org.
5 https://eprint.iacr.org/2019/436.pdf.
6 https://www.wireguard.com.
7 https://geti2p.net.

10.2.2 Cryptographic Algorithms and Key Generation

In accordance with the Signal protocol, WhatsApp uses state of the art cryptographic algorithms: X25519 (i.e., ECDH over Curve25519) for key agreement, AES-256 in CBC mode for message encryption,[8] SHA-256 for hashing, HMAC-SHA256 for message authentication, and HKDF for key derivation.

WhatsApp implements the X3DH protocol to initialize the root chain and the double ratchet mechanism to generate and update the root, chain, and message keys. Root and chain keys are each 32 bytes long, whereas message keys are 80 bytes long. More specif cally, a message key consists of the following components:

- A 32-byte AES-256 key used for message encryption;
- A 32-byte HMAC-SHA256 key used for message authentication;
- A 16-byte IV for CBC mode.

Remember that the sending and receiving chains are type II KDF chains, meaning that a new message key is generated from a chain key whenever the respective chain is ratcheted forward. As already mentioned in Section 9.4.3 (in the context of the Olm implementation used in Riot), the respective KDF is implemented with the HMAC-SHA256 construction and operates in the following two steps:

- First, the message key is computed as the HMAC-SHA256 value of the chain key and the constant byte 0x01;
- Second, the chain key is updated as the HMAC-SHA256 value of the chain key and the constant byte 0x02.

Remember that the same KDF is used in Olm, but that the two steps are executed in the reverse order—at least according to the Olm specif cation that is available online.[9] It is not clear whether the order matters or whether one order provides better security properties than the other. Consequently, we assume that the order does not matter.

10.2.3 Message Attachments

Another topic that is somewhat opaque to an E2EE messaging protocol but still needs to be addressed is the handling and transmission of large message attachments (e.g., audio, image, or video f les). If these attachments were transmitted in-band,

8 Note that—due to the use of Noise pipes—WhatsApp employs AES-GCM for message encryption at the transport layer.

9 https://gitlab.matrix.org/matrix-org/olm/blob/master/docs/olm.md.

then the respective transmission channels would have to be broadband. This may not be the case, and hence people are looking for possibilities to transmit the attachments out-of-band in some cryptographically protected form. For this purpose, WhatsApp uses a blob store that resides in the cloud. This means that the sending device uploads the attachment and the receiving device downloads it, but it always resides in a cryptographically protected form outside the two devices.

Let A be the sending device of a WhatsApp message with such an attachment, and B be the respective receiving device. A randomly selects two 32-byte secret keys: k_e for message encryption and k_a for authentication. A encrypts the attachment with k_e using AES-256 in CBC mode with a random IV, and appends a MAC of the ciphertext using k_a and the HMAC-SHA256 construction. Note that this refers to the encrypt-then-authenticate construction, and that this way of combining encryption and authentication is assumed to be secure. A uploads the now encrypted and authenticated attachment to the blob store, and transmits a normal E2EE message that contains k_e, k_a, a SHA-256 hash value of the encrypted blob, and a pointer to the blob (in the blob store) to B. B, in turn, can decrypt the message, retrieve the encrypted blob from the blob store, verify the SHA-256 hash value, verify the MAC with k_a, and decrypt the attachment with k_e. In the end, B has the required attachment in decrypted form, but the attachment has never resided outside A and B in unprotected form. This solves the problem of large attachments in a simple and straightforward way.[10]

10.2.4 Group Messaging

One of the major differences between Signal and WhatsApp refers to the way group messaging and respective chats are managed. Contrary to Signal, WhatsApp implements administered groups, meaning that only some specif c members of a group are authorized to administer it and to manage group memberships accordingly. They are the administrators of the group. The user who creates a group is always an administrator, but he or she can nominate arbitrarily many other users to become administrators, as well. It goes without saying that these nominations can also be revoked at some later point in time. In the current implementation, WhatsApp limits the maximum number of users in a group to 256.

In Section 9.2.4, we mentioned that Signal uses a client-side fan-out mechanism for group messaging, whereas WhatsApp uses a variant of the Signal protocol to implement some form of a server-side fan-out. This variant is known as *Sender*

10 The Facebook Messenger uses the same solution in the secret conversations mode. But instead of using AES-256 (for message encryption) and HMAC-SHA256 (for message authentication) separately, it uses AES-GCM as an AEAD scheme.

Keys, and it is conceptually similar to hybrid encryption. It basically works as follows:

- For the first time a group member A sends a message to a group, it randomly generates a 32-byte AES-256 chain key and an Ed25519 signature key pair, combines the chain key and the public signature key into a Sender Key message, and individually E2E-encrypts this message to each member of the group (using the normal pairwise E2EE messaging protocol). In the end, each group member has the Sender Key message and can extract the chain key (that is later used to derive message keys) and the public signature key (that is later used to verify signatures). This is done for each member of the group individually.
- For all subsequent messages A wants to send to the group, it derives a message key from the chain key and updates the chain key accordingly, encrypts the message with the now derived message key using AES-256 in CBC mode, uses the private signature key to digitally sign the ciphertext, and transmits the digitally signed ciphertext to the server. The server, in turn, performs the server-side fan-out for all group members, meaning that is sends the (same) digitally signed ciphertext to all members of the group. Each member can then use A's public signature key to verify the signature, the current chain key to derive the message key, and the message key to decrypt the message.

Each message key is derived from the latest chain key in a type II KDF chain. Note that his provides forward secrecy, but it does not provide PCS. Also note that the Sender Keys variant of the Signal protocol used in WhatsApp requires digital signatures, and that this moves away from the original intent of OTR and its use of MACs instead of digital signatures.

The WhatsApp server-side fan-out mechanism (or Sender Keys variant) is illustrated in Figure 10.1 in some simplified form (the figure is best compared to Figure 9.6). The figure does not show the Sender Key messages that are initially sent to the group members. $E_{SK}(m)$ refers to message m encrypted with SK (where SK stands for the message key that is derived from the chain key sent in the "Sender Key" message), whereas σ_A refers to A's signature. Note that $(E_{SK}(m), \sigma_A)$ is the same (encrypted and digitally signed) message distributed by the server S to all group members B, C, and D. Due to the use of Noise pipes, the encrypted messages sent from A to S, and then from S to B, S to C, and S to D are all different from each other.

The WhatsApp server-side fan-out mechanism has several advantages and disadvantages. The most important advantage is related to its efficiency and the number of messages that need to be transmitted between the sending device and

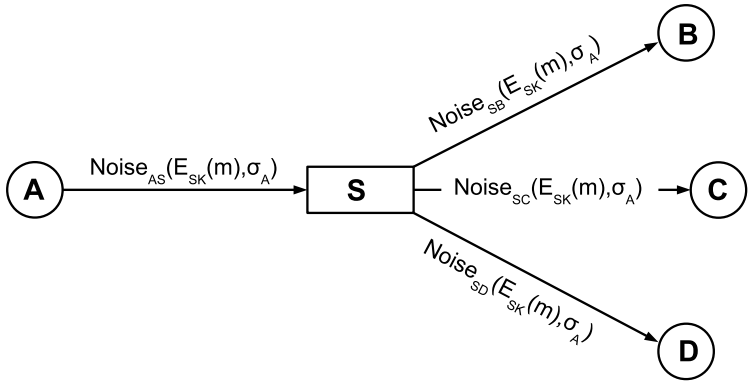


Figure 10.1 The WhatsApp server-side fan-out mechanism (or Sender Keys variant).

the server. The n messages that are required for a client-side fan-out (with n group members) reduce to just one in WhatsApp. Depending on n and the size of the message, this eff ciency gain is substantial. On the other hand, the most important disadvantage is related to the additional state[11] that is required and valid only as long as the group memberships don't change. More specif cally, if a new device joins the group, then each group member can provide the new device with the keying material that is needed to participate. In this case, there is no need to start from scratch. If, however, a device leaves the group, then all group members have to clear their state (i.e., chain keys and public signature keys) and start from scratch. Consequently, the way WhatsApp handles group messaging is eff cient as long as group leave operations do not occur too frequently.

Last but not least, we note that WhatsApp statuses and live location messages are also encrypted as group messages, but that a new and fast ratcheting algorithm is used for live location messages. This algorithm is described in [1]; it is beyond the scope of this book and not repeated here.

11 This state refers to the chain keys and the respective type II KDF chain, as well as the public signature keys that must be stored for all group members.

10.3 SECURITY ANALYSIS

First of all, it is important to note that the security analyses that have been done for the Signal protocol also apply to WhatsApp, and that WhatsApp is therefore assumed to be secure—at least from a cryptographic viewpoint. There are a few complementary analyses that have focused on some specif c aspects of WhatsApp (e.g., [2]), but these analyses have not found something particularly worrisome. In 2016, WhatsApp had a score of 6 out of 7 points on the Electronic Frontier Foundation's Secure Messaging Scorecard, where it only missed one point because the code was not open to independent review. These positive assessments, however, do not exclude the fact that vulnerabilities and implementation bugs may be found and exploited in some meaningful way. A prominent example that has attracted a lot of press attention is FakesApp, a vulnerability found by Check Point Research in August 2018 that may have allowed an adversary to fake valid-looking WhatsApp messages.[12] Another bug found in 2019 allowed a remotely operating adversary to install spyware on a WhatsApp installation by just making a call (that did not even have to be answered). More generally, any list of common vulnerabilities and exposures (CVE), such as the one maintained by MITRE,[13] can be searched through with the term WhatsApp to reveal vulnerabilities and implementation bugs—some of them may have security implications. In this regard, WhatsApp is not different from any other piece of software, and it always needs to be patched in a timely fashion (i.e., as soon as a patch is made available by WhatsApp).

With regard to group messaging, we already mentioned in Section 9.3 that the situation is more involved in a multi-party setting, and that a group of researchers has found subtle vulnerabilities and shortcomings in the way some E2EE messengers—including WhatsApp—handle groups and manage group memberships [3]. In the case of WhatsApp, all groups are administered, meaning that each group has at least one administrator who is in charge of managing group memberships. When such an administrator wishes to add a new member to the group, he or she sends a respective (group management) message to the server identifying the group and the member to add. The server then checks that the user is in fact an administrator, meaning that he or she is authorized to administer the group, and—in the positive case—sends a message to every member of the group indicating that they should add that particular user. These messages are sent by the server and are not digitally signed by the group administrator. This, in turn, suggests that a malicious server can also generate them, and hence that the server—or rather the party that operates the server—can add any user of its choice to any group. This clearly defeats the original purpose of E2EE messaging, namely to keep all messages private and accessible to only authorized

12 https://research.checkpoint.com/2018/fakesapp-a-vulnerability-in-whatsapp.
13 https://cve.mitre.org.

users. All members of a group get a notif cation message about the addition of a new member, but it is currently unknown how effective these messages really are in practice. Under certain circumstances, such messages tend to be overlooked.

An obvious possibility to mitigate this attack is to make sure that all (group management) messages are digitally signed by an administrator in charge. At f rst sight, this seems to solve the problem, but it is not so simple. Note that the WhatsApp server determines who the administrators are, so if the server wants to misbehave, then it can still introduce a fake administrator and have him or her sign the messages. But faking an administrator for a group is clearly a more intrusive attack than simply send out messages, so this countermeasure certainly helps mitigating the attack to some extent.

10.4 FINAL REMARKS

In this chapter, we have elaborated on the way the Signal protocol is implemented and used in WhatsApp. Due to the fact that similar people (i.e., software developers from Open Whisper Systems) have done the implementation, many (implementation) details are also similar. This is true, for example, for the way a user can establish a WhatsApp account and register a device,[14] as well as for the authentication ceremony used to make sure that a communicating peer is the one it claims to be. But there are some implementation details that are unique and specif c for WhatsApp, such as the use of Noise pipes (instead of TLS sessions), the Sender Keys variant to implement a server-side fan-out (instead of using a client-side fan-out), and the fact that groups are administered. These details do not negatively impact the security assessment, and hence WhatsApp can still be considered to be a good choice when it comes to E2EE messaging on the Internet. The major argument against the use of WhatsApp is related to fact that it belongs to Facebook and that one may try to minimize the data provided to this company. Assuming, however, that WhatsApp and its end-to-end encryption work as specif ed, the amount of data that is provided to Facebook is small. The company can learn social graphs and derive metadata about the communication behavior of its users, but it cannot derive information about the content of messages. This has been the goal of E2EE messaging in the f rst place, and WhatsApp seems to provide it. Deriving metadata is a privacy topic that is brief y addressed in Chapter 12. It is important but E2EE messaging is not primarily designed to protect against it.

14 In contrast to Signal, WhatsApp uses its own infrastructure to handle the SMS verif cation process (remember that Signal uses services powered by Twilio). If WhatsApp has access to the user's SMS inbox, then it can automatically enter the verif cation code sent to it. Otherwise, the user must enter the code manually.

References

[1] WhatsApp, "WhatsApp Encryption Overview," Technical white paper, December 19, 2017 (originally published April 5, 2016).

[2] Schrittwieser, S., et al., "Guess Who's Texting You? Evaluating the Security of Smartphone Messaging Applications," *Proceedings of the 19th Annual Symposium on Network and Distributed System Security (NDSS 2012)*, Internet Society, 2012.

[3] Rösler, P., Mainka, C., and J. Schwenk, "More is Less: On the End-to-End Security of Group Chats in Signal, WhatsApp, and Threema," *Proceedings of the 3rd IEEE European Symposium on Security and Privacy (Euro S&P 2018)*, 2018, pp. 415–429.

Chapter 11

Other E2EE Messengers

In this chapter, we overview and discuss a few other, i.e., non-Signal-based, E2EE messengers that are used in the f eld. In chronological order, this includes iMessage (2011) addressed in Section 11.1, Wickr (2012) in Section 11.2, Threema (2012) in Section 11.3, and Telegram (2013) in Section 11.4. In addition, there are many other E2EE messengers available and in use today, such as Hoccer,[1] SIMSme,[2] Dust,[3] Cyphr,[4] CoverMe,[5] Silence,[6] Surespot,[7] Pryvate,[8] Crypho,[9] SafeSlinger,[10] Line[11] with its letter sealing feature, KaKaoTalk[12] with its E2EE chatting option, and many more. All of these (E2EE) messengers are not further addressed in this book. If you are interested in their working principles and actual use, then you may refer to the many sources of information that are available online. But one has to be cautious here, because a particular E2EE messenger may also turn out to be a trap. In 2019, for example, it was revealed[13] that the widely used Emirati messaging app ToTok[14] is actually a spy tool that allows the government to supervise its citizens. Such a strategy is particularly successful if other E2EE messengers are banned in a country. The risk is pervasive: If the provider of a particular E2EE messaging app wants to

1 https://hoccer.com.
2 https://www.sims.me.
3 https://usedust.com.
4 https://www.goldenfrog.com/cyphr.
5 https://www.coverme.ws.
6 https://silence.im.
7 https://www.surespot.me.
8 https://www.pryvatenow.com.
9 https://www.crypho.com.
10 https://github.com/safeslingerproject.
11 https://line.me.
12 https://www.kakaocorp.com/service/KakaoTalk.
13 https://www.nytimes.com/2019/12/22/us/politics/totok-app-uae.html.
14 https://totok.ai.

cheat, then there are usually plenty of possibilities to do so, and it is therefore very important that the app provider is trustworthy and not subject to particular political interests. It is also very important that the users can choose between multiple apps and providers.

Contrary to most other chapters of this book, this chapter does not conclude with f nal remarks. This is because the respective remarks are compiled at the end of each section individually. This also means that the sections do not depend on each other, and can stand by themselves.

11.1 IMESSAGE

In 2011, Apple launched *iMessage* as a proprietary E2EE messaging service for Apple devices, such as iPhones, iPads, and other iOS devices, Mac computers, and Apple watches. According to [1] and contrary to most other messaging services discussed in this book, iMessage currently supports only asynchronous text messaging (with attachments, such photos, contacts, and locations), and therefore competes with traditional short and multimedia messaging services. There is no support for synchronous voice or video calling. This is because there are other Apple services like FaceTime that serve this purpose. In this outline, we only address iMessage and leave FaceTime aside.[15]

The iMessage service is based and deeply intertwined with the Apple push notif cation (APN) and directory (IDS) services. This suggests that an iMessage user is uniquely identif ed with an Apple ID,[16] and that all devices registered for a particular Apple ID are synchronized by default. Consequently, multidevice support is inherent to iMessage.

When a user turns on iMessage on a particular device, it generates the following two public key pairs (that are specif c for this device):

- A 1280-bit RSA key pair used for (hybrid) message encryption;
- A 256-bit ECDSA key pair from the NIST curve P-256 [4] used for message signing.

15 FaceTime uses standard technologies for real-time communications, such as the session initiation protocol (SIP), session traversal utilities for NAT (STUN), and SRTP, as well as standard cryptographic techniques for E2EE, such as AES-256 in CTR mode and HMAC-SHA1. FaceTime also supports group communications with up to 33 concurrent participants and some sophisticated cryptographic techniques, such as AES-SIV [2] and a Diff e-Hellman-based hybrid encryption scheme called elliptic curve integrated encryption scheme (ECIES) specif ed in [3] (where it is named ECIES-KEM).

16 Note that one or several phone numbers and e-mail addresses can be associated with an Apple ID. In either case, a phone number or e-mail address is verif ed using standard techniques.

While the private keys are stored in the device's Keychain that is protected by the operating system, the public keys are registered with the IDS and assigned to the device owner's Apple ID—together with the user's phone number and e-mail address, as well as the device's APN address. This is done for each device individually. In the end, an Apple ID is associated with a set of devices with distinct APN addresses and unique (RSA and ECDSA) public key pairs.

A user can start a new iMessage conversation by specifying a recipient. If he or she enters a phone number or e-mail address, then the device contacts the IDS to retrieve the APN addresses and public keys of all (receiving) devices associated with the addressee. Otherwise (i.e., if he or she enters a name) then the device f rst retrieves the phone numbers and e-mail addresses associated with that name from the user's Contacts app, and then gets the respective APN addresses and public keys from the IDS. In either case, the sending device is provided with a list of APN addresses and public keys for all devices associated with the intended recipient.

Let D be the sending device and $D_1, \ldots, D_n$ the $n > 0$ receiving devices for a particular message m. This message m is then individually encrypted for each of the n receiving devices (for which the APN addresses and public keys have been retrieved from the IDS as described above). For each receiving device D_i ($1 \leq i \leq n$), the sending device D randomly generates an 88-bit key k_i and uses it to construct a 40-bit value M_i that basically represents a MAC for the public keys and the message. The HMAC construction uses the SHA-256 hash function and is computed as follows:

$$M_i = \text{HMAC-SHA256}_{k_i}(pk_D \parallel pk_{D_i} \parallel m)|_{40} \tag{11.1}$$

In this notation, pk_D and pk_{D_i} refer to the sending and receiving devices' RSA public keys, and the output of the HMAC-SHA256 construction (that is 256 bits long) is truncated to 40 bits.

The concatenation of k_i and M_i (i.e., $k_i \parallel M_i$) sums up to 128 bits, and this value is used as a key to encrypt the message m with the AES in CTR mode. Hence, the f rst part of the ciphertext for the receiving device D_i, denoted as $c_i^{(1)}$, is constructed as follows:

$$c_i^{(1)} = \text{AES-CTR}_{k_i \parallel M_i}(m)$$

To allow D_i to decrypt the ciphertext after receiption, the encryption key $k_i \parallel M_i$ must also be encpyted with D_i's RSA public key pk_{D_i}. Here, iMessage uses a more secure version of RSA encryption (i.e., RSA-OAEP[17]) and hence the second part of

17 In contrast to normal RSA encryption, RSA-OAEP is known to be semantically secure.

the ciphertext for D_i, denoted as $c_i^{(2)}$, is constructed as follows:

$$c_i^{(2)} = \text{RSA-OAEP}_{pk_{D_i}}(k_i \parallel M_i)$$

The ciphertext c_i then consists of the two components, where the first component refers to the encrypted message and the second component refers to the encrypted message key:

$$c_i = (c_i^{(1)}, c_i^{(2)})$$

Furthermore, it is digitally signed by D using SHA-1 and its ECDSA private key. The resulting signature is denoted as σ_i. Using a forward secret TLS channel, D dispatches the pair (c_i, σ_i) to the APN service for delivery, and the service sends a respective APN to D_i. This is repeated for all receiving devices. Once a pair (c_i, σ_i) is delivered to D_i, it is deleted from the APN service. Unlike other APNs, iMessage messages are queued for delivery to offline devices (up to 30 days).

When D_i receives the APN, it captures the sending device's ECDSA public from the IDS service, verifies σ_i with it, RSA-OAEP-decrypts $c_i^{(2)}$ with its RSA private key sk_{D_i} to retrieve $k_i \parallel M_i$, verifies M_i according to equation (11.1), and—if the verification is successful—decrypts the message with the key $k_i \parallel M_i$ (using AES in CTR mode). This decryption must be done by each of the n receiving devices individually.

The APN service can handle messages only up to a maximal length (which is currently 4KB or 16KB, depending on the iOS version in use). It is unlikely that a text message is longer than this value. But if a message comprises an attachment, such as a photo, then the total message length may exceed the maximal length. In this case, the message is not delivered as an APN. Instead, it is AES-encrypted with a randomly chosen key k and stored in the cloud—typically the iCloud. The APN that is then sent to the recipient is still E2EE, but it includes an URI to the message stored in the cloud, a SHA-1 hash value of the encrypted message, and the key k required to decrypt it. It goes without saying that the receiving device can then decrypt the URI, retrieve the encrypted message from the cloud (using this URI), verify its SHA-1 hash value, and decrypt it with the key k.

In spite of its widespread use, iMessage has not experienced a lot of public scrutiny so far. Only in 2016 has a group of researchers published a paper in which they describe a chosen-ciphertext attack (CCA) against iMessage [5]. The attack exploits the facts that an adversary can replace the signature on a message with his or her own signature, and that AES-CTR encryption is malleable. This allows the adversary to craft arbitrary chosen ciphertexts and send them to the recipient. For each ciphertext, the recipient leaks one bit of information, namely whether the

underlying plaintext message is well formatted or not. This means that the attack is actually something like a format oracle attack. The attack is not very eff cient, since it requires about $2^{18} = 262,144$ chosen ciphertexts to compute a key. In an interactive setting, this is clearly infeasible. But in a noninteractive setting, the attack may represent a problem. To mitigate the attack, one may try to detect duplicate RSA signatures, employ certif cate pinning, or change the message format. The long-term solution is to use authenticated encryption, such as provided by AES in GCM mode instead of CTR mode.

In summary, we can say that iMessage has been a pioneer in the sense that it is the f rst messaging service used on a large scale that has built-in end-to-end encryption. Its design is simple and straightforward, and it has a lot in common with the conventional approaches and solutions for secure and E2EE messaging in an asynchronous setting, such as OpenPGP and S/MIME. It uses AES-128 in CTR mode and RSA-OAEP for digital envelopes and ECDSA for signatures, and is therefore state of the art. The only true novelty in iMessage is its inherent support for multiple devices per user (or Apple ID, respectively). But iMessage does not provide forward secrecy and PCS or deniability, and this can be seen as a major disadvantage in some application settings. Consequently, it is possible (and maybe even likely) that Apple will not only merge iMessage and FaceTime one day, but also move to Signal or a Signal-based E2EE messaging protocol in the future.

11.2 WICKR

Wickr[18] is a U.S.-based software company founded in 2012 to market a messaging app with the same name. The app is available for Apple iOS and Android, and the respective desktop version is available for Windows, Mac OS, and Ubuntu. The messaging app for personal use is called *Wickr Me*.[19] But Wickr also provides an enterprise solution called *Wickr Enterprise*,[20] a team collaboration solution called *Wickr Pro*,[21] and an integration gateway called *Wickr IO*,[22] as well as specif c solution for federal agencies, politics, and crisis communications. The core of all solutions and the focus of this outline is the Wickr messaging protocol that is specif ed in [6]. It is implemented in C and the respective source code is available on GitHub.[23]

18 https://wickr.com.
19 https://wickr.com/products/personal.
20 https://wickr.com/products/enterprise.
21 https://wickr.com/products/teams.
22 https://wickr.com/wickrio-integrations.
23 https://github.com/WickrInc/wickr-crypto-c.

According to [6], the Wickr messaging protocol is complemented by two additional layers of security: "First, app-server requests and responses are encrypted with a rotating shared secret key using AES 256 in CFB mode. Second, the Wickr app tunnels this AES encrypted data inside of TLS." While the second point can be translated into the statement that the Wickr messaging protocol is layered on top of TLS, the first point remains vague and is not further nailed down in the specification (it is, for example, unclear what rotating shared secret key means and how the rotation is achieved in the first place). In the following outline, we ignore these additional layers of security and only focus on the core functionalities of the Wickr messaging protocol. The cryptography used in Wickr is state of the art: ECC with the NIST curve P-521 [4] for key exchange (ECDHE) and digital signatures (ECDSA), AES-256 in GCM mode for authenticated encryption, and HKDF with SHA-256 or scrypt for key derivation.

Similar to iMessage, Wickr supports multiple devices[24] per user, meaning that a user may have one or several devices that operate in sync. Hence, the Wickr messaging protocol is rather a device-to-device protocol than it is a user-to-user protocol. If, for example, a user with two devices wants to send a message to another user with four devices, then the user's sending device has to transmit the encrypted message to five devices—the four devices associated with the other user and the sending user's own second device.

The Wickr messaging protocol requires user A to enroll himself or herself on one of his or her devices (using the Wickr app). During this enrollment process, the app generates an identifier for A, denoted as ID_A, that is called A's *root identifier* and is a SHA-256 hash value of some identification string. The app also randomly generates a *root identity public key pair* $(pk_A^{\text{ID}}, sk_A^{\text{ID}})$ that consists of a root identity public key pk_A^{ID} and a root identity private key sk_A^{ID}, as well as a bundle of three distinct secret keys $(k_A^{rsr}, k_A^{nsr}, k_A^{rb})$:

- The key k_A^{rsr} refers to A's *remote storage root key* and is used to remotely store account-level backup data on Wickr servers in encrypted form;
- The key k_A^{nsr} refers to A's *node storage root key* and is used to locally store data in encrypted form;
- The key k_A^{rb} refers to A's *recovery bundle key* and is used to remotely store data needed for recovery (e.g., identity keys) on Wickr servers in encrypted form.

24 Note that the Wickr documentation sometimes uses the term *node* to refer to a device. In fact, the terms device and node are used synonymously and interchangeably. In this book, however, we consistently use the term device.

The fact that k_A^{rsr} and k_A^{nsr} refer to root keys suggests that the actual encryption keys are derived from them. This is contrast to k_A^{rb} that stands for itself.

ID_A and pk_A^{ID} are public in nature, meaning that they are stored on Wickr servers and provided to communication peers—along with other prof le data—whenever needed. In contrast, sk_A^{ID}, k_A^{rsr}, and k_A^{nsr} are private and must be protected accordingly, but they still need to be shared among all devices employed by A. The respective triple $(sk_A^{\text{ID}}, k_A^{rsr}, k_A^{nsr})$ is called *recovery bundle*, and it is stored on Wickr servers in encrypted form. More specif cally, the recovery bundle is encrypted with AES-256 in GCM mode, using A's recovery bundle key k_A^{rb}. The key k_A^{rb}, in turn, is encrypted with a key that is the scrypt value[25] of A's passphrase, and the respective ciphertext is stored together with the recovery bundle. Note that this allows A to easily transfer his or her keying material to another device using his or her passphrase. Also note that this introduces the risk of somebody knowing A's passphrase is able to compromise the account. To mitigate this risk, it is possible to conf gure the Wickr app in a way that does not automatically store the recovery bundle key together with the recovery bundle key. Instead, the user has to store the key in a secure location. Similarly, the recovery bundle can also be maintained entirely off ine and provided only in situations that require access to sk_A^{ID}, k_A^{rsr}, or k_A^{nsr}. Either possibility is optional and—realistically speaking—not many users are going to make use of them.

Each time A wants to employ a new device D_i with $i \geq 1$, this device needs to be enrolled, as well. The respective device enrollment process is also handled by the Wickr app and requires—among other things—the recovery bundle $(sk_A^{\text{ID}}, k_A^{rsr}, k_A^{nsr})$ to be locally installed on D_i. If A enters the correct passphrase, then the key to decrypt k_A^{rb} can be compiled, and this key can then be used to decrypt the recovery bundle and install sk_A^{ID}, k_A^{rsr}, and k_A^{nsr} locally. Furthermore, the Wickr app randomly generates the following items that are unique for D_i:

- A *device identif er* ID_{D_i};
- A *device identity public key pair* $(pk_{D_i}^{\text{ID}}, sk_{D_i}^{\text{ID}})$;
- A secret *local storage device key* $k_{D_i}^{lsd}$ that is derived from k_A^{nsr} and some device-specif c data $data$ using a KDF (that implements the HKDF construction):

$$k_{D_i}^{lsd} = \text{KDF}(k_A^{nsr} \parallel data)$$

To make sure that it is A who is using D_i, the device identity public key $pk_{D_i}^{\text{ID}}$ is digitally signed with A's root identity private key sk_A^{ID}, denoted as $\text{Sign}(sk_A^{\text{ID}}, pk_{D_i}^{\text{ID}})$,

25 The crypt value is the output of the scrypt KDF function that takes A's passphrase as input.

and the respective device identity private key $sk^{\text{ID}}_{D_i}$ is used to sign messages sent out by D_i (on A's behalf). Both $pk^{\text{ID}}_{D_i}$ and $\mathsf{Sign}(sk^{\text{ID}}_{A}, pk^{\text{ID}}_{D_i})$ are stored in A's prof le, whereas $sk^{\text{ID}}_{D_i}$ is stored on D_i only. This is where $k^{lsd}_{D_i}$ comes into play: It is used by the Wickr app to create an encrypted container on D_i to store sensitive data, such as identity keys (including $sk^{\text{ID}}_{D_i}$), messages, and some other data. The container is transparently decrypted during an active session and its contents is used for normal operation. As soon as the user logs off, the container is reencrypted with the local storage device key that is then removed from persistent memory. The key is stored in encrypted form, so that it can be recovered the next time the user logs on. Similar to k^{rb}_{A}, the key used for encryption is derived from the user's passphrase with the scrypt KDF. So whenever the user enters his or her passphrase, the Wickr app can decrypt the container and transparently use it.

Similar to most E2EE messengers in use today, Wickr is based on the (elliptic curve) Diff e-Hellman key exchange protocol with identity keys that are long-lived and ephemeral keys that are short-lived. More specif cally, each device D_i has a device identity public key pair $(pk^{\text{ID}}_{D_i}, sk^{\text{ID}}_{D_i})$ as mentioned above and several ephemeral keys pairs. Let pk_{d_i} be a particular ephemeral public key and sk_{d_i} be the respective private key.[26] Similar to Signal's signed prekeys, each ephemeral public key pk_{d_i} is digitally signed with the device's identity private key $sk^{\text{ID}}_{D_i}$, denoted as $\mathsf{Sign}(sk^{\text{ID}}_{D_i}, pk_{d_i})$. The Wickr app running on D_i uploads several such digitally signed ephemeral public keys to the Wickr servers, and locally stores the respective private keys in a secure way (i.e., in the container mentioned above). The goal is to make sure that whenever somebody wants to send a message to this device, an ephemeral public key to perform the (elliptic curve) Diff e-Hellman key exchange is available in a dynamically sized pool on the Wickr servers and can be retrieved from there. If the pool gets exhausted, then the last key is reused until the user ref lls the pool. This is conceptually similar to the last resort prekey mechanism provided by Wire (Section 9.4.2).

If A wants to use D_i to send a message m to $n \geq 1$ other users $\mathrm{B}_1, \ldots, \mathrm{B}_n$, then the Wickr app (on D_i) retrieves the receiving users' prof le data, including each user B_i's root identity public key $pk^{\text{ID}}_{B_i}$, a list of $m \geq 0$ devices $D_1, \ldots, D_m$ registered for B_i, as well as the device identity public keys $pk^{\text{ID}}_{D_j}$ in digitally signed form (i.e., $(pk^{\text{ID}}_{D_j}, \mathsf{Sign}(sk^{\text{ID}}_{B_i}, pk^{\text{ID}}_{D_j}))$) for each such device D_j ($0 \leq j \leq m$). The app builds a list of receiving devices from the union of all devices registered for $\mathrm{B}_1, \ldots, \mathrm{B}_n$ and A. For each of these devices, the app retrieves an ephemeral public key with signature and identif er from the Wickr servers, and verif es the signature for each key individually. If X is such a device, then pk_x refers to one of

26 To keep the notation as simple as possible, we don't use an index to refer to a distinct ephemeral public key pair here.

X's ephemeral public keys and sk_X^{ID} refers to X's device identity private key. This means that the signature is $\mathsf{Sign}(sk_X^{\text{ID}}, pk_x)$, and hence that it can be verif ed by f rst checking the validity of $\mathsf{Sign}(sk_X^{\text{ID}}, pk_x)$ with X's device identity public key pk_X^{ID}, and then checking the validity of $\mathsf{Sign}(sk_Y^{\text{ID}}, pk_X^{\text{ID}})$ with Y's root identity public key pk_Y^{ID}—where Y is A or any of $\text{B}_1, \ldots, \text{B}_n$. The bottom line is that the app now has an ephemeral public key pk_x at hand for every receiving device X. This is the starting point for message encryption.

To encrypt m, the Wickr app on D_i randomly selects a message payload encryption key k_P and derives a packet header encryption key k_H from A's root and sending device identif ers (i.e., $k_H = \text{KDF}(ID_A \parallel ID_{D_i})$). Note that k_H can be derived by anybody who knows ID_A and ID_{D_i} (i.e., it does not depend on a secret). This is not particularly secure, but is not a problem here, because the key is going to be used only to encrypt data that is already encrypted (as explained below).

In addition to k_P and k_H that are independent from the receiving devices, message encryption requires another key k_X, called *exchange key*, that is shared with each receiving device X (so there is a distinct exchange key for every receiving device). This is where the ECDH key exchange protocol comes into play: The Wickr app randomly generates an ephemeral public key pair (pk_{d_i}, sk_{d_i}) for D_i, and combines its ephemeral private key sk_{d_i} with X's ephemeral public key pk_x. The resulting ECDH value is concatenated with pk_A^{ID}, pk_X^{ID}, and ID_X to derive k_X. This means that the derivation of k_X can be formally expressed as follows:

$$k_X = \text{KDF}(\text{ECDH}(sk_{d_i}, pk_x) \parallel pk_A^{\text{ID}} \parallel pk_X^{\text{ID}} \parallel ID_X)$$

Using this exchange key k_X, the message payload encryption key k_P is encrypted for X and compiled into key exchange data (KED) for X, denoted as KED_X, together with ID_X and an identif er for pk_{d_i}:

$$\text{KED}_X = E_{k_X}(k_P) \parallel ID_X \parallel ID_{pk_{d_i}}$$

The KED for all receiving devices are concatenated into a key exchange list (KEL). Finally, a packet header is created by f rst concatenating pk_{d_i} and KEL, and then encrypting the result with k_H. The result is the encrypted packet header (EPH) and it is generated as follows:

$$\text{EPH} = E_{k_H}(pk_{d_i} \parallel \text{KEL})$$

We mentioned earlier that—due to the way it is generated—k_H is not a particularly strong encryption key (because it can be derived by anybody who knows ID_A and ID_{D_i}). But it does not have to be strong, because it is used to encrypt data that is

already encrypted. This refers to the fact that KEL is a list of KED, and each KED is encrypted with a device-specif c exchange key.

In addition to the EPH, the Wickr app uses k_P to encrypt both the message metadata and the message payload (both referring to m in our simplif ed outline). The result is called encrypted message payload (EP) and it is generated as follows:

$$\text{EP} = E_{k_P}(m)$$

Finally, the app uses the device identity private key $sk_{D_i}^{\text{ID}}$ to digitally sign the concatenation of EPH and EP, and hence to create a packet signature (PS). Finally, a serialized packet is created by concatenating the version, some parameters referring to the cryptographic conf guration, EP, EPH, and PS. The resulting packet is sent to the Wickr servers from where it is dispatched to all receiving devices. Packaged with these deliveries are the identif ers for both the sending device (i.e., ID_{D_i}) and the sending user (i.e., ID_A), together with some other information. We omit the details here.

If the Wickr app on device X receives the delivery from the Wickr servers, it deserializes the packet and extracts the version, cryptographic conf guration, EP, EPH, and PS. It then uses D_i's identity public key $pk_{D_i}^{\text{ID}}$ to verify PS, and pk_A^{ID} to verify $\mathsf{Sign}(sk_A^{\text{ID}}, pk_{D_i}^{\text{ID}})$. The app then recalculates the packet header encryption key $k_H = \text{KDF}(ID_A \parallel ID_{D_i})$, and uses it to decrypt the EPH:

$$\text{EPH} = D_{k_H}(pk_{d_i} \parallel \text{KEL})$$

This allows the app to retrieve the appropriate pk_{d_i} and the KEL, from where it can extract its KED. Remember that X's KED (i.e., KED_X) is equal to $E_{k_X}(k_P) \parallel ID_X \parallel ID_{pk_{d_i}}$. This means that the app can use $ID_{pk_{d_i}}$ to identify pk_{d_i} and combine this value with its own ephemeral private key sk_x in an ECDH key exchange (i.e., $\text{ECDH}(pk_{d_i}, sk_x)$). The result is concatenated with pk_A^{ID}, pk_X^{ID}, and ID_X to recover k_X, and this key can then be used to decrypt $E_{k_X}(k_P)$, and hence to recover the message payload encryption key k_P. Finally, the app can use this key to decrypt m referring to both the message metadata and message payload. All short-lived keys are deleted, and the message payload is encrypted with X's local storage device key k_X^{lsd} to store it locally in encrypted form. Furthermore, the app carries out actions in accordance with the message metadata, such as deleting it after its time to live has expired.

While the Wickr messaging protocol is not based on the Signal protocol, it still has some similarities. For example, it also uses the ECDH key exchange protocol to generate new encryption keys. But instead of using a ratcheting mechanism to update the keys regularly and systematically, it invokes the ECDH key exchange protocol for every message and every receiving device. This is clearly less eff cient than the

use of Signal's double ratchet or any other ratcheting mechanism. Also similar to Signal, Wickr tries to hide metadata by encrypting the packet headers (this, by the way, is a feature that is optional in Signal and that we have not addressed in Chapter 9).

The big advantage of Wickr is its support of multiple device and users by default. The basic Wickr messaging protocol easily extends itself to the multi-device and group communication setting. In fact, there are two types of groups supported by Wickr: Managed groups that are called *rooms* and unmanaged groups that are called *conversations*. In a room, only administrators are authorized to change group memberships, whereas all users are authorized to do so in a conversation. So rooms refer to administered groups, whereas conversations refer to unadministered groups. As of this writing, it is not clear how Wickr scales to large rooms and conversations compared to other—typically Signal-based—E2EE messengers. The lack of ratcheting is certainly a disadvantage with regard to scalability.

11.3 THREEMA

Also in 2012, Manuel Kasper from Kasper Systems GmbH released a proprietary and closed source[27] E2EE messenger app named *Threema*.[28] The Threema servers are operated in Switzerland by a company called Threema GmbH.[29] The app is available for Android and iOS,[30] supports many languages, and can be downloaded from the respective app stores. Since version 2.1, the app also provides a *poll feature* that allows users to easily perform a poll with a predef ned set of choices. This feature is interesting and somewhat unique for Threema. Furthermore, there is a Web client called *Threema Web*,[31] a corporate version called *Threema Work*, and a gateway solution called *Threema Gateway*. The gateway solution can be used to integrate Threema messaging with other forms of communication. This outline focuses on the Threema app only, and does not address Threema Web, Work, or Gateway. The basis for the outline is a Cryptography Whitepaper [7] that is well written, comprehensive, and available online.[32]

27 While Threema is based on open source components, the resulting software is closed source. There are some attempts to develop open source implementations that can interoperate with Threema though, such as openMittsu (https://github.com/blizzard4591/openMittsu) that is a cross-platform open source implementation and desktop client for Threema.

28 The name is derived from the acronym EEEMA, standing for end-to-end-encrypting messaging application.

29 https://threema.ch.

30 There has also been a Windows Phone version of the app. Because the Windows Phone is not further developed, this version is a dead end and not further addressed here.

31 https://threema.ch/en/blog/posts/threema-web-whitepaper.

32 https://threema.ch/press-f les/2_documentation/cryptography_whitepaper.pdf.

While most E2EE messengers in use today employ a mobile phone number or e-mail address to uniquely identify a user, Threema doesn't employ any of these possibilities. Instead, it uses a randomly chosen *Threema ID* that consists of eight uppercase letters (i.e., A–Z) and decimal digits (i.e., 0–9), such as `2AZ5CEJ6`, and is completely independent from the mobile phone number or e-mail address. Hence, Threema can be used in a way that is totally anonymous.

When a user installs and uses the Threema app for the first time, the app generates a new Curve25519 public key pair, securely stores the private key on the device, and sends the respective public key to the Threema server. The Threema server stores the key in its repository, and assigns a Threema ID to the app (note that a user may have multiple Threema apps running on different devices). The Threema ID is returned to the app, where it is added to the private key. It goes without saying that the private key must be securely stored on the device, and that the security of the storage depends on the operating system in use. It also goes without saying that a user may revoke his or her Threema ID by visiting a Web site[33] and entering his or her Threema ID and revocation password (that he or she must have set beforehand).[34] Revoked IDs can no longer be used to log in, and messages cannot be sent to revoked IDs.

The Threema IDs and public keys of all users are stored in a repository on the server side. Optionally, a user can register a mobile phone number and/or e-mail address to his or her account. In either case, Threema—or the Threema directory server, respectively—verifies the mobile phone number or e-mail address by either sending an SMS message with a random 6-digit code that the user must enter and return back to the server,[35] or sending a verification e-mail message with a hyperlink that the user must open and confirm in a Web browser. In either case, the registration of a mobile phone number or e-mail address allows the user to be easily found by other users.

A user can obtain the public key for a Threema ID by querying the server using any of the following input values:

33 https://myid.threema.ch/revoke.

34 Revocation passwords are hashed with SHA-256 by the Threema app, but only the first 32 bits (4 bytes) are sent to the server and stored there. The rationale behind this truncation is that 4 bytes are sufficient to reliably authenticate the user, but not enough to determine the actual password. This mitigates the situation that an adversary may have captured the password file and now tries to mount an exhaustive password search. He or she does not have enough information to determine whether he or she has found the correct password for a given user (because there are hundreds of words that hash to the same 4 bytes). This mitigates the risk that a user may have reused his or her revocation password for other applications, as well. Without truncation, the adversary would have found a password that is also valid for other applications, and this would pose a security risk.

35 If the user cannot receive the SMS message, then he or she may choose to receive an automated phone call in which the code is provided by voice.

- A full (i.e., 8-character) Threema ID;
- A hash value of the mobile phone number[36] that may be associated with a Threema ID;
- A hash value of the e-mail address that may be associated with a Threema ID.

To avoid collisions with hash values used by other applications, Threema uses the HMAC-SHA256 construction with a distinct key for mobile phone numbers and another distinct key for e-mail addresses—both keys are not secret and provided in [7]. In addition to a mobile phone number and e-mail address, a user can also assign some complementary information to his or her Threema ID, such as a nickname or a prof le picture. This information, however, is not stored on the server side. Instead, it may be sent along with the encrypted message. In the case of a prof le picture, for example, the user can choose within the app whether and with whom he or she may want to share it. If he or she chooses to share it and afterwards sends a message to such a recipient, then the picture is processed as a normal image f le and sent to the recipient in encrypted form via the media server (as outlined below).

Similar to PGP's cumulative trust model or web of trust, the Threema app assigns a *verif cation level* indicator to every Threema ID and respective public key that appears locally. There are three possible verif cation levels:

- *Level 1*—indicated with one red dot: No matching contact (for the Threema ID and respective public key) is found in the user's contacts and address book (by mobile phone number or e-mail address).
- *Level 2*—indicated with two orange dots: A matching contact is found in the user's contacts and address book. Since the server routinely verif es mobile phone numbers and e-mail addresses, the user has some evidence that the other user is who he or she claims to be.
- *Level 3*—indicated with three green dots: The user has personally verif ed the Threema ID and public key by scanning the respective QR code.[37] In the positive case, the user has strong evidence that the other user is who he or she claims to be.

From a security perspective, level 3 is certainly the preferred choice. The Threema app, however, leaves it up to the user to decide whether he or she wants to go for this verif cation level. Many users will probably be satisf ed with a lower level.

36 The encoding of the phone number must be in line with ITU Recommendation E.164.

37 The QR code is displayed in the My ID section of the messenger app. It comprises the Threema ID and the (hexadecimal and lowercase representation of the 32-byte) public key.

Similar to most E2EE messengers in use today, Threema combines transport layer encryption—using the TLS protocol[38]—and message layer encryption—using a proprietary and custom-made protocol that is used to have the Threema app communicate with the *chat server*.

In addition to the chat server, Threema also operates a *directory server* that hosts the repository (as mentioned above), and a *media server* that hosts the media files (as discussed below). The Threema app uses TLS and HTTPS to securely communicate with the directory and media servers. Most requests to the directory server are anonymous and do not require user authentication. Only requests that require authentication (e.g., linking mobile phone numbers or e-mail addresses to a Threema IDs) invoke a challenge/response protocol that is based on the user's private key.

Similar to Wire (Section 9.4.2), the custom protocol to implement message layer encryption employs the cryptographic library *NaCl*. For public key cryptography, NaCl is based on ECC and supports Curve25519—in addition to another curve standardized by the U.S. NIST.[39] For secret key cryptography, it supports the block cipher AES and the stream ciphers Salsa20 and XSalsa20.[40] The cryptographic primitive needed by Threema is authenticated encryption, and NaCl provides two forms:

- Public-key authenticated encryption, using the `crypto_box` primitive;
- Secret-key authenticated encryption, using the `crypto_secretbox` primitive.

Public-key authenticated encryption means that public key cryptography is used to establish a secret key, and this key is then used for authenticated encryption. This is the cryptographic primitive mostly used in Threema, where the public key cryptography part is provided by an ECDH key exchange. Whenever the Threema app requires random values, it invokes the random generator or PRG provided by the mobile device's operating system, such as Yarrow (with SHA-1) for iOS or the Linux PRNG [8] for Android. To initially generate the (long-term) public key pairs, the user may be asked to provide some additional entropy by moving a finger on the screen.

Let A be a first user's Threema app, and B be a second user's Threema app. Let us further assume that A wants to send a message to B, and A and B have both generated their public key pairs and sent their public key to the repository (or

38 TLS version 1.2 with forward secure cipher suites (i.e., using ECDHE/DHE for key exchange) are supported, and certificate pinning is used for the servers' public keys to mitigate MITM attacks. This applies to the Threema directory and media servers.

39 It is worth mentioning that Curve25519 is not one of the curves recommended by the U.S. NIST.

40 XSalsa20 is based on Salsa20, but uses a 192-bit nonce (instead of 64 bits).

directory server, respectively). Also, A has obtained B's public key in authenticated form. The procedure to encrypt a message is as simple as it can possibly be: A performs an ECDH key exchange with its private key and B's public key. The result is hashed with HSalsa20[41] to derive a shared secret. A generates a random nonce, and then uses the XSalsa20 stream cipher with the shared secret as the key and the nonce to encrypt the plaintext message (with PKCS #7 padding to mitigate traff c analysis). Also, a portion of the key stream generated by XSalsa20 is used to form a MAC key, and this key is used in Poly1305 to compute a 128-bit MAC. Finally, A sends the MAC, the ciphertext, and the nonce (in the clear) to B. Again, note that this transmission is additionally secured using the TLS protocol. By reversing all steps and using A's public key and its own private key in the ECDH key exchange, B can decrypt the ciphertext and verify the MAC accordingly.

If A wants to send a large media f le to B, such as an image, video, or voice recording, then this f le is not sent directly via the chat protocol. Instead, A invokes authenticated encryption with XSalsa20 and Poly1305 with a randomly chosen 256-bit key k, and uploads the encrypted f le to the media server. The media server, in turn, assigns a unique ID for this upload, and returns it to A. A then sends an E2EE message to B that contains the ID and k. B can use the ID to retrieve the encrypted f le from the media server, and k to decrypt and authenticate it. Upon successful download, the media server can delete the f le to make sure that there is enough memory space for other f les.

When it comes to group messaging, Threema works similar to Signal[42] in the sense that the servers are unaware of groups, meaning that they do not know what groups exist and what users are members of what groups. Instead, when a user sends a message to a group, it is E2EE and sent to each member of the group individually (the upper limit for the number of group members is 50, so this is manageable). If the message includes large media f les, then the procedure mentioned above applies: The f les are encrypted with a randomly chosen secret key and uploaded only once, whereas the key—together with a reference to the uploaded f les—is distributed in E2EE form to all members of the group. With regard to the encryption of large media f les, Threema is thus similar to the Sender Keys variant of the Signal protocol (that is used in WhatsApp).

Another distinct feature of Threema allows a user to back up his or her private key, so that he or she can move the Threema ID to another device or restore it in case of emergency. The respective Threema ID and private key backup algorithm is summarized in Algorithm 13.1. It takes as input the user's private key sk, and it generates as output a backup string s that consists of 80 characters from the Base32

41 HSalsa20 is a hash function that is internally used in the Salsa20 stream cipher.

42 While a Signal group ID consists of 128 pseudorandomly chosen bits, a Threema group ID consists of the administrator's Threema ID and only 64 pseudorandomly chosen bits.

Algorithm 13.1 Threema ID and private key backup.

(sk)

choose password pw
$hash \leftarrow$ 2 most signif cant bytes of SHA-256(Threema ID $\|$ sk)
salt $\xleftarrow{r}$ $\{0,1\}^{64}$
$k \leftarrow$ PBKDF2(HMAC-SHA256, $pw, salt, 100000, 32$)
$c \leftarrow$ XSalsa20$_k$(Threema ID $\|$ sk $\|$ $hash$) with all-zero nonce
$s \leftarrow$ Base32($salt \| c$)

(s)

character set. First, the user has to choose a password that is at least 8 characters long. The algorithm then computes a hash value $hash$ that refers to the two most signif cant bytes of the SHA-256 hash of the user's Threema ID concatenated with the private key sk. The value $hash$ is used only during the restoration of the private key to verify with reasonable conf dence that the user-provided password is correct. The algorithm then randomly selects a 64-bit string $salt$ that is used—together with pw—to derive a key k (to later encrypt the private key). The key is derived with the *password-based key derivation function 2* (PBKDF2) version 2.1 that is specif ed, for example, in PKCS #5 and RFC 8018 [9]. PBKDF2 can be instantiated with several PRFs. Threema uses the HMAC-SHA256 construction with pw serving as key and $salt$ serving as message. The parameter 100000 means that the PRF is iterated 100000 times, and the parameter 32 means that 32 bytes are extracted from the result. These 32 bytes represent k, and this key is used to XSalsa20-encrypt the concatenation of the Threema ID, sk, and $hash$ with an all-zero nonce. The components are 8, 32, and 2 bytes long, so the resulting ciphertext c is also 42 bytes long. Together with the 8-byte salt this sums up to 50 bytes or 400 bits. Base32 enodes 5 bits into one character. This means that the 400 bits require 80 Base32-characters that can be split into groups of four characters and separated with dashes. A respective Threema backup string and QR code (that comprises the same information) is illustrated in Figure 11.1. To use the string or QR code, the password pw is required and must be entered on request.

In addition to this Threema ID and private key backup mechanism, Threema is complemented with an optional but more comprehensive server-based backup feature called *Threema Safe*. This feature allows a user to backup and restore his or her Threema ID and related data, or to move it to another device (only knowing the Threema ID and the respective Threema Safe password). As such, the backup doesn't comprise only the Threema ID and private key, but also the user's other prof le information (e.g., nickname, prof le photo, and linked mobile phone number

```
BIAK-5IJ4-LTLY-FKAZ-
74P2-FHT5-R4K4-3TJS-
EBKI-SLRY-QTYX-XNJU-
NGR5-VDMX-3Z6O-HBKN-
5WWZ-2QG4-4BRM-OXEN
```



Figure 11.1 Threema backup string and QR code.

and e-mail address), contact list (e.g., contact Threema IDs, public keys, names, and verif cation levels), group def nitions, distribution lists, and app settings. The messages and media data, however, are not part of a Threema Safe backup. Also, by default, the backups are stored on Threema servers, but the user can employ any other custom server as a backup store. In either case, the server cannot tell which backup belongs to which Threema user by only looking at the uploaded data.

To invoke a Threema Safe, the user must choose a password pw that is again at least 8 characters long. Because the data is going to be stored on the server side, the strength of the password is more important here. The Threema app therefore warns the user, if the password he or she has chosen is on a list of frequently occurring passwords. From the user's Threema ID and password, a Threema Safe master key mk is derived with the scrypt KDF. In short, this is a password-based key derivation function specif cally designed to make it expensive to perform large-scale custom hardware attacks by requiring large amounts of memory (it is used by some cryptocurrencies as a proof of work scheme). Using a unique parametrization of scrypt, mk is generated as

$$mk = \text{scrypt}(pw, \text{Threema ID}, 65536, 8, 1, 64)$$

where pw serves as passphrase, Threema ID as salt, 65536 as CPU/memory cost parameter value, 8 as block size parameter value, 1 as parallelization parameter

value, and 64 as output byte length. This means that the output is 64 bytes long, and that it can be split into two parts of 32 bytes each. One half refers to the backup ID, and the other half refers to the backup encryption key that is used for data encryption using the `crypto_secretbox` primitive mentioned above. Once the encrypted data is uploaded to the backup server, it can be referenced with the backup ID. If Threema Safe is enabled and a password is set, then the Threema app generates a backup once per day when the app is in use.

Contrary to Signal and Signal-like E2EE messengers, Threema does not provide forward secrecy and PCS at the message layer. Instead, it provides these features at the transport layer (by the use of TLS), but—strictly speaking—this is not end-to-end. If an adversary is able to compromise the long-term private key of a user and control the Threema servers, then he or she is also able to compromise this user's communications. This is much better than, for example, OpenPGP and S/MIME (that do not provide forward secrecy and PCS at all), but it is still not as good as providing forward secrecy and PCS at the message layer, and hence on an end-to-end basis. The reasons for this design choice are summarized in [7]. Most importantly, providing forward secrecy and PCS at the message layer requires some form of ratcheting, and it is claimed that this leads to lower reliability and more potential for mistakes, and hence it negatively affects the user experience. This claim can be controversially discussed. On the other hand, it is certainly correct that the "the risk of eavesdropping on any path through the Internet between the sender and the server, or between the server and the recipient, is orders of magnitude greater than the risk of eavesdropping on the server itself" [7]. Consequently, Threema provides a reasonable trade-off that mitigates many attacks that are relevant in practice.

The bottom line is that the overall assessment of Threema is highly positive. In 2019, for example, the IT security group[43] of the German Münster University of Applied Sciences reviewed the architecture and code and of both Threema apps (Android and iOS) and the Threema Safe feature.[44] The researchers discovered no high risk or critical vulnerability, but found a few low to medium risk issues that were quickly addressed by the Threema developers. So people continue to have a good feeling about the security of Threema. This good feeling is mainly rooted in the use of the NaCl library (that has a good reputation in the community) and Threema IDs (that are anonymous, and hence the Threema app can—in contrast to most other E2EE messengers—be used anonymously).

43 The group is headed by Prof. Dr. Sebastian Schinzel.

44 https://threema.ch/press-f les/2_documentation/security_audit_report_threema_2019.pdf.

11.4 TELEGRAM

In 2013, Nikolai and Pavel Durov[45] launched the *Telegram*[46] messenger that has as an estimated user base of almost 200 million worldwide. It supports one-to-one and group messaging on multiple devices that are operated in sync. The support for very large groups that have up to 200,000 members and channels, as well as the support for multiple devices per user are in fact the claimed distinguishing features of the Telegram messenger.

Contrary to many other E2EE messengers, the end-to-end encryption feature of Telegram is not activated by default, meaning that the user has to willingly select a *secret chat* if he or she wants to invoke E2EE messaging.[47] Hence, secret chats are Telegram's notion of E2EE messaging. Instead of using proven cryptographic primitives and mechanisms in some standardized protocol, the developers of Telegram opted to create an entirely new protocol, called MTProto,[48] that is currently available in version 2.0.[49] The protocol is relatively simple and straightforward, but it is described in a way that is difficult to understand. In what follows, we try to explain its working principles in simpler terms (partly following [10]). The protocol is built on weaker cryptographic primitives and mechanisms, but in a way that known attacks do not apply—at least it is hoped so. The client implementation is open source and available for all major platforms,[50] whereas the server implementation is not and remains proprietary. Similar to many other messengers, the Telegram messenger stores data in the cloud.

Each Telegram user must have an account that is bound to a particular phone number, and he or she can then register multiple devices for that account. For each device, the user has to confirm a five-digit registration code that is sent to the phone number via SMS or Telegram. This is a standard way of confirming that a particular device really belongs to a particular user, and is similar to what most other messengers do. Each device D must then execute the device registration protocol with the Telegram service S that is summarized in Protocol 11.1. D has no input, whereas S takes as input a prime p and a generator g that determine the group in which a Diffie-Hellman key exchange is performed, as well as an RSA public

45 Nikolai and Pavel Durov are two brothers who had previously launched the social network Vk.com that is popular in Russia. According to their own statements, Pavel supports Telegram financially and ideologically, whereas Nikolai's input is more technological.

46 https://telegram.org.

47 Note that groups and channels are not end-to-end encrypted in Telegram.

48 https://core.telegram.org/mtproto.

49 MTProto version 1.0 is deprecated and is currently being phased out. This also means that SHA-1 is replaced with SHA-256, a point Telegram has been heavily criticized in the past.

50 There is even a command line interface that supports the full functionality of Telegram.

Protocol 11.1 MTProto device registration protocol.

D		**S**
–		$p, g, (pk_S, sk_S)$
$r_d \xleftarrow{r} \{0,1\}^{128}$		
	$\xrightarrow{r_d}$	
		$r_s \xleftarrow{r} \{0,1\}^{128}$ generate 64-bit integer $n = pq$ $fp \leftarrow$ SHA-1$(pk_S)|^{64}$
	$\xleftarrow{r_s, n, fp}$	
$p, q \leftarrow$ factorize n use fp to determine pk_S $r'_d \xleftarrow{r} \{0,1\}^{256}$ $m \leftarrow (r_d, r_s, r'_d, n, p, q)$		
	$\xrightarrow{E_{pk_S}(m, h(m))}$	
		$x_s \xleftarrow{r} \mathbb{Z}_p$ $y_s \leftarrow g^{x_s} \pmod p$ derive k_{temp} from r_s and r'_d
	$\xleftarrow{E_{k_{temp}}(g, p, y_s)}$	
$x_d \xleftarrow{r} \mathbb{Z}_p$ $y_d \leftarrow g^{x_d} \pmod p$ derive k_{temp} from r_s and r'_d		
	$\xrightarrow{E_{k_{temp}}(y_d, r_d, r_s, h(y_d, r_d, r_s))}$	
$k \leftarrow {y_s}^{x_d} \pmod p$		$k \leftarrow {y_d}^{x_s} \pmod p$
(k)		(k)

key pair (pk_S, sk_S) that is used to encrypt data sent from D to S. In the current implementation, both p and the RSA modulus are 2,048 bits long.

The MTProto device registration protocol starts with D randomly selecting a 128-bit nonce r_d and sending it to S. S randomly selects another 128-bit nonce r_s and a 64-bit integer n that is the product of two primes p and q, and it constructs a f ngerprint fp from its public key (by taking the 64 least signif cant bits from the SHA-1 hash value of pk_S). The values r_s, n, and fp are sent to D in unencrypted form. As a proof of work, D factorizes n and decomposes p and q accordingly. Also, it uses fp to determine the appropriate public key pk_S employed by S, and it randomly selects another 256-bit nonce r'_d that—unlike r_d and r_s—is never sent in the clear. Instead, it is used as an encryption key as explained below. D creates a payload message m that comprises the three nonces r_d, r_s, and r'_d, as well as

n, p, and q. This message, along with its hash value (i.e., $h(m)$ for hash function h) is then encrypted with RSA and the server's public key pk_S. The resulting ciphertext $E_{pk_S}(m \parallel h(m))$ is sent to S, where it can be decrypted using sk_S and verif ed in terms of integrity (by verifying the hash value). S initiates a Diff e-Hellman key exchange by using p and g, randomly selecting a 2048-bit string to form a private Diff e-Hellman value x_s from $\mathbb{Z}_p$, and computing the respective public value $y_s = g^{x_s} \pmod p$. S encrypts all values required for the Diff e-Hellman key exchange (i.e., p, g, and y_s) using AES-256 and a temporary encryption key k_{temp} that is derived from r_s and r'_d (see below), and sends the resulting ciphertext to D.[51] As a special feature, Telegram uses an encryption mode that is called Inf nite Garble Extension (IGE) that is addressed below. D also derives k_{temp} from r_s and r'_d and uses it to decrypt the ciphertext. If decryption is successful and the parameters are valid,[52] then it randomly selects another 2048-bit string to form its private Diff e-Hellman value x_d from $\mathbb{Z}_p$, computes the public value $y_d = g^{x_d} \pmod p$, and sends it—together with r_d, r_s, and a hash value $h(y_d, r_d, r_s)$ to S in encrypted form (again using AES-256 in IGE mode with k_{temp} and some proper padding). S decrypts the message and verif es the parameters. If everything is f ne, then either side can compute the Diff e-Hellman value that yields the shared secret key k. D and S can now use this key whenever they want to communicate securely with each other.

The use of the IGE mode is unique in Telegram. It was f rst mentioned in 1978 [11] as a mode of operation for DES that provides inf nite garble extension, and later named and scientif cally examined in 2000 [12]. Outside Telegram and MTProto, IGE is rarely implemented and even more rarely used (it is, for example, implemented in OpenSSL [13], but it is not known to be used in the f eld). At its core, IGE is somewhat similar to the Propagating Cipher Block Chaining (PCBC) that was originally used in Kerberos version 4. IGE has the distinct property that errors are propagated indef nitely, meaning that an error garbles all subsequent blocks. This was already recognized in [11]: "Inf nite garble extension has the features that the originator can place in the f nal block a pattern expected by the recipient. If the recipient f nds the expected pattern at the end of the message, he is assured that the entire message, regardless of length, was received precisely as originated." IGE uses two blocks of IVs, where each block consists of 128-bits. This means that the IGE IV sums up to 256 bits.

51 This outline is simplif ed and the encrypted message also comprises a server timestamp used for synchronization and a SHA-1 hash value, and it is properly padded.

52 D verif es that (i) p is a safe prime, meaning that $q = (p-1)/2$ is also prime, (ii) p is appropriately sized, meaning that $2^{2047} < p < 2^{2048}$, (iii) g is equal to 2, 3, 4, 5, 6, or 7, (iv) g generates a cyclic subgroup of prime order q, and (iv) $1 < y_s < p - 1$. If any of these checks fails, then the protocol is aborted.

Also, we have not explained so far how the temporary encryption key k_{temp} is derived from r_s and r'_d. First, a stream s is generated as follows:

$$s = \text{SHA-1}(r'_d \parallel r_s) \parallel \text{SHA-1}(r_s \parallel r'_d) \parallel \text{SHA-1}(r'_d \parallel r'_d) \parallel r'_d$$

Each output of SHA-1 is 160 bits long, so the output length of this construction is $3 \cdot 160 + 256 = 480 + 256 = 736$ bits. The first 256 bits of s form the temporary encryption key k_{temp}, whereas the second 256 bits of s form the IV that is used in IGE mode encryption. The remaining 736 - 512 = 224 bits are discarded and not used.

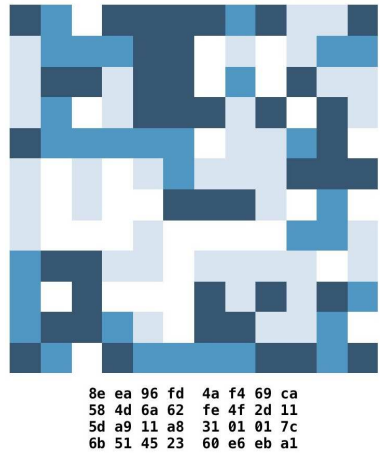


Figure 11.2 Telegram image to visualize a shared key.

If two Telegram users A and B have properly registered their devices (using the MTProto device registration protocol outlined above) and want to establish a secure chat, then they have to do a Diffie-Hellman key exchange to generate a master secret. The Diffie-Hellman parameters p and g are provided by the server, whereas the checks for these parameters are the same as the ones mentioned above. As a special feature, the server also provides some randomness that is fed into the generation process of the Diffie-Hellman values. This is to mitigate the risk that a device may have a poor random or pseudorandom generator in place. By default, the Diffie-Hellman key exchange is not authenticated, meaning that A and B must authenticate themselves afterwards. This is done by having A and B verify that they share the same key. The Telegram app therefore generates a fingerprint of the key that refers to its 128 least significant bits. These bits are visualized as an 8x8 grid,

where each cell has one of four colors. A cell therefore encodes 2 bits, and there are 8x8=64 cells in the grid. This means that each grid is able to encode 128 bits. An exemplary visualization of a key is shown in Figure 11.2. Users are intended to meet in person and compare their respective images. If they are the same, then they can be sure that the secret chat is secure, and hence that no MITM attack has been successfully mounted against them. Otherwise (i.e., if they are not the same) then there is real risk of having an MITM between them. Unfortunately, meeting in person often defeats the purpose of messaging. This leads many users to make a screenshot of the visualization of the f ngerprint and send it in the newly established unauthenticated session. As the MITM also has a f ngerprint for each user, and it is very easy for him or her to replace the screenshot with one of its own, the MITM mitigation technique may not be effective in practice. Also, note that the Diff e-Hellman key exchange is performed only at the beginning of a session and periodically repeated to refresh the session state and respective key. In fact, a new Diff e-Hellman key exchange is performed after the derivation of 100 AES keys or the old key has been in use for more than one week. This provide some lightweight form of forward secrecy and PCS. It is, however, not comparable to what can be achieved with ratcheting in other E2EE messaging protocols.

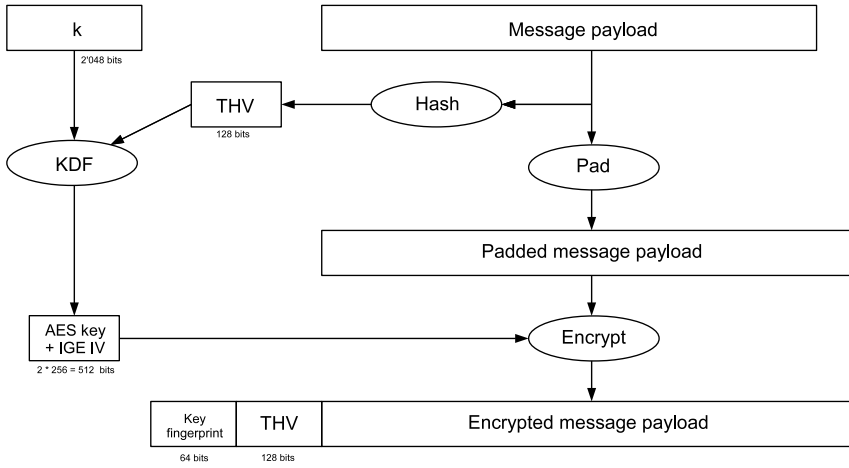


Figure 11.3 MTProto encryption.

The way MTProto encrypts a message payload is illustrated in Figure 11.3. The details of the message payload do not matter, and MTProto versions 1 and 2 are slightly different here (we omit the details). At the top, there is the message payload that is to be encrypted with key k (that is the key shared between the sender and the recipient, and that is the common output of the Diff e-Hellman key exchange). The message payload is f rst hashed (using SHA-1 in MTProto version 1 and SHA-256 in version 2), and the output is truncated to 128 bits. The respective value is called truncated hash value (THV) here. Both k and the THV are input to a KDF, and the output is a 256-bit AES key and two 128-bit IGE IVs, summing up to another 256 bits. This means that the output of the KDF is 512 bits. The message payload is padded and then encrypted using AES-256 in IGE mode with the AES key and the two IVs. The result is the encrypted message payload that gets two f eld that are prepended: A 64-bit key f ngerprint that refers to the key k in use and the 128-bit THV. This data structure is sent to the recipient. To decrypt it, the recipient takes the key f ngerprint to grab the appropriate key k. This value and the THV are then subject to the KDF, and the result is an AES-256 key and two IVs of the same total length. Using these parameters, the encrypted message payload can then be decrypted.

The details of MTProto are slightly more involved, and there are some complementary security mechanisms to mitigate specif c attacks, such as message counters to protect against replay and mirroring attacks, and an additional layer of encryption at the transport layer to provide obfuscation and make cryptanalysis more diff cult. With regard to its overall security, Telegram and the MTProto protocol are often criticized for not being as strong as its competitors. For example, the fact that the MTProto protocol is home-grown is heavily criticized in the community, because a general rule of thumb in applied cryptography suggests that one should never design a protocol on its own, but (re)use standardized protocols whenever possible—simply because they have seen a lot of public scrutiny. This is clearly not the case here, and only a few experts have analyzed the MTProto protocol and its implementation (e.g., [10, 14–17]). Most of them have expressed concerns about the quality and cryptographic strength of the protocol. For example, the use of a THV instead of a MAC construction or even authenticated encryption is very questionable and not seen elsewhere. Another source of concern is related to the fact that Telegram initially asks for the user's contact list that is then uploaded and stored on the server. This is similar to what other messengers do, and there are only a few messengers that are more cautious here, such as the Signal messenger.

References

[1] Apple, "Apple Platform Security," Fall 2019.

[2] Harkins, D., "Synthetic Initialization Vector (SIV) Authenticated Encryption Using the Advanced Encryption Standard (AES)," RFC 5297, October 2008.

[3] Shoup, V., "A Proposal for an ISO Standard for Public Key Encryption," Version 2.1, December 20, 2001.

[4] U.S. NIST, "Digital Signature Standard (DSS)," FIPS PUB 186-4, July 2013.

[5] Garman, C., et al., "Dancing on the Lip of the Volcano: Chosen Ciphertext Attacks on Apple iMessage," *Proceedings of the 25th USENIX Security Symposium*, USENIX Association, 2016, pp. 655–672.

[6] Howell, C, Leavy, T., and J. Alwen, "Wickr Messaging Protocol—Technical Paper," 2017.

[7] Threema, "Cryptography Whitepaper," January 16, 2019.

[8] Lacharme, P., et al., "The Linux Pseudorandom Number Generator Revisited," Cryptology ePrint Archive Report 2012/251, https://eprint.iacr.org/2012/251.

[9] Moriarty, K. (Ed.), Kaliski, B., and A. Rusch, "PKCS #5: Password-Based Cryptography Specif cation Version 2.1," RFC 8018, January 2017.

[10] Jakobsen, J.B., "A Practical Cryptanalysis of the Telegram Messaging Protocol," Master's Thesis, Aarhus University, September 2015.

[11] Campbell, C.M., "Design and Specif cation of Cryptographic Capabilities," in *Computer Security and the Data Encryption Standard, (D.K. Brandstad (Ed.))*, National Bureau of Standards Special Publications 500-27, U.S. Department of Commerce, February 1978, pp. 54–66, https://csrc.nist.gov/publications/detail/sp/500-27/archive/1978-02-01.

[12] Gligor, V.D., and P. Donescu, "On Message Integrity in Symmetric Encryption," unpublished manuscript, November 2000.

[13] Laurie, B., "OpenSSLs Implementation of Inf nite Garble Extension Version 0.1," August 2006.

[14] Jakobsen, J.B., and C. Orlandi, "On the CCA (in)Security of MTProto," *Proceedings of the 6th Workshop on Security and Privacy in Smartphones and Mobile Devices (SPSM 2016)*, ACM Press, New York, 2016, pp. 113–116.

[15] Sušánka, T., and J. Kokeš, "Security Analysis of the Telegram IM," *Proceedings of the 1st Reversing and Offensive-Oriented Trends Symposium (ROOTS 2017)*, Article No. 6, ACM Press, New York, 2017.

[16] Lee, J., et al., "Security Analysis of End-to-End Encryption in Telegram," *Proceedings of the Symposium on Cryptography and Information Security (SCIS 2017)*, 2017.

[17] Saribekyan, H., and A. Margvelashvili, "Security Analysis of Telegram," May 18, 2017, https://courses.csail.mit.edu/6.857/2017/project/19.pdf.

Chapter 12

Privacy Issues

In this chapter, we address a few privacy issues that are relevant for Internet messaging in general, and E2EE messaging in particular. More specif cally, we introduce the topic in Section 12.1, address self-destructing (or ephemeral) messaging and online presence indication as two exemplary technologies in Sections 12.2 and 12.3, and conclude with some f nal remarks in Section 12.4. There is a lot more to be said about privacy, but since this book is about security and not privacy, we can only explore the tip of the iceberg here.

12.1 INTRODUCTION

There are many def nitions of the term *privacy*. The greatest common denominator is that it refers to the ability of an individual or group to seclude themselves, or information about themselves, and to be left alone. The boundaries and content of what is considered private differ among cultures and people. There are some cultures that focus on social life in which there is hardly any privacy, whereas there are other cultures—typically the ones we know and live in—that emphasize and value the individuality of persons and consider privacy to be a fundamental human right. In either case, privacy is not primarily about protecting data, but rather about protecting persons against the misuse of data that may be stored and processed about them. The analogy is a rain protection that doesn't really protect the rain, but rather the persons that may be exposed to rain.

Because privacy is often considered to be a fundamental human right, many countries have a legal framework and a respective data privacy act in place that tries to protect their citizens against misuse of personal data (i.e., sensitive data stored about persons or groups of persons). The legislation of an appropriate data privacy act is a very challenging task, and different countries follow different approaches

here. In Europe, for example, the *general data protection regulation* (GDPR) has strengthened the privacy discussion since its enactment in 2016.[1] It is relevant today, also because it may stipulate draconic f nes for non-compliant actors.

In this chapter, we don't delve into legal issues that surround privacy and the legislation thereof. Instead, we raise a few topics that relate to privacy and that the user of a messaging app should be aware of and take into proper consideration.

- First, the user should be aware that the price for a messaging service in many cases is giving away his or her personal contact information. More specif cally, when the user installs the app on his or her smartphone, the app usually uploads the user's contacts to a central server. The advantage is that the user immediately sees who among his or her contacts is using the same app. This simplif es the use of the app considerably. The disadvantage, however, is that the service provider learns the social relationships of the user and can exploit this knowledge for other purposes, such as targeted advertising.
- Second, the user should be aware that many messaging apps backup messages automatically, and that these backups are often stored in the cloud. If the backups are encrypted, then they are sometimes encrypted in a way that can be decrypted not only by the user but also by the messaging service provider. The details of key management and recovery are often subtle here.
- Third, the user should be aware that messaging apps generate a lot of metadata, and that the messaging service provider may be tempted to misuse and commercialize this information. We partly revisit the issue of metadata when we address online presence indication in Section 12.3.

In many cases, there are settings that can be used to control these issues, but many people don't care about them (and leave the default setting unchanged). In addition to these awareness and conf guration issues, there are a few technologies—sometimes called *privacy-enhancing technologies* (PETs)—that can be used in the f eld to improve privacy. In the rest of this chapter, we look at two such PETs (i.e., self-destructing messaging and online presence indication) and we treat them as examples. There are many other PETs available in the realm of messaging, such as the use of anonymous identif ers (e.g., Threema IDs), the use anonymous remailers [1] that have come out of the cypherpunk movement [2, 3], or even the use of onion routing [4] and TOR (mentioned in Section 4.1.1) for messaging. We limit ourselves to the two topics mentioned above, mainly because the subject of the book is secure and E2EE messaging and not private messaging. There are some

1 https://eur-lex.europa.eu/eli/reg/2016/679/.

similarities between secret and private messaging, but their respective focus and perspective are still fundamentally different.

12.2 SELF-DESTRUCTING MESSAGING

Self-destructing messaging, or *ephemeral messaging*, refers to a type of messaging, in which messages are purposely short-lived, and in which the messaging system automatically deletes a message some time after consumption or after a certain number of views. The deletion occurs on all systems involved, including the sender's device(s), the receiver's device(s), and the server system(s). No lasting record of the conversation is to be kept, and hence self-destructing messaging can be seen as the digital analog to disappearing ink.

There are many use cases for self-destructing messaging. Whenever you want to communicate something to somebody else without giving this person the possibility to further disseminate it, you may think about using this type of messaging. The most prominent example of a messaging app that has pioneered and still supports self-destructing messaging is *Snapchat*[2] provided by Snap Inc.[3] In fact, Snapchat has become one of the major messaging apps especially for younger people because of this feature. Users can share content without being afraid that it gets further disseminated on the Internet.

With regard to the implementation of self-destructing messaging it is important to note that a message may exist in different copies on its delivery path. It is created on a sending device, potentially synchronized and replicated to many devices of the sender, sent through multiple servers, and f nally stored on one or several receiving devices. If proper end-to-end encryption is not in place, then self-destructing messaging is not feasible, because a copy of the message may then reside on every system on the path. So end-to-end encryption is a necessary but not suff cient requirement to implement self-destructing messaging: The sending and receiving messaging apps may still copy and store the message for later use, because they get in touch and must display it in decrypted form. It must therefore be ensured in one way or another that these apps have put in place and enforce proper access control and provide some form of message encapsulation. Only the authorized messaging app should have access to a message, and this access may even be f ne-grained—depending on the operating system in use. It may be possible to read the message, but it may not be possible to change it (e.g., by overwriting the allocated memory space). What makes things particularly involved is the fact that the access control mechanisms available depend on the operating systems in

2 https://www.snapchat.com.
3 https://www.snap.com.

use, and that the messaging app should run on multiple operating systems—or even versions of a particular operating system. This means that it cannot use a high-level application programming interface (API), but must generally take a deep dive into the internals of the operating systems. Unfortunately, this also makes the messaging apps depend on the actual systems in use. There is room for alternative technologies here. In 2009, for example, a system called *Vanish*[4] was proposed [5] to self-destruct data, such as messages, through the combined use of cryptographic techniques, P2P networking, and distributed hash tables (DHTs). Systems like Vanish are important to explore what is feasible in terms of self-destructing messaging, as it was attacked successfully only one year after its publication [6].

One problem that cannot be solved technically, at least not completely, is that a recipient of a message can always take a screenshot while the message is being displayed on one of his or her devices. There are a few mitigation techniques, but none of them is able to completely solve the problem. For example, prior to 2015, using Snapchat required the recipient to hold his or her f nger on the screen while viewing a message. This was to dissuade the use of screenshots. Because this requirement also handicapped legitimate use, the feature was later removed from Snapchat. It is still being used—in an even stronger form[5]—in the *Conf de*[6] messenger app, but again it negatively impacts user experience. What Snapchat does nowadays (instead on trying to make it more diff cult to take a screenshot) is to provide feedback to the original sender of a message (i.e., send back a notif cation whenever a recipient takes a screenshot while reading the message). This is not foolproof though, and there are many descriptions on the Internet that explain how to defeat the feedback mechanism. It is at least possible to use a secondary device to take the screenshot. The bottom line is that the feedback must be taken with a grain of salt: If the sender receives a notif cation, then he or she can be sure that the respective recipient has cheated and taken a screenshot. However, if he or she does not receive such a notif cation, then the status is largely unknown and the recipient may have circumvented the feedback mechanism altogether.

Today, almost every secure messenger app supports self-destructing messaging in one way or another. Besides Snapchat, this includes many of the messengers addressed in this book, including WhatsApp,[7] Wickr, and Telegram, but it also includes many other messengers, like Conf ne (mentioned above), DatChat,[8],

4 https://vanish.cs.washington.edu.

5 It is ensured that only one line of a message is unveiled at a time and that the message sender's name is not displayed simultaneously. Conf de claims that this patent-pending technology called *ScreenShield* is screenshot-proof.

6 https://getconf de.com.

7 WhatsApp will support self-destructing messaging in its recently announced *dark mode*.

8 https://www.datchat.net.

and Dust.[9] Some Web-based messaging solutions also support self-destructing messaging. If the sender and recipient(s) use the same solution, then self-destructing messaging is very simple to implement here. If they use different solutions, then the implementation is more involved. Gmail, for example, supports self-destructing messaging by leaving a message on its server and providing an (external) recipient with a temporarily valid link that can only be used to view the message. It is not possible to download the message to the recipient's system.

The bottom line is that self-destructing (ephemeral) messaging provides more privacy than normal messaging. The temporary nature of viewing an incoming message deters the chance that a text sent in anger or a photo sent in a lusty moment will cause embarrassment later. Unless the recipient is motivated to record a message in real time, using self-destructing messaging improves privacy considerably. In a world where privacy can't be guaranteed, it makes perfect sense to use self-destructing messaging—especially for personal use.

12.3 ONLINE PRESENCE INDICATION

Most messengers in use today are operated centrally and can therefore provide information about the online status of its users (i.e., the other persons that use the same messenger). The rationale behind this feature is that it is useful for a user to know whether his or her contacts are online, as well. On the other hand, however, it may also be in conf ict with the privacy requirements of the respective users. Consequently, there is a trade-off to make, and online presence indication is a controversially discussed topic for this reason.

What online presence indication provides is metadata. At f rst glance, this may seem innocent. But remember a famous quote from General Michael Hayden,[10] a former NSA and CIA director: "We kill people based on metadata." This suggests that metadata is not as innocent as it seems to be, and the research challenge is to implement online presence indication in a way that is as privacy-friendly as possible (i.e., without leaking too much information). If I am using a messenger and want to know whether another user is also online, then I reveal the information that I want to exchange messages with him or her. This yields metadata that can then be used, for example, in a social graph.

Against this background, the research question is to f nd a possibility to learn about the online status of another user without having to specify the identity of that user. This sounds paradox, but it can actually be solved with cryptographic techniques. There are currently two possibilities:

9 https://usedust.com.

10 https://www.nybooks.com/daily/2014/05/10/we-kill-people-based-metadata.

- On the one hand, one can use anonymity services, such as the ones provided by TOR or—more specif cally—TOR hidden services. An example of such a messenger is *Ricochet*.[11] A Ricochet user gets a unique address that looks like `ricochet:rs7ce36jsj24ogfw`. Other users can send contact requests to this address, asking to be added to the user's contact list. The user's Ricochet software establishes a TOR hidden service that can be used to rendezvous with the contacts without revealing the location or IP address. The user can also see when his or her contacts are online, and send them E2EE messages. Hence, a user's contact list is only known locally and is never exposed to a central server or network traff c monitor.
- On the other hand, one can also use technologies for private information retrieval (PIR). In cryptography, a PIR protocol allows a user to retrieve an item from a server in possession of a database without revealing which item is actually retrieved. This is exactly what is needed here. An example of a PIR protocol used for privacy-preserving online presence indication is the *Dragstuhl Privacy Preserving Presence Protocol Privacy* (DP5[12]) protocol originally proposed by Ian Goldberg, Nikita Borisov, and George Danezis in 2015 [7].[13] This is just an example of a protocol that serves this purpose, and it is possible and very likely that many other but similar protocols will be proposed and eventually prototyped in the future.

Using either of these technologies one can solve the research challenge mentioned above. The use of TOR hidden services is certainly possible, but it comes with many disadvantages mainly related to TOR. Hence, the use of a PIR protocol like DP5 looks more promising and avoids the necessity of using TOR altogether. It is certainly the preferred choice.

12.4 FINAL REMARKS

In addition to normal security requirements, like message authentication, integrity, and conf dentiality, people sometimes argue in favor of privacy. In many situations, however, it remains unclear what people really mean when they use the term privacy. Sometimes they refer to legal issues, but sometimes they also refer to technical ones. In addition to the awareness and conf guration issues mentioned in Section 12.1 (i.e., uploading contacts, storing backups, and deriving metadata) there are several PETs

11 https://ricochet.im.

12 Note that the f fth P is again standing for Privacy.

13 Remember from Chapter 8 that Ian Goldberg and Nikita Borisov are the coinventors of OTR messaging—together with Eric Brewer.

that have been discussed in the realm of messaging for many years. In this chapter, we have elaborated on two such PETs: Self-destructing (or ephemeral) messaging and online presence indication. One can reasonably expect either of these topics to become more important in the future. We have already seen many E2EE messengers provide a feature that allows users to declare self-destructing messages. This is likely to continue, although the technologies are not fool-proof and can be circumvented in many ways. But it is certainly a useful feature, because otherwise messages can be stored forever on the many devices that come in contact with them. In the realm of privacy-preserving online presence indication, we have just recognized the problem and we are still far away from having a solution that is widely deployed and used in the f eld. In contrast to self-destructing messaging, there is hardly any pressure from the providers' side. Unless users ask for it more vigorously, it is unlikely that the providers of E2EE messaging services will come up with solutions that are more appropriate and usable.

References

[1] Goldberg, I., Wagner, D., and E. Brewer, "Privacy-Enhancing Technologies for the Internet," *Proceedings of the 42nd IEEE COMPCON*, IEEE Computer Society, 1997, pp. 103–109.

[2] Narayanan, A., "What Happened to the Crypto Dream?," Part I, *IEEE Security and Privacy*, Vol. 11, No. 2, 2013, pp. 75-76.

[3] Narayanan, A., "What Happened to the Crypto Dream?," Part II, *IEEE Security and Privacy*, Vol. 11, No. 3, 2013, pp. 68–71.

[4] Reed, M.G., Syverson, P.F., and D.M. Goldschlag, "Anonymous Connections and Onion Routing," *IEEE Journal on Selected Areas in Communications*, Vol. 16 (1998), pp. 482–494.

[5] Geambasu, R., et al., "Vanish: Increasing Data Privacy with Self-Destructing Data," *Proceedings of the 18th USENIX Security Symposium*, USENIX, 2009, pp. 521–528.

[6] Wolchok, S., et al., "Defeating Vanish with Low-Cost Sybil Attacks Against Large DHTs," *Proceedings of the 17th Network and Distributed System Security Symposium (NDSS 2010)*, Internet Society, 2010.

[7] Borisov, N., Danezis, G., and I. Goldberg, "DP5: A Private Presence Service," *Proceedings on Privacy Enhancing Technologies*, De Gruyter Open, Volume 2015, Issue 2, pp. 4–24.

Chapter 13

Conclusions and Outlook

In this book, we have elaborated on secure and E2EE messaging on the Internet. More specif cally, we have introduced, discussed, and put into perspective technologies and protocols that can be used for this purpose. We started with a pair of technologies and protocols that have specif cally been designed for e-mail (i.e., OpenPGP and S/MIME). They use basic cryptographic primitives, like digital envelopes and signatures, and they are relatively simple and straightforward. Due to the asynchronous nature of e-mail, however, they do not allow the participants to perform an interactive key exchange, like a Diff e-Hellman key exchange, to provide forward secrecy and PCS. Also, the use of digital signatures provides nonrepudiation, but does not provide the opposite (i.e., repudiation or deniability).

Since the beginning of this century, the lack of forward secrecy and PCS on the one hand, and the inability to provide deniability or even plausible deniability on the other hand, has been criticized, for example, by the developers of OTR. Hence, OTR has really brought a paradigm shift in secure and E2EE messaging on the Internet, and hence the simplicity of the early solutions for secure messaging have been challenged ever since. Most solutions have deviated from only using digital envelopes and signatures, and use more sophisticated cryptographic primitives and mechanisms, such as AKE protocols, Diff e-Hellman or double ratchets, message authentication codes and authenticated encryption, and malleable encryption. Also, people are looking for alternatives to public key-based authentication ceremonies and respective trust models that are inherently more user-friendly (and hence more usable) than public key certif cates and f ngerprints. Examples include SMP and QR codes. In fact, there is quite a large and steadily increasing body of research in this area (e.g., [1–3]). We point out two approaches that look particularly promising here:

- Based on some early ideas related to what has been termed *social authentication* [4], some E2EE messaging apps, like Keybase[1] or rather Keybase Chat, explore a new approach: They pair a user's public key with several identities on social media and respective accounts (e.g., Twitter, Reddit, GitHub, ...). The user can then prove ownership of his or her public key by proving ownership of such accounts. The more accounts, the stronger the identity and the respective link to the public key. This, in turn, means that the key is going to be more trustworthy. Conf dante[2] is a research project that aims at building a highly usable E2EE mail client on top of Gmail and Keybase.
- In another line of research, people have come up with DLTs to handle public key certif cates and respective revocation information, such as the attack resilient public key infrastructure (ARPKI) [5], the CONsistent Identity Key Service (CONIKS[3]) [6], and an E2EE messaging extension to Google's certif cate transparency initiative already mentioned in Section 3.3.2.2 [7].

Besides these two approaches (that have not yet found their way into mainstream E2EE messengers yet), the culmination point and state of the art in secure and E2EE messaging on the Internet is certainly the Signal protocol that is used—as its name suggests—in the Signal messenger, as well as many other E2EE messengers (as discussed in previous chapters of this book). The Signal protocol equally supports synchronous and asynchronous applications, and can be equally used for instant messaging and e-mail. There is hardly any messaging-based use case for which the Signal protocol does not provide a viable solution. In 2019, for example, Facebook announced that more of its services (in addition to WhatsApp and secret conversations in the Facebook Messenger) are going to support E2EE in the future. This is a strong commitment, and it is possible and very likely that these services will also employ the Signal protocol in one way or another.

The success of E2EE messaging in general, and the Signal protocol in particular, has also revitalized and amplif ed interest in cryptographic research. People are looking for ways to improve the Signal and related protocols.

- For example, the upload of large batches of one-time public prekeys is not optimal, and people are looking for more eff cient cryptographic techniques to provide forward secrecy and PCS. Examples include *forward secure public key encryption* (FS-PKE) [8] and—more recently—*puncturable encryption*

1 Keybase is just a new type of groupware and collaboration software that is conceptually similar to Slack.
2 https://conf dante.cs.washington.edu.
3 https://coniks.cs.princeton.edu.

[9]. In this type of encryption, the secret key is punctured after each decryption operation, such that a given ciphertext can only be decrypted once.

- Some researchers try to exploit P2P techniques to come up with a messaging scheme that does not require single trusted entities. An example of this type is *Bitmessage*,[4] whose working principles—as its name suggests—are very similar to Bitcoin. Another example is *Elixxir*[5] promoted by David Chaum that yields a blockchain-based transaction platform to provide a secure version of WeChat or—more specif cally—WeChat Pay. Elixxir does not only end-to-end encrypt messages, but also protects the metadata. As such, it is not only a solution for E2EE messaging, but also for private messaging and paying. As of this writing, it is too early to tell whether solutions like these work suff ciently well in practice, and whether they are going to be accepted and supported in the f eld.

More recently (and similar to WeChat Pay and Elixxir), Facebook has announced that it wants to enter the digital payment business with a new stable coin called *Libra*. This announcement has initiated a political discussion about the future role of Facebook in this business. While some people argue that it is in the entrepreneurial freedom of Facebook to enter it, other people argue that the power of Facebook would become too large, if it entered it. In the end, it is going to be a political decision if and how far Facebook can go here. The respective political debates are going on and take time. From a purely technical viewpoint, Facebook is in a good position (with the Facebook Messenger and WhatsApp) to play a dominant role in the digital payment business.

In addition to the existing problems and research challenges itemized above, the use of encryption in general, and the use of E2EE messaging in particular, may also introduce some new problems and research challenges. Let us mention just two of them:

- First, E2EE messages cannot be inspected for malware and abuse, and this, in turn, means that some complementary protection mechanisms need to be implemented by the (receiving) end systems. While this partly works for malware detection (using existing endpoint security solutions), it is particularly more challenging for abuse detection. In 2016, Facebook therefore introduced a cryptographic solution for abuse detection and handling in its messenger—at least for E2EE messages sent in the secret conversations mode. Facebook coined the term *message franking* for its solution, and this term has prevailed

4 https://bitmessage.org.
5 https://elixxir.io.

and is now consistently used in the community. Informally speaking, a message franking system provides a proof that a particular (abusive) message really came from its claimed originator and was actually delivered by the messaging service in use. It is analogous to placing a cryptographic stamp (or tag) on the message without learning the message content. The original message franking mechanism employed by the Facebook Messenger is described in [10]. It employs two tags: A reporting tag generated by the sender of a message m, and a reporting tag generated by Facebook. In short, the sender of m randomly selects a 256-bit random nonce r and uses it as a key to compute the franking tag t_F for m' (where m' comprises m and r in some serialized form) using the HMAC-SHA256 construction:

$$t_F = \text{HMAC-SHA256}(r, m')$$

Note that r serves as a key and is also embedded in the input to the HMAC-SHA256 construction. After having computed t_F, the sender encrypts m' and sends the resulting ciphertext c' together with t_F to Facebook for delivery. When Facebook receives (c', t_F), it uses a static Facebook key k to compute the reporting tag t_R over t_F and some conversation context $context$ that comprises information, such as the sender and recipient identifiers and a timestamp:

$$t_R = \text{HMAC-SHA256}(k, t_F \parallel context)$$

Facebook delivers c' together with t_F and t_R to the intended recipient. The recipient, in turn, decrypts c', parses the resulting plaintext m' to retrieve r, and verifies t_F prior to displaying the message m. In the positive case, it locally stores m', r, t_F, t_R, and $context$ for later use. If, however, the verification of t_F fails, then the recipient discards the message without displaying it (and without storing anything). To report abuse, the recipient submits m', r, t_R, and $context$ to Facebook. Facebook then recomputes t_F (with m' and r) and verifies t_R (by recomputing it with k and comparing the result with the value submitted by the recipient of the message). This message franking mechanism is simple and straightforward, and there is a lot of room for improvement and optimization. In fact, the cryptographic research community has taken up the problem and has developed alternative mechanisms (e.g., [11]). As of this writing, however, the Facebook Messenger seems to be the only E2EE messenger that cares about message franking and provides a technical solution to it. This will likely change as E2EE messaging becomes more widely deployed in the future. The fact that E2EE messages can be abused in multiple ways, will make it necessary to come up with appropriate countermeasures.

- Second, some countries have started to censor and ban E2EE messages, and this has led people to think about (technical) possibilities to circumvent censorship. *Telex*[6] was an early attempt to protect the network infrastructure against censorship [12]. Telex has recently evolved into *Refraction Networking*[7] that is a technology that is now being deployed by some ISPs that operate in the f eld. Another technology that is frequently used is *domain fronting* [13] (as used, for example, by the Signal messenger), and f nding other versatile censorship circumvention techniques is another hotly debated topic in privacy-related research.

More recently, the IETF has started to address the topic and chartered a *message layer security* (MLS) WG within the security area.[8] The aim of the WG is to provide an architecture and a respective protocol that can be used for any messaging-type application—be it synchronous or asynchronous. The protocol should be able and optimized for large groups, possibly on the order of thousands of group members. Needless to say, the protocol should be able to provide state-of-the-art security in terms of forward secrecy and PCS. The work on MLS is fundamentally different from the work on TLS: While TLS focuses on a two-party setting with usually single devices and sessions that are short-lived, MLS is essentially the opposite (i.e., it focuses on an n-parties setting with $n \gg 2$ and multiple devices and sessions that are long-lived). Due to these differences, it makes a lot of sense to address the message security problem separately from the transport layer security problem, and this, in turn, has motivated the IETF to charter the new WG.

The basis for the work of the IETF MLS WG is the Signal protocol and some prior work related to group key exchange. Unfortunately, it is not known how to generalize the Diff e-Hellman key exchange protocol to more than three parties (e.g., [14]), and most group key exchange protocols require the participating parties to be permanently online and therefore only support the synchronous setting (there are so many protocols that we won't even start referencing them here). In 2018, a tree-based group key exchange protocol named *asynchronous ratcheting tree* (ART) was proposed that also supports the asynchronous setting [15], and since then the ART protocol has been improved in multiple ways. The resulting protocols are called TreeKEM [16], which stands for tree-based key encapsulation mechanism, and continuous group key agreement (CGKA) [17], and they feed the work of the IETF MLS WG. As of this writing, the WG has already provided a few Internet-Drafts (based on the ART, TreeKEM, and CGKA protocols), and the f rst

6 https://telex.cc.
7 https://refraction.network.
8 https://datatracker.ietf.org/wg/mls.

implementations have started to pop up, such as MLS++,[9] MLS*,[10] Molasses,[11] and Melissa[12] for Wire. It is, however, too early to tell, whether this standardization effort is going to be successful in the long term. So far, E2EE messaging has always been dominated by companies and organizations that are quick to implement and provide new features and solutions to the public. It is not an area in which conformance to standards has been the top priority—let's see whether this is going to change now. Group messaging remains an interesting topic that is going the make a difference. Some E2EE messengers will be early in adopting the MLS protocol (once it is specif ed and off cially released), whereas other E2EE messengers will stay with their originally designed and often proprietary protocol as long as they can and maybe not even care about the multi-party setting of group messaging in the f rst place.

References

[1] Herzber, A., and H. Leibowith,"Can Johnny Finally Encrypt? Evaluating E2E-Encryption in Popular IM Applications," *Proceedings of the Sixth International Workshop on Socio-Technical Aspects in Security and Trust (STAST 2016)*, ACM, 2016, pp. 17–28.

[2] Tan, J., et al.,"Can Unicorns Help Users Compare Crypto Key Fingerprints," *Proceedings of the 35th ACM Conference on Human and Computing Systems (CHI 2017)*, ACM, 2017, pp. 3787–3798.

[3] Vaziripor, E., et al.,"Is that you, Alice? A Usability Study of the Authentication Ceremony of Secure Messaging Applications," *Proceedings of the 13th Symposium on Usable Privacy and Security (SOUPS 2017)*, USENIX Association, Berkeley, CA, 2017, pp. 29–47.

[4] Vaziripor, E., et al.,"Social Authentication for End-to-End Encryption," *Proceedings of the 12th Symposium on Usable Privacy and Security (SOUPS 2016)*, USENIX Association, Berkeley, CA, 2016, 2-page position paper.

[5] Basin, D., et al.,"ARPKI: Attack Resilient Public-Key Infrastructure," *Proceedings of the ACM Conference on Computer and Communications Security (CCS 2014)*, ACM Press, New York, pp. 382–393.

[6] Melara, M., et al.,"CONIKS: Bringing Key Transparency to End Users," *Proceedings of the 24th USENIX Security Symposium (USENIX Security 2015)*, USENIX Association, Berkeley, CA, 2015, pp. 383–398.

[7] Ryan, M.D., "Enhanced Certif cate Transparency and End-to-End Encrypted Mail," *Proceedings of the Network and Distributed System Security Symposium (NDSS 2014)*, 2014, Brief ng Paper, https://www.ndss-symposium.org/ndss2014/programme/enhanced-certif cate-transparency-and-end-end-encrypted-mail.

9 https://github.com/cisco/mlspp.
10 https://www.fstar-lang.org.
11 https://github.com/trailofbits/molasses.
12 https://github.com/wireapp/melissa.

[8] Canetti, R., Halevi, S., and J. Katz,"A Forward-Secure Public-Key Encryption Scheme," *Proceedings of EUROCRYPT 2003*, Springer, LNCS 2656, 2003, pp. 255–271.

[9] Green, M.D., and I. Miers,"Forward Secure Asynchronous Messaging from Puncturable Encryption," *Proceedings of the IEEE Symposium on Security and Privacy*, IEEE, 2015, pp. 305–320.

[10] Facebook, "Messenger Secret Conversations Technical Whitepaper," July 8, 2016

[11] Grubbs, P., Lu, J., and T. Ristenpart,"Message Franking via Committing Authenticated Encryption," *Proceedings of CRYPTO 2017*, Springer, LNCS 10403, 2017, pp. 66–97.

[12] Wustrow, E., et al.,"Telex: Anticensorship in the Network Infrastructure," *Proceedings of the 20th USENIX Security Symposium*, USENIX Association, Berkeley, CA, 2011, p. 30.

[13] Fif eld, D., et al., "Blocking-Resistant Communication Through Domain Fronting," *Proceedings on Privacy Enhancing Technologies*, De Gruyter Open, Volume 2018, Issue 2, pp. 1–19.

[14] Joux, A., "A One Round Protocol for Tripartite Diff e-Hellman," *Journal of Cryptology*, Vol. 17, Issue 4, September 2004, pp. 263–276.

[15] Cohn-Gordon, K., et al.,"On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guaranteese," *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS 2018)*, ACM Press, New York, 2018, pp. 1802–1819.

[16] Bhargavan, K., Barnes, R., and E. Rescorla, "TreeKEM: Asynchronous Decentralized Key Management for Large Dynamic Groups," May 3, 2019, https://prosecco.gforge.inria.fr/personal/karthik/pubs/treekem.pdf.

[17] Alwen, J., et al., "Security Analysis and Improvements for the IETF MLS Standard for Group Messaging," Cryptology ePrint Archive, Report 1189, October 2019, https://eprint.iacr.org/2019/1189.

Appendix A

Mathematical Notation

X	set
$\|X\|$	cardinality (i.e., number of elements) of set X
$f : X \to Y$	function (mapping from elements of X to elements of Y)
f^{-1}	inverse function
$\mathrm{Perms}[X]$	set of all possible permutations of X
$\mathrm{Funcs}[X, Y]$	set of all possible functions that map elements of X to elements of Y
X	domain (of function f)
Y	range (of function f)
h	hash function
Σ	alphabet
$\{0,1\}$	binary alphabet
$\{0,1\}^l$	set of all binary strings of length l
$\{0,1\}^*$	set of all binary strings of arbitrary length
$\|s\|$	length (in bits) of string s
$s\|_a$	a leftmost bits of string s
$s\|^b$	b rightmost bits of string s
$\|$	string concatenation
$x \in X$	x is an element of X
$x \in_R X$	x is a random (i.e., randomly chosen) element of X
m	(plaintext) message
$\mathcal{M}$	(plaintext) message space
c	ciphertext
$\mathcal{C}$	ciphertext space
k	secret key in a secret key cryptosystem
$\mathcal{K}$	key space
pk	public key in a public key cryptosystem

sk	private key in a public key cryptosystem
E	family $\{E_k : k \in \mathcal{K}\}$ of encryption functions $E_k : \mathcal{M} \to \mathcal{C}$
D	family $\{D_k : K \in \mathcal{K}\}$ of decryption functions $D_k : \mathcal{C} \to \mathcal{M}$
t	authentication tag
$\mathcal{T}$	authentication tag space
A	family $\{A_k : k \in \mathcal{K}\}$ of authentication functions $A_k : \mathcal{M} \to \mathcal{T}$
V	family $\{V_k : K \in \mathcal{K}\}$ of verif cation functions $V_k : \mathcal{M} \times \mathcal{T}$ $\to \{valid, invalid\}$
$\leftarrow$	assignment
$\stackrel{r}{\leftarrow}$	assignment from a universal distribution
ϕ	Euler's totient function
$\mathbf{P}$	set of all primes
$\mathbf{P_k}$	set of all k-bit primes

Appendix B

Abbreviations and Acronyms

3DES	Triple DES
3DH	Triple Diff e-Hellman
AA	attribute authority
ACM	Association for Computing Machinery
AD	assoiated data
ADK	additional decryption key
AEAD	authenticated encryption with associated data
AES	Advanced Encryption Standard
AIM	AOL Instant Messenger
AKE	authenticated key exchange
ANSI	American National Standards Institute
APG	Android Privacy Guard
AOL	America Online
API	application programming interface
APN	Apple push notif cation
ARC	authenticated received chain
ARPKI	attack resilient public key infrastructure
ART	asynchronous ratcheting tree
ASCII	American Standard Code for Information Interchange
ASN.1	abstract syntax notation 1
bcc	blind carbon copy
BCP	best current practices
BER	basic encoding rules

BREACH	browser reconnaissance and exf ltration via adaptive compression of hypertext
CA	certif cation authority
CAA	certif cation authority authorization
CBC	cipher block chaining
cc	carbon copy
CCA	chosen-ciphertext attack
CCA2	adapative chosen-ciphertext attack
CDN	content delivery network
CFB	cipher feedback
CGKA	continuous group key agreement
CMAC	CBC-MAC
CMS	cryptographic message syntax
CONIKS	consistent identity key service
CR	carriage return
CRC	cyclic redundancy check
CRIME	compression ratio info-leak made easy
CRL	certif cate revocation list
CSP	certif cation service provider
CSR	certif cate signing request
CSS	content scrambling system
CTR	counter mode
CVE	common vulnerabilities and exposure
DAC	discretionary access control
DAKE	deniable AKE
DAKEZ	DAKE with zero knowledge
DANE	DNS-based authentication of named entities
DAP	directory access protocol
DDoS	distributed denial-of-service
DER	distinguished encoding rules
DES	Data Encryption Standard
DH	Diff e-Hellman
DHE	DH ephemeral
DHP	DH problem
DHT	distributed hash table
DIME	Dark Internet Mail Environment
DIT	directory information tree
DKIM	domainKeys identif ed mail

DLP	discrete logarithm problem data leakage prevention
DLT	distributed ledger technology
DMAP	dark mail access protocol
DMARC	domain-based message authentication, reporting, and conformance
DMS	defense messaging system
DMTP	dark mail transfer protocol
DN	distinguished name
DNS	domain name system
DNSSEC	DNS security extensions
DoD	Department of Defense
DoS	denial-of-service
DP5	Dragstuhl privacy preserving presence protocol privacy
DRAM	dynamic random access memory
DRM	digital rights management
DSA	digital signature algorithm
DSS	digital signature system
E2EE	end-to-end encrypted
EBCDIC	extended binary coded decimal information code
ECB	electronic code book
ECC	elliptic curve cryptography
ECDH	elliptic curve DH
ECDSA	elliptic curve DSA
ECIES	elliptic curve integrated encryption system
ECM	extended certif ed mail
ECMQV	elliptic curve Menezes-Qu-Vanstone
EdDSA	Edwards-curve DSA
EDT	eastern daylight time
EEEMA	end-to-end-encrypting messaging application
E-commerce	electronic commerce
EHE	Exchange hosted encryption
EHLO	extended HELLO
EKE	encrypted key exchange
E-mail	electronic mail
EMSEC	emission security
EP	encrypted payload
EPH	encrypted packet header

ESMTP	extended SMTP
ESS	enhanced security services
ETSI	European Telecommunications Standards Institute
FAQ	frequently asked questions
FIPS	Federal Information Processing Standard
FS	forward secrecy
FS-PKE	forward secure public key encryption
FTP	f le transfer protocol
FYI	for your information
gcd	greatest common denominator
GCHQ	Government Communications Headquarters
GDPR	general data protection regulation
GMT	Greenwich mean time
GOTR	group OTR
GPG	GNU Privacy Guard
HKDF	HMAC-based extract-and-expand key derivation function
HKP	HTTP keyserver protocol
HMAC	hashed MAC
HSTS	HTTP strict transport security
HTML	hypertext markup language
HTTP	hypertext transfer protocol
HTTPS	HTTP security
I2P	Invisible Internet Project
IACR	International Association for Cryptologic Research
IBE	identity-based encryption
ICMP	Internet control message protocol
ICSI	International Computer Science Institute
IDEA	International Data Encryption Algorithm
IEC	International Electrotechnical Committee
IEEE	Institute of Electrical and Electronic Engineers
IETF	Internet Engineering Task Force
IFIP	International Federation for Information Processing
IGE	inf nite garble extension
IGP	interior gateway protocol
IKE	Internet key exchange
IM	instant messaging

IMAP	Internet message access protocol
IMAPS	IMAP over SSL/TLS
IMF	Internet message format
IoT	Internet of things
IP	Internet protocol
IPsec	IP security
IRC	Internet relay chat
IRTF	Internet Research Task Force
IS	international standard
ISO	International Organization for Standardization
ISP	Internet service provider
IT	information technology
ITU-T	International Telecommunication Union — Telecommunication Standardization Sector
IV	initialization vector
JTC1	Joint Technical Committee 1
KDC	key distribution center
KDF	key derivation function
KEA	key exchange algorithm
KED	key exchange data
KEK	key-encryption key
KEL	key exchange list
KEM	key encapsulation mechanism
KOI	Kod Obmena Informatsiey
KTC	key translation center
KVAC	keyed-verif cation anonymous credentials
LAMPS	limited additional mechanisms for PKIX and SMIME
LAN	local area network
LDAP	lightweight directory access protocol
LDAPS	LDAP over SSL/TLS
LF	line feed
LLC	limited liability company
LMTP	local mail transfer protocol
LRA	local registration agent, also local registration authority
LSB	least signif cant bit
LSFR	linear feedback shift register
LZ77	Lempel-Ziv 77

MAC	message authentication code
MAN	metropolitan area network
MDA	mail delivery agent
MDC	modif cation detection code
MHS	message handling system
MIC	message integrity code
MIME	multipurpose Internet mail extensions
MITM	man-in-the-middle
MLA	mail list agent
MLS	message layer security
MMS	multimedia messaging service
MOSS	MIME object security services
mpOTR	multi-party OTR
MSA	message submission agent
MSP	message security protocol
MSRP	message session relay protocol
MSRPS	MSRP security
MTA	message transfer agent
MTA-STS	MTA strict transport security
MTS	message transfer system
MUA	message user agent
NIST	National Institute of Standards and Technology
NRUDT	notice-requested-upon-delivery-to
NSA	National Security Agency
NSE	nonsecret encryption
OAEP	optimal asymmetric encryption padding
OCR	optical character recognition
OCSP	online certif cate status protocol
OFB	output feedback
OID	object identif er
OpenPGP	open specif cation for PGP
ORAM	oblivious RAM
OSCAR	open system for communication in realtime
OSI	open systems interconnection
OTR	off-the-record
P2P	peer-to-peer

PAKE	password AKE
PBKDF	password-based key derivation function
PC	personal computer
PCBC	propagating cipher block chaining
PCS	post-compromise security
PEM	privacy enhanced mail
pEp	Pretty Easy Privacy
PET	privacy-enhancing technology
PFS	perfect forward secrecy
PGP	Pretty Good Privacy
PIN	personal identif cation number
PIR	private information retrieval
PKCS	public key cryptography standard
PKI	public key infrastructure
PKIX	public key infrastructure X.509
POP	post off ce protocol
PQC	post-quantum cryptography
PRBG	pseudorandom bit generator
PRG	pseudorandom generator
PS	packet signature
PSS	probabilistic signature scheme
PWM	Private WebMail
QR	quick response
RA	registration authority
RAM	random access memory
RCS	rich communication services
RFC	Request for Comments
RKE	ratcheted key exchange
RRT	return-receipt-to
RSA	Rivest, Shamir, and Adleman
RSASSA	RSA signature scheme with appendix
RTCWEB	real-time communication in WEB-browsers
RTF	rich text format
RTP	real-time transport protocol
SAFE	secure attached f le encryption
SCIMP	silent circle instant messaging protocol
SDA	self-decrypting archive

SDSI	simple distributed security infrastructure
SGX	software guard extensions
SHA	secure hash algorithm
SIMPLE	SIP for instant messaging and presence leveraging extensions
SIP	session initiation protocol
SIPS	SIP security (SIP over SSL/TLS)
SIV	synthetic initialization vector
SKIP	simple key-management for Internet protocol
SMAP	simple mail access protocol
SMIME	S/MIME mail security
S/MIME	secure MIME
SMP	socialist millionaires' problem
SMS	short message service
SMTP	simple mail transfer protocol
SNI	server name indication
SOA	service-oriented architecture
SoK	systematization of knowledge
SOML	send or mail
SPF	sender policy framework
SPKI	simple public key infrastructure
SRTP	secure real-time transport protocol (RTP over SSL/TLS)
SSMTP	secure SMTP
SSL	secure sockets layer
STD	(Internet) standard
STUN	session traversal utilities for NAT
TCP	transport control protocol
THV	truncated hash value
TIME	timing info-leak made easy
TLS	transport layer security
TOFU	trust of f rst use
TOR	The Onion Router
TS	technical specif cation
TTP	trusted third party
UA	user agent
UBE	unsolicited bulk e-mail
UC	University of California
UCS	universal character set
UDID	unique device ID

UMAC	universal MAC
URI	uniform resource identif er
URL	uniform resource locator
US	United States
USD	US dollar
UTC	universal time coordinated
UTF	UCS transformation format
UU	UNIX to UNIX
UUID	unique user ID
VoIP	voice over IP
VPN	virtual private network
WG	working group
WKD	Web key directory
WKS	Web key service
WWW	World Wide Web
W3C	World Wide Web consortium
XEP	XMPP extension protocol
XML	extensible markup language
XMPP	extensible messaging and presence protocol
XOF	extendable output function
XSF	XMPP Standards Foundation
XZDH	extended zero knowledge Diff e-Hellman

About the Author

Rolf Oppliger[1] received an M.Sc. and a Ph.D. in computer science from the University of Berne, Switzerland, in 1991 and 1993, respectively. After spending a year as a postdoctoral researcher at the International Computer Science Institute (ICSI) of UC Berkeley, he joined the federal authorities of the Swiss Confederation in 1995 and continued his research and teaching activities at several universities in Switzerland and Germany. In 1999, he received the venia legendi for computer science from the University of Zurich, Switzerland, where he still serves as an adjunct professor. Also in 1999, he founded eSECURITY Technologies Rolf Oppliger to provide scientif c and state-of-the-art consulting, education, and engineering services related to information security and began serving as the editor of Artech House's Information Security and Privacy Series. Dr. Oppliger has published numerous papers, articles, and books, holds a few patents, regularly serves as a program committee member of internationally recognized conferences and workshops, and is a member of the editorial board of some prestigious periodicals in the f eld. He is a senior member of the Association for Computing Machinery (ACM), the Institute of Electrical and Electronics Engineers (IEEE) and its Computer Society, as well as a member of the IEEE Computer Society and the International Association for Cryptologic Research (IACR). Besides, he has also served as the vice-chair of the International Federation for Information Processing (IFIP) Technical Committee 11 (TC11) Working Group 4 (WG4) on network security. His full curriculum vitae is available online.[2]

1 rolf-oppliger.ch and rolf-oppliger.com.
2 https://www.esecurity.ch/Flyers/cv.pdf.

Index

3DH protocol, 245

A5/1, 69
Abstract Syntax Notation 1, 167
Achilles' heel, 66, 80
active attack, 108
adaptive chosen-ciphertext attack, 136
additional decryption key, 127
Adi Shamir, 42
administered groups, 265
administrators, 265
advanced, 190
Advanced Encryption Standard, 65
AES, 65
Akamai, 257
algorithm, 43
Allo, xii
Amazon, 257
America Online, 33
Android Messages, 35
anonymous identif ers, 298
anonymous messaging, 156
anonymous remailers, 298
ANSI X9.17, 69
AOL Instant Messenger, 34
Apple ID, 272
Apple push notif cation, 272
application programming interface, 300
ASCII armor, 143
ASN.1, 89, 167
assets, 251
assoiated data, 230
asymmetric, 46, 72
asymmetric encryption system, 72, 73
asynchronous, 133
asynchronous ratcheting tree, 309
attack resilient public key infrastructure, 306
attribute authorities, 86
attribute certif cates, 86
authenticated Diff e-Hellman key exchange protocol, 84
authenticated encryption, xii, 68, 165, 284
authenticated encryption with associated data, 68
authenticated key exchange, 84
authenticated received chain, 32
authentication and key distribution system, 80
authentication ceremonies, 159
authentication functions, 66
authentication tag, 66
authenticity, 66
Autocrypt, 192
Axolotl, xii, 5, 222

backward secrecy, 113
base-64, 142
basic constraints extension, 91
Basic Encoding Rules, 167
BBS generator, 70
bidirectional asynchronous RKE, 242
birthday paradox, 61
Bitcoin mining, 250
BitLocker, 51
Bitmessage, 108, 307
block cipher, 65
blockchain, 225
Blowf sh, 132
boundary, 23
Brainpool curves, 79, 137
branch prediction analysis, 53
Brian Acton, 261

browser reconnaissance and exf ltration via adaptive compression of hypertext, 141
built-in security, 111

CAcert, 183
cache timing attacks, 53
Caesar cipher, 55
Caligula Word 97 macro virus, 158
Carter-Wegman MACs, 68
CAST, 132
CBC-MAC, 68
CCM, 68
certif cate, 85, 153
certif cate distribution scheme, 89
certif cate repositories, 86
certif cate revocation list, 91, 94
certif cate signing request, 183
Certif cate Transparency, 95, 306
certif cate-only MIME entity, 176
certif cates, 72
certif cation authorities, 86
certif cation chain, 93
certif cation path, 93
certif cation service providers, 86
CGKA, 309
ChaCha20, 65, 69, 250
ChaCha20-Poly1305, 178
chain key, 234
chat server, 284
ChatSecure, 223
chosen ciphertext attacks, 74
chosen protocol, 44
chosen-ciphertext attack, 274
cipher, 63
cipher block chaining, 65
cipher feedback, 65, 132
ciphertext, 63
ciphertext space, 63
client-side fan-out, 239
cloud computing, 14
CloudFlare, 257
CMAC, 68
code-based cryptography, 57
cold boot attack, 51
collision resistance, 60
collision resistant, 61
common vulnerabilities and exposures, 268
complete trust, 151
complexity theory, 58
Compression Ratio Infoleak Made Easy, 141
compromising emanations, 51
computational complexity theory, 48
computational security, 48
computer program, 44
conditional, 48
Conditional security, 48
Conf dante, 306
Conf de, 300
conf dentiality protection, 63
CONIKS, 306
CONsistent Identity Key Service, 306
constructive step, 55
content delivery network, 257
content scrambling system, 69
content type, 23
continuous group key agreement, 309
conversations, 223, 281
counter, 65
CoverMe, 271
Crypho, 271
cryptanalysis, 42
Cryptocat, 223
cryptographic, 61
cryptographic hash functions, 57
Cryptographic Message Syntax, 166
cryptographic scheme, 43
cryptographic stamp, 308
cryptographic system, 43
cryptography, 42
cryptology, 41
cryptosystem, 43
cumulative trust model, 89, 148
Curve25519, 79, 137, 227, 284
Curve448, 79, 84, 218, 227
Curve448-Goldilocks, 84
cyclic redundancy check, 143
Cyphr, 271

DAKE with zero knowledge, 218
Dark Internet Mail Environment, 36
Dark Mail Access Protocol, 36
Dark Mail Alliance, 36
Dark Mail Transfer Protocol, 36
dark mode, 300
Data Encryption Standard, 65
DatChat, 300

David Chaum, 307
decrypt, 63
decryption, 63
decryption functions, 63
Defense Messaging System, 2
def nitional step, 55
DEFLATE, 137, 178
delta CRL, 94
DeltaChat, 193
deniability, 202
deniable AKE, 218
denial-of-service, 109
detached signature, 140
deterministic, 44
device identif er, 277
device identity public key pair, 277
device registration protocol, 289
dictionary attack, 145
differential fault analysis, 53
Diff e-Hellman, 59
Diff e-Hellman group 5, 206
Diff e-Hellman key exchange, 81
Diff e-Hellman problem, 82
Diff e-Hellman ratchet, 213, 222
digest, 61
digital f ngerprinting, 43
digital signature, 76
digital signature algorithm, 79, 121
digital signature giving message recovery, 76
digital signature scheme, 43
digital signature system, 43, 66
digital signature with appendix, 76
digital watermarking, 43
direct, 190
direct trust model, 148
Directory Access Protocol, 31
directory information tree, 95
directory server, 284
discrete logarithm problem, 81, 121
distinguished name, 88
distributed denial-of-service, 109
distributed hash tables, 300
distributed ledger technology, 225
DKIM, 193
DNS Certif cation Authority Authorization, 95
DNS security, 33
DNS-based Authentication of Named Entities, 33, 95
DNSSEC, 191
domain, 58
domain fronting, 257, 309
domain name system, 15
domain-based message authentication, reporting, and conformance, 31
DomainKeys identf ed mail, 31
dot notation, 168
double ratchet, 214, 222
Dragstuhl Privacy Preserving Presence Protocol Privacy, 302
DSA, 79
DSS giving message recovery, 77
DSS with appendix, 76
Dust, 271
dynamic random access memory, 51

e-mail, 1
e-mail bombing, 109
e-mail message, 13
E0, 69
eSECURITY Technologies Rolf Oppliger, 325
eastern daylight time, 17
easy, 58
eavesdropping attacks, 106
EAX, 136
ECC, 79
ECDH, 84
ECDHE, 84
ECDSA, 79
ECIES, 272
ECIES-KEM, 272
Ed25519, 79, 137, 227, 251, 254
Ed448-Goldilocks, 218
EdDSA, 227
Edwards curves, 79
Edwards-curve DSA, 79
EFAIL, xii, 106, 119, 131, 165, 176
electronic codebook, 65
Electronic Frontier Foundation, 268
Electronic mail, 1
Elgamal, 74, 79, 121
Elgamal asymmetric encryption system, 82
Elixxir, 108, 307
elliptic curve cryptography, xii, 79
elliptic curve Diff e-Hellman, 84
elliptic curve Diff e-Hellman ephemeral, 84
elliptic curve DSA, 79

elliptic curve integrated encryption scheme, 272
emanations security, 51
emission security, 51
Encipher.it, 197
encrypt, 63
encrypted key exchange, 84
encrypted message payload, 280
encrypted packet header, 279
Encryption, 63
encryption functions, 63
End-To-End, 196
end-to-end encrypted, xii
end-to-end encryption, 5
end-to-end-encrypting messaging application, 281
enhanced security services, 167
Enigmail, 194
entropy, 184
ephemeral, 84, 227
ephemeral messaging, 299
Euler's theorem, 75
exchange key, 279
expected running time, 58
exponential key exchange, 81
extended Euclid algorithm, 75
extended SMTP, 24
eXtended Triple Diff e-Hellman, 229
Extended zero knowledge Diff e-Hellman, 218
extensible messaging and presence protocol, 4, 34

Facebook, 307
FaceTime, 272
FakesApp, 268
Fermat's little theorem, 75
f le, 125
FileVault, 51
f ngerprint, 61
f nite state machine, 69
Flow, 36
FlowCrypt, 197
forced-latency protocol, 84
format oracle attack, 275
forward secrecy, 112
forward secure public key encryption, 306
franking mechanism, 308
franking tag, 308
future secrecy, 113

GAIM, 205
GCM, 68
general data protection regulation, 298
GMAC, 68
Gmail, 196
GMime, 194
GNU Privacy Guard, 120
GNUnet, 196
GnuPG Project, 120
Gpg4win, 120, 146
graphical user interfaces, 4
Greenwich mean time, 17
greylisting, 31
group OTR, 204
group ratcheting, 254
GSM Association, 35

handshake key pair, 245
hard, 58
hardware security modules, 182
hash function, 60
hash ratchet, 222
hash-based (one-time) signature systems, 57
hashed MAC, 67
header f elds, 17
header section, 17
hierarchical trust model, 89, 148
HKDF, 250
HMAC-based extract-and-expand key derivation function, 227
Hoccer, 271
hosted S/MIME, 197
HSalsa20, 285
HTTP keyserver protocol, 155
HTTP strict transport security, 33
Huffman coding, 138
Hushmail, 197
hybrid cryptosystems, 99

iChat, 34
iCloud, 274
ICQ, 33
ICZ attack, 156
IDEA, 117
ideal system, 47
identity key, 226

identity key pair, 226
identity misbinding attack, 203, 241
identity-based encryption, 99
IETF, 12
iMessage, xii, 5, 272
Incognito Mode, xii
Inf nite Garble Extension, 291
information theory, 48, 56
information-theoretic security, 48
initialization vector, 133
initiator, 228
instant messaging, xi, 11
integrity, 66
intellectual property rights, 8
interlock protocol, 84
intermediate CAs, 93
International Computer Science Institute, 325
International Data Encryption Algorithm, 117
International Electrotechnical Committee, 87
International Organization for Standardization, 87
International Telecommunication Union, 1, 87
Internet Engineering Task Force, 2
Internet Key Exchange, 225
Internet mail architecture, 12
Internet Message Access Protocol, 14
Internet message format, 14
Internet messaging, 11
Internet relay chat, 33
Internet Research Task Force, 2
introducer, 97, 148, 152
invisible ink, 43
Invisible Internet Project, 263
IP security, 225
iPGMail, 120
IPsec, 2, 225
issuer, 90
ITU-T, 1, 87
ITU-T X.509, 87

Jabber, 4, 34
Jabber Software Foundation, 35
Jan Koum, 261
Java, 44
Jibe platform, 35
Joint Technical Committee 1, 87

K-9, 120, 193
KaKaoTalk, 271
Kasper Systems GmbH, 281
KDF chain, 231
KDF key, 231
Kerberos, 80
Kerckhoffs' principle, 54
key agreement, 81
key bundle, 225, 227
key derivation function, 231
key distribution center, 80
key distribution protocol, 81
key encapsulation mechanism, 309
key establishment protocol, 80, 81
key exchange data, 279
key exchange list, 279
key exchange protocol, 81
key ID, 122
key identif er, 96, 122
key legitimacy, 147, 150
key length, 64
key repository, 225
key revocation, 153
key revocation certif cate, 154
key servers, 154
key space, 63, 66
key usage extension, 91
Keybase, 306
Keybase Chat, 306
Keychain, 273
keyed-verif cation anonymous credentials, 241
KEYLEGIT, 150
Kleopatra, 146
Klíma-Rosa, 156
knapsack problem, 56

last resort prekey, 251
lattice-based cryptography, 57
Lavabit, 36
leaf certif cates, 93
leakage-resilient cryptography, 54
LEAP Encryption Access Project, 192
LEAP Platform, 192
Lempel-Ziv 77, 137
length, 124
letter sealing, 271
Lets Encrypt, 182
LFSR, 69
Libgcrypt, 205

Libra, 307
libsodium, 250
Lightning, 263
Lightweight Directory Access Protocol, 31
Limited Additional Mechanisms for PKIX and SMIME, 165
Line, 271
linear feedback shift register, 69
local mail transfer protocol, 15
local namespaces, 88
local registration agents, 86
local registration authorities, 86
local storage device key, 277
long key ID, 123

MAC, 66
Magma, 36
mail, 1
mail client, 13
mail exchanger, 15
mail list agent, 181
mailer, 13
Mailpile, 193
Mailvelope, 197
malware-in-the-middle, 83
man-in-the-middle, 33, 83, 108
Manuel Kasper, 281
Matrix, 35, 252
max-entropy, 184
Mazapp, 262
McAfee, 118
MD5, 61
media server, 284
Megolm, 254
Melissa, 310
Meltdown, 53
memory-hard, 250
message, 13
message authentication code, 66
message authentication system, 63, 66
message delivery agent, 15
message digest, 169
message franking, 307
message handling systems, 1
message integrity check, 175
message integrity code, 66
message key, 234
message layer security, 7, 36, 309
message part, 125
message payload encryption key, 279
Message Security Protocol, 1
message session relay protocol, 35
message stores, 12
message submission agents, 14
message transfer agents, 1, 12
message transfer system, 13
message user agents, 1
messaging, 11
messaging infrastructure, 12
meta-introducer, 152
MeTA1, 13
Microsoft Azure, 257
Microsoft Exchange, 2
millionaires' problem, 203
MIME object security services, 2
min-entropy, 184
MIT PGP Public Key Server, 155
MITRE, 268
mix network, 108
MLS*, 310
MLS++, 310
modif cation detection code, 136
modular exponentiation, 59
Molasses, 310
Molch, 223
monkey-in-the-middle, 83
MOSS, 2
Moxie Marlinspike, 222
mpOTR, 204, 239
MSN Messenger, 34
MSRP, 35
MSRP security, 35
MSRPS, 35
MTA Strict Transport Security, 33
MTProto, 289
muacrypt, 194
multiparty OTR, 204
multiprotocol attack, 44
multimedia messaging service, xi
multiple user ID attack, 158
multipurpose Internet mail extensions, 2, 14
multivariate cryptography, 57

NaCl, 250, 284
National Security Agency, 51
NIST curve P-256, 272

NIST curve P-521, 276
NIST curves, 137
node, 276
node storage root key, 276
Noise protocol framework, 263
nonce, 44
nonsecret encryption, 56
number used once, 44

object identif er, 90, 168
oblivious RAM, 256
OCB, 136
off-the-record, xii, 5, 202
Off ce 365, 165
OID, 90
Olm, 223, 252, 264
OMEMO, 35, 223
OMEMO Multi-End Message and Object Encryption, 223
one way, 57, 58
one-time prekey pairs, 226
one-time prekeys, 226
one-way functions, 57
one-wayness, 60
onion routers, 108
onion routing, 108, 298
Online Certif cate Status Protocol, 95
open, 2
Open Specif cation for Pretty Good Privacy, 3
open system for communication in realtime, 34
Open Whisper Systems, 222
oPenGP, 120
OpenKeychain, 120
openMittsu, 281
OpenPGP, 2, 3
OpenPGP certif cates, 89
OpenPGP CFB, 133, 141
OpenPGP message, 125
OPENPGPKEY resource record, 191
OpenSSL, 291
opportunistic encryption, 191, 228
optimal asymmetric encryption padding, 76
Opus audio codec, 225
originator, 13
OSI security architecture, 110
OTR messaging, 202
Outlook.com, 196
output feedback, 65
owner trust, 147, 150
OWNERTRUST, 150

P-256, 79, 137
P-384, 79, 137
P-521, 79, 137
P42, 1
packet header encryption key, 279
packet signature, 280
padding oracle attacks, 227
Pascal, 44
passive attack, 106
passive wiretapping, 106
password authenticated key exchange, 84
password-based key derivation function 2, 286
patents, 8
PathServer, 151
PEM, 2
perfect forward secrecy, 112
perfect secrecy, 112
PGP, 2
PGP CFB, 133
PGP Corporation, 119
PGP Global Directory, 155
PGP Universal Server, 192
Philip R. Zimmermann, 117
phishing, 45
physically observable cryptography, 54
Pidgin, 205
plaintext message, 63
plaintext message space, 63, 66
plausible deniability, 202
poll feature, 281
Poly1305, 68
polynomial, 58
Post Off ce Protocol, 14
post-compromise security, 113
post-quantum cryptography, 57
Postf x, 13
power analysis attacks, 53
power consumption, 53
pre-compromise security, 113
preimage resistance, 60
prekey bundle, 227
prekeys, 226
Pretty Easy Privacy, 192
Pretty Good Privacy, 2, 118
PRFs, 63

PRG, 68
PRGs, 63
privacy, 297
Privacy and Security Research Group, 2
Privacy enhanced mail, 2
privacy-enhancing technologies, 298
private, 72
private group messaging, 239
private information retrieval, 302
private keyring, 146
Private WebMail, 197
probabilistic, 44
probabilistic signature scheme, 177
probability theory, 48
programming language, 44
promiscuous mode, 107
Propagating Cipher Block Chaining, 291
Proposed Standards, 4
proprietary, 2
Proteus, 223, 250
protocol, 44
ProtonMail, 197
provable security, 49
Pryvate, 271
pseudorandom, 68
pseudorandom bit generator, 68
pseudorandom functions, 63
pseudorandom generators, 63
pseudorandom permutation, 71
public key, 72
public key certif cate, 85
public key cryptography, 73
public key cryptography standards, 163
public key cryptosystem, 46, 72
public key infrastructure, 72, 86
public keyring, 146
Public-Key Infrastructure X.509, 87
puncturable encryption, 306
PyAC, 194

qmail, 13
QR code, 237
quantum computing, 57
quick check, 132

Rabin, 59, 74
radix-64, 121, 143
Rakuten, 243
Rakuten Viber, 243
random bit generator, 62, 68
random function, 50, 70
random generator, 57, 62
random oracle, 50, 70
random oracle methodology, 50
random oracle model, 50
random permutation, 71
randomized, 44
range, 58
ratchet, 254
ratchet key pair, 245
ratcheted key exchange, 242
RC4, 65, 69
RCS, 35
real-time transport protocol, 35
receiving chain, 232
recipients, 13
recovery bundle, 277
recovery bundle key, 276
RedPhone, 222
Refraction Networking, 309
registration authorities, 86
remote attestation, 256
remote storage root key, 276
reporting tag, 308
Request for Comments, 4
REQUIRETLS, 33
responder, 228
revoker, 154
Rich Communication Services, 35
Ricochet, 302
Riot, 252, 264
Riot.im, 252
RIPEMD, 132
RIPEMD-160, 132
robustness principle, 177
rooms, 281
root CAs, 92
root chain, 232
root identif er, 276
root identity public key pair, 276
root key, 233
RSA, 59, 74, 79
RSA Data Security, 118
RSA patent, 166
RSA Security, 118, 163
RSA-OAEP, 76

RSA-PSS, 177
RSAREF cryptographic library, 118
RSASSA-PSS, 177
RTP, 35
running time, 58

S/MIME, 2, 163
S/MIME Mail Security, 3, 165
S/MIME versions 1–3, 165
SafeSlinger, 271
Salsa20, 65, 69, 284
science, 55
SCIMP, 222
ScreenShield, 300
screenshot-proof, 300
scrypt, 250
SDSI, 88
second-preimage resistance, 60
second-preimage resistant, 60
secret chat, 289
secret key, 72
secret key cryptography, 73
secret key cryptosystem, 46
secret key ratchet, 213
secret sharing, 147
SecuMail, 120
secure, 48
Secure Messaging Scorecard, 268
secure MIME, 2, 163
secure RTP, 35
Secure SMTP, 24
Secure Sockets Layer, 2, 81
secure Web server, 81
Secure/Multipurpose Internet Mail Extensions, 163
Security Area, 3
security labels extension, 180
security number, 237
seed, 68
self-decrypting archive, 142
Self-destructing messaging, 299
self-signature, 96
self-signed certif cate, 183
self-synchronizing, 133
semantically secure, 273
sender key, 239
Sender Keys, 239, 248, 266
sender policy framework, 31
sending chain, 232
Sendmail, 13
server name indication, 257
server-side fan-out, 239
service-oriented architectures, 14
session initiation protocol, 35
session key, 122
session key part, 126
session start message, 248
SHA-1, 61
SHA-2, 61
SHA-224, 61
SHA-256, 61
SHA-3, 61
SHA-384, 61
SHA-512, 61
Shannon entropy, 184
shareholders, 147
short key ID, 123
short message service, xi
side channel attacks, 52
SIGMA authenticated key exchange protocol, 203
Signal, xii, 5, 222
Signal Foundation, 223
Signal Messenger, 224
Signal protocol, 222
signatory, 76
signature part, 126
signature trust, 147, 150
signed prekey pair, 226
signed receipts extension, 179
signer, 76
signer attributes, 178
signing certif cates attribute, 181
SIGTRUST, 150
Silence, 271
Silent Circle, 243
Silent Circle Instant Messaging Protocol, 213
Silent Phone, 243
Simple Authentication and Security Layer, 29
Simple Distributed Security Infrastructure, 88
Simple Key-Management for Internet Protocol, 225
Simple Mail Access Protocol, 30
Simple Mail Transfer Protocol, 13
Simple Public Key Infrastructure, 88
SIMSme, 271

SIP, 35, 272
SIP for instant messaging and presence leveraging extensions, 35
SIP security, 35
SIPS, 35
SixChat secure messaging app, 34
Skype, 34, 223
Slack, 306
SMIME, 3
SMIMEA resource record, 191
Snapchat, 299
social authentication, 306
social engineering attacks, 45
socialist millionaires' problem, 203
socialist millionaires' protocol, 203
software guard extensions, 256
spam, 14, 31, 110
Spectre, 53
SQLCipher, 225
SQLite, 225
squaring generator, 70
SRTP, 35, 272
steganography, 43
stream cipher, 65
stretch function, 69
STUN, 272
subject, 91
subtype, 23
super-polynomial, 58
Surespot, 271
symmetric, 46
symmetric encryption system, 63
symmetric key, 222
Synchronizing Key Server, 155
synchronous, 11, 133
synthetic initialization vector, 227
systematization of knowledge, 5

tag, 123
tag space, 66
Tchap, 252
Telecommunication Standardization Sector, 87
Telegram, xii, 5, 289
Telex, 309
TEMPEST, 51
Tempest, 158
textbook versions, 74
TextSecure, xii, 5, 222
The Onion Router, 108
threats model, 47
Threema, xii, 5, 262, 281
Threema Gateway, 281
Threema GmbH, 281
Threema ID, 282, 298
Threema Safe, 286
Threema Web, 281
Threema Work, 281
timing attack, 52
timing info-leak made easy, 141
TOFU, 228
TOR, 298, 302
TOR hidden services, 302
ToTok, 271
Tox, 35
traff c analysis attacks, 107
Transport Layer Security, 2, 81
trapdoor, 59
trapdoor function, 56, 59
trapdoor one-way function, 59
TreeKEM, 309
Trevor Perrin, 222
Triple Diff e-Hellman, 229
TrueCrypt, 51
truncated hash value, 294
trust model, 89, 146
trust of f rst use, 192
trusted introducer, 152
trusted party, 148
trusted third parties, 86
TrustZone, 256
Tutanota, 198
Twilio, 224
Twof sh, 132

unconditional, 48
Unconditional security, 48
unique device ID, 243
unique user ID, 251
universal MAC, 68
universal time coordinated, 17
unkeyed cryptosystem, 46
unknown key-share attack, 241
unsolicited bulk e-mail, 14
usability, 159
user agents, 12
user ID, 96, 122

user identif er, 122

validity f eld, 150
validity period, 90
Vanish, 300
VeraCrypt, 51
verif cation functions, 66
verif cation level, 283
verif er, 76
Viacrypt, 118
Viber, xii, 5, 243
Viber Media, 243
Vk.com, 289
Volcano, 36
VPNs, 263

W3C, 35
weak collision resistance, 61
Web browser, 81
Web Key Directory, 190
Web Key Service, 191
web of trust, 97
Web-based messaging, 13
WebPG, 197
WebRTC, 35
WeChat, 5, 307
WeChat Pay, 307
WhatsApp, xii, 34, 239, 261
Whisper Systems, 222
Wickr, xii, 5, 275
Wickr Enterprise, 275
Wickr IO, 275
Wickr Me, 275
Wickr messaging protocol, 275
Wickr Pro, 275
Wire, xii, 5
WireGuard, 263
Wireshark, 224
World Wide Web Consortium, 35

X.400, 1
X.500 directory, 89
X.509, 87, 89
X.509 v1–v3, 89
X.509 version 1–3, 89
X25519, 85, 227, 250
X448, 85
X9.17, 144
XMPP extension protocol, 35
XMPP Standards Foundation, 35
XSalsa20, 284

Yahoo Messenger, 34
Yahoo Mail, 196
Yarrow, 69
yowsup, 262

Z-Base-32, 190
ZIP, 137
ZLIB, 178

Recent Titles in the Artech House Information Security and Privacy Series

Rolf Oppliger, Series Editor

Attribute-Based Access Control, Vincent C. Hu, David F. Ferraiolo, Ramaswamy Chandramouli, and D. Richard Kuhn

Biometrics in Identity Management: Concepts to Applications, Shimon K. Modi

Bitcoin and Blockchain Security, Ghassan Karame and Elli Androulaki

Bluetooth Security, Christian Gehrmann, Joakim Persson and Ben Smeets

Computer Forensics and Privacy, Michael A. Caloyannides

Computer and Intrusion Forensics, George Mohay, et al.

Defense and Detection Strategies against Internet Worms, Jose Nazario

Demystifying the IPsec Puzzle, Sheila Frankel

Developing Secure Distributed Systems with CORBA, Ulrich Lang and Rudolf Schreiner

Electric Payment Systems for E-Commerce, Second Edition, Donal O'Mahony, Michael Peirce, and Hitesh Tewari

End-to-End Encrypted Messaging, Rolf Oppliger

Evaluating Agile Software Development: Methods for Your Organization, Alan S. Koch

Fuzzing for Software Security Testing and Quality Assurance, Second Edition, Ari Takanen, Jared DeMott, Charlie Miller, and Atte Kettunen

Homeland Security Threats, Countermeasures, and Privacy Issues, Giorgio Franceschetti and Marina Grossi, editors

Identity Management: Concepts, Technologies, and Systems, Elisa Bertino and Kenji Takahashi

Implementing Electronic Card Payment Systems, Cristian Radu

Implementing the ISO/IEC 27001 ISMS Standard, Second Edition, Edward Humphreys

Implementing Security for ATM Networks, Thomas Tarman and Edward Witzke

Information Hiding Techniques for Steganography and Digital Watermarking, Stefan Katzenbeisser and Fabien A. P. Petitcolas, editors

Internet and Intranet Security, Second Edition, Rolf Oppliger

Introduction to Identity-Based Encryption, Luther Martin

Java Card for E-Payment Applications, Vesna Hassler, Martin Manninger, Mikail Gordeev, and Christoph Müller

Multicast and Group Security, Thomas Hardjono and Lakshminath R. Dondeti

Non-repudiation in Electronic Commerce, Jianying Zhou

Outsourcing Information Security, C. Warren Axelrod

Privacy Protection and Computer Forensics, Second Edition, Michael A. Caloyannides

Role-Based Access Control, Second Edition, David F. Ferraiolo, D. Richard Kuhn, and Ramaswamy Chandramouli

Secure Messaging on the Internet, Rolf Oppliger

Secure Messaging with PGP and S/MIME, Rolf Oppliger

Security Fundamentals for E-Commerce, Vesna Hassler

Security Technologies for the World Wide Web, Second Edition, Rolf Oppliger

SSL and TLS: Theory and Practice, Second Edition, Rolf Oppliger

Techniques and Applications of Digital Watermarking and Content Protection, Michael Arnold, Martin Schmucker, and Stephen D. Wolthusen

User's Guide to Cryptography and Standards, Alexander W. Dent and Chris J. Mitchell

For further information on these and other Artech House titles, including previously considered out-of-print books now available through our In-Print-Forever® (IPF®) program, contact:

Artech House
685 Canton Street
Norwood, MA 02062
Phone: 781-769-9750
Fax: 781-769-6334
e-mail: artech@artechhouse.com

Artech House
16 Sussex Street
London SW1V 4RW UK
Phone: +44 (0)20 7596-8750
Fax: +44 (0)20 7630-0166
e-mail: artech-uk@artechhouse.com

Find us on the World Wide Web at: www.artechhouse.com